



Guidewire PolicyCenterTM

Gosu Reference Guide

Release 10.1.2

© 2021 Guidewire Software, Inc.

For information about Guidewire trademarks, visit <http://guidewire.com/legal-notice>.

Guidewire Proprietary & Confidential — DO NOT DISTRIBUTE

Product Name: Guidewire PolicyCenter

Product Release: 10.1.2

Document Name: Gosu Reference Guide

Document Revision: 14-June-2021

Contents

Guidewire Documentation	11
About PolicyCenter documentation	11
Support	13

Part 1

Gosu language basics

1 Gosu introduction	17
Basic Gosu	18
Overview of control flow.	18
Overview of blocks.	20
Overview of enhancements	21
Overview of collections.	21
Overview of access to Java types	22
Overview of Gosu classes and properties.	22
Overview of interfaces	26
List and array expansion operator *.	27
Overview of comparisons	27
Gosu character set and case sensitivity	28
Compound assignment statements	28
Delegating interface implementation with composition	29
Overview of concurrency	30
Overview of exceptions.	30
Overview of annotations	31
Overview of templates	32
Overview of XML and XSD support.	32
Overview of web service support for consuming WSDL	33
Running Gosu programs and calling other classes	34
Guidewire internal classes	35
More about the Gosu type system	35
Compile-time error prevention	36
Type inference	36
Intelligent code completion and other Gosu editor tools	36
Null safety for properties and other operators.	37
Generics in Gosu	38
Gosu primitive types.	39
Gosu statement terminators.	39
Gosu comments.	40
Gosu reserved words	41
Legal variable names	42
Gosu generated documentation (Gosudoc)	42
Code coverage support.	42
Notable differences between Gosu and Java	42
2 Gosu types.	49
Primitive Gosu types	49
Gosu objects	50
Object instantiation in Gosu.	50

Object property assignment	51
Object property access	52
Object methods in Gosu	53
Using Boolean values in Gosu.	53
Using characters and String data in Gosu	54
String variables can have content, be empty, or be null	55
Setting values of String properties in entity instances	55
Methods on String objects	56
String utilities	56
Escaping special characters in strings	56
Gosu String templates	57
Numeric literals in Gosu.	58
Using date and time values in Gosu	59
Entity types.	60
Keys of Guidewire entity instances.	61
Typekeys and typelists	61
Getting all available typekeys from a typelist	62
Getting information from a typekey	62
Gosu array types	63
Creating a Gosu array	63
Initializing a Gosu array with default values	63
Accessing elements of a Gosu array	63
Iterating the elements of a Gosu array	64
Gosu arrays and the expansion operator.	64
Array list access with array index notation.	65
Entity arrays	66
Casting arrays	68
Using Gosu to access Java types	69
Compatibility with earlier Gosu releases	70
3 Operators and expressions	71
Gosu operators	71
Operator precedence	71
Standard Gosu expressions	73
Arithmetic expressions in Gosu	73
Equality expressions	78
Evaluation expressions	81
Logical expressions	81
Using the operator new in object expressions.	83
Relational expressions.	87
Unary expressions	89
Conditional ternary expressions	90
Special Gosu expressions	92
Function calls.	92
Static method calls	93
Static property paths	93
Entity and typekey type literals	93
Handling null values in expressions	94
Null-safe property access.	94
Null-safe method access	96
Null-safe default operator	96
Null-safe indexing for arrays, lists, and maps.	97
Null-safe math operators	97
Handling null values in logical expressions	97
Best practices for using null-safe operators	98

List and array expansion expressions (*)	98
4 Gosu building blocks	99
Gosu statements	99
Statement lists	99
Using the operator new as a statement.	100
Gosu variables	100
Variable type declaration	101
Variable assignment	101
Gosu conditional execution and looping	105
if - else statements	105
for statements.	105
while statements.	107
do...while statements	108
Examples of Gosu conditional iteration and looping	108
switch statements	110
Gosu functions	111
Named arguments and argument defaults.	112
Public and private functions.	113
Importing types and package namespaces.	114
Packages always in scope	115
Importing static members	115
5 Exception handling	117
Handling exceptions with try/catch/finally	117
Catching exceptions	118
Throwing exceptions	119
Object life-cycle management with using clauses	120
Assigning variables inside a using expression declaration	121
Passing multiple items to a using statement	121
Disposable objects and using clauses	122
Closeable objects and using clauses	123
Reentrant objects and using clauses	124
Returning values from using clauses	125
Optional use of a finally clause with a using clause.	126
Assert statements.	127
6 Intervals	129
What are intervals?	129
Reversing interval order	130
Granularity of interval steps and units	130
Writing your own interval type	131
Custom iterable intervals using sequenceable items	131
Custom iterable intervals using manually written iterators	133
Example: Color interval written with manual iterators.	134
Custom non-iterable interval types	136
7 Calling Java from Gosu	137
Core Java types in Gosu	138
Using Java types in Gosu	139
Using Java get, set, and is methods as Gosu properties	139
Using Java static members and static import in Gosu	141
Using Java generics	141
Using other Java features in Gosu	142
Using a Java properties file.	143

8	Classes	145
	Creating and instantiating classes	146
	Extending another class to make a subclass	146
	Constructors	147
	Static methods and variables	148
	Using public properties instead of public variables	148
	Creating a new instance of a class	149
	Naming conventions for packages and types	149
	Using uses declarations to import a type	150
	Properties	150
	Defining properties	151
	Overriding a property definition in a subclass	152
	Property paths are null tolerant	153
	Designing APIs around null-safe property paths	155
	More property examples	155
	Modifiers	157
	Access modifiers	157
	Override modifier	158
	Abstract modifier	158
	Final modifier	160
	Static modifier	162
	Inner classes	164
	Named inner classes	164
	Referring to members of the outer class	165
	Static inner classes	165
	Accessing an inner class from a class that extends the outer class	166
	Anonymous inner classes	166
9	Enumerations	169
	Working with enumerations	171
	Create an enumeration	172
	Extract information from an enumeration	172
10	Interfaces	173
	What is an interface?	173
	Defining and using an interface	174
	Create and implement an interface	175
	Defining and using properties on interfaces	176
	Modifiers and interfaces	177
	Create an interface with getters, setters, and methods	177
	Static properties and methods on interfaces	178
11	Blocks	181
	What are blocks?	181
	Basic block definition and invocation	182
	Block return types	183
	Invoking blocks	183
	Variable scope and capturing variables in blocks	184
	Argument type inference shortcut	185
	Block type literals	185
	Blocks and collections	187
	Blocks as shortcuts for anonymous classes	187
12	Collections	189
	Basic lists	189
	Type inference and list initialization	190

Getting and setting list values	190
Basic hash maps	191
Getting and setting values in a hash map	192
Special enhancements on maps	192
Wrapped maps with default values	192
List expansion (*).	194
Array flattening to a single dimensional array	195
Working with collections	196
Finding data in collections	196
Sorting lists or other comparable collections	197
Mapping data in collections	198
Iterating across collections	198
Partitioning collections	199
Converting lists, arrays, and sets	200
Flat mapping a series of collections or arrays	200
Sizes and length of collections and strings are equivalent	201
Collections and Gosu enhancements	201
Enhancements on Gosu collections and related types	201
List of enhancement methods and properties on collections	201
List of enhancement methods and properties on maps	211
13 Enhancements	215
Using enhancements	216
Creating a new enhancement	216
Syntax for defining enhancements	217
14 Coding style	219

Part 2

Advanced Gosu features

15 Annotations	227
Annotating a class, method, type, class variable, or argument	227
Function argument annotations	228
Annotations provided by Gosu	229
Getting annotations at run time	232
Annotation access at run time	232
Getting multiple annotations of multiple types	233
Getting annotation data from a type at run time	233
Defining your own annotations	234
Limiting annotation usage to specific code contexts	235
Setting retention of an annotation	236
16 Dimensions	237
Dimensions expressions example (Score)	238
Built-in dimensions	239
17 Generics	241
Overview of Gosu generics	241
Compatibility of Gosu generics with Java generics	242
Parameterizing a class	244
Parameterizing a method	246
Generic methods that reify a type	246
Restricting generics	248
Using generics with collections and blocks	250

Multiple dimensionality generics	251
Generics with custom containers	252
18 Dynamic types and expando objects	255
Dynamic type	255
Expando objects	257
IExpando interface	258
The Expando class	258
Concise syntax for dynamic object creation	259
19 Structural types	263
Overcoming the limitations of nominal typing.	263
Create a structural type with getters, setters, and methods.	265
Adding static constant variables to a structural type	266
Assignability of an object to a structural type	268
Assignability of a structural type to another type	268
Using enhancements to make types compatible with structures	268
20 Composition	271
Delegation syntax	272
Composition and delegates	273
Overriding methods independent of the delegate class.	273
Declaring delegate implementation type in the variable definition	273
Using one delegate for multiple interfaces	274
Using composition with built-in interfaces	274
21 Working with XML in Gosu	275
Introduction to the XML element in Gosu.	275
XmlElement core property reference.	277
Contents of an XML element	278
Create an XmlElement	278
Accessing XSD type properties	279
Exporting XML data	280
Export-related methods on an XML element	280
XML serialization options reference and examples	281
Serialization performance and element sorting	283
Parsing XML data into an XML element	283
Referencing additional schemas during parsing	284
Creating many qualified names in the same namespace	285
XSD-based properties and types	286
Important concepts in XSD properties and types.	286
XSD generated type examples	290
Automatic insertion into lists	291
XSD list property example.	292
Customizing XSD type code generation to exclude types	293
Exclude XSD types from code generation	294
Special handling of very large XSD enumerations	294
Getting data from an XML element	295
Manipulating elements and values	295
XML attributes.	298
XML simple values	298
Methods to create XML simple values	299
XSD to Gosu simple type mappings	301
Facet validation	302
Accessing the nilness of an element	302
Automatic creation of intermediary elements	303

Default and fixed attribute values	303
Substitution group hierarchies	304
Element sorting for XSD-based elements	305
Sorting of XSD-based elements already in the correct order.	307
Sorting of XSD-based elements matching multiple correct orders.	308
Built-in schemas	308
The metaschema XSD that defines an XSD	309
Using a local XSD for an external namespace or XSD location	309
Use a local XSD for an external namespace URI or XSD location URL	309
Schema access type	310
Conversions from XSD types to Gosu types	311
22 Working with JSON in Gosu	313
Dynamic access to JSON objects	314
JSON APIs for URLs	315
Generating URLs from JSON data	315
Structural type-safe access to JSON objects	317
Create a JSON structural type from an example object	317
Problematic property names for JSON structural types	318
Concise dynamic object creation syntax for JSON coding	319
Useful JSON API methods	321
23 Templates	323
Template expressions	323
Running Gosu statements in a template scriptlet	324
Escaping special characters for templates	325
Using template files	326
Creating and running a template file	327
Create and use a template file.	327
Template parameters	329
Support and use parameters in a template file.	329
Maximum file-based template size	330
Extending a template from a class.	330
Template comments	331
Template export formats	331
24 Bundles and database transactions	333
When to use database transaction APIs.	333
Overview of bundles	335
Accessing the current bundle	336
Warning about transaction class confusion.	336
Adding entity instances to bundles.	336
Making an entity instance writable by adding to a bundle	337
Moving a writable entity instance to a new writable bundle	337
Getting the bundle of an existing entity instance	338
Getting an entity instance from a public ID or a key.	338
Creating new entity instances in specific bundles.	339
Committing a bundle explicitly in very rare cases	339
Removing entity instances from the database	340
Determining what data changed in a bundle	341
Detecting property changes on an entity instance	343
Getting changes to entity arrays in the current bundle	343
Getting added, changed, or deleted entities in a bundle	344
Running code in an entirely new bundle	344
Creating a bundle for a specific PolicyCenter user	346
Effects of exception handling on database commits	346

Bundles and published web services	346
Entity cache versioning, locking, and entity refreshing.	346
Entity instance versioning and the entity touch API	346
Record locking for concurrent data access	347
User interface bundle refreshes.	347
Details of bundle commit steps	347
25 Type system	349
Basic type checking	349
Retrieving the type by using typeof.	349
Testing the type by using typeis	350
Basic type coercion	351
Automatic type downcasting	351
When automatic downcasting occurs	352
Type metadata properties and methods	353
Accessing general type metadata	354
Using feature literals	355
Chaining feature literals by using feature literal syntax	355
Binding a property to an instance by using feature literal syntax.	355
Binding argument values by using feature literal syntax	356
Using feature literal syntax to convert method references to blocks.	356
Primitive and boxed types.	356
Compound types	356
Using reflection.	357
Getting properties by using reflection	358
Retrieving methods by using reflection	359
Comparing types by using reflection.	360
Java type reflection.	360
Type system class	361
Calling Gosu from Java by using reflection	362
26 Concurrency and thread safety.	365
Request and session scoped variables.	367
Concurrent lazy variables	368
Concurrent cache.	369
Create a thread-safe cache	370
Concurrency with monitor locks and reentrant objects	370
27 Checksums	373
Creating a fingerprint.	374
Extending a fingerprint.	375
28 Gosu programs and the command-prompt tool	377
Gosu programs	378
The structure of a Gosu program	378
Write and run a Gosu program	378
Gosu command-prompt-tool	379
Setting up the Gosu command-prompt tool	379
Command-prompt tool and options.	379
Command-prompt arguments.	380
Create and run a Gosu program that processes arguments	380

Guidewire Documentation

About PolicyCenter documentation

The following table lists the documents in PolicyCenter documentation:

Document	Purpose
<i>InsuranceSuite Guide</i>	If you are new to Guidewire InsuranceSuite applications, read the <i>InsuranceSuite Guide</i> for information on the architecture of Guidewire InsuranceSuite and application integrations. The intended readers are everyone who works with Guidewire applications.
<i>Application Guide</i>	If you are new to PolicyCenter or want to understand a feature, read the <i>Application Guide</i> . This guide describes features from a business perspective and provides links to other books as needed. The intended readers are everyone who works with PolicyCenter.
<i>Upgrade Guide</i>	Describes the overall upgrade process, and describes how to upgrade your configuration and database. The intended readers are implementation engineers who must merge base application changes into existing application extensions and integrations.
<i>Configuration Upgrade Tools Guide</i>	Describes the tools and functionality provided by the Guidewire InsuranceSuite Configuration Upgrade Tools. The intended readers are implementation engineers who must merge base application changes into existing application extensions and integrations. Visit the Guidewire Community to access the <i>Configuration Upgrade Tools Guide</i> , which is available for download, separately from the main documentation set, with the Configuration Upgrade Tools.
<i>Installation Guide</i>	Describes how to install PolicyCenter. The intended readers are everyone who installs the application for development or for production.
<i>System Administration Guide</i>	Describes how to manage a PolicyCenter system. The intended readers are system administrators responsible for managing security, backups, logging, importing user data, or application monitoring.
<i>Configuration Guide</i>	The primary reference for configuring initial implementation, data model extensions, and user interface (PCF) files for PolicyCenter. The intended readers are all IT staff and configuration engineers.
<i>PCF Format Reference</i>	Describes PolicyCenter PCF widgets and attributes. The intended readers are configuration engineers. See the <i>Configuration Guide</i> .
<i>Data Dictionary</i>	Describes the PolicyCenter data model, including configuration extensions. The dictionary can be generated at any time to reflect the current PolicyCenter configuration. The intended readers are configuration engineers.
<i>Security Dictionary</i>	Describes all security permissions, roles, and the relationships among them. The dictionary can be generated at any time to reflect the current PolicyCenter configuration. The intended readers are configuration engineers.
<i>Globalization Guide</i>	Describes how to configure PolicyCenter for a global environment. Covers globalization topics such as global regions, languages, date and number formats, names, currencies, addresses, and phone numbers. The intended readers are configuration engineers who localize PolicyCenter.
<i>Rules Guide</i>	Describes business rule methodology and the rule sets in Guidewire Studio for PolicyCenter. The intended readers are business analysts who define business processes, as well as programmers who write business rules in Gosu.
<i>Guidewire Contact Management Guide</i>	Describes how to configure Guidewire InsuranceSuite applications to integrate with ContactManager and how to manage client and vendor contacts in a single system of record. The intended readers are PolicyCenter implementation engineers and ContactManager administrators.
<i>Best Practices Guide</i>	A reference of recommended design patterns for data model extensions, user interface, business rules, and Gosu programming. The intended readers are configuration engineers.
<i>Integration Guide</i>	Describes the integration architecture, concepts, and procedures for integrating PolicyCenter with external systems and extending application behavior with custom programming code. The intended

Document	Purpose
	readers are system architects and the integration programmers who write web services code or plugin code in Gosu or Java.
<i>REST API Client Guide</i>	Describes how to use the InsuranceSuite REST API Client to make outbound HTTP calls to internal or third-party REST services.
<i>Java API Reference</i>	Javadoc-style reference of PolicyCenter Java plugin interfaces, entity fields, and other utility classes. The intended readers are system architects and integration programmers.
<i>Gosu Reference Guide</i>	Describes the Gosu programming language. The intended readers are anyone who uses the Gosu language, including for rules and PCF configuration.
<i>Gosu API Reference</i>	Javadoc-style reference of PolicyCenter Gosu classes and properties. The reference can be generated at any time to reflect the current PolicyCenter configuration. The intended readers are configuration engineers, system architects, and integration programmers.
<i>ISBTF and GUnit Testing Guide</i>	Describes the tools and functionality provided by InsuranceSuite for testing application behavior during an initial implementation or an upgrade. The guide covers functionality related to Behavior Testing Framework, GUnit, and Gosu functionality designed specifically for application testing. There are two sets of intended readers: business analysts who will assist in writing tests that describe the desired application behavior; and technical developers who will write implementation code that executes the tests.
<i>Glossary</i>	Defines industry terminology and technical terms in Guidewire documentation. The intended readers are everyone who works with Guidewire applications.
<i>Advanced Product Designer Guide</i>	Advanced Product Designer is a tool that helps you, a business user, design, simulate, and deploy an insurance product. The intended readers are business analysts who understand insurance products, business systems analysts who liaise between business analysts and IT, project managers, and IT who provides technical expertise in areas such as programming, testing, and databases.
<i>Product Model Guide</i>	Describes the PolicyCenter product model. The intended readers are business analysts and implementation engineers who use PolicyCenter or Product Designer. To customize the product model, see the <i>Product Designer Guide</i> .
<i>Product Designer Guide</i>	Describes how to use Product Designer to configure lines of business. The intended readers are business analysts and implementation engineers who customize the product model and design new lines of business.
<i>REST API Framework</i>	Describes the Guidewire InsuranceSuite framework that provides the means to define, implement, and publish REST API contracts. It also describes how the Guidewire REST framework interacts with JSON and Swagger objects. The intended readers are system architects and integration programmers who write web services code or plugin code in Gosu or Java.

Conventions in this document

Text style	Meaning	Examples
<i>italic</i>	Indicates a term that is being defined, added emphasis, and book titles. In monospace text, italics indicate a variable to be replaced.	A <i>destination</i> sends messages to an external system. Navigate to the PolicyCenter installation directory by running the following command: <pre>cd installDir</pre>
bold	Highlights important sections of code in examples.	<pre>for (i=0, i<someArray.length(), i++) { newArray[i] = someArray[i].getName() }</pre>
narrow bold	The name of a user interface element, such as a button name, a menu item name, or a tab name.	Click Submit .

Text style	Meaning	Examples
monospace	Code examples, computer output, class and method names, URLs, parameter names, string literals, and other objects that might appear in programming code.	The getName method of the IDoStuff API returns the name of the object.
<i>monospace italic</i>	Variable placeholder text within code examples, command examples, file paths, and URLs.	Run the startServer <i>server_name</i> command. Navigate to <code>http://server_name/index.html</code> .

Support

For assistance, visit the Guidewire Community.

Guidewire customers

<https://community.guidewire.com>

Guidewire partners

<https://partner.guidewire.com>

Gosu language basics

Gosu is a powerful and easy-to-use object-oriented language. Gosu combines the best features of Java, including compatibility with existing Java libraries, and adds significant improvements like blocks and powerful type inference that change the way you write code. Now you can write readable, powerful, and type-safe type code built on top of the Java platform.

Many of the configuration and integration points in the base configuration of PolicyCenter use Gosu classes or expressions.

To integrate with external systems, you can use native web service and XML support built directly into the language. You can work with XSD types or external APIs like native objects.

For these reasons and more, large companies all around the world use Gosu every day in their production servers for their most business-critical systems.

The next step for you is to write your first Gosu program and become familiar with the Guidewire Studio application and the Gosu editor in Studio.

See also

- *Configuration Guide*
- *Integration Guide*

Gosu introduction

Gosu is a general-purpose programming language built on top of the Java Virtual Machine (JVM). Because Gosu uses the JVM, Gosu has the following characteristics:

- Java compatible, so you can use Java types, extend Java types, and implement Java interfaces
- Object-orientation
- Easy to learn, especially for programmers familiar with Java
- Imperative paradigm

As well as the many features that Gosu shares with Java, Gosu includes the following additional features:

- *Static typing*, which helps you find errors at compile time.
- *Type inference*, which simplifies your code and preserves static typing.
- *Blocks*, which are in-line functions that you can pass around as objects. Some languages call these closures or lambda expressions.
- *Enhancements*, which add functions and properties to other types, even Java types. Gosu includes built-in enhancements to common Java classes, some of which add features that are unavailable in Java (such as blocks).
- *Generics*, which abstracts the behavior of a type to work with multiple types of objects. The Gosu generics implementation is 100% compatible with Java, and adds further powerful improvements.
- JSON support and dynamic typing.
- *Structural types*, which relate classes to one another based on shared characteristics rather than interfaces or inheritance.
- *Dimension types*, which include a unit as well as a value.
- *Composition*, which delegates the implementation of an interface to another class.
- Transactions for database integrity.
- XML/XSD support.
- Web service (SOAP) support.
- String templates.
- Concurrency.
- Checksums.

Large companies around the world use Gosu every day in production systems for critical applications.

Basic Gosu

The following Gosu program illustrates basic Gosu code using a familiar Hello World example. The example uses the built-in `print` function to write the text "Hello World" to the default output stream or console:

```
print("Hello World")
```

Gosu uses the Java type `java.lang.String` as its native `String` type to manipulate texts. You can create in-line `String` literals just as in Java. In addition, Gosu supports native in-line templates, which simplify common text substitution coding patterns.

To declare a variable in the simplest way, use the `var` statement followed by the variable name. Typical Gosu code also initializes the variable by using the equal sign followed by any Gosu expression:

```
var x = 10
var y = x + x
```

Although this example does not specify types for the variables, Gosu is statically typed. All variables have a compile-time type that Gosu enforces at compile time, even though this example has no explicit type declaration. Gosu automatically assigns the type `int` to these variables. Gosu infers the type `int` from the expressions on the right side of the equal signs on lines that declare the variables. Inferring the type of an item from the expression that initializes the item is called *type inference*.

Type inference simplifies Gosu code compared to other statically typed programming languages. Type inference makes typical Gosu code easy to read but retains the power and safety of static typing. For example, take the common pattern of declaring a variable and instantiating an object.

In Gosu, this pattern looks like:

```
var c = new MyVeryLongClassName()
```

This line is equivalent to the following Java code:

```
MyVeryLongClassName c = new MyVeryLongClassName();
```

The Gosu version is more readable and concise than the Java version.

Gosu also supports explicit type declarations of variables during declaration by adding a colon character and a type name. The type name could be a language primitive, a class name, or interface name. For example:

```
var x : int = 3
```

Explicit type declarations are required if you are not initializing the variable on the same statement as the variable declaration. Explicit type declarations are also required for all class variable declarations.

From the previous examples, you might notice another difference between Gosu and Java: no semicolons or other line ending characters. Semicolons are unnecessary in nearly every case, and the standard style is to omit them.

[See also](#)

- “Generics in Gosu” on page 38
- “Type inference” on page 36
- “More about the Gosu type system” on page 35
- “Gosu statement terminators” on page 39

Overview of control flow

Gosu has all the common control flow structures, including improvements on the Java versions.

[if statements](#)

Gosu has the familiar `if`, `else if`, and `else` statements:

```
if (myRecord.Open and myRecord.MyChildList.length > 10) {
    // Some logic
} else if (not myRecord.Open) {
```

```
// Some more logic
} else {
  // Yet more logic
}
```

Gosu permits the more readable English words for the Boolean operators: `and`, `or`, and `not`. Alternatively, you can use the symbolic versions from Java (`&&`, `||`, and `!`).

for loop

The `for` loop in Gosu is similar to the Java 1.5 syntax:

```
for (ad in addressList) {
  print(ad.ID)
}
```

This syntax accepts arrays or any `Iterable` object. Although the syntax does not specify a type for `ad`, the variable is strongly typed. Gosu infers the type based on the iterated variable's type. In the previous example, if `addressList` has type `Address[]`, then `ad` has type `Address`. If the `addressList` variable is `null`, Gosu skips the `for` statement entirely, and generates no error. In contrast, Java throws a null pointer exception if the iterable object is `null`.

If you need an index within the loop, use the following syntax to access the zero-based index:

```
for (ad in addressList index i) {
  print(ad.ID + " has index " + i)
}
```

Intervals

Gosu has native support for intervals. An *interval* is a sequence of values of the same type between a given pair of endpoint values. For example, the set of integers beginning with 0 and ending with 10 is an integer interval. A *closed interval* contains the starting and ending values, in this example, including 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10. The Gosu syntax for this closed interval is `0..10`. A typical use for intervals is in writing concise `for` loops:

```
for (i in 1..10) {
  print(i)
}
```

An *open interval* excludes the value at one or both ends of the interval. The Gosu syntax `1|..|10` defines an interval that is open on both sides, and contains the values from 2 through 9.

Intervals do not need to represent numbers. Intervals can be of types including numbers, dates, or other abstractions such as names. Gosu accepts the shorthand syntax of the two periods, `(. .)` for intervals of dates and common number types. You can also add custom interval types that support iterable comparable sequences. If your interval type implements the required interfaces, you can use intervals of that type in `for` loop declarations:

```
for (i in new ColorInterval("red", "blue")) {
  print(i)
}
```

Gosu does not have a direct general purpose equivalent of the Java three-part `for` declaration:

```
for (i =1 ; i <20 ; ++i)
```

In practice, using intervals makes most use of this pattern unnecessary. If necessary, you can use a Gosu `while` loop to duplicate this pattern.

To use intervals with `for` loops, the interval must be iterable. Custom non-iterable intervals are useful mainly for math and theoretical work. For example, a non-iterable interval could represent non-countable values such as the infinite set of real numbers between two other real numbers.

switch statement

The Gosu `switch` statement can test any type of object, with a special default case at the end:

```
var x = "b"

switch (x) {
  case "a":
```

```

    print("a")
    break
    case "b":
        print("b")
        break
    default:
        print("c")
}

```

In Gosu, you must put a `break` statement at the end of each case to jump to the end of the `switch` statement. Otherwise, Gosu continues to the next case in the series. In the previous example, if you remove the `break` statements, the code prints both "b" and "c". Java has the same requirement. Some other languages do not require the `break` statement to prevent continuing to the next case.

See also

- “Intervals” on page 129

Overview of blocks

Gosu supports in-line functions that you can pass around as objects, which are called *blocks*. Other languages call blocks *closures* or *lambda expressions*.

Blocks are useful as method parameters, in which the method’s implementation generalizes a task or algorithm but callers provide code to implement the details of the task. For example, Gosu adds many useful methods to Java collections classes that take a block as a parameter. That block could return an expression, such as a condition against which to test each item, or could represent an action to perform on each item.

For example, the following Gosu code makes a list of strings, sorts the list by `String` length, and then iterates across the result list to print each item in order:

```

var strings = { "aa", "dddd", "c" }
strings.sortBy(\ str -> str.length).each(\ str -> { print(str) })

```

Block shortcut for one-method interfaces

If an anonymous inner class implements an interface that has exactly one method, you can use a Gosu block to implement the interface. The Gosu block is an alternative to using an explicit anonymous class. You can use a Gosu block for interfaces that are defined in either Gosu or Java. For example:

```

_callbackHandler.execute(\ -> { /* Your Gosu statements here */ })

```

See also

- “Blocks” on page 181
- “Gosu block shortcut for anonymous classes implementing an interface” on page 168

Define a block

A block defines an in-line function. For example, a block named `square` multiplies a value by itself and returns the result.

Procedure

1. Start with the `\` character.
2. Optionally add a list of arguments as *name:type* pairs.
3. Add the `->` characters, which mark the beginning of the block’s body.
4. Finally, add either a statement list surrounded by braces: `{` and `}`, or a Gosu expression.

Example

The following block multiplies a number with itself to square the number:

```
var square = \ x : Integer-> x * x // No need for braces for expressions, but must use for statements
var myResult = square(10)          // Call the block
```

The value of `myResult` in this example is 100.

Overview of enhancements

Gosu provides a feature called *enhancements*, which supports adding functions and properties to other types. Enhancements are especially powerful for enhancing native Java types, and types defined in other people's code. For example, Gosu includes built-in enhancements on collection classes and interfaces, such as `java.util.List`, that improve power and readability of collections code. For example, the following code takes a list of `String` objects, and uses a block to sort the list by the length of each `String`. The code then uses another block in the iteration across the result list to print each item:

```
strings.sortBy(\ str -> str.length).each(\ str -> print(str))
```

The `sortBy` and `each` methods in this example are Gosu enhancement methods on the `List` interface. Both methods return the result list, which makes them useful for chaining methods in series.

See also

- “Enhancements” on page 215
- “Blocks” on page 181

Overview of collections

Gosu provides several features for usage of collections like lists and maps. Gosu directly uses the built-in Java collection classes like `java.util.ArrayList` and `java.util.HashMap`. This usage makes Gosu straightforward to use for interaction with pre-existing Java classes and libraries.

In addition, Gosu adds the following features:

- Shorthand syntax for creating lists and maps that is natural to read and still uses static typing:

```
var myList = { "aa", "bb" }
var myMap = { "a" -> "b", "c" -> "d" }
```

- Shorthand syntax for getting and setting elements of lists and maps

```
var myList = { "aa", "bb" }
myList[0] = "cc"
var myMap = { "a" -> "b", "c" -> "d" }
var mappedToC = myMap["c"]
```

- Gosu includes enhancements that improve Java collection classes. Some enhancements enable you to use Gosu features that are unavailable in Java. For example, the following Gosu code initializes a list of `String` objects and then uses enhancement methods that use Gosu blocks, which are in-line functions.

```
// Use Gosu shortcut to create a list of type ArrayList<String>
var myStrings = { "a", "abcd", "ab", "abc" }

// Sort the list by the length of the String values:
var resortedStrings = myStrings.sortBy(\ str -> str.length)

// Iterate across the list and run arbitrary code for each item:
resortedStrings.each(\ str -> print(str))
```

Notice how the collection APIs are chainable. Alternatively, for readability, you can put each step on a separate lines. The following example declares some data, then uses a block to search for a subset of the items, and then sorts the results.

```
var minLength = 4
var strings = { "yellow", "red", "blue" }
var sorted = strings.where(\ s -> s.length() >= minLength).sort()
print(sorted)
```

See also

- “Overview of blocks” on page 20

Overview of access to Java types

Gosu provides full access to Java types. Gosu is built on top of the Java language and runs within the Java Virtual Machine (JVM). Gosu loads all Java types, so you have full direct access to Java types, such as classes, libraries, and primitive (non-object) types. You can use your favorite Java classes or libraries directly from Gosu with the same syntax as for native Gosu objects.

The most common Java object types retain their fully qualified name but are always in scope, so you do not need to fully qualify their names in typical code. Examples of classes that are always in scope are `Object`, `String`, and `Boolean`. If your code has ambiguity because of similarly named types also in scope, you must fully qualify these types, such as `java.lang.String`. Other Java types are available to Gosu code, but must be fully qualified, for example `java.io.DataInputStream`.

For example, for standard Gosu coding with lists of objects, use the Java type `java.util.ArrayList`. The following simple example uses a Java-like syntax:

```
var list = new ArrayList<String>()
list.add("Hello Java, from Gosu")
```

Gosu includes transformations on Java types that make your Gosu code clearer, such as turning getters and setters into Gosu properties.

Access to Java types from Gosu includes:

- Instantiation of Java classes with the `new` keyword
- Implementing Java interfaces
- Manipulating Java objects as native Gosu objects
- Calling object methods on instantiated objects
- Exposing object methods that look like getters and setters as properties
- Calling static methods on Java types
- Providing built-in extensions and improvements of many common Java types by using Gosu enhancements
- Support for your own extensions of Java types and interfaces
- Support for Java primitive types

See also

- “Packages always in scope” on page 115
- “Overview of enhancements” on page 21
- “Calling Java from Gosu” on page 137

Overview of Gosu classes and properties

Gosu supports object-oriented programming using classes, interfaces and polymorphism. Also, Gosu is fully compatible with Java types, so Gosu types can extend Java types, or implement Java interfaces.

At the top of a class file, use the `package` keyword to declare the package of this class. A *package* defines a namespace. To import specific classes or package hierarchies for later use in the file, add lines with the `uses` keyword. This is equivalent to the Java `import` statement. Gosu supports exact type names, or hierarchies with the `*` wildcard symbol:

```
uses gw.example.MyClass      // Exact type
uses gw.example.queues.jms.* // Wildcard symbol * specifies a hierarchy
```

To create a class, use the `class` keyword, followed by the class name, and then define the variables, then the methods for the class. To define one or more constructor (object instance initialization) methods, use the `construct` keyword. The following example shows a simple class with one constructor that requires a `String` argument:

```
class ABC {
    construct(id : String) {
    }
}
```

You can optionally specify that your class implements one or more interfaces.

To create a new instance of a class, use the `new` keyword in the same way as in Java. Pass any constructor arguments in parentheses. Gosu decides what version of the class constructor to use based on the number and types of the arguments. For example, the following calls the constructor for the ABC class defined earlier in this topic:

```
var a = new ABC("My initialization string")
```

Gosu improves on this basic pattern and introduces a standard compact syntax for property initialization during object creation. For example, suppose you have the following Gosu code:

```
var myFileContainer = new my.company.FileContainer()
myFileContainer.DestFile = jarFile
myFileContainer.BaseDir = dir
myFileContainer.Update = true
myFileContainer.WhenManifestOnly = ScriptEnvironment.WHEN_EMPTY_SKIP
```

After the first line, four more lines contain the object variable name, which is repeated information.

You can use Gosu object initializers to simplify this code to only a couple of lines:

```
var myFileContainer = new my.company.FileContainer() { :DestFile = jarFile, :BaseDir = dir,
:Update = true, :WhenManifestOnly = ScriptEnvironment.WHEN_EMPTY_SKIP }
```

You can also choose to list each initialization on its own line, which uses more lines but is more readable:

```
var myFileContainer = new my.company.FileContainer() {
:DestFile = jarFile,
:BaseDir = dir,
:Update = true,
:WhenManifestOnly = ScriptEnvironment.WHEN_EMPTY_SKIP
}
```

Unlike in Java, for special case usage you can optionally omit the type name entirely in a `new` expression if the type is known from its context. Do not omit the type name in typical code. However, for dense hierarchical structure, omitting the type name can make your Gosu code more readable.

[See also](#)

- “Overview of interfaces” on page 26
- “Creating and instantiating classes” on page 146
- “Using the operator `new` in object expressions” on page 83
- “Importing types and package namespaces” on page 114

Functions

Declare a function by using the `function` keyword. Gosu uses the term *method* for a function that is part of another type. In Gosu, types follow the variable or function definition, separated by a colon. In contrast, Java types precede the variable or parameter name with no delimiter. To return a value, add a statement with the `return` keyword followed by the value. The following simple function returns a value:

```
public function createReport(user : User) : Boolean {
    return ReportUtils.newReport(user, true)
}
```

Method invocation in Gosu looks familiar to programmers of imperative languages, particularly Java. Just use the period symbol followed by the method name and the argument list in parentheses:

```
obj.createReport(myUser)
```

Pass multiple parameters, which can be Gosu expressions, delimited by commas, just as in Java:

```
obj.calculate(1, t.Height + t.Width + t.Depth)
```

In some cases, such as in-line functions in Gosu programs, functions are not attached to a class or other type. You call such functions directly in your code. A rare globally available function for any Gosu code is `print`. Call that function with a `String` argument to write data to the system console or other default output stream. For example, the following line prints text to the console:

```
print("Hello Gosu!")
```

Gosu supports access control modifiers (`public`, `private`, `internal`, and `protected`). These modifiers have the same meaning as in Java. For example, if you mark a method `public`, any other code can call that method.

Gosu also supports static methods, which are methods on the type rather than on object instances.

If the return type for a method is not `void`, all possible code paths in the method must contain a `return` statement and return a value. The set of all paths includes all outcomes of conditional execution, such as `if` and `switch` statements. This requirement is identical to the analogous requirement in Java. The Gosu editor notifies you at compile time if your code violates this requirement.

See also

- “Access modifiers” on page 157
- “Static members” on page 26
- “Gosu functions” on page 111

Class variables and properties

Gosu supports instance variables and static variables in class declarations in basically the same way Java does, although the syntax is slightly different. Use the `var` keyword in the class definition, and declare the type explicitly. Note that variables are `private` by default in Gosu.

```
var _id : String // Variables are private by default
```

One difference between Gosu and some languages, including Java, is full support in Gosu for properties. A *property* provides a dynamic getter method and a dynamic setter method for its value. To set or get properties from an object, use natural syntax. Type the period (`.`) character followed by the property name just as for an object variable:

```
var s = myobj.Name
myobj.Name = "John"
```

Internally, Gosu calls the property getter and setter methods. In addition, Gosu has a special null-safety behavior with pure property paths, which have the form `obj.Property1.Property2.Property3`.

Define a property accessor function, which is a property getter, by using the declaration `property get` instead of `function`. Define a setter function by using function declaration `property set` instead of `function`. These property functions dynamically get or set the property, depending on whether the definition is `property get` or `property set`. Properties can be read/write, read-only, or write-only. Gosu provides a special shortcut to expose internal variables as public properties with other names. Use the syntax as `PROPERTY_NAME` in the definition for a variable. This syntax separates the internal implementation of the variable from how you expose the property to other code:

```
var _name : String as Name // Exposes the _name field as a readable and writable 'Name' property
```

This syntax is a shortcut for creating a `property get` function and a `property set` function for each variable. This syntax is the standard and recommended Gosu style for designing public properties. Do not expose actual class variables as `public`, although for compatibility with Java, Gosu does support exposing these variables.

The following code defines a simple Gosu class:

```
package example                // Declares the package (namespace) of this class

class Person {

    var _name : String as Name    // Exposes the _name field as a readable and writable 'Name' property
    var _id : String              // Variables are private by default

    // Constructors are like functions called construct but omit the function keyword.
    // You can supply multiple method signatures with different numbers or types of arguments
```



```

construct(id : String){
    _id = id
    _name = ""
}

property get ID() : String {
    // _id is exposed as a read only 'ID' property
    return _id
}

// Comment out the property set function to make ID a read-only property:
property set ID(id : String) {
    _id = id
}

// Functions are public by default
function printOut() {
    print(_name + ":" + _id)
}
}

```

With this class, you can use concise code like the following:

```

n.ID = "12345"    // Set a property
print(n.ID)      // Get a property
n.Name = "John"  // Set a property -- See the "as Name" part of the class definition!
print(n.Name)    // Get a property -- See the "as Name" part of the class definition!

```

[See also](#)

- “Property accessor paths are null safe” on page 26

Java getter and setter methods become properties in Gosu

For methods on Java types that look like getters and setters, Gosu exposes methods on the type as properties rather than methods. Gosu uses the following rules for methods on Java types:

- If the method name starts with `set` and takes exactly one argument, Gosu exposes this method as a setter for the property. The property name matches the original method but without the prefix `set`. For example, suppose the Java method signature is `setName(String thename)`. Gosu exposes this method as a property set function for the property named `Name` of type `String`.
- If the method name starts with `get` and takes no arguments and returns a value, Gosu exposes this method as a getter for the property. The property name matches the original method but without the prefix `get`. For example, suppose the Java method signature is `getName()` and it returns a `String`. Gosu exposes this method as a property `get` function for the property named `Name` of type `String`.
- If the method name starts with `is`, takes no arguments, and returns a Boolean value, the rules are similar to the rules for `get`. Gosu exposes this method as a getter for the property. The property name matches the original method but without the prefix `is`. For example, suppose a Java method signature is `isVisible()`. Gosu exposes this method as a property `get` function for a Boolean property named `Visible`.

If the Java code has a setter and a getter, Gosu makes the property readable and writable. If the setter is absent, Gosu makes the property read-only. If the getter is absent, Gosu makes the property write-only.

For example, consider a Java class called `Circle` with the following method declarations:

```

public double getRadius()
public void setRadius(double dRadius)

```

Gosu exposes these methods as the `Radius` property, which is readable and writable. With this property, you can use straightforward code such as:

```

circle.Radius = 5    // Property SET
print(circle.Radius) // Property GET

```

[See also](#)

- “Using Java `get`, `set`, and `is` methods as Gosu properties” on page 139

Property accessor paths are null safe

One notable difference between Gosu and some other languages is that property accessor paths in Gosu are tolerant of unexpected null values. This feature affects only expressions separated by period characters that access a series of instance variables or properties such as:

```
obj.Property1.Property2.Property3
```

In most cases, if any object to the left of the period character is null, Gosu does not throw a null pointer exception (NPE) and the expression returns null. Gosu null-safe property paths tend to simplify real-world code. Gosu null-tolerant property accessor paths are a very good reason to expose data as properties in Gosu classes and interfaces rather than as setter and getter methods.

Gosu provides additional null-safe operators. For example, specify default values with code like:

```
// Set display text to the String in the txt variable, or if it is null use "(empty)"
var displayText = txt ?: "(empty)"
```

WARNING In Studio, Gosu code in the Gosu Scratchpad has different compiler behavior than any other part of Studio. If you run code in Gosu Scratchpad using the **Run**  or **Debug**  buttons, then the code compiles and runs locally in Studio with different behavior. Most notably, the behavior of null safety for properties in the period operator is different. In nearly all other contexts in Studio, the period operator is null safe for properties. However, in Gosu Scratchpad in Studio, Gosu is not null-safe. Code that might not throw an exception in a Gosu class might throw an exception in Gosu Scratchpad. If Studio is connected to a server running with a DCEVM and you click **Run in Debug Process** , then Studio sends the Gosu code to the server. The PolicyCenter server uses the standard null-safe behavior for the period operator.

See also

- “Null safety for properties and other operators” on page 37

Static members

Gosu supports static members on a type. Static members include variables, functions, property declarations, and static inner classes on a type. A *static* member exists only once, on the type, not on instances of the type.

To declare a type as static for a new Gosu class, use the `static` keyword, as in Java.

The syntax for using a static member is to use the period (.) character followed by the property name or method call after a type reference, which is the type name. For example, the following code accesses a static property and calls a static method:

```
var myVar = MyClass.MY_CONST // Get a static property value
MyClass.staticMethodName() // Call a static method
```

Alternatively, to use the static members without qualifying them with the class name, you use the syntax of the `uses` statement that imports static members. For example, the following code is equivalent to the previous lines:

```
uses MyClass#*
...
var myVar = MY_CONST // Get a static property value
staticMethodName() // Call a static method
```

See also

- “Modifiers” on page 157
- “Importing static members” on page 115

Overview of interfaces

Gosu supports interfaces, including full support for Java interfaces. An interface is a set of method signatures that a type must implement. An interface defines a contract that specifies the minimum set of functionality to be

considered compatible. To define an interface, use the `interface` keyword, then the interface name, and then a set of method signatures without function bodies. The following Gosu code uses the `interface` keyword to define a simple interface:

```
package example

interface ILoadable {
    function load()
}
```

A class can implement the interface with the `implements` keyword followed by a comma-delimited list of interfaces. A class that implements an interface contains all methods in the interface, as shown in the following code:

```
package example

class LoadableThing implements ILoadable {

    function load() {
        print("Loading...")
    }
}
```

[See also](#)

- “Interfaces” on page 173

List and array expansion operator `*`.

Gosu provides an operator for array expansion and list expansion that expands and flattens complex object graphs. This expansion can be useful and powerful, enabling you to extract one specific property from all objects several levels down in an object hierarchy. The expansion operator is an asterisk followed by a period. For example:

```
names*.length
```

Using the expansion operator on a list retrieves a property from every item in the list and returns all instances of that property in a new list. The expansion operator works similarly with arrays.

Let us consider the previous example `names*.length`. Assume that `names` contains a list of `String` objects, and each one represents a name. All `String` objects contain a `length` field. The result of the previous expression is a list containing the same number of items as the original list. However, each item is the length (the `String.length` property) of the corresponding name `String` object.

Gosu uses Gosu generics, an advanced type feature, to infer the type of the list as the appropriate parameterized type. Similarly, Gosu infers the type of the result array if you call the operator on an array.

[See also](#)

- “Generics in Gosu” on page 38
- “List expansion (`*`.)” on page 194

Overview of comparisons

In general, the comparison operators work as you might expect if you are familiar with most programming languages. Some notable differences are:

- The operators `>`, `<`, `>=`, and `<=` operators work with all objects that implement the `Comparable` interface, not just with numbers.
- The standard equal comparison `==` operator implicitly uses the `equals` method on the first (leftmost) object. This operator does not check for pointer equality. It is `null` safe in that if the value on either side of the operator is `null`, Gosu does not throw a null pointer exception.

Note: In contrast, in the Java language, the `==` operator evaluates to `true` if and only if both operands have exactly the same reference value. The Java expression evaluates to `true` if both terms refer to the same object in memory. This behavior provides the expected result for primitive types like integers. For reference types, the references are not usually what you want to compare. Instead, to compare value equality, Java code typically uses `object.equals()`, rather than the `==` operator.

- In some cases, you do need to compare identity references, to determine whether two objects reference the same in-memory object. Gosu provides a special equality operator called `===` (three equal signs) to compare object equality. This operator compares whether both references point to the same in-memory object. The following examples illustrate some differences between `==` and `===` operators:

Expression	Output	Description
<pre>var x = 1 + 2 var str = x as String print(str == "3")</pre>	true	These two comparison arguments reference the same value but different objects. Using the double-equals operator returns true.
<pre>var x = 1 + 2 var str = x as String print(str === "3")</pre>	false	These two comparison arguments reference the same value but different objects. Using the triple-equals operator returns false.

See also

- “Property accessor paths are null safe” on page 26

Gosu character set and case sensitivity

Because Gosu runs within a JVM, Gosu shares the same 16-bit Unicode character set as Java. By supporting this character set, your Gosu code can represent virtually any character in any human language.

The Gosu language is case sensitive. Write all Gosu as case-sensitive code matching the declaration of the language element. For example, if a type is declared as `MyClass`, you cannot use the type as `myClass` or `myCLASS`.

There are standard conventions for how to capitalize different language elements. For example, local method names have an initial lowercase letter. Type names, including class names have an initial uppercase letter.

See also

- “Coding style” on page 219

Compound assignment statements

Gosu supports all operators in the Java language, including bit-oriented operators. Additionally, Gosu has compound operators such as:

- `++` – The increment-by-one operator, which is supported only after the variable name
- `+=` – The add-and-assign operator, which is supported only after the variable name followed by a value to add to the variable
- Similarly, Gosu supports `--` (decrement-by-one) and `-=` (subtract-and-assign)
- Gosu supports additional compound assignment statements that mirror other common operators.

For example, to increment the variable `i` by 1:

```
i++
```

These operators always form statements, not expressions. For example, the following Gosu is valid:

```
var i = 1
while (i < 10) {
    i++
    print(i)
}
```

The following Gosu is invalid because statements are impermissible in an expression, which Gosu requires in a `while` statement:

```
var i = 1
while (i++ < 10) { // Compilation error!
    print(i)
}
```

Gosu supports the increment and decrement operator only after a variable, not before a variable, so `i++` is valid but `++i` is invalid. The `++i` form exists in other languages to support expressions in which the result is an expression that you pass to another statement or expression. In Gosu, these operators do not form an expression, so you cannot use increment or decrement in `while` declarations, `if` declarations, and `for` declarations.

[See also](#)

- “Variable assignment” on page 101

Delegating interface implementation with composition

Gosu supports the language feature called composition by using the `delegate` and `represents` keywords in variable definitions. A class uses *composition* to delegate responsibility for implementing an interface to a different object. This compositional model supports implementation of objects that are proxies for other objects, or encapsulation of shared code independent of the type inheritance hierarchy. The syntax for using the composition feature looks like the `MyWindow` class in the following code:

```
interface IClipboardPart {
    function load()
}

class ClipboardPart implements IClipboardPart {
    override function load() {
        print("Gosu is loaded!")
    }
}

class MyWindow implements IClipboardPart {
    delegate _clipboardPart represents IClipboardPart

    construct() {
        _clipboardPart = new ClipboardPart(this)
        this.load()
    }
}
```

In this example, the class definition for `MyWindow` uses the `delegate` keyword to delegate implementation of the `IClipboardPart` interface. To complete the delegation, the constructor for the `MyWindow` class creates an object of the `ClipboardPart` class. This object is the delegate. It has the delegate’s name, `_clipboardPart`. When a class delegates the implementation of an interface, it must construct the delegate it declares. Upon doing so, the class fulfills indirectly its implementation duty.

The class that defines a delegate object must implement the interface that the delegate represents. In the example code, the `ClipboardPart` class defines the delegate object, `_clipboardPart`. This definition implements the sole method that the `IClipboardPart` interface declares, `load()`.

When instantiating the `MyWindow` class, the class constructor in the example code calls the constructor for the class that defines the delegate object, `_clipboardPart`. Upon its construction, `_clipboardPart` has access to all of the methods that its defining class, `ClipboardPart`, implements, including the `load()` method. Upon constructing its delegate, the `MyWindow` class has access to the `load()` method as well.

The class constructor in the example code then calls the `load()` method. This method in turn calls the `print()` method. The `print()` method then writes the sentence “Gosu is loaded!” to the system console or default output stream.

You can use a delegate to represent (provide methods for) multiple interfaces for the enclosing class. In the `delegate` statement, specify a comma-separated list of interfaces. For example:

```
private delegate _employee represents ISalariedEmployee, IOfficer
```

The Gosu type system handles the type of the variable in the previous example by using a special kind of type called a compound type.

See also

- “Composition” on page 271

Overview of concurrency

If more than one Gosu thread interacts with data structures that another thread needs, you must ensure that you protect data access to avoid data corruption. The requirement to support concurrent access from multiple threads is called *concurrency*. Code that can safely get or set concurrently accessed data is called *thread-safe*.

Gosu provides the following concurrency APIs:

Support for Java monitor locks, reentrant locks, and custom reentrant objects

Gosu provides access to Java-based classes for monitor locks and reentrant locks in the Java package `java.util.concurrent`. Gosu provides straightforward access to these classes with `using` clauses that also properly handle cleanup if exceptions occur. Additionally, Gosu provides a straightforward way to create custom Gosu objects that support reentrant object handling, as shown in the following example. The following Gosu code that uses the `using` keyword shows the compact readable syntax for using Java-defined reentrant locks.

```
// In your class definition, define a static variable lock
static var _lock = new ReentrantLock()

// A property get function uses the lock and calls another method for the main work
property get SomeProp() : Object {
    using (_lock) {
        return _someVar.someMethod() // Do your work here and Gosu synchronizes it, and handles cleanup
    }
}
```

Scoping classes (pre-scoped maps)

Scope-related utilities in the class `gw.api.web.Scopes` provide synchronization and protect access to shared data. These APIs return `Map` objects that you can use to safely get and put data by using different scope semantics.

Lazy concurrent variables

The `LazyVar` class in `gw.util.concurrent` implements what is known as a lazy variable. Gosu does not construct a *lazy variable* until the first time any code uses the variable. For example, the following code is part of a class definition that defines the object instance. For example, Gosu does not run the following Gosu block that creates an `ArrayList` until the first usage of the variable:

```
var _lazy = LazyVar.make{\-> new ArrayList<String>() }
```

Concurrent cache

The `Cache` class in `gw.util.concurrent` declares a cache of values that you can look up quickly and in a thread-safe way. A `Cache` object provides a concurrent cache similar to a Least Recently Used (LRU) cache. To use the cache, call the `get` method with an argument of the input value, which is the key. If the value is in the cache, the `get` method returns the value from the cache. If the value is not cached, Gosu calls the block and calculates the value from the key, and then caches the result. For example:

```
print(myCache.get("Hello world"))
```

See also

- “Concurrency and thread safety” on page 365

Overview of exceptions

Gosu supports the full feature set for Java exception handling, including `try/catch/finally` blocks. However, unlike Java, no exceptions are checked. Standard Gosu style is to avoid checked exceptions where possible. You can throw any exception in Gosu, but if the exception is not a `RuntimeException`, Gosu wraps the exception in a `RuntimeException`.

Catching exceptions

The following Gosu code is a simple try/catch/finally:

```
try {
    user.enter(bar)
} catch (e : Exception) {
    print("Failed to enter the bar!")
} finally {
    // Clean-up code here...
}
```

To catch only specific exceptions, specify a subclass such as `IOException` instead of `Exception`. For example:

```
try {
    doSomethingThatMayThrowIOException()
} catch (e : IOException) {
    // Handle the IOException
}
```

Throwing exceptions

In Gosu, throw an exception with the `throw` statement, which is the `throw` keyword followed by an object. The following example creates an explicit `RuntimeException` exception:

```
if (user.Age < 21) {
    throw new RuntimeException("User is not allowed in the bar")
}
```

You can also pass a non-exception object to the `throw` statement. If you pass a non-exception object, Gosu first coerces it to a `String`. Next, Gosu wraps the `String` in a new `RuntimeException`, which has a `Message` property that contains the `String` data. Your error-handling code can use the `Message` property for logging or displaying messages to users.

You could rewrite the previous `throw` code example as the concise code:

```
if (user.Age < 21) {
    throw "User is not allowed in the bar"
}
```

Overview of annotations

Gosu annotations are a syntax to provide metadata about a Gosu class, constructor, method, class variable, or property. An annotation can control the behavior of the item or the documentation for the item.

This code demonstrates adding a `@Throws` annotation to a method to indicate what exceptions it throws.

```
class MyClass{

    @Throws(java.text.ParseException, "If text is invalid format, throws ParseException")
    public function myMethod() {
        ...
    }
}
```

You can define custom annotations, and optionally have your annotations take arguments. If there are no arguments, you can omit the parentheses.

You can get annotations from types at run time.

Gosu supports named arguments syntax for annotations:

```
@MyAnnotation(a = "myname", b = true)
```

See also

- “Annotations” on page 227

Overview of templates

Gosu provides a syntax that supports in-line dynamic templates. Use a template to combine static text with values from variables or other calculations Gosu evaluates template values at run time. For example, suppose you want to display text with a calculation in the middle of the text:

```
var s = "One plus one equals ${ 1 + 1 }."
```

If you print this variable, Gosu displays:

```
One plus one equals 2.
```

Template expressions can include variables and dynamic calculations. Gosu substitutes the run-time values of the expressions in the template. The following line is an example of a method call inside a template:

```
var s2 = "The total is ${ myVariable.calculateMyTotal() }."
```

At compile time, Gosu ensures all template expression are valid and type safe. At run time, Gosu runs the template expression, which must return a `String` value or a type that can cast to a `String`.

In addition to in-line Gosu templates, Gosu supports a powerful file-based approach for Gosu templates with optional parameter passing. Any use of the parameters is validated for type-safety, just like any other Gosu code. For example, use a template to generate a customized notification email, and design the template to take parameters. Parameters could include type safe references to the recipient email address, the sender email address, and other objects. Insert the parameters directly into template output, or call methods or get properties from parameters to generate your customized email report.

See also

- “Templates” on page 323

Overview of XML and XSD support

Gosu provides support for Extensible Markup Language (XML). XML files describe complex structured data in a text-based format with strict syntax for data interchange. For more information on XML, refer to the World Wide Web Consortium web page <http://www.w3.org/XML>.

Gosu can parse XML by using an existing XML Schema Definition (XSD) file to produce a statically typed tree with structured data. Alternatively, Gosu can read or write to any XML document as a structured tree of untyped nodes. In both cases, Gosu code interacts with XML elements as native in-memory Gosu objects assembled into a graph, rather than as text data.

All the types from the XSD become native Gosu types, including element types and attributes. All these types appear naturally in the namespace defined by the part of the class hierarchy that you place the XSD. In other words, you put your XSDs side-by-side next to your Gosu classes and Gosu programs.

Suppose you put your XSD in the package directory for the package `mycompany.mypackage` and your XSD is called `mySchema.xsd`. Gosu lowercases the schema name because the naming convention for packages is lowercase. Gosu creates new types in the hierarchy:

```
mycompany.mypackage.myschema.*
```

For example, the following XSD file is called `driver.xsd`:

```
<xs:element name="DriverInfo">
  <xs:complexType>
    <xs:sequence>
      <xs:element ref="DriversLicense" minOccurs="0" maxOccurs="unbounded"/>
      <xs:element name="PurposeUse" type="String" minOccurs="0"/>
      <xs:element name="PermissionInd" type="String" minOccurs="0"/>
      <xs:element name="OperatorAtFaultInd" type="String" minOccurs="0"/>
    </xs:sequence>
    <xs:attribute name="id" type="xs:ID" use="optional"/>
  </xs:complexType>
</xs:element>
<xs:element name="DriversLicense">
  <xs:complexType>
```



```

<xs:sequence>
  <xs:element name="DriversLicenseNumber" type="String"/>
  <xs:element name="StateProv" type="String" minOccurs="0?"/>
  <xs:element name="CountryCd" type="String" minOccurs="0?"/>
</xs:sequence>
<xs:attribute name="id" type="xs:ID" use="optional"/>
</xs:complexType>
</xs:element>

```

The following Gosu code uses XSD-based types to manipulate XML objects:

```

uses xsd.driver.DriverInfo
uses xsd.driver.DriversLicense
uses java.util.ArrayList

function makeSampleDriver() : DriverInfo {
  var driver = new DriverInfo(){PurposeUse = "Truck"}
  driver.DriversLicenses = new ArrayList<DriversLicense>()
  driver.DriversLicenses.add(new DriversLicense(){CountryCd = "US", :StateProv = "AL"})
  return driver
}

```

The example to follow uses the XSD file, `Credentials.xsd`. In particular, the code relies upon the following portion of the XSD file:

```

<xs:complexType name="CredentialsType">
  <xs:sequence>
    <xs:element name="username" type="xs:string" minOccurs="1" maxOccurs="1"/>
    <xs:element name="password" type="xs:string" minOccurs="1" maxOccurs="1"/>
  </xs:sequence>
  <xs:attribute name="key" type="xs:string"/>
</xs:complexType>

<xs:element name="CredentialsElem" type="tns:CredentialsType"/>

<xs:complexType name="CredentialsArrayType">
  <xs:sequence>
    <xs:element ref="tns:CredentialsElem" minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>

<xs:element name="CredentialsArray" type="tns:CredentialsArrayType"/>

```

In this example, the following Gosu code creates a first element defined in the XSD file. This first element is of type `CredentialsArray`. Next, the code creates a second element defined in the same XSD file. This second element is of type `CredentialsElem`. The code then adds the second element as a child to the first element:

```

uses gw.xsd.credentials.*

var e = new CredentialsArray()

// Create a new child element that is legal in the XSD, and add it as a child
var c1 = new CredentialsElem()
c1.setUsername("Harry Truman")
c1.setPassword("password")
e.addChild(c1)

```

See also

- “Working with XML in Gosu” on page 275

Overview of web service support for consuming WSDL

Gosu code can import web services (SOAP APIs) from external systems and call these services as a SOAP client, which is an API consumer. The Gosu language handles all aspects of object serialization, object deserialization, basic authentication, and SOAP fault handling.

The following example uses a hypothetical web service `SayHello`.

```

// Get a reference to the service in the package namespace of the WSDL
var service = new example.gosu.wsi.myservice.SayHello()

// Set security options

```

```
service.Config.Http.Authentication.Basic.Username = "jms"
service.Config.Http.Authentication.Basic.Password = "b5"

// Call a method on the service
var result = service.helloWorld()
```

See also

- *Integration Guide*

Running Gosu programs and calling other classes

To use Gosu, the initial file that you run must be a Gosu program. A Gosu program file has the `.gsp` file name extension. Gosu code in a program can call out to other Gosu classes and other types, such as Java classes.

You can run Gosu programs (`.gsp` files) directly from the command line. You cannot run a Gosu class file or other types of file directly. If you want to call a Gosu class or other type of file, make a Gosu program that uses those other types.

In Java, you define a `main` method in a class and tell Java which main class to run. This method calls out to other classes as needed.

In Gosu, your main Gosu program (`.gsp` file) can call any necessary code, including Gosu or Java classes. If you want to mirror the Java style, your `.gsp` file can contain a single line that calls a `main` method on a Gosu class or Java class.

To tell Gosu where to load additional classes, do either of the following:

- Use the `Class-Path` attribute in the manifest of a JAR file.
- Use the `-classpath` option on the command-prompt tool.

You can combine these two options for greater flexibility and to support very long class paths.

Using the Class-Path attribute in the manifest of a Java library file

You can add a `Class-Path` attribute line in the `MANIFEST.MF` file, which is the manifest file in any Java library file. The value of the `Class-Path` attribute has an arbitrary length, so you can use this attribute if you need a class path that is longer than a command prompt supports.

The `Class-Path` attribute can contain one or more URLs for JAR file names. To specify multiple JAR files in the class path, use a space character to separate the file names. To specify an absolute path to a local file use the `file:/` prefix.

The following lines show a manifest file that contains a `Class-Path` attribute. The class path contains a library in the same directory as the library that contains the manifest, a library in a subdirectory, and a library at an absolute path.

```
Manifest-Version: 1.0
Class-Path: MyUtils.jar classfiles/MyClasses.jar file:/C:/javilib/AppHelpers.jar
Created-By: 1.8.0_92 (Oracle Corporation)
```

Using the -classpath option on the command-prompt tool

To use other Gosu classes, Java classes, or Java libraries, you perform the following steps.

1. Create a package-style hierarchy for your class files on your disk. For example, if the root of your files is `PolicyCenter/MyProject/`, put the class files for the Gosu class `com.example.MyClass` at the location `PolicyCenter/MyProject/com/example/MyClass.gs`.
2. At the command prompt, tell Gosu where to find your other Gosu classes and Java classes by adding the `-classpath` option to the `gosu` command.

Typically you place Java classes, Gosu classes, or libraries in subdirectories of your main Gosu program.

For example, suppose you have a Gosu program at this location:

```
C:\gosu\myprograms\test1\test.gsp
```

Copy your class file for the class `mypackage.MyClass` to the location:

```
C:\gosu\myprograms\test1\src\mypackage\MyClass.class
```

Copy your library files to the location:

```
C:\gosu\myprograms\test1\lib\mylibrary.jar
```

For this example, add two directories to the class path with the following command:

```
test.gsp -classpath "src;lib"
```

[See also](#)

- “Command-prompt tool and options” on page 379

Guidewire internal classes

Some code packages contain Guidewire internal classes that are reserved for Guidewire use only. Gosu code written to configure PolicyCenter must never use an internal class nor call any method of an internal class for any reason. Future releases of PolicyCenter can change or remove an internal class without notification.

The following packages contain Guidewire internal classes.

- All packages in `com.guidewire.*`
- Any package whose name or location includes the word `internal`

Gosu configuration code can safely use classes defined in any `gw.*` package, except for those packages whose name or location includes the word `internal`.

Some Gosu classes are visible in Studio, but are not intended for use. You can distinguish these Gosu classes because they have no visibility annotations (neither `@Export` nor `@ReadOnly`) and they are not in a `gw` package. Do not use these methods in configuration. The methods are unsupported and may change or be withdrawn without notice.

More about the Gosu type system

This topic further describes the Gosu type system and its advantages for programmers. Gosu is a statically typed language, rather than a dynamically typed language. In a *statically typed* language, all variables must be assigned a type at compile time. Gosu enforces this type constraint at compile time and at run time. If any code violates type constraints at compile time, Gosu flags this violation as a compile error. If your code violates type constraints at run time, for example, by making an invalid object type coercion, Gosu throws an exception.

Static typing in Gosu provides several benefits:

- Compile-time error prevention
- Intelligent code completion

Although Gosu is a statically typed language, Gosu supports a concept of generic types, called *Gosu generics*. You can use generics in special cases to define a class or method so that it works with multiple types of objects. Gosu generics are especially useful to design or use APIs that manipulate collections of objects. Programmers familiar with the Java implementation of generics quickly become comfortable with Gosu generics, although there are minor differences.

[See also](#)

- “Intelligent code completion and other Gosu editor tools” on page 36
- “Type system” on page 349
- “Variable type declaration” on page 101
- “Generics” on page 241

Compile-time error prevention

Static typing allows you to detect most type-related errors at compile time. This increases reliability of your code at run time. This is critical for real-world production systems. When the Gosu editor detects compilation errors and warnings, it displays them in the user interface as you edit Gosu source code.

For example, functions (including object methods) take parameters and return a value. The information about the type of each parameter and the return type is known at compile time. During compilation, Gosu enforces the following constraints:

- Calls to this function must take as parameters the correct number of parameters having the appropriate types.
- Within the code for the function, code must always treat an object as the appropriate type. For example, you can call methods or get properties from the object, but only methods or properties declared for that compile-time type. It is possible to cast the value to a different type, however. If the run-time type is not a subtype of the compile-time type, run-time errors can occur.
- If code that calls this function assigns a variable to the result of this function, the variable type must match the return type of this function.

For example, consider the following function definition.

```
public function getLabel(person: Person) : String {
    return person.LastName + ", " + person.FirstName
}
```

If you write code that calls this method and passes an integer instead of a `Person`, the code fails with a type mismatch compiler error. The compiler recognizes that the parameter value is not a `Person`, which is the contract between the function definition and the code that calls the function.

Similarly, Gosu ensures that all property access on the `Person` object (`LastName` and `FirstName` properties) are valid properties on the class definition of `Person`. If the code inside the function calls any methods on the object, Gosu also ensures that the method name that you are calling actually exists on the type.

Within the Gosu editor, violations of these rules become compilation errors. By finding this large class of problems at compile time, you can avoid unexpected run-time failures.

Type inference

Gosu supports type inference, in which Gosu can often infer the type of an item without requiring explicit type declarations in the Gosu code. For instance, Gosu can determine the type of a variable from its initialized value:

```
var length = 12
var list = new java.util.ArrayList()
```

In the first line, Gosu infers the `length` variable has the type `int`. In the second line, Gosu infers the type of the list variable is of type `ArrayList`. In most cases, it is unnecessary to declare a variable's type if Gosu can determine the type of the initialization value.

The Gosu syntax for explicit declarations of the type of the variable during declaration is:

```
var c : MyClassName = new MyClassName()
```

For typical code, Gosu coding style is to omit the type and use type inference to determine the variable's type.

Another standard Gosu coding style is to use a coercion on the right side of the expression with an explicit type. For example, suppose you use a class called `Vehicle`, which has a subclass `Car`. If the variable `v` has the compile-time type `Vehicle`, the following code coerces the variable to the subtype:

```
var myCar = v as Car
```

Intelligent code completion and other Gosu editor tools

When you type code into the Gosu editor, the editor uses its type system to help you write code quickly and easily. The editor also uses the type system to preserve the constraints for statically typed variables. When you type the period (.) character, the editor displays a list of possible properties or subobjects that are allowed at that location in the code.

Similar to the use of the period character, the Gosu editor has a **Complete Code** feature. Choose this tool to display a list of properties or objects that could complete the current code where the cursor is. If you try to enter an incorrect type, Gosu displays an error message immediately so that you can fix your errors at compile time.

Refactoring

Static typing makes it much easier for development tools to perform smart code refactoring.

See also

- *Configuration Guide*

Null safety for properties and other operators

One notable difference between Gosu and some other languages is that property accessor paths in Gosu are null-tolerant, also called null-safe. Only expressions separated by period (.) characters that access a series of instance variables or properties support null safety, such as the following form:

```
obj.PropertyA.PropertyB.PropertyC
```

In most cases, if any object to the left of the period character is null, Gosu does not throw a null-pointer exception (NPE) and the expression returns null. Gosu null-safe property paths tend to simplify real-world code. Often, a null expression result has the same meaning whether the final property access is null or earlier parts of the path are null. For such cases in Gosu, do not check for the null value at every level of the path. This conciseness makes your Gosu code easier to read and understand.

For example, suppose you have a variable called `house`, which contains a property called `walls`, and that object has a property called `windows`. The syntax to get the `windows` value is:

```
house.Walls.Windows
```

In some languages, you must be aware that if `house` is null or `house.Walls` is null, your code throws a null-pointer exception. The following common coding pattern avoids the exception:

```
// Initialize to null
var x : ArrayList<Windows> = null

// Check earlier parts of the path for null to avoid a null-pointer exceptions (NPE)
if (house != null and house.Walls != null) {
    x = house.Walls.Windows
}
```

In Gosu, if earlier parts of a pure property path are null, the expression is valid and returns null. The following Gosu code is equivalent to the previous example and a null-pointer exception never occurs:

```
var x = house.Walls.Windows
```

By default, method calls are not null-safe. If the right side of a period character is a method call, Gosu throws a null-pointer exception (NPE) if the value on the left side of the period is null.

For example:

```
house.myaction()
```

If `house` is null, Gosu throws an NPE. Gosu assumes that method calls might have side effects, so Gosu cannot skip the method call and return null.

Gosu provides a variant of the period operator that is always explicitly null-safe for both property access and method access. The null-safe period operator has a question mark before the period: `?.`

If the value on the left of the `?.` operator is null, the expression evaluates to null.

For example, the following expression evaluates left-to-right and contains three null-safe property operators:

```
obj?.PropertyA?.PropertyB?.PropertyC
```

Null-safe method calls

A *null-safe method call* does not throw an exception if the left side of the period character evaluates to null. Gosu just returns null from that expression. Using the `?.` operator calls the method with null safety:

```
house?.myaction()
```

If house is null, Gosu does not throw an exception. Gosu returns null from the expression.

Null-safe versions of other operators

Gosu provides null-safe versions of other common operators:

- The null-safe default operator (`? :`). You use this operator to specify an alternative value if the value to the left of the operator evaluates to null. For example:

```
var displayName = Book.Title ? : "(Unknown Title)" // Return "(Unknown Title)" if Book.Title is null
```

- The null-safe index operator (`?[ ]`). Use this operator with lists and arrays. The expression returns null if the list or array value is null at run time, rather than throwing an exception. For example:

```
var book = bookshelf?[bookNumber] // Return null if bookshelf is null
```

- The null-safe math operators (`?+`, `?-`, `?*`, `?/`, and `?%`). For example:

```
var displayName = cost ?* 2 // Multiply by 2, or return null if cost is null
```

Design code for null safety

Use null-safe operators where appropriate. These operators make code more concise and simplify the handling of edge cases.

You can design your code to take advantage of this special language feature. For example, expose data as properties in Gosu classes and interfaces rather than as setter and getter methods.

See also

- “Handling null values in expressions” on page 94
- “Properties” on page 150

Generics in Gosu

Generics abstract the behavior of a type to support working with multiple types of objects, and are especially useful for implementing collections such as lists and maps in a type-safe way. At compile time, each use of the collection can specify the specific type of its items. For example, instead of just referring to a list of objects, you can refer to a list of Address objects or a list of Vehicle objects. To specify a type, add one or more parameters types inside angle brackets (`<` and `>`). This syntax that specifies the type of a collection or another item of unknown type is called *parameterizing a generic type*. For example:

```
uses java.util.*

var mylist = new ArrayList<Date>()
var mymap = new Map<String, Date>() // A map that maps String to Date
```

In this example, `ArrayList<Date>` is an array list of date objects. The `Map<String, Date>` is a map that maps String to Date.

The Gosu generics implementation is compatible with the Java 1.5 generics implementation, and adds these improvements:

- Gosu type inference greatly improves readability. You can omit unnecessary type declarations, which is especially important for generics because the syntax tends to be verbose.
- Gosu generics support array-style variance of different generic types. In Java, this code results in a compilation error:

```
ArrayList<Object> mylist;
mylist = new ArrayList<String>()
```

In comparison, this similar Gosu code works:

```
var mylist : ArrayList<Object>
mylist = new ArrayList<String>()
```

- Gosu types support reified generics. *Reified generics* preserve generic type information at run time. In complex cases, you can check the exact type of an object at run time, including any parameterization. In contrast, Java discards this information completely after compilation, so it is unavailable at run time.

Note: Even in Gosu, parameterization information is unavailable for all native Java types, because Java does not preserve this information beyond compile time. For example, the runtime type of `java.util.List<Address>` in Gosu returns the unparameterized type `java.util.List`.

- Gosu includes shortcut initialization syntax for common collection types so you do not need to see the generics syntax, which tends to be verbose. For example:

```
var strlist = { "a", "list", "of", "Strings" }
```

Although this code does not specify a type, the `strlist` variable is statically typed. Gosu detects the types of objects that you use to initialize a variable and uses type inference to determine that `strlist` has the type `java.util.ArrayList<java.lang.String>`. This generics syntax represents a list of `String` objects.

See also

- “Generics” on page 241

Gosu primitive types

Gosu supports the following primitive types: `int`, `char`, `byte`, `short`, `long`, `float`, `double`, `boolean`. Additionally, Gosu supports `null`, which is a special value that means an empty object value. This set of primitive types is the full set that Java supports.

Every Gosu primitive type (other than the special value `null`) has an equivalent object type that the Java language defines. For example, for `int` there is the `java.lang.Integer` type that descends from the `Object` class. This category of object types that represent the equivalent of primitive types are called *boxed primitive* types. In contrast, primitive types are also called *unboxed primitives*. In most cases, Gosu converts between boxed and unboxed primitives in typical code. However, they are slightly different types, just as in Java, and on rare occasion these differences are important, especially with respect to the ability to represent `null`.

Boxed primitives can represent `null` but unboxed primitives cannot represent `null`. Converting a boxed primitive to an unboxed primitive has a special behavior if the variable or expression has the value `null` at run time. For `Boolean` values, the `null` becomes `false`. For numeric values, the `null` becomes `0` or the closest equivalent, such as the `char` value `0`.

Gosu statement terminators

The recommended way to terminate a Gosu statement and to separate statements is:

- A new-line character, also known as the invisible `\n` character

Although not recommended, you may also use the following to terminate a Gosu statement:

- A semicolon character `;`
- White space, such as space characters or tab characters

In general, use new-line characters to separate lines so that Gosu code looks cleaner.

For typical code, omit semicolons because they are unnecessary in almost all cases. Standard Gosu style uses semicolons between multiple Gosu statements when they are all on one line. For example, a short Gosu block definition uses this style. However, even in those cases, semicolons are optional in Gosu.

Valid and recommended

```
// Separated with newline characters
print(x)
```

```
print(y)

// If defining blocks, use semicolons to separate statements
var adder = \ x : Integer, y : Integer -> { print("I added!"); return x + y; }
```

Valid but not recommended

```
// NO - Do not use semicolon
print(y);

// NO - Do not rely on whitespace for line breaks. It is hard to read and errors are common.
print(x) print(y)

// NO - Do not rely on whitespace for line breaks. It is hard to read and errors are common.
var pnun = Policy.PolicyNumber cnum = Claim.ClaimNumber
```

IMPORTANT In almost all cases, omit semicolon characters in Gosu. However, standard Gosu style uses semicolons between multiple Gosu statements on one line, such as in short Gosu block definitions.

Invalid statements

```
var pnun = Policy.PolicyNumbercnum = Claim.ClaimNumber
```

See also

- “Blocks” on page 181

Gosu comments

Add comments to your Gosu code to explain its programming logic. Gosu supports several styles of comments.

Block comments	Use block comments to describe the purpose of classes and methods. A block comment begins with a slash/asterisk (/*) and ends with an asterisk/slash (*). <pre>/* * This is a block comment. * Use block comments at the beginnings of files and before * classes and functions. */</pre> Use proper capitalization and punctuation in block comments.
Single-line comments	Use single-line comments to describe one or more statements within a function or method. A single-line comment begins with double slashes (//) as the first non-whitespace characters on the line. <pre>if (condition) { // Handle the condition ... return true }</pre> If you cannot fit your comment on a single line, use a sequence of single-line statements.
Trailing comments	Use trailing comments to briefly describe single lines of code. A trailing comment begins with double slashes (//) as the first non-whitespace characters after the code that the comment describes. <pre>if (a == 2) { return true // Desired value of 'a' } else { return false // Unwanted value of 'a' }</pre>

If you place two or more trailing comments on lines of code in a block, indent them all to a common alignment

You can use some styles of comments to temporarily disable lines or blocks of code during development and testing. This technique is known as commenting out code.

Commenting out a single line of code	Use a single-line comment delimiter (//) to comment out a line of code. <pre> if (x = 3) { // y = z This line is commented out for testing. y = z + 1 This line will execute for testing. } </pre>
Commenting out a block of code	Use a pair of block comment delimiters (/*, */) to comment out a block of code, even if the block you want to comment out contains block comments. <pre> /* /* * The function returns the sum of its parts. */ public function myMethod(int A, int B) { return A + B } */ </pre> The preceding function cannot be called, because it is commented out. Gosu permits the nested block comment within the commented out block of code.

See also

- *Rules Guide*

Gosu reserved words

Gosu reserves a number of keywords for specialized use. The following list contains all the keywords recognized by Gosu. Gosu does not use some of the keywords in the list in the current Gosu grammar. Gosu does not prevent the use of some keywords in contexts outside their Gosu language meaning. Nonetheless, take care not to use any of the following words other than in their Gosu grammar usage. Using a keyword in a non-standard manner creates code that is difficult to understand or fails to compile.

- abstract
- accessors
- and
- annotation
- as
- assert
- block
- boolean
- break
- byte
- case
- catch
- char
- class
- classpath
- construct
- contains
- continue
- finally
- float
- for
- foreach
- function
- get
- hide
- if
- implements
- in
- index
- Infinity
- int
- interface
- internal
- iterator
- length
- long
- protected
- public
- readonly
- receiver
- reified
- represents
- return
- set
- short
- startswith
- static
- statictypeof
- structure
- super
- switch
- this
- throw
- transient

- | | | |
|---------------|------------|--------------|
| • default | • NaN | • true |
| • delegate | • new | • try |
| • do | • not | • typeas |
| • double | • null | • typeis |
| • else | • or | • typeloader |
| • enhancement | • out | • typeof |
| • enum | • outer | • uses |
| • eval | • override | • using |
| • extends | • package | • var |
| • false | • param | • void |
| • field | • print | • where |
| • final | • private | • while |
| • finally | • property | |

Legal variable names

Gosu variable names must conform to the following rules.

The first character of the name can be:

- Any letter
- \$
- _

Subsequent characters can be:

- Any letter
- Any digit
- \$
- _

Gosu generated documentation (Gosudoc)

You use the `gwb` tool to generate the Gosu documentation from the command line:

```
gwb gosudoc
```

This command generates the documentation at `PolicyCenter/build/gosudoc/index.html`.

The formatting of generated Gosu documentation has Javadoc style. This Gosudoc contains the output of the Gosu type system.

Code coverage support

Code coverage tools analyze the degree of testing of programming code. For Gosu code in Studio, PolicyCenter supports code coverage tools that use Java class files as input to bytecode analysis. Gosu compiles to Java class files.

Direct your code coverage tools to the following directory:

```
PolicyCenter/modules/configuration/out/classes
```

This feature requires tools that use Java class files, not source code, as input.

Notable differences between Gosu and Java

The following tables briefly summarize notable differences between Gosu and Java, with particular attention to requirements for converting existing Java code to Gosu. If the Gosu column contains multiple code examples, these

examples show the multiple ways in which Gosu can represent the Java code in the Java column. If the Required Change? column contains “Required”, you must make this change to convert existing Java code to Gosu. If this column contains “Optional”, that item is either an optional feature, a new feature, or Gosu optionally permits the Java syntax for this feature.

The following table shows general differences between Gosu and Java.

General difference	Java	Gosu	Required change?
Omit semicolons in most code. Gosu supports the semi-colon, but standard coding style is to omit it. One exception is in block declarations with multiple statements.	<pre>x = 1;</pre>	<pre>x = 1</pre>	Optional.
Print to console by using the print function. For compatibility with Java code that is converted to Gosu, you can optionally call the Java class <code>java.lang.System</code> print methods.	<pre>System.out.println("hello world");</pre>	Gosu style: <pre>print("hello world")</pre> Java compatibility style: <pre>System.out.println("hello world")</pre>	Optional.
For Boolean operators, optionally use more natural English versions. Gosu also supports the symbolic versions that Java uses.	<pre>(a && b) c</pre>	Gosu style: <pre>(a and b) or c</pre> Java compatibility style: <pre>(a && b) c</pre>	Optional.
Null-safe property accessor paths.	<pre>// Initialize to null ArrayList<Windows> x = null // Check earlier parts of // path for // null, to avoid // null pointer // exception if (house != null and house.Walls != null) { x = house.Walls.Windows }</pre>	<pre>var x = house.Walls.Windows</pre>	Optional. Do not make this change for cases in which you rely on throwing null pointer exceptions.

The following table shows the differences in functions and variables between Gosu and Java.

Function or variable difference	Java	Gosu	Required change?
In function declarations: - Use the keyword <code>function</code>. - List the type after the name, delimited by a colon, for parameters. - List the type after the parentheses that contain the parameters, delimited by a colon, for the return value.	<pre>public int addNumbers(int x, String y) { ... }</pre>	<pre>public function addNumbers(x : int, y : String) : int { ... }</pre>	Required

Function or variable difference	Java	Gosu	Required change?
In variable declarations, use the var keyword. Typically, you can rely on Gosu type inference and omit explicit type declaration. To explicitly declare the type, list the type after the variable, delimited by a colon. You can also coerce the expression on the right side, which affects type inference.	<pre>Auto c = new Auto();</pre>	Type inference: <pre>var c = new Auto()</pre> Explicit: <pre>var c : Auto = new Auto()</pre> Type inference with coercion: <pre>var c = new Auto() as Vehicle</pre>	Required
In declarations of variable argument functions, also called vararg functions, Gosu does not support the special Java syntax that indicates a variable number of arguments by using ... after the parameter type. Instead, design your methods to use arrays or collections. - To call variable argument Java functions, pass an array of the declared type. - To pass no values for the optional parameter, you can pass either an empty array {} or no value. Note: Gosu array initialization syntax is useful for calling these types of methods. This example demonstrates how to use a collection in a method declaration and call it using standard Gosu list initializer syntax: <pre>static function setValues(preparedStatement: PreparedStatement, values: List<Object>) MyClass.setValues(ps) MyClass.setValues(ps, {}) MyClass.setValues(ps, {value}) MyClass.setValues(ps, {my, many, values})</pre>	<pre>public String format(Object... args); public String errorMessage(Object... args);</pre>	Method call using array initializer syntax: <pre>var c = format({"aa", "bb"})</pre> Function declaration: <pre>public function format(args : Object[]) : String</pre> Method call with no arguments: <pre>var e = errorMessage()</pre>	Required
Gosu supports the unary operator assignment statements ++ and --, with the following restrictions: - Only use the operator after the variable, such as i++. - These operators form only statements, not expressions. Gosu supports other compound assignment statements, such as +=, -=.	<pre>if (++i > 2) { ... }</pre>	<pre>i++ if (i > 2) { ... }</pre>	Required
In an inner class, referring to methods, fields, and properties in the outer class uses the keyword outer and does not require specification of the outer class name.	<pre>String oldId = MyOuterClass.this._id</pre>	Gosu style: <pre>var oldId = outer._id</pre> Java compatibility style: <pre>var oldId = MyOuterClass.this._id</pre> If there is no ambiguity with an identifier in the inner class, you can omit the outer. syntax. <pre>var oldId = _id</pre>	Optional

Function or variable difference	Java	Gosu	Required change?
Calling a default method on an interface uses map-access operator syntax.	<code>IJourney.super.travel()</code>	<code>super[IJourney].travel()</code>	Required

The following table shows the differences in the type system between Gosu and Java.

Type system difference	Java	Gosu	Required change?
For coercions, use the <code>as</code> keyword. Optionally, for compatibility, Gosu supports Java-style coercion syntax.	<code>int x = (int) 2.1;</code>	Gosu style: <code>var x = 2.1 as int</code> Java compatibility style: <code>var x = (int) 2.1</code>	Optional.
Gosu downcasts to a more specific type in <code>if</code> and <code>switch</code> statements. Omit casting to the specific type.	<code>Object x = "nice"; Int s1 = 0; if (x instanceof String) { s1 = ((String) x).length; }</code>	<code>var x : Object = "nice" var s1 = 0 if (x typeis String) { s1 = x.length // Downcast }</code>	Optional.
Check if an object is a specific type or its subtypes by using <code>typeis</code> , which is similar to the Java <code>instanceof</code> operator.	<code>myobj instanceof String;</code>	<code>myobj typeis String</code>	Required.
To reference the type directly, use <code>typeof</code> . Direct comparisons to a type do not match on subtypes. Consider using <code>typeis</code> for this type of comparison rather than <code>typeof</code> .	<code>myobj.class</code>	<code>typeof myobj</code>	Optional.
In Gosu, type names are first-class symbols for the type. Do not get the <code>class</code> property from a type name.	<code>Class sc = String.class;</code>	<code>var sc = String</code>	Required.
Types defined natively in Gosu as generic types preserve their type information, including parameterization, at run time. This feature is called reified generics. In contrast, Java removes this information. This feature is called type erasure. From Gosu, Java types lack parameterization even if instantiated in Gosu. For native Gosu types, Gosu preserves type parameterization at run time.	<code>ArrayList<String> mylist = new ArrayList<String>(); System.out.println(mylist)</code> This code prints: <code>class java.util.ArrayLi st</code>	<code>var mylist = new ArrayList<String>() print(typeof mylist)</code> This code prints: <code>java.util.ArrayLi st<java.lang.Object></code> Note that <code>ArrayList</code> is a Java type. For native Gosu types as the main type, Gosu preserves the parameterization as runtime type information. In the following example, assume <code>MyClass</code> is a Gosu class:	Optional for typical use consuming existing Java types. If your code checks type information of native Gosu types, remember that Gosu has reified generics.

Type system difference	Java	Gosu	Required change?
		<pre>var mycustom = new MyClass<String>() print(typeof mycustom)</pre> This code prints: my.package.MyClass<java.lang.String>	
Gosu generics support array-style variance of different generic types. In Java, this usage is a compilation error, even though it is natural to assume that it works.	In Java, this code causes a compilation error: <pre>ArrayList<Object> mylist; mylist = new ArrayList<String>();</pre>	The analogous Gosu code works: <pre>var mylist : ArrayList<Object> mylist = new ArrayList<String>()</pre>	Optional.

The following table shows the differences in defining classes between Gosu and Java.

Defining classes difference	Java	Gosu	Required change?
Declare that you use specific types or package hierarchies with the keyword <code>uses</code> rather than <code>import</code> .	<pre>import com.abc.MyClass;</pre>	<pre>uses com.abc.MyClass</pre>	Required
Declare that you use specific static type members with the keyword <code>uses</code> rather than <code>import</code> . Use the <code>#</code> symbol before the member name or <code>*</code> symbol, not the period (<code>.</code>) symbol.	<pre>import com.abc.MyClass.MY_CONST import com.abc.MyClass.*;</pre>	<pre>uses com.abc.MyClass#MY_CONST uses com.abc.MyClass#*</pre>	Required
To declare one or more class constructors, write them as functions named <code>construct</code> but omit the keyword <code>function</code> . Gosu does not support Java-style constructors.	<pre>class ABC { public ABC(String id){ } }</pre>	<pre>class ABC { construct(id : String) { } }</pre>	Required

The following table shows the differences in control flow between Gosu and Java.

Control flow difference	Java	Gosu	Required change?
The <code>for</code> loop syntax in Gosu is different for iterating across a list or array. Use the same Gosu syntax for iterating with any iterable object, which is an object that implements <code>Iterable</code> . Optionally, add index <code>indexVar</code> before the closing parenthesis to create a zero-based index variable. If the object to iterate across is <code>null</code> , Gosu skips the loop and does not throw an exception. Java does throw an exception in this case.	<pre>int[] numbers = { 1,2,3 }; for (int item : numbers) { ... }</pre>	<pre>var numbers : int[] = {1,2,3} for (item in numbers) { ... }</pre>	Required
The <code>for</code> loop syntax in Gosu is different for iterating a loop for an integer number of times. The loop variable contains the zero-based index. Gosu has native support for intervals, which are sequences of values of the same type between a given pair of end-point values. For example, the set of integers beginning with 0 and ending with 10 has the shorthand syntax <code>0..10</code>. Intervals are useful for writing concise <code>for</code> loops.	<pre>for (int i=1; i<20; i++){ ... }</pre>	<pre>for (item in 20) { ... }</pre> Using Gosu intervals: <pre>for (i in 1..20) { ... }</pre>	Required

Control flow difference	Java	Gosu	Required change?
Gosu does not support the <code>for</code> (<code>initialize;compare;increment</code>) syntax that Java uses. You can duplicate this syntax by using a <code>while</code> statement.		Verbose style: <pre>var i = 0 while (i < 20) { ... i++ }</pre>	

The following table shows optional enhancements that Java does not support and that Gosu provides.

Difference	Java	Gosu	Required change?
Gosu enhancements, which support adding methods and properties to any type, including Java types.	Not applicable	<pre>enhancement StrLenEnhancement : java.lang.String { public property get PrettyLength() : String { return "length : " + this.length() } }</pre>	Optional
Gosu blocks, which are in-line functions that act like objects. These blocks are especially useful with the Gosu collections enhancements. Blocks can also be useful as a shortcut for implementing one-method interfaces.	Not applicable	<pre>\ x : Integer -> x * x</pre>	Optional
Native XML support and XSD support.	Not applicable	<pre>var e = schema.parse(xmlText)</pre>	Optional
Native support for consuming web services with syntax similar to native method calls.	Not applicable	<pre>extAPI.myMethod(1, true, "c")</pre>	Optional
Native String templates and file-based templates with type-safe parameters.	Not applicable	<pre>var s = "Total = \${ x }."</pre>	Optional
Gosu uses the Java-based collections APIs but improves upon them: - Simplified initialization syntax that preserves type safety. - Simpler array-like syntax for getting and setting values in lists, maps, and sets. - Additional methods and properties to improve functionality of the Java classes. Some enhancements use Gosu blocks for concise, flexible code. For new code, use the Gosu style initialization and APIs. You can call the more verbose Java style for compatibility.	<pre>List<String> strs = new ArrayList<String>(Arrays.asList("a", "ab", "abc"));</pre>	Simplified initialization: <pre>var strs = {"a", "ab", "abc"}</pre> Array-like "set" and "get": <pre>strs[0] = "b" var val = strs[1]</pre> New APIs on Java collections types: <pre>strList.each(\ str -> { print(str) })</pre>	Optional
List and array expansion operator.	Not applicable	<pre>names*.length</pre>	Optional

See also

- “Property accessor paths are null safe” on page 26
- “Variable assignment” on page 101
- “Basic type checking” on page 349
- “Enhancements” on page 215
- “Blocks” on page 181
- “Blocks as shortcuts for anonymous classes” on page 187
- “Working with XML in Gosu” on page 275
- “Templates” on page 323
- “Collections” on page 189
- “List expansion (*.)” on page 194
- *Integration Guide*

Gosu types

The Gosu language provides primitive data types and standard object types. The Gosu language also supports the use of Java types.

[See also](#)

- “Type system” on page 349

Primitive Gosu types

Gosu supports the following Java primitive types, which are provided for compatibility with existing Java code. From Gosu, you can access the Java object versions (non-primitives) of the Java primitive types. For example, `java.lang.Boolean` is an object type that wraps the behavior of the `boolean` primitive. Primitive types perform better in terms of performance and space compared to their object versions.

The following table compares primitive types and object types.

Type	Primitive types	Object types
Extends from <code>Object</code> class		•
Can reference an object		•
A variable of this type could contain <code>null</code> at run time		•
Exposes methods		•
Exposes properties		•
Can be a member of a collection		•

The following table lists the Java primitives that you can access from Gosu. The table mentions IEEE 754, which is the Institute of Electrical and Electronics Engineers standard for binary floating-point arithmetic.

Primitive	Type	Value
<code>boolean</code>	Boolean value	true or false
<code>byte</code>	Byte-length integer	8-bit two's complement

Primitive	Type	Value
char	Single character	16-bit Unicode
double	Double-precision floating point number	64-bit (IEEE 754)
float	Single-precision floating point number	32-bit (IEEE 754)
int	Integer	32-bit two's complement
long	Long integer	64-bit two's complement
short	Short integer	16-bit two's complement

See also

- http://en.wikipedia.org/wiki/IEEE_754

Gosu objects

The root type for all object types in Gosu is the Java class `java.lang.Object`. An object encapsulates some data as variables and provides properties and methods, which are functions that act on the object. Because `java.lang.Object` is always in scope, your code uses `Object` unless a similarly named type is in scope and requires disambiguation.

You can create classes that extend the root object type `Object`. If you do not declare a new class to extend a specific class, your new class extends the `Object` class.

Do not use the root object type `Object` to create objects directly. In some cases, you need to declare a variable that uses the type `Object` to support a variety of object subclasses. For example, you can define a collection to contain a heterogeneous mix of object types, all of which extend the root object type `Object`.

Example

```
var a : Address
var map = new java.util.HashMap()
```

See also

- “Bundles and database transactions” on page 333
- *Configuration Guide*

Object instantiation in Gosu

A Gosu object is an instance of a type. A type can be a class or other construct exposed to Gosu through the type system. A class is a collection of object data and methods. To *instantiate* the class means to use the class definition to create an in-memory copy of the object with its own object data. Other code can get or set properties on the object. Other code can call methods on the object, which are functions that perform actions on that unique instance of the object.

In PolicyCenter, entity instances are special objects defined through the data model configuration files. PolicyCenter persists data in a database at special times in the object life cycle.

Use the Gosu new operator to create an instance from a class definition or other type that can be instantiated.

Example

For example:

```
new java.util.ArrayList() // Create an instance of an ArrayList.
new Integer[5]           // Create an array of integers.
new LossCause[3]         // Create an array of loss cause typecodes.
```

See also

- “Bundles and database transactions” on page 333
- “Using the operator new in object expressions” on page 83
- *Configuration Guide*

Object property assignment

Property assignment is similar to variable assignment in Gosu.

Syntax

```
<object-property-path> = <expression>
```

Some properties are write-protected. Consider the following Gosu code:

```
Activity.UpdateTime = "Feb 17, 2017"
```

That code causes the following error:

```
Property, UpdateTime, of class Activity, is not writable
```

Other properties are read-protected but can be written.

Example

```
myObject.Prop = "Test Value"  
var startTime = myObject.UpdateTime
```

Property assignment triggering instantiation of intermediate objects

In general, if you try to assign a property on a variable and the variable evaluates to null at run time, Gosu throws a null pointer exception. However, if you set a property by using property path syntax that contains at least two objects, you can instruct Gosu to automatically instantiate an intermediate object in the path.

For example, suppose the following conditions hold:

- You have Gosu classes called `AClass` and `BClass`.
- `AClass` has a property called `B` that contains a reference to an instance of class `BClass`.
- The `BClass` class has a property called `Name` that contains a `String` value.
- Your code has a variable called `a` that contains an instance of type `AClass`.

At run time, if `a.B` contains a non-null reference, you can predict what the following code does:

```
a.B.Name = "John"
```

Gosu first evaluates `a.B`, and then on the result object sets the `Name` property to the value "John".

If the `AClass.B` property supports instantiation of intermediate objects, the same code works even if `a.B` is null at run time.

At run time, if Gosu detects that `a.B` is null:

1. Gosu instantiates a new instance of `BClass`.
2. Gosu sets `a.B` to the new instance.
3. Gosu sets `a.B.Name` property on the new instance of `BClass`.

For all Guidewire entity types, properties that contain a foreign key reference support automatic instantiation of intermediate objects.

You can add instantiation of intermediate objects to any Gosu class property. On the line before the property declaration, add the annotation:

```
@Autocreate
```

See also

- “Handling null values in expressions” on page 94
- “Annotating a class, method, type, class variable, or argument” on page 227

Object property access

Property accessor syntax

Gosu retrieves a property’s value by using the period operator. You can chain this expression with additional property accessors. Gosu handles null values in the expression to the left of the period differently from many other languages. Your code uses Gosu behavior to handle null values to avoid null pointer exceptions (NPE).

Syntax

```
object.PROPERTY_NAME
```

Examples

Expression	Result
Claim.Contacts.Attorney.Name	Pam Trujillo
Claim.Addresses[0].State	New Mexico

Associative array syntax

For most types other than Gosu arrays, you can use associative array syntax to access the elements, or properties, of an object. With *associative array syntax*, you access an object property with a String value in square brackets that contains the property name instead of an index number.

The following code uses associative array syntax to get the postal code of a contact.

```
contact["PostalCode"]
```

Use associative array syntax if a property name is unknown at compile but can be determined at run time with input from users. If the property name does not exist at run time, Gosu throws an exception.

WARNING Gosu cannot check associative array syntax at compile time to be certain that a property name is accurate or that the property exists. Be careful whenever you use associative array syntax to catch unexpected run time errors. Use property path expressions, which provide type-safe property access, instead of associative array syntax whenever possible.

Associative arrays on objects in Gosu are similar to instances of the Java map class `java.util.Map`. Associative array syntax for property access works with most classes, including the Map class and types that do not take array-style index numbers.

You cannot use associative array syntax with String objects in Gosu. A String object behaves like an ordered list of characters and requires array index notation to access its individual character elements.

Syntax

```
object[PROPERTY_NAME_STRING]
```

Examples

The following code demonstrates single associative array syntax to get the `StreetAddress` property on a `Person` object. At run time, the code is equivalent to the property path expression `person.StreetAddress`. The code might evaluate to `123 Main Street`.

```
person["StreetAddress"]
```

The following code demonstrates double associative array syntax to get the `City` property on the `Address` object that is a property of a `Person` object. At run time, the code is equivalent to the property path expression `person.Address.City`. The code might evaluate to `Birmingham`.

```
person["Address"]["City"]
```

The following code uses single associative array syntax to set the `City` property on an `Address` object.

```
newAddress["City"] = "Birmingham"
```

[See also](#)

- “Handling null values in expressions” on page 94
- “Null-safe property access” on page 94
- “Collections” on page 189

Object methods in Gosu

An object property can be any valid data type, including an array, a function, or another object. An object function is generally called a *method*. Invoking a method on an object is similar to accessing an object property, with the addition of parentheses after the name to denote the function. Gosu uses the dot notation to call a method on a object instance.

[Syntax](#)

```
object.METHOD_NAME()
```

[Example](#)

Expression	Result
<code>claim.isClosed()</code>	Returns a Boolean value indicating the status of <code>Claim</code>
<code>claim.resetFlags()</code>	Resets flags for this <code>claim</code>

[See also](#)

- “Handling null values in expressions” on page 94
- “Static method calls” on page 93

Using Boolean values in Gosu

From Gosu code, two types represent the values `true` and `false`:

- The Java primitive type `boolean`. Possible values are `true` and `false`.
- The Java `Boolean` object, which is an object wrapper around the primitive type. Possible values for variables declared to the `Boolean` data type are `true`, `false`, and `null`. The fully qualified type name is `java.lang.Boolean`. Because `java.lang.Boolean` is always in scope, your code uses `Boolean` unless a similarly named type is in scope and requires disambiguation.

For both `boolean` and `Boolean`, Gosu coerces values to `true` or `false`. The following table describes coercion rules.

Value	Type of value	Coerces to	Note
1	int	true	
0	int	false	

Value	Type of value	Coerces to	Note
"true"	String	true	If you coerce String data to either Boolean or boolean, the String data itself is examined. It coerces to true if and only if the text data has the value "true".
"false" or any non-null String value other than "true"	String	false	See note for column value "true".
null	See note	If coerced to the boolean type, the result depends on the type of the declared variable. For some types, including Object and number types such as Integer, null coerces to the value false. For other types, coercion is prohibited at compile time. If coerced to the Boolean type, value remains null.	The boolean type is a primitive and can never contain null. The Boolean type is an object type, and variables of that type can contain null. Be careful to check for null values for variables declared as String. A null value might indicate uninitialized data or other unexpected code paths.

Notice the following differences between primitive and object types:

- null coerced to a variable of type Boolean stores the original value null.
- null coerced to a variable of type boolean stores the value false because primitive types cannot be null. For some variable types, this coercion is prohibited at compile time.

Be careful to check for null values for variables declared as String to avoid ambiguity in your code. A null value might indicate uninitialized data or other unexpected code paths.

Example

```
var hasMoreMembers : Boolean = null
var isDone = false
```

Using characters and String data in Gosu

To represent a single character in Gosu, use the primitive type char. To type a single character literal, surround it with single quotation marks, such as 'x'.

To represent a sequence of characters in Gosu, use the Java type java.lang.String. Because java.lang.String is always in scope, your code uses String unless a similarly named type is in scope and requires disambiguation. Create a String object by enclosing a string of characters in beginning and ending quotation marks. Example values for the String data type are "hello", "123456", and "" (the empty string). If you need to type a quotation mark character in a String literal, escape the quotation mark character by putting a backslash before it, such as "hello \\"Bob\\"".

Alternatively, you can use single quotation marks instead of double quotation marks. This style is useful if you want to type String literals that contain the double quotation mark character because you do not need to escape quotation mark characters:

```
var c = 'Hello "Bob"'
```

If you use single quotation marks and the literal has exactly one character, Gosu infers the type to be char instead of String.

String variables can have content, be empty, or be null

It is important to understand that the value `null` represents the absence of an object reference and it is distinct from the empty String value `""`. The two are not interchangeable values. A variable declared to type `String` can hold the value `null`, the empty String `""`, or a non-empty String.

To test for a populated String object versus a null or empty String object, use the `HasContent` method. When you combine it with the null-tolerant property access in Gosu, `HasContent` returns `false` if either the value is `null` or an empty String object.

Compare the behavior of properties `HasContent` and `Empty`:

```
var s1 : String = null
var s2 : String = ""
var s3 : String = "hello"

print("s1 has content = " + s1.HasContent)
print("s2 has content = " + s2.HasContent)
print("s3 has content = " + s3.HasContent)

print("s1 is empty = " + s1.Empty)
print("s2 is empty = " + s2.Empty)
print("s3 is empty = " + s3.Empty)
```

This code prints:

```
s1 has content = false
s2 has content = false
s3 has content = true
s1 is empty = false
s2 is empty = true
s3 is empty = false
```

If the variable holds an empty String or `null`, the `HasContent` method returns `false`. Using the `HasContent` method is more intuitive than using the `Empty` property in typical cases where `null` represents absence of data.

Setting values of String properties in entity instances

Setting a value on a String data type property on an entity instance causes special behavior that does not happen for String properties on classes. When you set a String value on a database-backed entity type property:

- Gosu removes spaces from the beginning and end of the String value. This behavior is configurable.
- If the result is the empty String `""`, Gosu sets the value to `null` instead of the empty String. This behavior is not configurable

In both cases, this change happens immediately as you set the value. This change is not part of committing the data to the database. If you get the property value after setting it, the value is already updated.

For example:

```
var obj = new Temp1()           // New normal Gosu or Java object
var entityObj = new Address()  // New entity instance

// Set String property on a regular object
obj.City = "      San Francisco      "
display("Gosu", obj.City)
obj.City = "      "
display("Gosu", obj.City)
obj.City = null
display("Gosu", obj.City)

// Set String property on an entity instance
entityObj.City = "      San Francisco      "
display("entity", entityObj.City)
entityObj.City = "      "
display("entity", entityObj.City)
entityObj.City = null
display("entity", entityObj.City)

function display (intro : String, t : String) {
    print (intro + " object " + (t == null ? "NULL" : "\"" + t + "\""))
}
```

This code prints:

```
Gosu object "      San Francisco      "
Gosu object ""
Gosu object NULL
entity object "San Francisco"
entity object NULL
entity object NULL
```

Note that the entity property has no initial or trailing spaces in the first case, and is set to null in the other cases. If you want to test a `String` variable to see if it has content or is either null or empty, use the `HasContent` method.

Configuring whitespace removal for entity text properties

You can control whether PolicyCenter trims whitespace on a database-backed `String` property on an entity type. Set the `trimwhitespace` column parameter in the data model definition of the `String` column. For columns that you define as `type="varchar"`, Gosu trims leading and trailing spaces by default.

To prevent PolicyCenter from trimming whitespace before committing a `String` property to the database, include the `trimwhitespace` column parameter in the column definition, and set the parameter to `false`. The XML definition for a column that does not trim whitespace looks like the following lines:

```
<column
  desc="Primary email address associated with the contact."
  name="EmailAddress1"
  type="varchar">
  <columnParam name="size" value="60"/>
  <columnParam name="trimwhitespace" value="false"/>
</column>
```

The parameter controls only whether PolicyCenter trims leading and trailing spaces. You cannot configure whether PolicyCenter coerces an empty string to null.

For both trimming and converting empty `String` values to null, the change happens immediately that you set the value.

See also

- “String variables can have content, be empty, or be null” on page 55
- “Setting values of String properties in entity instances” on page 55

Methods on String objects

Gosu provides various methods to manipulate strings and characters. For example:

```
var str = "bat"
str = str.replace( "b", "c" )
print(str)
```

This code prints:

```
cat
```

Type `"new String()"` into the Gosu Scratchpad and then press period (.) to see the full list of methods.

String utilities

You can access additional `String` methods in the API class `gw.api.util.StringUtil`. Type `gw.api.util.StringUtil` into the Gosu Scratchpad and press period to see the full list of methods.

For example, to perform search and replace, instead of the Java native method `replace` on `java.lang.String`, you can use the `StringUtil` method `substituteGlobalCaseInsensitive`.

Escaping special characters in strings

In Gosu strings, the backslash character (`\`) indicates that the immediately following character requires special handling. Because the backslash is used to “escape” the usual meaning of the character in the string, the backslash is

called an *escape character*. The combination of the backslash and its following character is called an *escape sequence*.

For example, you use the backslash escape character to insert a quotation mark into a string without terminating the string. The following list describes some common uses for the backslash in Gosu strings.

Sequence	Result
\\	Inserts a backslash into the string without forcing an escape sequence.
\"	Inserts a double-quotation mark into the string without terminating it. Note: This escape sequence is not used in embedded code inside Gosu templates. In such cases, do not escape the quotation mark characters.
\n	Inserts a new line into the string so that the remainder of the text begins on a new line if printed.
\t	Inserts a tab into the string to add horizontal space in the line if printed.

Examples

```
Claim["ClaimNumber"]
var address = "123 Main Street"
"LOGGING: \n\"Global Activity Assignment Rules\""
```

See also

- “Templates” on page 323

Gosu String templates

In addition to simple text values enclosed by quotation marks, you can embed Gosu code in the definition of a `String` literal. At compile time, Gosu uses its native type checking to ensure that the embedded Gosu code is valid and type safe.

Embedding a Gosu expression into text

Use the following syntax to embed a Gosu expression in the `String` text:

```
${ EXPRESSION }
```

If the expression does not return a value of type `String`, Gosu attempts to coerce the result to the type `String`.

For example, suppose you need to display text with a calculation in the middle of the text:

```
var mycalc = 1 + 1
var str = "One plus one equals " + mycalc + "."
print(str)
```

Instead of this multiple-line code, embed the calculation directly in the `String` as a template:

```
var str = "One plus one equals ${ 1 + 1 }."
print(str)
```

This code prints:

```
One plus one equals 2.
```

Embedding Gosu statements into text

To embed one or more statements, which do not return a value, in `String` text, you use a template scriptlet. Use the two-character text `<%` to begin the scriptlet. Use the two-character text `%>` to end the scriptlet:

```
<% STATEMENT LIST %>
```

Gosu runs the statements in the scriptlet before evaluating the text in the string.

The following code shows the use of scriptlets in a `String` value:

```
var str = "<% for (i in 1..5) { var odd = (i % 2 == 1) %>${i} is ${ (odd?"odd":"even")} }\n<% } %>"
print(str)
```

This code prints:

```
1 is odd
2 is even
3 is odd
4 is even
5 is odd
```

Quotation marks in templates

Within a code expression or statement inside a template, do not escape quotation mark characters by using a backslash character. The following line is valid Gosu code:

```
var str = "<% var myvalue = {"a", "b"} %>"
```

The following line is invalid because of incorrect escaping of the internal quotation marks:

```
var badStr = "<% var bar = {"a\"", "\"b\""} %>"
```

See also

- “Templates” on page 323

Numeric literals in Gosu

Gosu natively supports numeric literals of the most common numeric types, as well as a binary and hexadecimal syntax. Gosu uses the syntax to infer the type of the value.

For example:

- Gosu infers that the following variable has type `BigInteger` because the right side of the assignment uses a numeric literal `4bi`. That literal means “4, as a big integer”.

```
var aBigInt = 4bi
```

- Gosu infers that the following variables have type `Float` because the right side of the assignment uses a numeric literal with an `f` after the number.

```
var aFloat = 4f
var anotherFloat = 4.0f
```

You can omit the suffix of a numeric literal if the type is declared explicitly such that no type inference is necessary. Gosu does not support floating point hexadecimal literals.

The following table lists the suffix or prefix for different numeric, binary, and hexadecimal literals.

Type	Suffix or prefix	Examples
byte	suffix: b or B	var aByte = 4b
short	suffix: as short	var aShort = 4 as short
int	none	var anInt = 4
long	suffix: l (lowercase L) or L	var aLong = 4L
float	suffix: f or F	var f1 = 4f var f2 = 4.0f var f3 = 4.0e3
double	suffix: d or D	var aDouble = 4d
BigInteger	suffix: bi or BI	var aBigInt = 4bi

Type	Suffix or prefix	Examples
BigDecimal	suffix: bd or BD	var aBigD = 4bd var anotherBigD = 4.0bd
int as binary or mask	prefix: 0b or 0B	var maskVal1 = 0b0001 // 0 and 1 only var maskVal2 = 0b0010 var maskVal3 = 0b0100
int as hexadecimal	prefix: 0x or 0X	var aColor = 0xFFFF // 0 through F only

Scientific notation and floating point

Gosu supports the use of scientific notation to represent large or small numbers. *Scientific notation* represents a number as a coefficient, which is a number greater than or equal to 1 and less than 10, and a base, which is always 10.

For example, consider the number: $1.23 * 10$ to the 11th power.

The number 1.23 is the coefficient. The number 11 is the exponent, which means the power of 10. The base number 10 is always written in exponent form. Gosu represents the base number as the letter *e*, which stands for *exponent*.

You can use the scientific notation anywhere you can use a `float` literal or `double` literal.

Examples

```
var result1 = 9.2 * 3
var result2 = 2.057e3

print (result1)
print (typeof result1)

print (result2)
print (typeof result2)
```

This code prints:

```
27.599999999999998
double
2057.0
double
```

Using date and time values in Gosu

Gosu date handling is simpler than Java provides in the `java.util.Date` class. Gosu and Guidewire enhancements also simplify coding in comparison to the `DateUtil` class. For example, the following code using the `DateUtil` class can be simplified:

```
var minDaysNeeded = 2
var nonRenewalNotificationLeadTime = 10
var newDate = DateUtil.addDays( DateUtil.currentDate(), minDaysNeeded + nonRenewalNotificationLeadTime )
print(newDate)
```

Compare the usage of the `DateUtil.addDays` method usage above to its equivalent in the following example:

```
var minDaysNeeded = 2
var nonRenewalNotificationLeadTime = 10
var newDate = Date.CurrentDate.addDays( minDaysNeeded + nonRenewalNotificationLeadTime )
print(newDate)
```

In both cases the output is:

```
Sat Aug 24 17:43:16 PDT 2019
```

The `Date` class defines many useful methods and static properties, such as `Date.Today` and `Date.Yesterday`.

Gosu enables you to iterate over a range of dates, which is often useful when testing date-sensitive code. The `Date.to` enhancement method returns a `DateRangeIterator`, as shown in the following example:

```
var start = Date.CurrentDate
var end = Date.CurrentDate.addDays(10)
for (d in start.to( end )) {
    print(d)
}
```

The output is:

```
Mon Aug 12 17:44:21 PDT 2019
Tue Aug 13 17:44:21 PDT 2019
Wed Aug 14 17:44:21 PDT 2019
Thu Aug 15 17:44:21 PDT 2019
Fri Aug 16 17:44:21 PDT 2019
Sat Aug 17 17:44:21 PDT 2019
Sun Aug 18 17:44:21 PDT 2019
Mon Aug 19 17:44:21 PDT 2019
Tue Aug 20 17:44:21 PDT 2019
Wed Aug 21 17:44:21 PDT 2019
Thu Aug 22 17:44:21 PDT 2019
```

The default iteration step, used in the example above, is one day. To change the step increment, call one of the `DateRangeIterator.by` methods, such as `byMonth`:

```
var start : Date = Date.CurrentDate
var end : Date = Date.CurrentDate.addDays(200)
for (d in start.to( end ).byMonth()) { //To step by month
    print(d)
}
```

The output is:

```
Mon Aug 12 17:45:18 PDT 2019
Thu Sep 12 17:45:18 PDT 2019
Sat Oct 12 17:45:18 PDT 2019
Tue Nov 12 17:45:18 PST 2019
Thu Dec 12 17:45:18 PST 2019
Sun Jan 12 17:45:18 PST 2020
Wed Feb 12 17:45:18 PST 2020
```

Note: The `DateRangeIterator` works in both directions. When starting with a later date, the iterator iterates backwards by the specified step value.

Entity types

An entity is a type of object that is constructed from the data model configuration files. Like other types of objects, entities have data in the form of object properties, and actions in the form of object domain methods.

Some Guidewire API methods take an argument of type `IEntityType`. The `IEntityType` type is an interface, not a class. All Guidewire entity types implement this interface. To use an API that requires an `IEntityType`, pass an entity instance to the API. For example, to pass an `Address` to an API that takes an `IEntityType` as the method argument, use code such as:

```
myClass.myMethod(Address)
```

In rare cases, you might need to get the type from an entity dynamically, for example to get the subtype of an entity instance. If you have a reference to an entity instance, get its `Type` property. For example, if you have a variable `ad` that contains an `Address` entity instance, use code such as the following:

```
myClass.myMethod(ad.Type)
```

WARNING The base configuration includes visible usages of the entity instance method `setFieldValue`. The `setFieldValue` method is reserved for Guidewire internal use. Calling this method can cause serious data corruption or errors in application logic that may not be immediately apparent. You must obtain explicit prior approval from Guidewire Support to call `setFieldValue` as part of a specific approved workaround.

See also

- *Configuration Guide*

Keys of Guidewire entity instances

Key is a Guidewire data model entity base type that uniquely identifies an entity instance. The internal ID property of an entity such as `Policy.ID` has type `Key`. Provide the ID of an entity instance in methods that use `Key` objects as parameters.

Do not confuse a key, which has the type `Key`, and a `typekey`, which has the type `Typekey`.

Example

```
var id = myEntityInstance.ID // id has type Key
```

See also

- “Typekeys and typelists” on page 61

Typekeys and typelists

Many Guidewire entity data model entities contain properties that contain typekeys. A typekey is similar to an enumeration. A typelist encapsulates a list of typekeys. Each typekey has a unique immutable code as a `String`, in addition to a user-readable (and localized) name. The code for a typekey is also known as a typecode.

For example, the typelist called `AddressType` contains typekey values `BUSINESS`, `HOME`, and `OTHER`, which are available as constants on the typelist type with some changes to the name.

Typelist Literals

In most cases, to get a literal for a typelist, you can type the typelist name in the appropriate programming context. Ambiguity about the name or package of the typelist might occur in some programming contexts. To resolve this ambiguity, type the fully qualified syntax `typekey.TYPELISTNAME`. For example, `typekey.AddressType`.

Typecode Literals

To reference an existing typecode from Gosu, access the typecode by using the typelist name and the typecode value in capital letters and the prefix `TC_`:

```
TYPELISTNAME.TC_TYPECODENAME
```

For example, to select from the `AddressType` typelist a reference to the typekey with code `BUSINESS`, use:

```
AddressType.TC_BUSINESS
```

In the data model configuration, the typecode definition has an optional `identifierCode` attribute, which changes how the application generates the programmatic typekey name in Gosu. The value of the `identifierCode` attribute can include only characters that are valid for a Gosu identifier such as a class or variable name. For example, you cannot include a hyphen (-) character in the value of the `identifierCode` attribute. The maximum length of the identifier code value is three characters fewer than the maximum length of a Gosu identifier name. For practical purposes, this length is unlimited.

PolicyCenter uses the following rules to generate the typecode literal:

- If the optional `identifierCode` attribute exists, PolicyCenter transforms the `identifierCode` attribute to upper case, and adds the prefix `TC_`. Using the `identifierCode` attribute ensures that all identifier names are unique, which prevents name conflicts among typecode literals.
- If the `identifierCode` attribute does not exist for a typecode, PolicyCenter generates the typecode literal from the code attribute. PolicyCenter transforms the code attribute to upper case, replaces invalid characters with underscores (`_`), and adds the prefix `TC_`. For example, codes `abc`, `123`, and `a-b` become the identifiers `TC_ABC`, `TC_123`, and `TC_A_B`, respectively. The substitution of invalid characters by underscores might cause two codes to become the same identifier. For example, codes `A_B` and `A-B` both become `TC_A_B`, which is a conflict that causes compilation errors.

Getting all available typekeys from a typelist

You can programmatically get a list of typekeys in a typelist. You can choose whether to get all typekeys, including both retired and non-retired typekeys, or just non-retired typekeys. You can retire a typekey in the data model configuration files.

To get typekeys from a typelist, call the `getTypeKeys` method:

```
TYPENAME.getTypeKeys(includeRetiredTypekeys)
```

The Boolean argument indicates whether to include obsolete typekeys that are marked as retired. If the argument is set to `true`, the return value includes retired typekeys. Otherwise, the method returns only typekeys that are not retired.

For example:

```
// Prints all typekeys in the AddressType typelist
print(AddressType.getTypeKeys(false))

// Prints the code of the first typekey in the array
print(AddressType.AllTypeKeys[0].Code)
```

This code prints:

```
[Billing, Business, Home, Other]
billing
```

Getting information from a typekey

From Gosu, if you have a reference to a typekey, you can get the following properties from the typekey:

Property	Description	Context to use	Example
Code	A non-localized String representation that represents this typekey. Typically you would use this code for exporting this typekey property to an external system. This name is not intended to be a display name, and must contain no space characters and might use abbreviations.	If comparing values, sending values to a remote system, or storing values for later comparison	business
DisplayName	A localized version of the display name, based on the current language settings.	For display to a user. This might include sending mail or other notifications in the local language.	Affaires
UnlocalizedName	The unlocalized version of the display name. This does not vary on the current language settings.	Only in debugging or for compatibility with earlier product releases.	Business
Description	A description of this typekey value's meaning	If you need a description (non-localized) from the data model configuration files.	Business address type

If your application is multi-lingual and manipulates typekeys, choose very carefully whether you want to get the `DisplayName` property, the `UnlocalizedName` property, or the `Code` property. In almost all cases, use the `Code` if you

might store or compare values later on or use the `DisplayName` if you are displaying something to the user. The `UnlocalizedName` property exists for debugging reasons and compatibility reasons and in general do not use it. Instead, use `Code` or `DisplayName`.

To extract display name information, you can use `myCode.DisplayName`

For example:

```
print(AddressType.TC_BUSINESS.DisplayName)
```

This code prints:

```
Business
```

Gosu array types

An *array* in Gosu is a collection of data values or objects of the same type. A number, or *index*, identifies the position of each element in the array. Index values for arrays begin with zero and increment by one. The index value 0 identifies the first element of an array.

Creating a Gosu array

Use square brackets after a type name to create an array. Gosu constrains elements of the array to values or objects of that type. For example, the expression `new String[]` creates an array of `String` elements.

Whenever you create an array, you must set its size in terms of a fixed number of elements. You can set the size explicitly by including the number of elements within the square brackets. The following statement explicitly sets the size of a `String` array to four elements.

```
var stringArray = new String[4]
```

Alternatively, you can set the size of an array implicitly by providing a list of values or objects with which to initialize the array. The size of the array is the number of items in the list. The following statement implicitly sets the size of a `String` array to four elements.

```
var stringArray = new String[] {"one", "two", "three", "four"}
```

Initializing a Gosu array with default values

You can create an array with all of its elements set to the same default value by calling the `makeArray` Gosu enhancement method on the `java.utils.Arrays` class. Pass the default value and the size of the array. Gosu sets the type of the array to the same type as the default value. The following statement creates a `String` array of five elements, with each element initialized to the value `no`.

```
var stringArray = java.util.Arrays.makeArray("no", 5)
```

Accessing elements of a Gosu array

In Gosu, you access elements of an array with array index notation. Place the index value for the element you want to access within square brackets that follow the array name. Index values for array elements begin with zero and increment by one. The following code uses the index value 2 to access the third element of an array.

```
// Create an array with four elements.
var stringArray = new String[] {"one", "two", "three", "four"}

// Print the the third element of the array.
print ("Third array element contains the value " + stringArray[2] + ".")
```

The output from this code is:

```
The third array element contains the value three.
```

Examples

The following code accesses the first exposure on a claim.

```
Claim.Exposures[0]
```

The following code creates an array of five elements and then accesses the third element, the character “c”.

```
gw.api.util.StringUtil.splitWhitespace( "a b c d e" )[2]
```

Iterating the elements of a Gosu array

In Gosu, you can iterate through the elements of an array in a for loop. You declare a temporary iteration variable to hold an element of the array during each loop iteration, and you specify the name of the array through which to iterate. The following code declares the iteration variable `element` and then uses it to print each element of the array.

```
var stringArray = new String[] {"one", "two", "three", "four"}

for (element in stringArray) {
    print(element)
}
```

The output from this code is:

```
one
two
three
four
```

See also

- “Iteration in for statements” on page 106

Gosu arrays and the expansion operator

Gosu includes a special operator for array expansion and list expansion. The expansion operator is an asterisk followed by a period. Array expansion is valuable if you need a single one-dimensional array or list through which you can iterate.

The return value is as follows:

- If you use the expansion operator on an array, Gosu gets a property from every item in the array and returns all instances of that property in a new, single-dimensional, read-only array.
- If you use the expansion operator on a list, Gosu gets a property from every item in the list and returns all instances of that property in a new, single-dimensional, read-only list.

You can access elements in the new array or list individually with index notation, and you can iterate through the elements in a for loop. You cannot modify the elements of the new array or list.

The following code creates a `String` array. `String` objects in Gosu have a `length` property that contains the number of characters. The code uses the expansion operator on the array to create an array of string lengths for elements of the original array. Because the `length` property is of type `int`, the expansion array also is of type `int`.

```
var stringArray = new String[] {"one", "two", "three", "four"}
var lengthArray = stringArray*.length

for (element in lengthArray) {
    print(element)
}
```

The output from this code is:

```
3
3
5
4
```


Suppose you have an array of `Book` objects, each of which has a `String` property `Name`. You could use array expansion to extract the `Name` property from each item in the array. Array expansion creates a new array containing just the `Name` properties of all books in the array.

If a variable named `myArrayOfBooks` holds your array, use the following code to extract the `Name` properties:

```
var nameArray = myArrayOfBooks*.Name
```

The `nameArray` variable contains an array whose length is exactly the same as the length of `myArrayOfBooks`. The first item is the value `myArrayOfBooks[0].Name`, the second item is the value of `myArrayOfBooks[1].Name`, and so on.

Suppose you wanted to get a list of the groups to which a user belongs so that you can display the display name property of each group. Suppose a `User` object contains a `MemberGroups` property that returns a read-only array of groups to which the user belongs. In this case, the Gosu syntax `user.MemberGroups` returns an array of `Group` objects, each one of which has a `DisplayName` property. To get the display name property from each group, use the following Gosu code

```
user.MemberGroups*.DisplayName
```

Because `MemberGroups` is an array, Gosu expands the array by the `DisplayName` property on the `Group` component type. The result is an array of the names of all the `Groups` to which the user belongs. The type is `String[]`.

The result might look like the following:

```
["GroupName1", "GroupName2", "GroupName14", "GroupName22"]
```

The expansion operator also applies to methods. Gosu uses the type on which the method runs to determine how to expand the type:

- If the original object is an array, Gosu creates an expanded array.
- If the original object is a list, Gosu creates an expanded list.

The following example calls a method on the `String` component of the `List` of `String` objects. Gosu generates a list of initials by extracting the first character in each `String`.

```
var s = {"Fred", "Garvin"}

// Generate the character list [F, G]
var charList = s*.charAt( 0 )
```

Important notes about the expansion operator:

- The generated array or list itself is always read-only from Gosu. You cannot assign values to elements of the array, such as setting `nameArray[0]`.
- The expansion operator `*` applies only to array expansion, not for standard property accessing.
- When using the `*` expansion operator, only component type properties are accessible.
- When using the `*` expansion operator, array properties are not accessible.
- The expansion operator applies to arrays and to any `Iterable` type and all `Iterator` types. The operator preserves the type of the array or list. For instance, if you apply the `*` operator to a `List`, the result is a `List`. All other expansion behavior is the same as for arrays.

See also

- “Enhancements on Gosu collections and related types” on page 201

Array list access with array index notation

In Gosu, you can access elements of Java language array lists by using array index notation. You can iterate through Java array lists in a `for` loop the same as a Gosu array. The following code creates a Java array list, populates the list, and updates one element in the list by using array index notation. Then the code uses a `for` loop to iterate through the elements of the list and print them.

```
// Create a Java array list.
var list = new java.util.ArrayList()
```

```
//Populate the Java list with values.
list.add("zero")
list.add("one")
list.add("two")
list.add("three")
list.add("four")

//Assign a value to a element of the Java list.
list[3] = "threeUPDATED"

//Iterate through the Java list the same way as a Gosu array.
for (element in list) {
    print(element)
}
```

The output for this code is:

```
zero
one
two
threeUPDATED
four
```

In many situations, using collection classes, such as `java.util.ArrayList` or `java.util.HashMap`, is a better choice than using Gosu arrays. For example, the sizes of lists and maps grow and shrink as you add and remove members from them. The `ArrayList` and `HashMap` classes are the native Java array lists and hash maps. Gosu adds enhancement methods to these types or their parent classes/interfaces, such as the interface `java.util.List`.

See also

- “for statements” on page 105
- “Collections” on page 189

Entity arrays

In the data model definition of an entity type, you can define a property that is an array of read-only entity instances of another specific type.

By default, array fields on entities are unordered indexed arrays in Gosu. An indexed entity array acts like a typical array in that you can get an object by using a numeric index offset from the beginning. For example, `myArray[2]`. Because the items are unordered, do not rely on the meaning or long term position of a specific element. However, use numeric ordering to loop across the array with looping syntax, such as a `for` loop.

Gosu defines other enhancement methods and properties on arrays, such as `each`, `firstWhere`, `HasElements`, and `map`. Use enhancement methods and properties to write concise powerful Gosu code. However, be aware that bringing many entities into memory can be resource intensive if the entity array size is large.

For example, in PolicyCenter, on a `User` entity type definition file in `user.eti`, the property `Roles` is an array of role objects that relate to a user:

```
<array arrayentity="UserRole" name="Roles" ... />
```

Gosu adds an `addTo...` method and a `removeFrom...` method to the containing entity type to help manage the elements of each associative array defined on the entity. In the preceding example, Gosu adds the methods `addToRoles` and `removeFromRoles` to the entity type `User`.

Optionally, entity arrays support an associative syntax like a map.

Entity array `addTo...` methods

An `addTo...` method, such as the `addToRoles` method described in the example above, sets a foreign key on each element in an entity array that points back to the entity that owns the array. You must call the `addTo...` method on newly created array elements in order for the Rule Engine to commit them to the database. As a best practice, Guidewire strongly recommends that you always call the appropriate `addTo...` method to relate new array elements to the entity that owns the array.

IMPORTANT Entity arrays are inherently unordered with a non-deterministic order each time they are generated. For example, the order in which an `addTo...` method adds elements to its entity array is not necessarily the order in which a `for` loop retrieves them.

Entity array `removeFrom...` methods

The `removeFrom...` method behaves differently depending on whether the elements of an entity array are a retireable type:

- If the element type of the array is retireable, the `removeFrom...` method retires the element.
- If the element type of the array is not retireable, the `removeFrom...` method deletes the element.

The following example removes elements in an array in which the primary contributing factor was driver error.

```
for (factor in claim.ContributingFactors) {
    if (factor.Primary == "Driver Factors") {
        claim.removeFromContributingFactors(factor)
    }
}
```

In this next example, the `Modifiers` entity is an array of objects that relate to a payment plan:

```
<array name="Modifiers"
  arrayentity="Modifier"
  owner="true">
  <link-association>
    <typelist-map field="BillingInstructionType"/>
  </link-association>
</array>
```

If you have created an array of `Modifier` objects called `paymentPlan`, the method `removeFromModifiers` removes the specified element from that array:

```
paymentPlan.removeFromModifiers(aModifier)
```

Note: If the `paymentPlan` array already has a `Modifier` element for a particular `BillingInstructionType`, adding a new element for that same `BillingInstructionType` does not retire the old modifier, but rather appends the object to the array. To retire a modifier you must call its `removeFromModifiers` method.

To remove an element from all arrays containing it, call that element's `remove` method.

Optional associative entity arrays

By default, entity arrays are unordered indexed arrays. Optionally, you can define an entity array to support associative array syntax. An associative array acts like a container with key-value mapping similar to `java.util.HashMap`. Get items from an associative array by providing a key instead of a numeric index. For example, `myArray["en_US"]`. In the data model definition files, you can make associative arrays by adding the `<array-association>` subelement to the `<array>` element.

The modified array field still behaves as an indexed array but you can optionally get items by using associative array syntax.

See also

- “Enhancements on Gosu collections and related types” on page 201
- *Configuration Guide*

Loading a multidimensional entity array

The `ArrayLoader` class allows you to load a multidimensional array with at most a single pull from the PolicyCenter database.

Assuming no data is stored in cache, when you load a multidimensional array manually, you must query the PolicyCenter database for each array element. Thus, for manual array loading, you must query the database $N \times M$

times. Here, N is the number of array elements in the first dimension array, and M is the number of array elements in each of the N second dimension arrays.

This number of queries arises from the fact that a multidimensional array is really an array of pointers. The array is not a data structure where all of the array elements are combined in the same location. Hence, manual array loading degrades performance due to the overhead bandwidth requirement for the $N*M$ queries and due to contention with other database queries.

The `ArrayLoader` class eliminates this challenge by combining the $N*M$ queries into at most single query. The special case where all of the required data is stored in cache requires no database queries. Thus, the `ArrayLoader` class reduces the overhead bandwidth requirement and the risk of database contention for loading a multidimensional array.

The primary methods in the `ArrayLoader` class are three overloaded constructors. The first constructor has the following signature:

```
public ArrayLoader(IArrayPropertyInfo arrayPropertyInfo, Iterable<P> parents)
```

The `arrayPropertyInfo` parameter contains metadata about the entity type to be loaded into an array. This metadata includes primarily the table name and the column name for the entity type to be queried. The `parents` parameter includes the entity instances from the table.

For example, a desired multidimensional array might contain a collection of policies and the coverables for those policies. The `arrayPropertyInfo` parameter would instruct the database to pull the coverables for the policies in the collection. The `parents` parameter contains the collection of policies

The second constructor has the following signature:

```
public ArrayLoader(IArrayPropertyInfo arrayPropertyInfo, P[] parents)
```

This constructor is logically equivalent to the first constructor. The only difference is that the second constructor represents the `parents` parameter in array form as opposed to an `Iterable` form.

The signature for the third constructor is as follows:

```
public ArrayLoader(IArrayPropertyInfo arrayPropertyInfo, Iterable<Key> parentIds, Bundle bundle)
```

For this constructor the `parents` parameter contains parent keys as opposed to complete entity instances. All three constructors use only the key from the entity instances. The third constructor takes the parent keys directly from the method invocation as opposed to the collection of entity instances. The `bundle` parameter is the ultimate destination for the children in the second dimension of the multidimensional array.

Casting arrays

The `as` keyword can be used to cast an array type to another compatible array type. For example, `String` is a subtype of `Object`, so an array of type `String` can be cast to an array of type `Object`.

```
var objArray : Object[]
objArray = new String[] { "a", "b" } as Object[] // Can cast String[] to Object[]
```

The reverse operation of casting an `Object` to a `String` is invalid, as demonstrated below.

```
var objArray2 = new Object[1]
objArray2[0] = "aString"
var strArray = objArray2 as String[] // Run-time error attempting to cast Object[] to String[]
```

The compiler can detect and produce a compilation error when two referenced types are incompatible. For example, an `Integer[]` can never be converted to a `String[]`, so the following code statements produce a compilation error.

```
var strArray2 : String[]
strArray2 = new Integer[] { 2, 3 } as String[] // Compilation error attempting to cast Integer[] to String[]
```

In some cases, you know that all the objects in an array are of a particular subtype of the defined type of the array. In other cases, you filter an array to retrieve only objects of a particular subtype. As shown in a preceding code example, you cannot use the `as` keyword to convert an array to an array of a subtype. Instead, you can use the Gosu enhancement method `cast` to make an array of the subtype. You can then perform operations on the new array that

are specific to the subtype and not available to the parent type. If you filter an array by element type and do not use the `cast` method, the result retains the type of the original array.

The following code demonstrates how to use the `cast` method.

```
var objArray3 = new Object[1]
objArray3[0] = "aString"
var strArray3 = objArray3.cast(String) // String array

var objArray4 = new Object[2]
objArray4[0] = "anotherString"
objArray4[1] = 42
var strArray4 = objArray4.where{\aid -> aid typeis String}.cast(String) // String array
var objArray5 = objArray4.where{\aid -> aid typeis String} // Object array
```

You cannot use the `cast` method to convert incompatible types. The compiler does not detect this type of error. The following code produces a run-time error.

```
var strArray5 : String[]
strArray5 = new Integer[] {2, 3}.cast(String) // Run-time error attempting to cast Integer[] to String[]
```

Using Gosu to access Java types

Gosu provides full access to Java types. Gosu is built on top of the Java language and runs within the Java Virtual Machine (JVM). Gosu loads all Java types, so you have full direct access to Java types, such as classes, libraries, and primitive (non-object) types. You can use your favorite Java classes or libraries directly from Gosu with the same syntax as for native Gosu objects.

The most common Java object types retain their fully qualified name but are always in scope, so you do not need to fully qualify their names in typical code. Examples of classes that are always in scope are `Object`, `String`, and `Boolean`. If your code has ambiguity because of similarly named types also in scope, you must fully qualify these types, such as `java.lang.String`. Other Java types are available to Gosu code, but must be fully qualified, for example `java.io.DataInputStream`.

For example, for standard Gosu coding with lists of objects, use the Java type `java.util.ArrayList`. The following simple example uses a Java-like syntax:

```
var list = new ArrayList<String>()
list.add("Hello Java, from Gosu")
```

Gosu includes transformations on Java types that make your Gosu code clearer, such as turning getters and setters into Gosu properties.

Access to Java types from Gosu includes:

- Instantiation of Java classes with the `new` keyword
- Implementing Java interfaces
- Manipulating Java objects as native Gosu objects
- Calling object methods on instantiated objects
- Exposing object methods that look like getters and setters as properties
- Calling static methods on Java types
- Providing built-in extensions and improvements of many common Java types by using Gosu enhancements
- Support for your own extensions of Java types and interfaces
- Support for Java primitive types

See also

- “Packages always in scope” on page 115
- “Overview of enhancements” on page 21
- “Calling Java from Gosu” on page 137

Compatibility with earlier Gosu releases

Gosu provides an unsupported type that exists for compatibility with earlier Gosu releases.

Array

The `Array` type exists for compatibility with earlier Gosu releases. For new code, instead use standard array syntax with bracket notation with a specific type, such as `Integer[]`.

See also

- “Gosu array types” on page 63

Operators and expressions

Gosu provides many operators that are similar to operators in the Java language. Gosu also provides additional operators that are specific to Gosu itself. You use these operators, along with constants and variables, to create Gosu expressions.

Gosu operators

Gosu uses standard programming operators to perform a wide variety of mathematical, logical, and object manipulation operations. If you are familiar with the C, C++ or Java programming languages, you might find that Gosu operators function similar to those other languages. Gosu evaluates operators within an expression or statement in order of precedence.

Gosu operators take either a single operand (unary operators), two operands (binary operators), or three operands (a ternary operator). The following list provides examples of each operator type:

Operator type	Arguments	Examples of this operator type
unary	1	• <code>-3</code> • <code>typeof "Test"</code> • <code>new ArrayList<Address>()</code>
binary	2	• <code>5 - 3</code> • <code>a and b</code> • <code>2 * 6</code>
ternary	3	• <code>3*3 == 9 ? true : false</code>

[See also](#)

- “Operator precedence” on page 71

Operator precedence

The following list orders the Gosu operators from highest to lowest precedence. Gosu evaluates operators with the same precedence from left to right. The use of parentheses can modify the evaluation order as determined by operator precedence. Gosu first evaluates an expression within parentheses, then uses that value in evaluating the remainder of the expression.

Operator	Description
. [] () ?. ?[]	Property access, array indexing, function calls and expression grouping. The operators with the question marks are the null-safe operators.
new	Object creation, object reflection.
,	Array value list, as in { <i>value1</i> , <i>value2</i> , <i>value3</i> } Argument list, as in (<i>parameter1</i> , <i>parameter2</i> , <i>parameter3</i>)
as	Casting a value to a specific type.
+ -	Unary operands for positive value and negative value.
~ ! not typeof eval	Bit-wise OR, logical NOT, type of, eval(<i>expression</i>).
typeof	Determine if an object is a specific type or subtype.
* / %	Multiplication, division, modulo division.
<< >> >>>	Bitwise shifting.
+ - ?+ ?-	Addition, subtraction, string concatenation. The versions with the question marks are the null-safe versions.
< <= > >=	Less than, less than or equal, greater than, greater than or equal
== === != !==	Equality and inequality operators for values and references.
&	Bitwise AND.
^	Bitwise exclusive OR.
	Bitwise inclusive OR.
&& and	Logical AND. The two variants are equivalent.
 or	Logical OR. The two variants are equivalent.
? : ?:	Ternary conditional. For example: <code>3*3 == 9 ? true : false</code> Null-safe default operator

Operator	Description
<code>= += -= *= /= %= &=</code>	Assignment operator statements. These operators apply to Gosu statements, not expressions.
<code>&&= ^= = = <<=</code>	
<code>>>= >>>=</code>	

See also

- “Handling null values in expressions” on page 94
- “Null-safe math operators” on page 97
- “Equality expressions” on page 78
- “Gosu variables” on page 100

Standard Gosu expressions

A Gosu expression results in a single value. A Gosu expression is categorized by the type of operator used in constructing it. Arithmetic expressions use arithmetic operators (+, -, *, /). Logical expressions use logical operators (AND, OR, NOT). Other topics contain descriptions and examples of Gosu-supported expressions and how to use them. Gosu also supports type-cast expressions and type-checking expressions.

Expressions can also be statements. For example, the value of `obj.lookupObj()` can be used as:

- An argument in a method call
- A standalone statement within a statement block

Note: A statement block is also an expression, because its resulting value can be passed as parameter to another method for subsequent execution.

Arithmetic expressions in Gosu

Gosu defines arithmetic expressions corresponding to all the common arithmetic operators, which are:

- Addition and Concatenation Operator (+)
- Subtraction Operator (-)
- Multiplication Operator (*)
- Division Operator (/)
- Arithmetic Modulo Operator (%)

Using big decimal arithmetic in Gosu

Gosu supports Java big decimal arithmetic on the +, -, *, /, and % arithmetic operators. If the left-hand or right-hand operand of the operator is a Java `BigDecimal` or `BigInteger`, the result is `BigDecimal` or `BigInteger` also. This coercion is especially important for accuracy, which currency values often require.

Overflow errors

By default, Gosu arithmetic expressions do not check for overflow errors. Optionally you can enable checked arithmetic, which generates Gosu exceptions for overflow errors.

See also

- “Checked arithmetic for add, subtract, and multiply” on page 76

Arithmetic expression types

Addition and concatenation operator (+)

The + operator performs arithmetic addition or string concatenation using either two numeric data types or two String data types as operands. If you add two numeric types, the result is numeric. If you add two String objects, the result is a String. Note the following:

- If both operands are numeric, the + operator performs addition on numeric types.
- If either operand is a String, Gosu converts the operand that is not a String to a String. The result is the concatenation of the two strings.

Expression	Result
3 + 5	8
8 + 7.583	15.583
"Auto" + "Policy"	"AutoPolicy"
10 + "5"	"105"
"Room " + 1	"Room 1"

IMPORTANT By default, the addition operator does not check for overflow errors. Optionally, you can enable checked arithmetic, which generates Gosu exceptions for overflow errors.

Subtraction operator (-)

The - operator performs arithmetic subtraction, using two numeric values as operands. The result is the same type as the input types.

Expression	Result
9 - 2	7
8 - 3.359	4.641

IMPORTANT By default, the subtraction operator does not check for overflow errors. Optionally, you can enable checked arithmetic, which generates Gosu exceptions for overflow errors.

Multiplication operator (*)

The * operator performs arithmetic multiplication, using two numeric values as operands. The result is the same as the input types.

Expression	Result
2 * 6	12
12 * 3.26	39.12
"9" * "3"	27

IMPORTANT By default, the multiplication operator does not check for overflow errors. Optionally, you can enable checked arithmetic, which generates Gosu exceptions for overflow errors.

Division operator (/)

The / operator performs arithmetic division using two numeric values as operands. The result is the same as the input types. The result of floating-point division follows the specification of IEEE arithmetic.

Expression	Result
10 / 2	5
1 / 0	Infinity
0 / 0	NaN
0/1	0

Arithmetic modulo operator (%)

The % operator performs arithmetic modulo operations, using numeric values as operands. The result is the same type as the input types. The result of a modulo operation is the remainder if the numerator divides by the denominator.

Expression	Result
10 % 3	1
2 % 0.75	0.5

Bitwise AND (&)

The & operator performs a binary bitwise AND operation on the value on the left side of the operator and the value on the right side of the operator.

For example, 10 & 15 evaluates to 10. The decimal number 10 is 1010 binary. The decimal number 15 is 1111 binary. In binary, this expression does a bitwise AND between value 1010 and 1111. The result is binary 1010, which is decimal 10.

In contrast, 10 & 13 evaluates to 8. The decimal number 10 is 1010 binary. The decimal number 13 is 1101 binary. In binary, this expression does a bitwise AND between value 1010 and 1101. The result is binary 1000, which is decimal 8.

Bitwise inclusive OR (|)

The | (pipe character) operator performs a binary bitwise inclusive OR operation with the value on each side of the operator.

For example, 10 | 15 evaluates to 15. The decimal number 10 is 1010 binary. The decimal number 15 is 1111 binary. In binary, this code does a binary bitwise inclusive OR with value 1010 and 1111. The result is binary 1111, which is decimal 15.

The expression 10 | 3 evaluates to 11. The decimal number 10 is 1010 binary. The decimal number 3 is 0011 binary. In binary, this does a bitwise OR between value 1010 and 0011. The result is binary 1011, which is decimal 11.

Bitwise exclusive OR (^)

The ^ (the caret character) operator performs a binary bitwise exclusive OR operation with the values on both sides of the operator.

For example, 10 ^ 15 evaluates to 5. The decimal number 10 is 1010 binary. The decimal number 15 is 1111 binary. In binary, this code does a binary bitwise exclusive OR with value 1010 and 1111. The result is binary 0101, which is decimal 5.

Bitwise left shift (<<)

The << operator performs a binary bitwise left shift with the value on the left side of the operator and value on the right side of the operator.

For example, `10 << 1` evaluates to 20. The decimal number 10 is 01010 binary. In binary, this code does a binary bitwise left shift of 01010 one bit to the left. The result is binary 10100, which is decimal 20.

The expression `10 << 2` evaluates to 40. The decimal number 10 is 001010 binary. In binary, this code does a binary bitwise left shift of 001010 one bit to the left. The result is binary 101000, which is decimal 40.

Bitwise right shift and preserve sign (>>)

The >> operator performs a binary bitwise right shift with the value on the left side of the operator and value on the right side of the operator. For signed values, the >> operator sets the high-order bit with its previous value for each shift. This preserves the sign (positive or negative) of the result. For signed integer values, this is the usually the appropriate behavior. Contrast this behavior with the >>> operator.

For example, `10 >> 1` evaluates to 5. The decimal number 10 is 1010 binary. In binary, this code does a binary bitwise right shift of 1010 one bit to the right. The result is binary 0101, which is decimal 5.

The expression `-10 >> 2` evaluates to -3. The decimal number -10 is 11111111 11111111 11111111 11110110 binary. This expression does a binary bitwise right shift two bits to the right, filling in the top two bits with a 1 because the original number was negative. The result is binary 11111111 11111111 11111111 11111101, which is decimal -3.

Bitwise right shift and clear sign (>>>)

The >>> operator performs a binary bitwise right shift with the values on both sides of the operator. The >>> operator sets the high-order bit for each shift to zero. For unsigned integer values, this behavior is the usually the desirable one. Contrast this behavior with the >> operator.

For example, `10 >>> 1` evaluates to 5. The decimal number 10 is 1010 binary. In binary, this code does a binary bitwise right shift of 1010 one bit to the right. The highest value bit is set to 0. The result is binary 0101, which is decimal 5.

The expression `-10 >>> 2` evaluates to 1,073,741,821. The decimal number -10 is 11111111 11111111 11111111 11110110 binary. This expression does a binary bitwise right shift two bits to the right, filling in the top two bits with a 0. The result is binary 00111111 11111111 11111111 11111101, which is decimal 1,073,741,821.

See also

- “Null-safe math operators” on page 97

Checked arithmetic for add, subtract, and multiply

By default, numeric values could exceed their defined bounds in arithmetic operations. For example, if you multiplied the maximum integer value by 2, the result by definition exceeds the range of integer values. Not only is the result value incorrect, the result could be positive when you expect it to be negative, or negative when you expect it to be positive. Because no Gosu exceptions occur, writing code to protect against overflow errors, which could cause unexpected behavior or security issues, is difficult.

Gosu includes an optional feature called checked arithmetic. If *checked arithmetic* is enabled, Gosu behavior for the standard arithmetic operators changes for addition, subtraction, and multiplication. In nearly all cases, the result is the same and the behavior is the same. In the rare case that arithmetic overflow occurs, Gosu throws the exception `ArithmeticException`.

To enable checked arithmetic, set Java system property `checkedArithmetic` to `true` when launching Studio and when launching the application server.

Gosu checked arithmetic includes protection only for the operators `+`, `-`, and `*`. There is no protection for division, which only affects the expression `Integer.MIN_VALUE / -1`.

In some cases, arithmetic overflow behaviors are desirable for operators `+`, `-`, and `*`. For example, some common hash algorithms rely on arithmetic overflow. To handle typical use cases in overridden `hashCode` methods, Gosu always compiles `hashCode` methods with checked arithmetic disabled.

For other cases in which arithmetic overflow behaviors are desirable, you can use three Gosu operators that ensure unchecked arithmetic independent of the Java system property `checkedArithmetic`. The new operators are the standard arithmetic operators prefaced with an exclamation point character: `!+`, `!-`, and `!*`.

For example, with checked arithmetic enabled:

```
var four = new Integer(4)
var y = Integer.MAX_VALUE * 2 + four // This line throws ArithmeticException
```

In contrast, the following example uses the unchecked arithmetic operator `!*`, which is only for special circumstances in which overflow is desirable:

```
var four = new Integer(4)
var x = Integer.MAX_VALUE !* 2 + four // This line does not throw ArithmeticException
print(x) // Print "2"
```

Because the arithmetic in the second example is unchecked, the result that prints 2 successfully is possibly unexpected and invalid.

See also

- *Configuration Guide*

Numeric type conversions with arithmetic operations

For arithmetic operations in which the operands are `Short` or `Byte` boxed types, the result is stored as in `Integer`:

```
var shortVal1:Short = 12
var byteVal1:Byte   = 14
var shortVal2:Short = 23
var byteVal2:Byte   = 31

var shortSum = shortVal1 + shortVal2
var byteSum  = byteVal1  + byteVal2
var comboSum = shortVal1 + byteVal1
```

The following code tests the resultant types:

```
print(typeof shortSum)
print(typeof byteSum)
print(typeof comboSum)
```

Output:

```
int
int
int
```

Similarly, for arithmetic operations in which the operands are `short` or `byte` primitive types, the result is stored as in `int`:

```
var shortVal1:short = 12
var byteVal1:byte   = 14
var shortVal2:short = 23
var byteVal2:byte   = 31

var shortSum = shortVal1 + shortVal2 // Result is an int
var byteSum  = byteVal1  + byteVal2  // Result is an int
var comboSum = shortVal1 + byteVal1   // Result is an int
```

For arithmetic operations in which one operand is a primitive type and the other is a boxed version of the same type, the result is stored as the boxed type. For example, the expression `int <operand> Integer` returns an `Integer`, where `<operand>` is a `+`, `-`, `*`, `/`, or `%` arithmetic operator. In the following example, operands of type `int` and `Integer` are added:

```
var val1:int = 12
var val2:Integer = 14
var sumVal1 = val1 + val2
var sumVal2 = val2 + val1
```

The following code tests the resultant types:

```
print(typeof sumVal1)
print(typeof sumVal2)
```

Output:

```
class java.lang.Integer
class java.lang.Integer
```

For arithmetic operations in which one operand is of type `float`, `Float`, `double`, or `Double` and the other operand is of type `BigInteger`, the result is stored as a `BigDecimal`:

```
var val1:float      = 12.8
var val2:double     = 14.3
var val3:Float      = 3.4
var val4:Double     = 52.6
var val5:BigInteger = 31
var sumVal1 = val1 + val5
var sumVal2 = val2 + val5
var sumVal3 = val3 + val5
var sumVal4 = val4 + val5
```

The following code tests the resultant types:

```
print(typeof sumVal1)
print(typeof sumVal2)
print(typeof sumVal3)
print(typeof sumVal4)
```

Output:

```
class java.math.BigDecimal
class java.math.BigDecimal
class java.math.BigDecimal
class java.math.BigDecimal
```

Note: If one of the operands in an arithmetic operation is `null`, a `java.lang.NullPointerException` is thrown:

```
var val1:Integer = null
var val2 = 12
var total = val1 + val2 // Throws a null pointer exception
```

Equality expressions

Equality expressions return a `Boolean` value (`true` or `false`) indicating the result of the comparison between the two expressions. Equality expressions consist of the following types:

- Relational equality operator (`==`)
- Object equality operator (`===`)
- Inequality operator (`!=`)
- Object inequality operator (`!==`)

Relational equality operator (`==`)

The `==` operator tests for relational equality. The operands can be of any compatible types. The result is always `Boolean`. For object types, the following rules apply in the following order:

- If both objects implement the `Comparable` interface, the `==` operator calls the `Comparable` interface method `compareTo`. Gosu calls the `compareTo` method on the object to the left of the `==` operator.
- Otherwise, the `==` operator calls the native Java method `object.equals()` to compare values. Gosu calls the `equals` method on the object to the left of the `==` operator.

IMPORTANT To determine whether two objects are the same in-memory object, instead use the `===` or `!==` operators.

In the Java language, the `==` operator evaluates to `true` if and only if both operands have exactly the same reference value, which refers to the same object in memory. This behavior works well for primitive types like integers. For reference types, you usually do not want to compare the object in memory. Instead, to compare value equality, Java code typically uses `object.equals()`, not the `==` operator. The `object.equals()` method includes the comparison of object reference values, but also compares property values.

If you use the `==` operator for comparison of reference types, Gosu calls `object.equals()`. In most cases, this behavior is what you want for reference types.

Syntax

```
a == b
```

Examples

Expression	Result
<code>7 == 7</code>	<code>true</code>
<code>"3" == 3</code>	<code>true</code>
<code>3 == 5</code>	<code>false</code>

Object equality operator (===)

You compare identity references, rather than the object values, to determine whether two objects reference the same in-memory object. You use the Gosu operator `===` (three equal signs) to compare object equality.

The operands can be of any compatible types. The result type is always `Boolean`.

Examples comparing `==` and `===`

Expression	Output	Description
<code>print("3" == "3")</code>	<code>true</code>	The two <code>String</code> objects contain the same value.
<code>print("3" == new String("3"))</code>	<code>true</code>	The two <code>String</code> objects contain the same value.
<code>print("3" == "4")</code>	<code>false</code>	The two <code>String</code> objects contain different values.
<code>print("3" === "4")</code>	<code>false</code>	Gosu represents the two <code>String</code> literals as separate objects in memory (as well as separate values).
<code>var x = 1 + 2</code> <code>var s = x as String</code> <code>print(s == "3")</code>	<code>true</code>	These two variables reference the same value but different objects, so the double-equals operator returns <code>true</code> .
<code>var x = 1 + 2</code> <code>var s = x as String</code> <code>print(s === "3")</code>	<code>false</code>	These two variables reference the same value but different objects, so the triple-equals operator returns <code>false</code> .
<code>print("3" === "3")</code>	<code>true</code>	This example is harder to understand. By just looking at the code, it seems like these two <code>String</code> objects would be different objects. However, the Gosu compiler detects that the objects are the same <code>String</code> . Gosu optimizes the code to point to the same <code>String</code> object in memory for both usages of the <code>String</code> literal <code>"3"</code> .
<code>print("3" === new String("3"))</code>	<code>false</code>	The two <code>String</code> objects both contain the value <code>"3"</code> , but are not the same object.

Inequality operator (!=)

The != operator tests for relational inequality. The operands can be of any compatible types. The result type is always Boolean.

For object types, the following rules apply in the following order:

- If both objects implement the Comparable interface, the != operator calls the Comparable interface method compareTo to compare values. Gosu calls the compareTo method on the object to the left of the == operator.
- Otherwise, the != operator calls the native Java method *object.equals()* to compare values. Gosu calls the equals method on the object to the left of the == operator.

IMPORTANT To compare whether two objects are the same in-memory object, instead use the === or !== operators.

Syntax

```
a != b
```

Examples

Expression	Result
7 != 7	false
"3" != 3	false
3 != 5	true

Object inequality operator (!==)

The !== operator tests for object inequality. It returns the opposite value of the === operator, which tests for object equality not value equality.

The operands can be of any compatible types. The result type is always Boolean.

Syntax

```
a !== b
```

Examples comparing != and !==

Expression	Output	Description
print("3" != "4")	true	The two String objects contain different values.
print("3" !== "4")	true	Gosu represents the two String literals as separate objects in memory (as well as separate values).
var x = 1 + 2 var s = x as String print(s != "3")	false	These two variables reference the same value but different objects. If you use the == operator, it returns true.
var x = 1 + 2 var s = x as String print(s !== "3")	true	These two variables reference the same value but different objects. If you use the === operator, it returns false.
print("3" != "3")	false	The two String objects contain the same value. Compare to the examples in the following rows.
print("3" != new String("3"))	false	The two String objects contain the same value.

Expression	Output	Description
<code>print("3" != "3")</code>	false	This example is harder to understand. By just looking at the code, it seems like these two <code>String</code> objects would be different objects. However, in this case, the Gosu compiler detects they are the same <code>String</code> at compile time. Gosu optimizes the code for both usages of a <code>String</code> literal to point to the same object in memory for both usages of the "3".
<code>print("3" != new String("3"))</code>	true	The two <code>String</code> objects both contain the value "3", but are not the same object.

Evaluation expressions

The `eval()` expression evaluates Gosu source at run time, which enables dynamic execution of Gosu source code. Gosu executes the source code within the same scope as the call to `eval()`.

Syntax

```
eval(Expression)
```

Examples

Expression	Result
<code>eval("2 + 2")</code>	4
<code>eval(3 > 4 ? true : false)</code>	false

Logical expressions

Gosu logical expressions use standard logical operators to evaluate the expression in terms of the `boolean` values of `true` and `false`.

Gosu supports the following logical expressions:

- Logical AND
- Logical OR
- Logical NOT

Gosu evaluates logical expressions from left to right and uses the following rules to test the expressions for possible short-circuit evaluation:

- `true OR any-expression` always evaluates to `true` – Gosu only runs and evaluates *any-expression* if the expression before the `OR` is false. If Gosu determines that the expression before the `OR` evaluates to `true`, the second expression is not evaluated.
- `false AND any-expression` always evaluates to `false` – Gosu only runs and evaluates *any-expression* if the expression before the `AND` is true. If Gosu determines that the expression before the `AND` evaluates to `false`, the second expression is not evaluated.

Logical AND

Gosu uses either `and` or `&&` to indicate a logical AND expression. The operands must be of the `Boolean` data type or any type convertible to `Boolean`. The result is always `Boolean`.

Syntax

```
a and b
a && b
```

Examples

Expression	Result
(4 > 3) and (3 > 2) (true/true) = true	
(4 > 3) && (2 > 3) (true/false) = false	
(3 > 4) and (3 > 2) (false/true) = false	
(3 > 4) && (2 > 3) (false/false) = false	

Logical OR

Gosu uses either `or` or `||` to indicate a logical OR expression. The operands must be of the Boolean data type or any type convertible to Boolean. The result is always Boolean.

Syntax

```
a or b
a || b
```

Examples

Expression	Result
(4 > 3) or (3 > 2) (true/true) = true	
(4 > 3) (2 > 3) (true/false) = true	
(3 > 4) or (3 > 2) (false/true) = true	
(3 > 4) (2 > 3) (false/false) = false	

Logical NOT

To indicate a logical negation (a logical NOT expression), use either the keyword `not` or the exclamation point character (`!`). The operand must be of the Boolean data type or any type convertible to Boolean. The result is always Boolean.

Syntax

```
not a
!a
```

Examples

Expression	Result
!true	false
not false	true
!null	true
not 1000	false

The following examples illustrate how to use the NOT operator.

Bad example

The following example demonstrates how not to use the logical NOT operator.

```
if (not liabilityCov.Limit == line.BOPLiabilityCov.Limit) {
    return true
}
```

This example causes an error at run time because Gosu associates the NOT operator with the variable to its right before evaluating the expression. This association causes the expression to become:

```
(false == line.BOPLiabilityCov.Limit)
```

This comparison causes a class-cast exception, as follows:

```
'boolean (false)' is not compatible with Limit
```

Better example

The following example demonstrates how to use the NOT operator correctly.

```
if (not (liabilityCov.Limit == line.BOPLiabilityCov.Limit)) {
    return true
}
```

In this example, the extra parentheses force the desired comparison, and associate the NOT operator with the correct expression.

Preferred example

Use the `!=` operator rather than NOT for writing code of this type.

```
if (liabilityCov.Limit != line.BOPLiabilityCov.Limit) {
    return true
}
```

The expression had no need to use the NOT operator. The final code expression is somewhat simpler and does exactly what is required.

typeis expressions

Gosu uses the operator `typeis` to test the type of an object against a named type. The result is `true` if the object has that type or a subtype.

See also

- “Using Boolean values in Gosu” on page 53
- “Testing the type by using `typeis`” on page 350

Using the operator `new` in object expressions

Gosu uses the `new` operator to create an instance of a type. The type can be a Gosu class, a Java class, a Guidewire entity type, or an array. Creating a new instance is also called *instantiation*. Instantiation allocates memory to a new object of that type and initializes the object. After you instantiate an object, you can call object methods or access instance fields and properties.

At least one constructor (creation function) must be exposed on a type to construct an instance of the type with the `new` operator. You can have multiple constructors with different types and numbers of arguments.

Although creating a new object is typically used as an expression, it can also optionally be a statement if the return value is not needed.

Syntax for typical cases

```
new javaType (argument_list)      // The optional argument list contains constructor arguments.
new gosuType (argument_list)      // The optional argument list contains constructor arguments.
new arrayType [size]
new arrayType [] {array_Value_List} // This syntax allows declaring a list of initial array members.
```

If you pass arguments to the `new` operator, Gosu passes those arguments to the constructor. There might be multiple constructors defined, in which case Gosu uses the types and numbers of objects passed as arguments to determine which constructor to call.

Examples

Expression	Result
<code>new java.util.HashMap(8)</code>	Creates an instance of the HashMap Java class.
<code>new String[12]</code>	Creates a String array that has 12 members with no initial values.
<code>new String[] { "a", "b", "c" }</code>	Creates a String array that has three members, initialized to "a", "b", and "c".

See also

- “Using the operator `new` as a statement” on page 100

Special behaviors for entity types during instantiation

There are special behaviors for instantiating Guidewire entity types, which are Gosu types that you define by editing data model XML configuration files.

Required Fields During Instantiation

In data model configuration XML files, you can declare required properties for an entity type during object instantiation. To specify a required entity property is required, set the attribute called `required` to the value `true`. The default is `false`. The required attribute is supported on the `<column>`, `<typekey>`, and `<foreignkey>` definitions of an entity type.

The `required` attribute does not overlap with the behavior of the attribute called `nullok`. The `nullok` attribute affects only the value of that property at commit time, which is the time the object is written to the database. In contrast, the `required` attribute affects only object instantiation. If the required property is an object type, the value must be non-null at the time of object instantiation to prevent Gosu throwing an exception.

Code that uses the `new` operator to instantiate an entity type must include all required properties when constructing an instance of that entity as arguments to the constructor. If the constructor has other constructor arguments, the required properties appear after those other arguments.

For example, suppose an entity called `MyEntity` has one required parameter that is has type `Address`. The expression `new MyEntity()` is a compilation error because the required property is missing from the argument list. To fix the error, get a reference to a non-null `Address` entity instance and pass it to the constructor, such as:

```
var e = new MyEntity(myAddress)
```

Optional bundle argument during instantiation

By default, Gosu creates the new entity instance in the current database transaction. Gosu represents the database transaction as a set of entity instance changes encapsulated in a Gosu object called a *bundle*. In nearly all PolicyCenter contexts, this is the best behavior. For example, PolicyCenter correctly rolls back all changes to the database in that transaction if any errors occur before the entire set of changes commit to the database.

Providing a specific bundle during entity instantiation is appropriate only in rare cases in application logic. For example, there are rare cases where there is no current database transaction or an action must be handled asynchronously in a different thread. For these rare cases, pass the bundle as the final argument to the `new` operator expression.

The following table lists the behavior of the `new` operator with different parameters:

Extra database transaction parameter to new operator	Meaning	Example
No extra parameter	Create the entity in the current database transaction, which is the current bundle. In almost all cases, use this approach.	<code>new Note()</code>

Extra database transaction parameter to new operator	Meaning	Example
Reference to a bundle	Create the entity instance in the current database transaction indicated by the bundle passed directly to the new operator. The bundle must be writable, for example it cannot be a read-only bundle associated with a database query result.	<code>new Note(myPolicy.Bundle)</code>
Reference to a Guidewire entity	Create the entity instance in the same database transaction as the entity passed as a parameter. The bundle must be writable, for example it cannot be a read-only bundle associated with a database query result.	<code>new Note(myPolicy)</code>

WARNING Be extremely careful if you use bundle and transaction APIs to override the default behavior for bundle management.

Interaction between bundle parameter and required fields

If an entity type has required fields and you need the optional instantiation parameter for the bundle, the constructor argument list must include the bundle parameter before any required properties.

See also

- “Bundles and database transactions” on page 333

Optional omission of type name with the new keyword

If the type of the object is determined from the programming context, you can omit the type name entirely in the object creation expression with the new keyword.

WARNING Omitting the type name with the new keyword is strongly discouraged in typical code. Omit the type name only for XML manipulation and dense hierarchical structures with long type names. Some types imported from XSDs have complex and hard-to-read type names. The following simple examples demonstrate the concepts, but do not promote usage of this syntax in simple cases.

For example, first declare a variable with an explicit type. Next, assign that variable a new object of that type in a simple assignment statement that omits the type name:

```
// Declare a variable explicitly with a type.
var s : String

// Create a new empty string.
s = new()
```

You can also omit the type name if the context is a method argument type:

```
class SimpleObj {
}

class Test {
    function doAction (arg1 : SimpleObj) {
    }
}

var t = new Test()

// The type of the argument in the doAction method is predetermined,
// and therefore you can omit the type name if you create a new instance as a method argument.
t.doAction(new())
```

The following example uses both local variables and class variables:

```
class Person {
    private var _name : String as Name
```

```

    private var _age : int as Age
}

class Tutoring {
    private var _teacher : Person as Teacher
    private var _student : Person as Student
}

// Declare a variable as a specific type to omit the type name in the "new" expression
// during assignment to that variable.
var p : Person
var t : Tutoring
p = new()    // type name omitted
t = new()    // type name omitted

// If a class var or other data property has a declared type, optionally omit the type name.
t.Teacher = new()
t.Student = new()

```

See also

- “Object initializer syntax” on page 86

Object initializer syntax

Object initializers support setting properties on newly created objects immediately after new expressions. Use object initializers for compact and clear object declarations. Object initializers are especially useful in combination with data structure syntax and nested objects.

Special syntax for initializing lists, collections, and maps

Gosu uses specialized initializer syntax and rules for creating new lists, collections, and maps.

See also

- “Basic lists” on page 189
- “Basic hash maps” on page 191

Simple object initializers

A simple object initializer looks like the following:

```
var sampleClaim = new Claim() {:ClaimId = "TestID"}
```

Object initializers comprise one or more property initializer expressions, separated by commas, and enclosed by braces. A property initializer expression is the following, in order: a colon (:), a property name, an equal sign (=), and a value or any expression that results in a value.

```
:propertyName = value
```

For example, suppose you have the following code, which sets properties on a new file container by using assignment statements that follow the new statement:

```

var myFileContainer = new my.company.FileContainer() // Create a new file container.

myFileContainer.DestFile = jarFile                // Set the properties on the new file container.
myFileContainer.BaseDir = dir
myFileContainer.Update = true
myFileContainer.WhenManifestOnly = ScriptEnvironment.WHEN_EMPTY_SKIP

```

The following sample code is functionally equivalent to the preceding code but uses an object initializer to make the assignments within the bounds of in the new statement:

```

var myFileContainer = new my.company.FileContainer() { // Create a new file container,
    :DestFile = jarFile, :BaseDir = dir, :Update = true, // and set its properties at creation time.
    :WhenManifestOnly = ScriptEnvironment.WHEN_EMPTY_SKIP
}

```

Nested object initializers

You can use object initializers to create and initialize properties on nested objects within new statements. If an object property in a property initializer expression is itself an object, the value you assign can be a new object with its own object initializer.

For example, suppose you have the following code:

```
var testSet = new TestSet()           // Create a test set.
testSet.name = "Root"                // Set the name of the test set.

var test = new Test()                 // Create and initialize a test.
test.name = "test1"

testSet.tests.add(test)               // Add the new test to the array of the test set.
testSet.tests.add(new Test())         // Create another test and add it to the array.
testSet.tests.get(1).final = true     // Set the final property on the second test in the array.
testSet.tests.get(1).type = new TestType() // Create a test type and assign the type property
                                      // on the second test in the array to that type.

var testStyle = new TestStyle()       // Create and initialize a test style.
testStyle.color = Red;                // Gosu infers which enum class is appropriate.
testStyle.number = 5

testSet.style = testStyle              // Set the style property of the test set.

return testSet.toXML()                // Convert the test set to XML.
```

You can rewrite the preceding code by using nested new object expressions with their own object initializers to reflect visually the nested object structure that the code creates.

```
var testSet = new TestSet() {         // Create a test set.
    :name = "Root",                  // Set the name of the test set.
    :tests = {
        new Test() { :name = "test1", // Create and initialize a test and add it to the array.
        new Test() {                 // Create another test and add it to the array
            :final = true,            // Set the final property to true on the second test.
            :type = new TestType()    // Create a test type and set the type property
        }                             // on the second test in the array to that type.
    },
    :style =
        new TestStyle() {            // Create and initialize a test style and set the
            :color = Red,              // style of the test set to that style.
            :number = 5
        }
}
```

Nested object initializers are especially useful when constructing in-memory XML data structures.

Relational expressions

Gosu relational operators support all types of objects that implements the `java.lang.Comparable` interface, not just numbers. Relational expressions return a `Boolean` value (`true` or `false`) indicating the result of a comparison between two expressions. Relational expressions use the following operators:

- `>` operator
- `>=` operator
- `<` operator
- `<=` operator

Gosu supports the use of multiple relational operators to compare multiple values. Add parentheses around each comparison expression. For example, the following expression ultimately evaluates to `true`:

```
((1 <= 2) <= (3 > 4)) >= (5 > 6)
```

The first compound expression evaluates to `false` (`(1 <= 2) <= (3 > 4)`) as does the second expression (`5 > 6`). However, the larger expression tests for greater than or equal. Therefore, because `false` is equal to `false`, the entire expression evaluates to `true`.

> operator

The > operator tests two expressions and returns true if the left value is greater than the right value. The operands can be numeric, String, or java.util.Date data types. The result is Boolean.

Syntax

```
expression1 > expression2
```

Examples

Expression	Result
8 > 8	false
"zoo" > "apple"	true
5 > "6"	false
currentDate > policyEffectiveDate	true

>= operator

The >= operator tests two expressions and returns true if the left value is greater than or equal to the right value. The operands can be numeric, String, or java.util.Date data types. The result is Boolean.

Syntax

```
expression1 >= expression2
```

Examples

Expression	Result
8 >= 8	true
"zoo" >= "zoo"	true
5 >= "6"	false
currentDate >= policyEffectiveDate	true

< operator

The < operator tests two expressions and returns true if the left value is less than the right value. The operands can be numeric, String, or java.util.Date data types. The result is Boolean.

Syntax

```
expression1 < expression2
```

Examples

Expression	Result
8 < 5	false
"zoo" < "zoo"	false
5 < "6"	true

Expression	Result
<code>currentDate < policyEffectiveDate</code>	false

<= operator

The <= operator tests two expressions and returns true if the left value is less than or equal to the right value. The operands can be numeric, String, or java.util.Date data types. The result is Boolean.

Syntax

```
expression1 <= expression2
```

Examples

Expression	Result
<code>8 <= 5</code>	false
<code>"zoo" <= "zoo"</code>	true
<code>5 <= "6"</code>	true
<code>currentDate <= policyEffectiveDate</code>	false

Unary expressions

Gosu supports the following unary (single operand) expressions:

- Numeric negation
- Bit-wise NOT
- Typeof expressions

The following sections describe these expressions. The value of a typeof expression cannot be fully determined at compile time. For example, an expression at compile time might resolve as a supertype. At run time, the expression may evaluate to a more specific subtype.

Numeric negation

Gosu uses the unary - operator before a number to indicate numeric negation. The operand must be a number data type. The result is always the same type as the original number.

Syntax

```
-value
```

Examples

Expression	Result
<code>-42</code>	-42
<code>-(3.14 - 2)</code>	-1.14

Bit-wise NOT

The bit-wise NOT operator treats a numeric value as a series of binary digits (bits) and inverts them. This behavior is different from the logical NOT operator (!), which treats the entire numeral as a single Boolean value. In the following example, the logical NOT operator assigns a Boolean value of true to x if y is false, or false if y is true:

```
x = !y
```

In the following example, the bit-wise NOT operator (~) treats a numerical value as a set of bits and inverts each bit, including the sign operator.

```
z = ~7
```

The decimal number 7 is the binary value 0111 with a positive sign bit. By using the bit-wise NOT, the expression ~7 evaluates to the decimal value -8. The binary value 0111 reverses to 1000, the binary value for 8, and the sign bit changes as well to produce -8.

Use the bit-wise NOT operation to manipulate a bit mask. A *bit mask* is a number or byte field that maintains the state of many items by flags mapping to each bit in the field.

Typeof expressions

Gosu uses the operator `typeof` to determine meta information about the type to which an expression evaluates. The operand can be any valid data type. The result is the type of the expression.

See also

- “Basic type checking” on page 349

Conditional ternary expressions

A conditional ternary expression uses the Boolean value of one expression to decide which of two other expressions to evaluate. A question mark (?) separates the conditional expression from the alternatives, and a colon (:) separates the alternatives from each other. In some programming languages, ternary expressions are known as using the *conditional operator* or *ternary operator*, which has three operands instead of two.

If your test is for a null value, you can use the null-safe default operator to produce even more concise code.

Syntax

```
conditionalExpression ? trueExpression : falseExpression
```

The second and third operands that follow the question mark (?) must be of compatible types.

At run time, Gosu evaluates the first operand, the conditional expression. If the conditional expression evaluates to true, Gosu evaluates the second operand, the expression that follows the question mark. Gosu ignores the third operand, the expression that follows the colon. Conversely, if the conditional expression evaluates to false, Gosu ignores the second operand, the true expression, and evaluates the third operand, the false expression.

Examples

Ternary expression	Evaluation result at run time
<code>x > y ? x : y</code>	The smaller of the two values, x and y
<code>print(myNumberVar > 10 ? "Bigger than 10" : "10 or less")</code>	• If the value of <code>myNumberVar</code> is greater than 10, Gosu prints <code>Bigger than 10</code>. • If the value of <code>myNumberVar</code> is 10 or less, Gosu prints <code>10 or less</code>.

Ternary expression types at run time and compile time

At run time, the type of a ternary expression is the type of the true expression or the false expression, depending on the result of evaluating the conditional expression. The true and false expressions often, but not always, evaluate to the same type.

IMPORTANT If the true and false expression types evaluate to different runtime types, you must ensure that your code that uses the result checks the type before performing type-specific actions.

At compile time, if the true and false expressions are of different types, Gosu reconciles the type of the ternary expression to the type of their nearest common ancestor. Gosu requires that the types of the true and false expressions in a ternary expression be compatible so that Gosu can reconcile their types. If the expressions have no common ancestor, the type at compile time of the ternary expression is `Object`.

For example, consider the following ternary expression.

```
var ternaryResult = aContact.Status == "new" ? "hello" : false
```

In the example, the true expression is of type `String` and the false expression is of type `Boolean`. If at run time the contact is new, `ternaryResult` is of type `String`, and its value is `hello`. If the contact is not new, `ternaryResult` is of type `Boolean`, and its value is `false`. Although the true expression and the false expression are of different types, their types are compatible because `String` and `Boolean` both descend from `Object`. The following example explicitly sets the type to `Object`.

```
// The type of a ternary expression is the common ancestor type of its true and false expressions.  
var ternaryResult : Object = aContact.Status == "new" ? "hello" : false
```

At run time, the type evaluation of the ternary expression and the `ternaryResult` variable depends on the current state of the system. Different type checking keywords produce different results. For example, if the contact in the following example is new, `ternaryResult` is of type `String`.

```
var ternaryResult = aContact.Status == "new" ? "hello" : false  
  
print(ternaryResult typeis Object)  
print(ternaryResult typeis String)  
print(ternaryResult typeis Boolean)  
print(typeof ternaryResult)
```

The preceding code produces the following output.

```
true  
true  
false  
java.lang.String
```

If the contact in the example is not new, `ternaryResult` is of type `Boolean`. The preceding code produces the following output.

```
true  
false  
true  
java.lang.Boolean
```

Ternary expression type of a null value

If the result of a ternary expression is null, the runtime type of the result is `void`.

The following example demonstrates this behavior:

```
var testNotNull = false ? null : 7  
print ("Result is ${testNotNull} ... type = ${typeof testNotNull}")  
  
var testNull = true ? null : 7  
print ("Result is ${testNull} ... type = ${typeof testNull}")
```

The output is the following lines:

```
Result is 7 ... type = java.lang.Integer  
Result is null ... type = void
```

Primitive type coercion and ternary expressions

If the true or false expression in a ternary expression is of a primitive type, such as `int` or `boolean`, Gosu first coerces the primitive type to its boxed version. For example, Gosu coerces the primitive type `boolean` to its boxed type `Boolean`. Then, Gosu searches the type hierarchy for a common ancestor type of both the true and false expressions.

The following examples use a primitive `float` type and a boxed `Float` type to demonstrate how a ternary expression evaluates these types:

```
var prim = 7 as float
var boxed : Float = prim

var r = false ? null : prim
print ("Result is ${r} ... type = ${typeof r}")

var r2 = false ? null : boxed
print ("Result is ${r2} ... type = ${typeof r2}")
```

Running the code shows that the type is `java.lang.Float` for the primitive and boxed expressions:

```
Result is 7 ... type = java.lang.Float
Result is 7 ... type = java.lang.Float
```

Nested use of ternary expressions

Gosu supports the nesting of ternary expressions. The second and third operands, the true and false expressions, can themselves be ternary expressions. The ternary operator is syntactically right-associative.

For example, the nested ternary expression `a ? b : c ? d : e ? f : g` evaluates with the explicit order of precedence `a ? b : (c ? d : (e ? f : g))`. At run time, the ternary expression reduces to one of the expressions `b`, `d`, `f`, or `g`.

See also

- “Logical expressions” on page 81
- “Null-safe default operator” on page 96

Special Gosu expressions

The Gosu syntax supports the following types of specialized Gosu expressions:

- Functions
- Static methods
- Static property paths
- Entity literals
- Typekey literals

Function calls

This expression calls a function with an optional list of arguments and returns the result.

Syntax

```
functionName(argumentList)
```

Examples

Expression	Result
<code>now()</code>	Current Date
<code>concat("limited-", "coverage")</code>	"limited-coverage"

Static method calls

This expression calls a static method on a type with an optional list of arguments and returns the result.

Syntax

```
typeExpression.staticMethodName(argumentList)
```

Examples

Expression	Result
<code>java.lang.System.currentTimeMillis()</code>	Current time
<code>java.util.Calendar.getInstance()</code>	Java Calendar

See also

- “Modifiers” on page 157

Static property paths

Gosu uses the dot-separated path rooted at a Type expression to retrieve the value of a static property.

Syntax

```
TypeExpression.StaticProperty
```

Examples

Expression	Result
<code>Claim.TypeInfo</code>	Claim typeInfo
<code>LossCause.TC_HAIL</code>	"hail" typekey
<code>java.util.Calendar.FRIDAY</code>	Friday value

See also

- “Modifiers” on page 157

Entity and typekey type literals

Gosu supports referencing an entity or typekey type by relative name, or by fully qualified name:

- A fully qualified entity name begins with “entity.”.
- A fully qualified typekey begins with “typekey.”.

Syntax

```
[entity.]typeName
[typekey.]typeName
```

Examples

Expression	Result
Claim	Claim type
entity.Claim	Claim type
LossCause	LossCause type
typekey.LossCause	LossCause type
java.lang.String	String type
int	int type

Handling null values in expressions

Many operations in Gosu are null-safe. These operations do not throw a null pointer exception (NPE) at run time if they encounter a null value. Gosu provides a set of null-safe operators to protect other operations from these exceptions.

Null-safe property access

A property path expression in Gosu is a series of property accesses in series, for example `x.P1.P2.P3`. There are two different operators you can use in Gosu to get property values:

- The standard period operator (`.`), which can access properties or invoke methods. The standard period operator is only null-safe for properties, and not for method invocations.
- The null-safe period operator (`?.`), which can access properties or invoke methods in a null-safe way. For properties, this is the same as the standard period operator. For methods, you must use this operator if you want a null-safe method invocation.

See also

- “Property assignment triggering instantiation of intermediate objects” on page 51
- “Null-safe method access” on page 96

How the standard period operator handles null

The standard period (`.`) operator has a special behavior that some languages do not have. If any object property in a property path expression evaluates to null, in Gosu the entire path evaluates to null. A null value at any location in an object path short-circuits evaluation and results in a null value, with no exception being thrown. This feature is called null-safe short circuiting for a property path expression.

For example, suppose that you have an expression similar to the following:

```
var groupType = claim.AssignedGroup.GroupType
```

Remember that if any element in the path evaluates to null, the entire expression evaluates to null. If `claim` is null, the result is null. Also, if `claim.AssignedGroup` is null, the result is null.

If the expression contains a method call, the rules are different. The period operator does not default to null if the left side of the period is null. With the standard period operator, if the value on the left of the period is null, Gosu throws a null pointer exception (`NullPointerException`). In other words, method calls are not null-safe because the method could have side effects.

Note: Technically speaking, property access from a dynamic property `get` function could generate side effects, but this is strongly discouraged. If a property accessor has any side effects, convert the accessor into a method.

Example 1

Suppose that you have an expression similar to the following:

```
claim.AssignedGroup.addEvent("abc")
```

In this case, if either `claim` or `claim.AssignedGroup` evaluate to `null`, Gosu throws a `NullPointerException`. The method call follows the `null` value.

Example 2

Suppose that you have an expression similar to the following:

```
claimant.getSpecialContactRelationship().Contact.Name
```

If `Contact` is `null`, the expression evaluates to `null`. Similarly, if `getSpecialContactRelationship()` evaluates to `null`, the expression evaluates to `null`.

However, remember that evaluation in the expression is left-to-right. If `claimant` is `null`, the expression throws an exception because Gosu cannot call a method on `null`.

Example 3

For those cases in which Gosu expects a Boolean value (for example, in an `if` statement), a `null` value coerces to `false` in Gosu. This is true regardless of whether the expression's value was short-circuited. For example, the following `if` statement prints "Benefits decision not made yet", even if `claim` or `claim.BenefitsStatusDcsn` is `null`:

```
if( not claim.BenefitsStatusDcsn ) {  
    print( "Benefits decision not made yet" )  
}
```

Primitives and null-safe paths

Gosu null-safety for property paths does not work if the type of the entire expression is a Gosu primitive type. This is equivalent to saying it does not work if the type of the last item in the series of property accesses has a primitive type. For example, suppose you use the property path:

```
a.P1.P2.P3
```

By using null-safe paths, Gosu returns `null` if any of the following are `null`:

- `a`
- `a.P1`
- `a.P1.P2`

However, if the type of the `P3` property is `int` or `char` or another primitive type, then the expression `a.P1.P2.P3` is not null safe.

Primitive types (in contrast to object types, which are descendents of `Object`) can never contain the value `null`. Thus, Gosu cannot return `null` from that expression, and any casting from `null` to the primitive type would be meaningless. Therefore, Gosu throws a null pointer exception in those conditions.

WARNING In Studio, Gosu code in the Gosu Scratchpad has different compiler behavior than any other part of Studio. If you run code in Gosu Scratchpad using the **Run**  or **Debug**  buttons, then the code compiles and runs locally in Studio with different behavior. Most notably, the behavior of null safety for properties in the period operator is different. In nearly all other contexts in Studio, the period operator is null safe for properties. However, in Gosu Scratchpad in Studio, Gosu is not null-safe. Code that might not throw an exception in a Gosu class might throw an exception in Gosu Scratchpad. If Studio is connected to a server running with a DCEVM and you click **Run in Debug Process** , then Studio sends the Gosu code to the server. The PolicyCenter server uses the standard null-safe behavior for the period operator.

See also

- “Property assignment triggering instantiation of intermediate objects” on page 51

How the null-safe period operator handles null

In contrast to the standard period character operator, the null-safe period operator `?.` always returns `null` if the left side of the operator is `null`. This works both for accessing properties and for invoking methods. If the left side of the operator is `null`, Gosu does not evaluate the right side of the expression.

The null-safe operator is particularly important for invoking methods because for methods, the standard period operator throws an exception if the left side of the period is `null`.

For property access in Gosu, the standard period and the null-safe period operator are equivalent. The period operator is already null-safe.

See also

- “Property assignment triggering instantiation of intermediate objects” on page 51

Null-safe method access

By default, method calls are not null-safe. If the right side of a period character is a method call, Gosu throws a null-pointer exception (NPE) if the value on the left side of the period is `null`.

A null-safe method call does not throw an exception if the left side of the period character evaluates to `null`. Gosu just returns `null` from that expression. Using the `?.` operator calls the method with null safety. This operator behaves in the same way as the null-safe period operator for properties.

See also

- “How the standard period operator handles null” on page 94

Null-safe default operator

Sometimes you need to return a different value based on whether an expression evaluates to `null`. The Gosu operator `?:` results in the value of the expression to the left of the operator if that value is non-null, avoiding evaluation of expression to the right. If the value of the expression to the left of the operator is `null`, Gosu evaluates the expression to the right and returns that result.

For example, suppose there is a variable `str` of type `String`. At run time, the value contains either a `String` or `null`. Perhaps you want to pass the input to a display routine. However, if the value of `str` is `null`, you want to use a default value rather than `null`. Use the `?:` operator as follows:

```
var result = str ?: "(empty)" // Return str, but if the value is null return a default string
```

You can chain the null-safe default operator to get the first non-null value in a set of items. If every value in the chain is `null`, the result of the chain is `null`. For example, the following code gets the first day of the week that has an assigned task or returns `null` if no day has a task:


```
var firstTask = dayTask[0] ?: dayTask[1] ?: dayTask[2] ?: dayTask[3] ?: dayTask[4]
```

Null-safe indexing for arrays, lists, and maps

For objects such as arrays and lists, you can access items by index number, such as `myArray[2]`. Similarly, with maps (java.util.Map objects), you can pass the key value to obtain the value. For example with a `Map<String, Integer>`, you could use the expression `myMap["myvalue"]`. The challenge with indexes is that if the object at run time has the value `null`, code like this throws a null pointer exception.

Gosu provides an alternative version of the indexing operator that is null-safe. Instead of typing just the indexing subexpression, such as `[2]`, after an object, prefix the expression with a question mark character. For example:

```
var v = myArray?[2]
```

If the value to the left of the question mark is `null`, the entire expression for the operator returns `null`. If the left-hand-operand is not `null`, Gosu evaluates the index subexpression and indexes the array, list, or map. Finally, Gosu returns the result, just as for the typical use of the brackets for indexing lists, arrays, and maps.

Null-safe indexing does not protect against array-index out-of-bounds exceptions. If the array is non-`null`, but the index is greater than or equal to the size of the array, Gosu throws an out-of-bounds exception at run time. Any value of the index causes this exception if the array is empty rather than `null`. To test for an empty array, use the `HasElements` property.

Null-safe math operators

Gosu provides null-safe versions of common math operators.

For example, the standard operators for addition, subtraction, multiplication, division, and modulo are as follows: `+`, `-`, `*`, `/`, and `%`. If you use these standard operators and either side of the operator is `null`, Gosu throws a `NullPointerException` exception.

In contrast, the null-safe operators are the same symbols but with a question mark (?) character preceding it. In other words, the null-safe operators are: `?+`, `?-`, `?*`, `?/`, and `?%`.

Handling null values in logical expressions

If your data can provide `null` values, you need to be aware of how Gosu handles these values in logical expressions. Using the null-safe operators can propagate `null` values from their original sources into other objects and variables in your code.

- In an equality test, Gosu returns `true` if both sides of the operator are `null` and `false` otherwise.
- In a Boolean expression that uses the `and`, `or`, or `not` operators, Gosu treats `null` as a value of `false`.

Be particularly aware of the possibility of `null` values in code that uses the `and`, `or`, or `not` operators. If you use null-safe method operators in logical expressions, your code can produce unexpected results:

```
var nullStr : String // A string that contains the null value

// Test whether the string is empty
if (nullStr?.isEmpty()) {
    print("Empty string") // This message does not appear even though there is nothing in the string.
    // Code here to put a value into an empty string does not execute for the null string.
}
// Test whether the string is not empty
if (!nullStr?.isEmpty()) {
    print("Got here!") // This message does appear, but the string has no contents.
    // If you put code here to use the string value, you risk a null-pointer exception.
}
```

See also

- *Integration Guide*

Best practices for using null-safe operators

Be aware of the consequences of using the null-safe operators. If the null-safe expression returns a value of `null`, your subsequent code must be able to handle that value. If the argument on the left of a null-safe operator is `null`, the expression returns a value of `null`, or its equivalent for a primitive type. For example, using a null-safe operator in an expression that returns an `int` value produces a value of `0` if the argument on the left of the operator is `null`.

Do not use null-safe operators merely to avoid any null-pointer exceptions. Using these operators is a signal to other code developers that the code path can accept and return `null` values. Use the null-safe operators only if your code does not need to perform additional actions for an object path expression that evaluates to `null`.

Do not use null-safe operators if you know that an object is never going to be `null`. For example, if you create a new object, subsequent code has no need to use null-safe operators on that object.

```
uses gw.api.util.DateUtil

var startTime = DateUtil.currentDate()
// No need to use null-safe operators to call methods on startTime
var terminationTime = startTime.addHours(2) // CORRECT
var halfTime = startTime?.addHours(1) // DO NOT USE THIS FORM. startTime is never null.
```

Similarly, if a function in a class uses the `this` name to reference the current object, do not use the null-safe operators, such as `this?.methodName()`. The `this` object never has a value of `null`.

List and array expansion expressions (*.)

Gosu includes a special operator for array expansion and list expansion. The expansion operator is an asterisk followed by a period. For example, the following line of code gets the length property of every item in an array called `names`:

```
names*.length
```

See also

- “List expansion (*.)” on page 194

Gosu building blocks

The following types of Gosu building blocks are necessary in writing more complex Gosu code to perform operations required by your business logic:

- Gosu statements
- Gosu variables
- Gosu conditional execution and looping
- Gosu functions

Gosu statements

A Gosu expression has a value, but Gosu statements do not. The result of a Gosu expression can be passed as an argument to a function. A Gosu statement does not have a result that can be passed as an argument to a function.

For example, the following lines are all Gosu expressions as each results in a value:

```
5 * 6
typeof 42
exists (var e in Claim.Exposures where e == null)
```

The following lines are all Gosu statements:

```
print(x * 3 + 5)
for (i in 10) { ... }
if (a == b) { ... }
```

Note: Do not confuse statement lists with expressions or Gosu blocks. *Blocks* are anonymous functions that Gosu can pass as objects, even as function arguments.

[See also](#)

- “Blocks” on page 181

Statement lists

A statement list is a list containing zero or more Gosu statements enclosed by braces ({}).

The Gosu standard is always to omit semicolon characters at the ends of lines of Gosu code. Code is more readable without optional semicolons. In the rare cases in which you type multiple statement lists on one line, such as within block definitions, do use semicolons to separate statements.

Syntax

```
{ statement-list }
```

Multi-line example (no semicolons)

```
{  
  var x = 0  
  var y = myfunction( x )  
  
  print( y )  
}
```

Single-line example (semicolons)

```
var adder = \ x : int, y : int -> { print("I added!"); return x + y; }
```

See also

- “Coding style” on page 219

Using the operator new as a statement

Use the new operator to instantiate an object. Although it is often used as an expression, new can also be a statement. For some types, this functionality may not be useful. However, if the constructor for the object triggers code that saves a copy of the new object, the return value from new may be unnecessary. Ignoring the return value and using new as a statement may permit more concise code in these cases.

For example, suppose that a constructor for a class that represents a book registers itself with a bookshelf object and saves the new object. Some code might merely create the book object and pass the bookshelf as a constructor argument:

```
new gw.example.Book( bookshelfReference, author, bookID )
```

See also

- “Using the operator new in object expressions” on page 83

Gosu variables

A Gosu variable is a storage container for a certain type of data. As you create and assign a Gosu variable, you need to consider the type of the variable as well as its value.

Strong typing of variables

If you specify a type for a variable in its declaration in Gosu code, Gosu thereafter considers the variable to be strongly typed. Any attempt to assign an incompatible value to the variable generates a type-mismatch error. Similarly, if you set a value for a variable in the variable declaration, but do not specify the variable type, Gosu strongly types the variable to the type of the provided initial value.

The only way to declare a variable without a strong type is to initialize it with a null value without a specifying a type. However, even if you do not assign a type to a variable in its declaration, the variable takes on the type of the first value assigned to it.

Character set available for variable names

Gosu uses the same rules for variable names as Java and supports a standard ASCII subset for those names. The first character can be:

- A letter
- \$
- _

Subsequent characters can be:

- A letter
- A digit
- \$
- _

Gosu also supports Unicode characters above the extended ASCII range. For example:

- `var русский_язык = 1.0`
- `var Español = 2.0`
- `var 日本語_$ = 3.0`
- `print(русскийязык * Español * 日本語$)`

Variable type declaration

Use the following syntax to declare a Gosu variable.

Syntax

```
var identifier [ : type-literal ] = expression
var identifier : type-literal [ = expression ]
```

Examples

```
var age = 42
var age2 : int
var age3 : int = 42
var c : Claim
```

Variable assignment

Gosu uses the standard programming assignment operator = to assign the value on the right side of the statement to the item on the left side of the statement.

Syntax

```
variable = expression
```

Examples

```
count = 0
time = DateUtil.currentDate()
```

Compound assignment operators

Gosu supports compound assignment operators that perform an action and assign a value in a single statement. The following table lists each compound operator and describes its behavior. The examples assume that the variable `i` is previously declared as `type int`.

Operator	Description	Examples
=	Simple assignment of the value on the right side of the operator to the variable on the left side.	<pre>i = 10</pre> Assigns the value 10.
+=	Adds the value of the variable on the left side of the operator to the value on the right side. Next, Gosu assigns this result to the variable.	<pre>i = 10 i += 3</pre> Assigns the value 13.
-=	Subtracts the value of the variable on the left side of the operator from the value on the right side. Next, Gosu assigns this result to the variable.	<pre>i = 10 i -= 3</pre> Assigns the value 7.
*=	Multiplies the value of the variable on the left side of the operator by the value on the right side. Next, Gosu assigns this result to the variable.	<pre>i = 10 i *= 3</pre> Assigns the value 30.
/=	Divides the value of the variable on the left side of the operator by the value on the right side. Next, Gosu assigns this result to the variable.	<pre>i = 10 i /= 3</pre> Assigns the value 3. For the <code>int</code> type, there is no fraction. If you used a floating-pointing type, the value would be 3.333333.
%=	Divides the value of the variable on the left side of the operator by the value on the right side, and returns the remainder. Next, Gosu assigns this result to the variable.	<pre>var i = 10 i %= 3</pre> Assigns the value 1. This value is 10 (3.3333 as <code>int</code>)*3
=	Performs a logical OR operation on the value of the variable on the left side of the operator and the value on the right side. Next, Gosu assigns this result to the variable. Gosu accepts the primitive type <code>boolean</code> or the object type <code>Boolean</code> on either side of the operator	<pre>var a = false var b = true a = b</pre> Assigns the value <code>true</code> .
&&=	Performs a logical AND operation on the value of the variable on the left side of the operator and the value on the right side. Next, Gosu assigns this result to the variable. Gosu accepts the primitive type <code>boolean</code> or the object type <code>Boolean</code> on either side of the operator	<pre>var a = false var b = true a &&= b</pre> Assigns the value <code>false</code> .
&=	Performs a bitwise AND operation on the value of the variable on the left side of the operator and the value on the right side. Next, Gosu assigns this result to the variable.	<pre>i = 10 i &= 15</pre> Assigns the value 10. The decimal number 10 is 1010 binary. The decimal number 15 is 1111 binary. This code does a bitwise AND between value 1010 and 1111. The result is binary 1010, which is decimal 10. Contrast with this example: <pre>i = 10 i &= 13</pre> Assigns the value 8. The decimal number 10 is 1010 binary. The decimal number 13 is 1101 binary. This does a bitwise AND between value 1010 and 1101. The result is binary 1000, which is decimal 8.
^=	Performs a bitwise exclusive OR operation on the value of the variable and value on the right side of the operator. Next, Gosu assigns this result to the variable on the left side.	<pre>i = 10 i ^= 15</pre> Assigns the value 5.

Operator	Description	Examples
		The decimal number 10 is 1010 binary. The decimal number 15 is 1111 binary. This code does a bitwise exclusive OR with value 1010 and 1111. The result is binary 0101, which is decimal 5. Contrast with this example: <pre>i = 10 i ^= 13</pre> Assigns the value 7. The decimal number 10 is 1010 binary. The decimal number 13 is 1101 binary. This does a bitwise AND between value 1010 and 1101. The result is binary 0111, which is decimal 7.
=	Performs a bitwise inclusive OR operation on the value of the variable on the left side of the operator and the value on the right side. Next, Gosu assigns this result to the variable.	<pre>i = 10 i = 15</pre> Assigns the value 15. The decimal number 10 is 1010 binary. The decimal number 15 is 1111 binary. This code does a bitwise inclusive OR with value 1010 and 1111. The result is binary 1111, which is decimal 15. Contrast with this example: <pre>i = 10 i = 3</pre> Assigns the value 11. The decimal number 10 is 1010 binary. The decimal number 13 is 1101 binary. This does a bitwise AND between value 1010 and 1101. The result is binary 0111, which is decimal 11.
<<=	Performs a bitwise left shift on the value of the variable on the left side of the operator. The number of bit shifts is the value on the right side of the operator. Next, Gosu assigns this result to the variable.	<pre>i = 10 i <<= 1</pre> Assigns the value 20. The decimal number 10 is 01010 binary. This code does a bitwise left shift of 01010 one bit to the left. The result is binary 10100, which is decimal 20. Contrast with this example: <pre>i = 10 i <<= 2</pre> Assigns the value 40. The decimal number 10 is 001010 binary. This code does a bitwise left shift of 001010 one bit to the left. The result is binary 101000, which is decimal 40.
>>=	Performs a bitwise right shift on the value of the variable on the left side of the operator. The number of bit shifts is the value on the right side of the operator. Next, Gosu assigns this result to the variable.	<pre>i = 10 i >>= 1</pre> Assigns the value 5. The decimal number 10 is 1010 binary. This code does a bitwise right shift of 1010 one bit to the right. The result is binary 0101, which is decimal 5. Contrast with this example: <pre>i = -10 i >>= 2</pre> Assigns the value -3.
	IMPORTANT For signed values, this operator sets the high-order bit with its previous value for each shift. This behavior preserves the sign (positive or negative) of the result. For signed integer values, this behavior is usually appropriate. Contrast this operator with the >>>= operator.	

Operator	Description	Examples
		The decimal number -10 is 11111111 11111111 11111111 11110110 binary. This code does a bitwise right shift two bits to the right, filling in the top sign bit with the 1 because the original number was negative. The result is binary 11111111 11111111 11111111 11111101, which is decimal -3.
>>>=	Performs a bitwise right shift on the value of the variable on the left side of the operator. The number of bit shifts is the value on the right side of the operator. Next, Gosu assigns this result to the variable on the left side. IMPORTANT This operator sets the high-order bit for each shift to zero. For unsigned integer values, this behavior is usually appropriate. Contrast this operator with the >>= operator.	<pre>i = 10 i >>>= 1</pre> Assigns the value 5. The decimal number 10 is 1010 binary. This code does a bitwise right shift of 1010 one bit to the right. The result is binary 0101, which is decimal 5. Contrast with this example: <pre>i = -10 i >>>= 2</pre> Assigns the value 1073741821. The negative decimal number -10 is 11111111 11111111 11111111 11110110 binary. This code does a bitwise right shift two bits to the right, with no filling of the top bit. The result is binary 00111111 11111111 11111111 11111101, which is decimal 1073741821. The original was a negative number, but in this operator that bit value is filled with zeros for each shift.
++ unary operator	Adds one to the value of a variable. This operator is also known as the increment-by-one operator. The unary ++ operator must always appear after the variable name.	<pre>i = 10 i++</pre> Assigns the value 11.
-- unary operator	Subtracts one from the current value of a variable. This operator is also known as the decrement-by-one operator. The unary -- operator must always appear after the variable name.	<pre>i = 10 i--</pre> Assigns the value 9.

Comparison of compound assignments and expressions

The table above lists a variety of compound assignment operators, such as ++, --, and +=.

These operators form statements rather than expressions.

The following Gosu is valid:

```
while (i < 10) {
  i++
  print(i)
}
```

The following Gosu is invalid because statements are not permissible in an expression, which Gosu requires in a while statement:

```
while (i++ < 10) { // Compilation error!
  print(i)
}
```

Gosu supports the increment and decrement operator only after a variable, not before a variable. The operation `i++` is valid but `++i` is invalid. The `++i` form exists in other languages to support expressions in which the result is an expression that you pass to another statement or expression. In Gosu, these operators do not form an expression. Thus, you cannot use increment or decrement in while declarations, if declarations, and for declarations.

Gosu conditional execution and looping

Gosu uses the multiple constructions to perform program flow, including the following:

- `if...else` statements
- `for` statements
- `while` statements
- `do...while` statements
- `switch` statements

Each of these statement types provides a different type of program control flow. However, all work by first determining whether an expression is true or false, then using that information to determine subsequent program action.

if - else statements

The most commonly used statement block within the Gosu language is the `if` block. The `if` block uses a multi-part construction. The `else` block is optional.

Syntax

```
if (<expression>) <statement>
[ else <statement> ]
```

Example

```
if (a == b) { print("a equals b") }

if (a == b || b == c) { print("a equals b or b equals c") }
else { print("a does not equal b and b does not equal c") }

if (a == b) { print("a equals b") }
else if (a == c) { print("a equals c") }
else { print("a does not equal b, nor does it equal c") }
```

To improve the readability of your Gosu code, Gosu automatically downcasts after a `typeis` expression if the type is a subtype of the original type. This is particularly useful for `if` statements and similar Gosu structures. Within the Gosu code bounded by the `if` statement, you do not need to do casting (as *TYPE* expressions) to that subtype. Because Gosu confirms that the object has the more specific subtype, Gosu implicitly considers that variable's type to be the subtype, at least within that block of code.

See also

- “Basic type checking” on page 349

for statements

The `for` statement block uses a multipart construction.

Syntax

```
for ( [var] identifier in expression [ index index-identifier ] [ iterator iterator-identifier ] )
{ statements }
```

The scope of *identifier* is limited to the statement block itself. The keyword `var` before that local variable identifier is optional.

The *expression* expression in the `in` clause must evaluate to one of the following:

An array

An iterator

An object that implements the Java `Iterable` interface. Iteration starts with the initial member and continues sequentially until terminating at the last member.

A Java list or collection class, such as `java.util.ArrayList`

Iteration starts with the initial member and continues sequentially until terminating at the last member.

A String object

Gosu treats the string as a list of characters.

A Gosu interval

The iterator part is valid only for an expression that is iterable.

The number of times that Gosu executes the statements in the body of the loop block depends on the value of the expression and on the start and end values of the iterator:

- If the expression evaluates at run time to `null`, the body of the loop block is not executed.
- If the start and end values of the iterator are identical, the `for` loop is executed once.
- The loop is executed the same number of times if the iterator's start and end values are reversed. If the start value is greater than the end value, the looping variable subtracts from the start value until the end value is reached.

Gosu provides backwards compatibility for the use of an older Gosu style `foreach` statement. Better style is to use the `for` statement instead.

See also

- “Intervals” on page 129
- “Array list access with array index notation” on page 65

Iteration in for statements

There are several ways to iterate through members of a list or array using the `for` statement.

Use automatic iteration to iterate through a collection, such as a number sequence, an array, or a list. Iteration starts with the initial value or member, and continues sequentially until terminating at the last value or member.

Collection iteration with a variable

The standard form of iteration is to use an iteration variable. Use the following syntax:

```
for (member in OBJ) { ... }
```

The following lines show examples of iterating over a collection by using a variable:

```
for (prop in myListOfProperties)
for (name in {"John", "Mary", "David"})
for (i in 0..100) // i iterates through the values 0 to 99
```

Iteration with no variable for numerical intervals only

If the object to loop across is a numerical iterator type, the variable declaration is optional. For example:

```
for (1..10) {
    print("hello!")
}
```

Iteration with an index

Use index iteration if you need to determine the exact position of a particular element within an array or list. This technique adds an explicit variable to contain the index value. This can be useful to read members of the array or list in a non-sequential fashion using array notation. Specify iteration with an index variable with the following syntax:

```
for (member in OBJ index Loopcount)
```

Example:

```
// This code prints the index of the highest score in an array of test scores.
// This particular example prints "3".

var testScores = new int[] {91, 75, 97, 100, 89, 99}
print(getIndexOfHighestScore(testScores))

function getIndexOfHighestScore(scores : int[]) : int {
    var highIndex = 0

    for (score in scores index i) {
        if (score > scores[highIndex]) { highIndex = i }
    }

    return highIndex
}
```

The result of running this code is the following line:

```
3
```

Iteration with an iterator

Use this type of iteration if you need to access the functionality of the iterator that processes the items in an array or collection. The iterator is only available if the expression that the `for` loop evaluates is an iterable type. For example, the iterator is not available if the expression is a `String`, as shown in the following line:

```
for (letter in "abcde")
```

You can use the `hasNext` method on the iterator to determine whether the `for` loop is processing the last item.

Most collections do not support modification during iteration and throw a `ConcurrentModificationException` on attempted modifications. Gosu does support object removal by using the `remove` method on the iterator loop variable, as shown in this example:

```
var strs = {"a", "b", "", "c"}
for (c in myList iterator iter) {
    if (c.length() == 0) {
        iter.remove()
    } else {
        ...
    }
}
for (c in strs) {
    print(c)
}
```

The output from this code is the following:

```
a
b
c
```

while statements

Gosu evaluates the `while()` expression, which must evaluate to `true` or `false`, and uses the Boolean result to determine the next course of action:

- If the expression is initially `true`, Gosu executes the statements in the statement block repeatedly until the expression becomes `false`. At this point, Gosu exits the `while` statement and continues statement execution at the next statement after the `while` statement.
- If the expression is initially `false`, Gosu does not execute any of the statements in the statement block, and continues statement execution at the next statement after the `while` statement.

Syntax

```
while (expression) {
    statements
}
```

If the `while()` expression tests whether an iterator has more items available, the statements in the statement block can process the items that the iterator provides. The `Iterator.next` method returns an item of type `Object`. You must downcast that item to the expected type before processing it. Because downcasting could fail at run time if the actual type is not compatible with the expected type, you must consider the risks of that technique. In the following example, the object retrieved from the `iterator.next` method is cast to the type of object required, which is `Payment` in this case.

```
var iterator = this.getPaymentsIterator(true)
while (iterator.hasNext()) {
    var payment = (iterator.next() as Payment)
    // Your code here
}
```

do...while statements

The `do...while` block is similar to the `while` block in that it evaluates an expression and uses the `Boolean` result to determine the next course of action. The principal difference is that Gosu tests the expression for validity after executing the statement block, instead of prior to executing the statement block. The statements in the statement block execute at least once when execution first accesses the block.

- If the expression is initially `true`, Gosu executes the statements in the statement block repeatedly until the expression becomes `false`. At this point, Gosu exits the `do...while` block and continues statement execution at the next statement after the `do...while` statement.
- If the expression is initially `false`, Gosu executes the statements in the statement block once, then evaluates the condition. If nothing in the statement block has changed so that the expression still evaluates to `false`, Gosu continues statement execution at the next statement after the `do...while` block. If action in the statement block causes the expression to evaluate to `true`, Gosu executes the statement block repeatedly until the expression becomes `false`, as in the previous case.

Syntax

```
do {
    <statements>
} while (<expression>)
```

Example

```
// Print the digits
var i = 0

do {
    print(i)
    i = i + 1
} while (i < 10)
```

Examples of Gosu conditional iteration and looping

The following examples illustrate different techniques for iterating through the members of an array or list.

Example: Printing all letters in a string with their index

```
for (letter in gw.api.util.StringUtil.splitWhitespace("a b c d e") index i) {
    print("Letter " + i + ": " + letter)
}
```

The output of this code is the following lines:

```
Letter 0: a
Letter 1: b
Letter 2: c
Letter 3: d
Letter 4: e
```

Example: Printing all Policy properties by using reflection

```
for (prop in PolicyGosu.Type.TypeInfo.Properties) {
    print(prop)
}
```

The output of this code is the following lines:

```
Class
Bundle
OriginalVersion
IntrinsicType
Changed
ChangedFields
DisplayName
ChangedProperties
NewlyImported
ID
PublicID
...
```

Example: Ensuring that array elements have a minimum value

```
var colors = {"red", "orange", "yellow", "green", "blue", "indigo", "violet"}
var blueshift = new String()

var r : int = (Math.random() * 6) as int
blueshift = colors[r]

// Shift the color at least one step to the blue end of the spectrum
do {
    r += 1
    blueshift = colors[r]
} while (r < 3) // Minimum color is "green"

print(blueshift)
```

Running this code multiple times produces lines similar to the following:

```
blue
green
green
green
green
blue
violet
green
blue
indigo
```

switch statements

Gosu evaluates the `switch` expression, and uses the result to choose one course of action from a set of multiple choices. Gosu evaluates the expression, and then iterates through the case expressions in order until it finds a match.

- If a case value equals the expression, Gosu executes its accompanying statement list. Statement execution continues until Gosu encounters a `break` statement, or the `switch` statement ends. Gosu continues to the next case and executes multiple case sections if you omit the `break` statement.
- If no case value equals the expression, Gosu skips to the default case, if one exists. The default case is a case section with the label `default:` rather than `case VALUE:`. The default case must be the last case in the list of sections.

The `switch` statement block uses a multi-part construction. The `default` statement is optional. In most cases, you implement a default case to handle any unexpected conditions.

Syntax

```
switch (<expression>) {
  case label1 :
    [statementlist1]
    [break]
  [ ...
  [ case labelN :
    [statementlistN]
    [break] ] ]
  [ default :
    [statementlistDefault]]
}
```

Example

```
switch (strDigitName) {
  case "one":
    strOrdinalName = "first"
    break
  case "two":
    strOrdinalName = "second"
    break
  case "three":
    strOrdinalName = "third"
    break
  case "five":
    strOrdinalName = "fifth"
    break
  case "eight":
    strOrdinalName = "eighth"
    break
  case "nine":
    strOrdinalName = "ninth"
    break
  default:
    strOrdinalName = strDigitName + "th"
}
```

To improve the readability of your Gosu code, Gosu automatically downcasts the object after a `typeis` expression if the type is a subtype of the original type. This is particularly valuable for `if` statements and similar Gosu structures such as `switch`. Within the Gosu code bounded by the `if` or `switch` statement, you do not need to do casting (as `TYPE` expressions) to that subtype for that case. Because Gosu confirms that the object has the more specific subtype, Gosu implicitly considers that variable's type to be the subtype for that block of code. There are several special cases that turn off the downcasting.

See also

- “Basic type checking” on page 349

Gosu functions

Functions encapsulate a series of Gosu statements to perform an action and optionally return a value. Generally speaking, functions exist attached to a type. For example, declaring functions within a class. As in other object-oriented languages, functions declared on a type are also called *methods*.

In the context of a Gosu program (a `.gsp` file), you can declare functions at the top level, without attaching them explicitly to a class. You can then call this function from other places in that Gosu program.

Note: The built-in `print` function is special because it is always in scope, and is not attached to a type. It is the only true global function in Gosu.

Gosu does not support functions defined within other functions. However, you can use the Gosu feature called blocks to do something similar.

Unlike Java, Gosu does not support variable argument functions (so-called `vararg` functions), meaning that Gosu does not support arguments with “...” arguments.

Gosu permits you to specify only type literals for a function’s return type. Gosu does not support other expressions that might evaluate (indirectly) to a type.

Gosu requires that you provide the return type in the function definition, unless the return type is `void` (no return value). If the return type `void`, omit the type and the colon before it. Also, any `return` statement must return a type that matches the declared function return type. A missing return type or a mismatched return value generates a compiler error.

Syntax

```
[modifiers] function IDENTIFIER( argument-declaration-list ) [:type-literal] {
    function-body
}
```

Examples

```
function square( n : int ) : int {
    return n * n
}

// Compile error "Cannot return a value from a void function."
private function myfunction() {
    return "test for null value"
}

function fibonacci( n : int ) : int {
    if (n == 0) { return 0 }
    else if (n == 1) { return 1 }
    else {return fibonacci( n - 1 ) + fibonacci( n - 2 ) }
}

function concat ( str1:String, str2:String ) : String {
    return str1 + str2
}
```

If the return type is not `void`, all possible code paths must return a value in a method that declares a return type. In other words, if any code path contains a `return` statement, Gosu requires a `return` statement for all possible paths through the function. The set of all paths includes all outcomes of conditional execution, such as `if` and `switch` statements.

For example, the following method is invalid:

```
// Invalid...
class MyClass {
    function myfunction(myParameter) : boolean {
        if (myParameter == 1) {
            return true
        }
        if (myParameter == 2) {
            return false
        }
    }
}
```

```
}
}
```

Gosu generates a “Missing Return Statement” error for this function and you must fix this error. The Gosu compiler sees two separate `if` expressions for a total of four total code paths. Even if you believe the function is always used with `myParameter` set to value 1 or 2 but no other value, you must fix the error. To fix the error, rewrite the code so that all code paths contain a `return` statement.

For example, you can fix the earlier example using an `else` clause:

```
class MyClass {
    function myfunction(myParameter) : boolean {
        if (myParameter == 1) {
            return true
        } else {
            return false
        }
    }
}
```

Similarly, if you use a `switch` statement, consider using an `else` section.

This strict requirement for `return` statements mirrors the analogous requirements in the Java language.

[See also](#)

- “What are blocks?” on page 181
- “Modifiers” on page 157

Named arguments and argument defaults

Gosu provides some optional features that make function calls very readable. These optional features include the following:

- Named arguments in function calls
- Default argument values in function declarations

It is also possible to call a function with multiple arguments and explicitly name only some of the arguments.

Calling named arguments

In your code that calls functions, you can specify argument names explicitly rather than relying on matching the declaration order of the arguments. This helps make your code more readable. For example, typical method calls might look like the following:

```
someMethod(true, false)
```

If you pass series of one or more comma-separated arguments, it is difficult or impossible to tell what each value represents. It can be difficult to visually confirm the argument order in code that was written long ago or written by others.

Alternatively, for each argument you can explicitly specify the argument name. Instead of just the argument value, pass the following items in this order:

1. A colon character
2. The argument name
3. The equal sign
4. The value

For example:

```
someMethod(:redisplay=true, :sendUpdate=false)
```

Defining argument defaults

You can optionally provide default argument values in function declarations. By providing a default argument value, you permit a function caller optionally to omit that argument. To declare a default, follow the argument name with

an equal sign and then the value. If the function caller passes the argument, the passed-in value overrides any declared default value.

For example, consider the following function that prints `String` values with a prefix:

```
class MyClass {
    var _names : java.util.ArrayList<String>

    construct(strings : java.util.ArrayList<String>) {
        _strings = strings
    }

    function printWithPrefix(prefix : String = " ---> ") {
        for (n in _strings) {
            print(prefix + n) // used a passed-in argument, or use the default " ---> " if omitted
        }
    }
}
```

In the `printWithPrefix` declaration, the `prefix` value has the default value `" ---> "`. To use the default values, call the `printWithPrefix` method and omit the optional arguments.

The following example calls the `printWithPrefix` method using the default value:

```
var c = new MyClass({"hello", "there"})

// Because the argument has a default, it is optional -- you can omit it
c.printWithPrefix()
```

The following example calls the `printWithPrefix` method providing an explicit value:

```
var c = new MyClass({"hello", "there"})

// Specify the parameter explicitly (ignores default) as traditional unnamed argument
c.printWithPrefix(" next string is:")

// Specify the parameter explicitly (ignores default) as named argument
c.printWithPrefix(:prefix= " next string is:")
```

The Gosu named arguments feature requires that the method name is not already overloaded on the class.

Interaction of named arguments and argument defaults

When you call a function with a multiple arguments, you can explicitly name some of the arguments and not others. The following are the rules for the calling convention of functions using the named argument convention and argument defaults:

- The named arguments can appear in any order.
- For any non-named arguments in the function call, the order must match in left-to-right order any arguments without defaults. Gosu considers any additional passed-in non-named arguments as representing the arguments with defaults, passed in the same order (left-to-right) as they are declared in the function.

If you use a named parameter in a function call, all following parameters must be named parameters.

Public and private functions

A function is public by default, meaning that it can be called from any Gosu code. In contrast, a private function can be called only within the library in which it is defined. For example, suppose you have the following two functions defined in a library:

```
public function funcA() {
    ...
}

private function funcB() {
    ...
}
```

Because `funcA()` is defined as public, it can be called from any other Gosu expression. However, `funcB()` is private, and therefore is not valid anywhere except within the library.

For example, a function in another library could call `funcA()`, but it could not call the private `funcB()`. Because `funcA()` is defined in the same library as `funcB()`, however, `funcA()` can call `funcB()`.

Do not make any function public without good reason. Therefore, mark a function as private if it is defined only for use inside the library.

See also

- “Modifiers” on page 157

Importing types and package namespaces

To use types and namespaces in Gosu scripts without fully qualifying the full class name including the package, use a Gosu `uses` statement. The `uses` statement behaves similarly to the Java language’s `import` statement. To indicate that you are importing static members of a class, you use a `#` character and the members to import after the type name.

Always put `uses` statements at the beginning of the file or script. Do not add these lines within a Gosu method declaration.

Syntax

After the `uses` statement, specify a package namespace or a specific type such as a fully qualified class name.

```
uses type
uses namespace
uses type#static member
uses type#all static members
```

Namespaces can be specified with an asterisk (*) character to indicate a hierarchy. For example:

```
uses toplevelpackage.subpackage.*
```

Example 1

The following code uses a fully qualified type name.

```
var myInputStream = new java.io.FileInputStream()
```

Instead, you can use the following code that declares an explicit type with the `uses` operator.

```
// This "uses" expression provides access to the java.io.FileInputStream class
uses java.io.FileInputStream

// Use this simpler expression without specifying the full package name:
var myInputStream = new FileInputStream()
```

Example 2

The following code uses a fully qualified type name.

```
var myInputStream = new java.io.FileInputStream()
```

Instead, you can use the following code that declares a package hierarchy with the `uses` operator.

```
// This "uses" expression provides access to all the classes in the java.io package hierarchy
uses java.io.*

// Use this simpler expression without specifying the full package name:
var myInputStream = new FileInputStream()
```

Example 3

The following code uses a qualified type name. Because classes in the `java.lang` package are always in scope, you do not need to specify `java.lang`.

```
var circleRadius = 2.0
var circleCircumference = circleRadius * circleRadius * Math.PI
```

Instead, you can use the following code that declares a single static field from the `java.lang.Math` class.

```
// This "uses" expression provides access to the PI constant in the java.lang.Math class
uses java.lang.Math#PI

var circleRadius = 2.0
// Use this simpler expression without specifying the class name:
var circleCircumference = circleRadius * circleRadius * PI
```

Example 4

The following code uses a qualified type name. Because classes in the `java.lang` package are always in scope, you do not need to specify `java.lang`.

```
var mySquareArea = 1000
var mySideLength = Math.sqrt(mySquareArea)
```

You can use any of the angle constants in your code.

```
// This "uses" expression provides access to all the static members in the java.lang.Math class
uses java.lang.Math##*

var mySquareArea = 1000
// Use this simpler expression without specifying the class name:
var mySideLength = sqrt(mySquareArea)
```

Packages always in scope

Some built-in packages are always in scope. You do not need to use fully qualified type names or the `uses` operator for types in these packages, which include the following:

- `gw.lang.*`
- `java.lang.*`
- `java.util.*`
- `dynamic.*`
- The default, empty package
- `entity.*`
- `typekey.*`
- `pcf.*` – only for expressions inside PCF files.
- `productmodel.*`

List

The Gosu type system performs special treatment for the type `List`. Gosu resolves `List` to `java.util.List` in general use but to `java.util.ArrayList` if code creates an untyped list. For example, the following code creates an `ArrayList`:

```
var x = {}
```

Importing static members

The Gosu `uses` statement supports importing static features from a type so you can use them later in your code without verbose syntax. Static features are members of a type that are defined with the `static` keyword, and so appear on the type itself rather than an instance of the type. The Gosu static imports feature is similar to the Java equivalent, but with more concise syntax that does not require the `static` keyword during import.

Static features that you can import include:

- Static properties
- Static fields
- Static methods

Gosu static imports are particularly useful to support the Gosu binding expressions feature.

To import static features, at the top of the file, write a `uses` line that specifies a type, followed by the `#` character, then the static member name. For the static member name, list the property name, the field name, or a method signature with type names only (omit the argument names). The use of the `#` character in this case reflects the Gosu feature syntax for a property or method on a type. This syntax is sometimes called *feature literal* syntax.

To import all static features for a type, use the `*` character instead of feature literal syntax for a specific member.

The following example imports one method:

```
uses org.junit.Assert#assertEquals(String, Object, Object)
```

The following example imports all static features from a class

```
uses gw.util.money.IMoneyConstants#* // Imports all static features from IMoneyConstants
```

Note: Use static imports with caution. Excessive use of unqualified names of static members from outside the scope of a class creates code that is confusing to read.

Example usage of static imports

Create the following example class `example.TestClass`.

```
package example

class TestClass {

    // Static field (a field directly on the type, not an instance of it)
    static public var OneHundred : Integer = 100

    // Static method (a method directly on the type, not an instance of it)
    static function helloWorld() {
        print("Hello world")
    }
}
```

Note that the class contains a static field and a static method. Without using static imports, Gosu code in other classes needs to qualify the field or method:

```
uses example.TestClass

print(2 + TestClass.OneHundred)

TestClass.helloWorld()
```

If you import the static members from the type, you can omit the type name before using static members:

```
// Import all static members from a type (note the #* at the end)
uses example.TestClass#*

// You can now use a static field without specifying what type it is on
print(2 + OneHundred)

// You can call a static function without specifying what type it is on
helloWorld()
```

Exception handling

Gosu supports the standard exception handling constructions in typical use in many programming languages such as the following:

- `try...catch...finally` blocks
- `throw` statements

In addition to these types of exception handling statements, Gosu also provides a special `using` statement that is a more compact form of the `try...catch...finally` block.

Handling exceptions with `try/catch/finally`

The `try/catch/finally` blocks provides a way to handle some or all of the possible errors that may occur in a given block of code during run time. If errors occur that the script does not handle, Gosu provides its normal error message, as if no error handling existed.

The `try` block contains code where an error can occur, while the `catch` block contains the code to handle any error that does occur:

- If an error occurs in the `try` block, Gosu passes program control to the `catch` block to handle the error.
- In the `catch` block, Gosu passes as an argument to `catch` block a value that represents the exception that occurred in the `try` block. If an error is thrown from Java code, the value that the `catch` block gets is the exception thrown. For exceptions directly thrown from Gosu code, any value that is not a subclass of `RuntimeException` is wrapped in a `RuntimeException` object.
- If no error occurs, Gosu does not execute the `catch` block.
- If the error cannot be handled in the `catch` block associated with the `try` block where the error occurred, use the `throw` statement. The `throw` statement rethrows the exception to a higher-level error handler.

After Gosu runs all statements in the `try` block or after running the error handling code in the `catch` block, Gosu runs the `finally` block.

Gosu executes the code inside the `finally` block, even if a `return` statement occurs inside the `try` or `catch` blocks, or if an error is thrown from a `catch` block. It is important to understand that , Gosu always runs the `finally` block.

In a `finally` block, Gosu does not permit the `return`, `break`, or `continue` statements.

Syntax

```
try
  <try statements>
[catch ( exception )
  <catch statements>]
```

```
[finally
  <finally statements>]
```

Example

```
try {
  print( "Outer TRY running..." )
  try {
    print( "Nested TRY running..." )
    throw "an error"
  }
  catch (e : Exception) {
    print( "Nested CATCH caught "+e )
    throw e + " rethrown"
  }
  finally { print( "Nested FINALLY running..." ) }
}
catch (e : Exception) { print( "Outer CATCH caught " + e ) }
finally { print( "Outer FINALLY running" ) }
```

Output

```
Outer TRY running...
Nested TRY running...
Nested CATCH caught gw.lang.parser.EvaluationException: an error
Nested FINALLY running...
Outer CATCH caught gw.lang.parser.EvaluationException: gw.lang.parser.EvaluationException: an error
    rethrown
Outer FINALLY running
```

See also

- “Throwing exceptions” on page 119
- “Catching exceptions” on page 118

Catching exceptions

Gosu allows you to catch all general exceptions, or test for and catch specific types of exceptions.

The standard syntax for catch is the following, which catches all exceptions by specifying the class `Exception`:

```
catch (e : Exception)
```

To catch a single specific exception type, specify a subclass such as `IOException` instead of `Exception`. The technique of catching a named exception is called *checked exceptions*. The recommended Gosu coding style is not to use checked exceptions, although this technique is supported.

The following code is an example of checked exceptions:

```
try {
  doSomething()
}
catch (e : IOException) {
  // Handle the IOException
}
```

Add a `finally` block at the end to perform cleanup code that runs for both error and success code paths:

```
try {
  doSomething()
}
catch (e : IOException) {
}
finally {
  // PERFORM CLEANUP HERE
}
```

Throwable

The class `Throwable` is the superclass of all errors and exceptions in the Java language. Best practice is to catch `Exception`, not `Throwable`. Use `catch(e : Exception)` not `catch(e : Throwable)`. Catching `Throwable` can

catch serious infrastructure problems like `OutOfMemoryException` or `AssertionFailedException`. Typical code cannot handle those types of exceptions, so catch `Exception` so that these serious infrastructure exceptions propagate upward.

Throwing exceptions

The `throw` statement generates an error condition that you can handle through the use of `try/catch/finally` blocks.

WARNING Do not use `throw` statements as part of regular non-error program flow. Use these statements only for handling actual error conditions.

The following example throws an explicit `RuntimeException` exception:

```
if ( user.Age < 21 ) {  
    throw new RuntimeException("User is not allowed in the bar")  
}
```

You can also pass a non-exception object to the `throw` statement. If you pass a non-exception object, Gosu first coerces it to a `String`. Next, Gosu wraps the `String` in a new `RuntimeException`, which has a `Message` property that contains the `String` data. Your error-handling code can use the `Message` property for logging or displaying messages to users.

You could rewrite the previous `throw` code example as the concise code:

```
if ( user.Age < 21 ) {  
    throw "User is not allowed in the bar"  
}
```

Syntax

```
throw <expression>
```

In the following example, notice how the error message changes if the value of `x` changes from 0 to 1.

Example

```
uses java.lang.Exception  
  
doOuterCode() // Call outer code  
  
function doOuterCode() {  
    try {  
        doInnerCode(0)  
        doInnerCode(1)  
    }  
    catch (e : Exception) {  
        print( e.Message + " -- caught in OUTER code" )  
    }  
}  
  
function doInnerCode(x : int) {  
    print("For value ${x}...")  
    try {  
        if ( x == 0 ) {  
            throw "x equals zero"  
        } else {  
            throw "x does not equal zero"  
        }  
    }  
    catch (e : Exception) {  
        if ( e.Message == "x equals zero" ) {  
            print(e.Message + " -- caught in INNER code")  
        } else { throw e }  
    }  
}
```

This example prints:

```

For value 0...
x equals zero -- caught in INNER code.
For value 1...
x does not equal zero -- caught in OUTER code

```

Object life-cycle management with using clauses

If you have an object with a complete life cycle in a particular extent of code, you can simplify your code with the `using` statement. The `using` statement is a more compact syntax and less error-prone way to work with resources than using `try/catch/finally` clauses. With `using` clauses:

- Cleanup always occurs without requiring a separate `finally` clause.
- You do not need to explicitly check whether variables for initialized resources have null values.
- Code for locking and synchronizing resources is simplified.

For example, suppose you want to use an output stream. Typical code would open the stream, then use it, then close the stream to dispose of related resources. If something goes wrong while using the output stream, your code must close the output stream and perhaps check whether it successfully opened before closing it. In Gosu or standard Java, you could use a `try/finally` block like the following to clean up the stream:

```

OutputStream os = SetupMyOutputStream() // Insert your code that creates your output stream
try {
    // Do something with the output stream
}
finally {
    os.close();
}

```

You can simplify that code by using the Gosu `using` statement:

```

using ( var os = SetupMyOutputStream() ) {
    // Do something with the output stream
} // Gosu disposes of the stream after it completes or if there is an exception

```

The basic form of a `using` clause is as follows:

```

using ( ASSIGNMENT_OR_LIST_OF_STATEMENTS ) {
    // Do something here
}

```

The parentheses after the `using` keyword can contain either a Gosu expression or a list of one or more Gosu statements delimited by commas, not semicolons.

Several categories of objects work with the `using` keyword: disposable objects, closeable objects, and reentrant objects. If you try to use an object that does not satisfy the requirements of one of these categories, Gosu displays a compiler error.

If Gosu detects that an object is in more than one category, at run time, Gosu considers the object to be in only one category. Gosu selects the category by the following precedence: disposable, closeable, reentrant. For example, if an object has a `dispose` and `close` method, Gosu only calls the `dispose` method.

Use the `return` statement to return values from `using` clauses.

If Gosu successfully evaluates and initializes all variables, each variable is examined in order. For each object, if the object has an `enter` action, that action is taken:

- If the object implements the `Lock` interface, Gosu calls `Lock.lock()`
- If the object implements the `IReentrant` interface, Gosu calls `IReentrant.enter()`
- If the object has a method named `lock` and it takes no parameters, Gosu calls the `lock` method.
- If the object was cast to the `IMonitorLock` interface, Gosu synchronizes on the variable

If the enter action completes without an exception, Gosu guarantees that the corresponding exit action for that object will run. If no enter action applies to the object, Gosu guarantees that the exit action for the object will run. Exit actions are called in the reverse order of declaration in the using clause. Exit actions are as follows:

- If the object implements the `IDisposable` interface, Gosu calls `IDisposable.dispose()`
- If the object has a method named `dispose` and it takes no parameters, Gosu calls the method.
- If the object implements the `Closeable` interface, Gosu calls `Closeable.close()`
- If the object has a method named `close` and it takes no parameters, Gosu calls the method.
- If the object implements the `IReentrant` interface, Gosu calls `IReentrant.exit()`
- If the object implements the `Lock` interface, Gosu calls `Lock.unlock()`
- If the object has a method named `unlock` that takes no parameters, Gosu calls the `unlock` method.
- If the object was cast to the `IMonitorLock` interface, Gosu unsynchronizes on the variable

If an exception occurs during the execution of an exit action of an object, the exit actions defined before that exit action run, and then the exception throws. If an exception occurs in more than one exit action, the outermost exception, which occurs later in the code, takes precedence and is thrown to the surrounding code.

Assigning variables inside a using expression declaration

The using clause supports assigning a variable inside the declaration of the using clause. This feature is useful if you want to reference the expression that you pass to the using expression from inside the using clause.

For example, suppose you call a method that returns a file handle and you pass that handle to the using clause as the lock. From within the using clause contents, you probably want to access the file so that you can iterate across its contents.

To simplify this kind of code, use the `var` keyword to assign a variable to the expression:

```
using ( var VARIABLE_NAME = EXPRESSION ) {
    // Code that references the VARIABLE_NAME variable
}
```

For example:

```
using ( var out = new FileOutputStream( this, false ) ) {
    out.write( content )
}
```

Passing multiple items to a using statement

You can pass one or multiple items in the using clause expression. You must separate each item by a comma character, not a semicolon character as in a typical Gosu statement list. You cannot pass more than one reentrant object, which is an object that implements `IReentrant` or `IMonitorLock` interfaces or has a `lock` method.

For example:

```
using ( _file1, _file2, _file3 ) {
    // Do your main work here
}
```

To pass multiple reentrant objects, nest multiple using clauses, such as:

```
using ( lock1 ) {
    using ( lock2 ) {
        ...
    }
}
```

You can combine the multiple item feature with the ability to assign variables. Gosu runs any statements, including variable assignment, at run time and uses the result as an object to manage in the using clause. Within each comma-delimited assignment statement, you do not need a return statement.

For example:

```
using ( var lfc = new FileInputStream(this).Channel,
        var rfc = new FileInputStream(that).Channel ) {
```

```

var lbuff = ByteBuffer.allocate(bufferSize)
var rbuff = ByteBuffer.allocate(bufferSize)

while (lfc.position() < lfc.size()) {
    lfc.read(lbuff)
    rfc.read(rbuff)

    if (not Arrays.equals(lbuff.array(), rbuff.array())) {
        return true
    }

    lbuff.clear()
    rbuff.clear()
}
return false
}
}

```

Gosu ensures that all objects are properly cleaned up. Gosu cleans up only the objects that initialized without throwing exceptions or otherwise having errors such as returning null for a resource.

If you choose to initialize multiple resources and some but not all objects in the list successfully initialize, Gosu performs the following actions:

1. Gosu skips initialization for any subsequent items not yet initialized, in other words the items that appear later in the initialization list.
2. Gosu skips execution of the main part of the using clause.
3. Gosu cleans up exactly the set of objects that initialized without errors. In other words, Gosu releases, closes, or disposes the object, depending on the type of object.

For example, suppose you try to acquire three resources, and the first one succeeds but the second one fails. Gosu does not attempt to acquire the third resource. Then, Gosu cleans up the one resource that did acquire successfully.

See also

- “Assigning variables inside a using expression declaration” on page 121

Disposable objects and using clauses

Disposable objects are objects that Gosu can dispose to release all system resources. For Gosu to recognize a valid disposable object, the object must have one of the following attributes:

- Implements the Gosu `IDisposable` interface, which contains only a single method called `dispose` with no arguments. Use a type that implements `IDisposable` if possible due to faster runtime performance.
- Has a `dispose` method with no arguments, even if it does not implement the `IDisposable` interface. This approach is slower at run time, because Gosu must use reflection (examining the type at run time) to find the method.

A type’s `dispose` method must release all resources owned by its base types by calling its parent type’s `dispose` method.

To ensure that resources clean up appropriately even under error conditions, you must design your `dispose` method so Gosu can call it multiple times without throwing an exception, even if the object has already been disposed.

The following example shows a basic disposable object:

```

// Create a simple disposable class that implements IDisposable
class TestDispose implements IDisposable {
    construct(){
        print("LOG: Created my object!")
    }

    override function dispose() {
        print("LOG: Disposed of my object! Note that you must support multiple calls to dispose.")
    }
}

```

The following code tests this object:

```
using (var d = new TestDispose()) {  
    print("LOG: This is the body of the 'using' statement.")  
}
```

This code prints:

```
LOG: Created my object!  
LOG: This is the body of the 'using' statement.  
LOG: Disposed of my object! Note that you must support multiple calls to dispose.
```

You can pass multiple objects to a using clause. In such cases, exit actions (closing, disposing, and unlocking) happen in reverse order of the object declarations.

Closeable objects and using clauses

Closeable objects include objects such as data streams, reader or writer objects, and data channels. Many of the classes in the `java.io` package are closeable. For Gosu to recognize a valid closeable object, the object must have one of the following attributes:

- Implements the `java.io.Closeable` interface, which contains only a single method called `close` with no arguments. Use a type that implements `Closeable` if possible for faster runtime performance.
- Has a `close` method with no arguments, even if it does not implement the `java.io.Closeable` interface. This approach is slower at run time, because Gosu must use reflection (examining the type at run time) to find the method.

A type's `close` method must release all resources owned by its base types by calling its parent type's `close` method. To ensure that resources clean up appropriately even under error conditions, you must design your `close` method so Gosu can call it multiple times without throwing an exception, even if the object is already closed.

The following example creates a new Java file writer instance (`java.io.PrintWriter`) and uses the more verbose `try` and `finally` clauses:

```
var writer = new PrintWriter("c:\\temp\\test1.txt")  
try {  
    writer.write( "I am text within a file." )  
}  
finally {  
    if ( writer != null ) {  
        writer.close()  
    }  
}
```

You can write more readable Gosu code with the `using` keyword:

```
using (var writer = new PrintWriter("c:\\temp\\test1.txt")) {  
    writer.write("I am text within a file.")  
}
```

You can list multiple variables in the `using` clause:

```
uses java.io.FileReader  
uses java.io.PrintWriter  
uses gw.util.StreamUtil  
  
using (var reader = new FileReader("c:\\temp\\test1.txt"),  
      var writer = new PrintWriter("c:\\temp\\test2.txt")) {  
    writer.write( StreamUtil.getContent( reader ) )  
}
```

You can pass multiple objects to a using clause. In such cases, exit actions (closing, disposing, and unlocking) are executed in reverse order of the object declarations.

JDBC resources and using clauses

The following example demonstrates how to use a using clause with a JDBC (Java Database Connection) object.

```
uses java.sql.*  
  
...
```

```
function sampleJdbc( con : Connection ) {
    using (var stmt = con.createStatement(),
        var rs = stmt.executeQuery("SELECT a, b FROM TABLE2")) {
        rs.moveToInsertRow()
        rs.updateString(1, "AINSWORTH")
        rs.insertRow()
    }
}
```

Reentrant objects and using clauses

A *reentrant* object supports multiple concurrent uses of a single lock in a single thread. Reentrancy maintains a count of the number of times the thread sets and releases the lock. In reentrant lock use, the thread does not deadlock on setting an already locked lock, nor release the lock when other code in the thread still retains that lock. Reentrant objects help manage safe access to data that is shared by reentrant or concurrent code execution. For example, if you must store data that is shared by multiple threads, ensure that you protect against concurrent access from multiple threads to prevent data corruption. The most prominent type of shared data is class static variables, which are variables that are stored on the Gosu class itself.

For Gosu to recognize a valid reentrant object, the object must have at least one of the following attributes:

- The object implements the `java.util.concurrent.locks.Lock` interface. For example, the following Java classes in that package implement this interface: `ReentrantLock`, `ReadWriteLock`, `Condition`.
- You cast the object to the Gosu interface `IMonitorLock`. You can cast any arbitrary object to `IMonitorLock`. It is useful to cast Java monitor locks to this Gosu interface.
- The object implements the Gosu class `gw.lang.IReentrant`. This interface contains two methods with no arguments: `enter` and `exit`. Your implementation code must properly lock or synchronize data access as appropriate during the `enter` method and release any locks in the `exit` method.

The profiler tag class (`gw.api.profiler.ProfilerTag`) implements the `IReentrant` interface. This class adds hints to the Gosu profiler about what actions happen in a block of code.

For blocks of code that use locks by implementing `java.util.concurrent.locks.Lock`, a `using` clause simplifies the code. The `using` statement always cleans up the lock, even if the code throws an exception.

```
uses java.util.concurrent

// In your class variable definitions...
var _lock : ReentrantLock = new ReentrantLock()

...

function useReentrantLockNew() {
    using ( _lock ) {
        // Do your main work here
    }
}
```

Similarly, you can cast any object to a monitor lock by adding `as IMonitorLock` after the object. For example, the following method code uses the object itself, by using the keyword `this`, as the monitor lock:

```
function monitorLock() {
    using ( this as IMonitorLock ) {
        // Do stuff
    }
}
```

This approach is equivalent to a `synchronized` block in the Java language.

The following code uses the `java.util.concurrent.locks.ReentrantLock` class using more verbose `try` and `finally` statements. This approach is not recommended:

```
// In your class variable definitions...
var _lock : ReentrantLock = new ReentrantLock()

function useReentrantLockOld() {
    _lock.lock()
    try {
        // Do your main work here
    }
}
```

```
finally {  
    lock.unlock()  
}  
}
```

Alternatively, you can do your change in a Gosu block. Guidewire does not recommend this approach. Returning the value from a block imposes more restrictions on how you implement return statements. It is typically better to use the using statement structure shown earlier in this topic.

A using clause supports passing multiple objects. If you pass multiple objects to the using clause, exit actions (closing, disposing, unlocking) happen in reverse order of the object declarations.

See also

- [http://en.wikipedia.org/wiki/Monitor_\(synchronization\)](http://en.wikipedia.org/wiki/Monitor_(synchronization))
- “Object life-cycle management with using clauses” on page 120
- “Concurrency and thread safety” on page 365
- “Using profiler tags” on page 125

Using profiler tags

The profiler tag class (`gw.api.profiler.ProfilerTag`) hints to the Gosu profiler what actions happen within a block of code. To create a new profiler tag, pass a String value that defines the profiler tag name to the `ProfilerTag` constructor:

```
new ProfilerTag("EventPayloadXML")
```

To use the profiler tag, pass it to a using clause, as shown in the following code:

```
using ( new ProfilerTag("EventPayloadXML") ) {  
    // Do your main work here...  
  
    // Use ProfilerTag methods if needed  
    tag.setCounterValue( "test", 3 )  
}
```

See also

- *System Administration Guide*

Returning values from using clauses

You can return values from using clauses by using the standard return statement. If you return a value from a using clause, Gosu considers the clause complete, and therefore calls your object’s final life-cycle management method to clean up your resources:

- For disposable objects, Gosu calls the `dispose` method.
- For closable objects, Gosu calls the `close` method.
- For a reentrant or lock object, Gosu calls the `exit` method.

The following Gosu example opens a file using the Java `BufferedReader` class and reads lines from the file until the line matches a regular expression. If code in the while loop finds a match, it immediately returns the value and skips the rest of the code within the using clause.

```
uses java.io.File  
uses java.io.BufferedReader  
uses java.io.FileReader  
  
function containsText(file : File, regExp : String) : boolean {  
    using (var reader = new BufferedReader(new FileReader(file))) {  
        var line = reader.readLine()  
        while (line != null) {  
            if (line.matches(regExp)) {  
                return true  
            }  
        }  
    }  
}
```

```

        line = reader.readLine() // Read the next line
    }
}
return false
}

```

See also

- “Disposable objects and using clauses” on page 122
- “Closeable objects and using clauses” on page 123
- “Reentrant objects and using clauses” on page 124

Optional use of a finally clause with a using clause

In some cases, you need to perform additional cleanup that is not handled automatically by one of the closable, reentrant, or disposable types. To add arbitrary cleanup logic to the using clause, add a **finally** clause after the using clause. The Gosu using clause with the **finally** clause is similar to the try-with-resources statement of Java 1.7 but without a catch clause.

Important notes about the **finally** clause:

- The body of the using clause always runs before the **finally** clause.
- If exceptions occur that are not caught, Gosu still runs the **finally** clause.
- Your **finally** clause can access objects created in the using clause, but all resources are already cleaned up by the time the **finally** clause begins. For example, closeable objects are already closed and disposable objects are already disposed.

For example, create a disposable class:

```

// Create a disposable class -- it implements IDisposable
class TestDispose implements IDisposable {

    var _open : boolean as OpenFlag

    construct(){
        print("LOG: Created my object!")
        _open = true
    }

    override function dispose() {
        print("LOG: Disposed of my object! Note that you must support multiple calls to dispose.")
        _open = false
    }
}

```

The following code creates a disposable object in a using clause, and throws an exception in the body of the using clause. Note that the **finally** clause still runs:

```

using (var d = new TestDispose()) {
    print("LOG: In the body of the 'using' statement.")
    print("value of d.OpenFlag = ${d.OpenFlag}")

    print(3 / 0) // THROW AN EXCEPTION (divide by zero)
}
finally {
    print("LOG: In the finally clause.")
    print("value of d.OpenFlag = ${d.OpenFlag}")
}
print("LOG: Outside the 'using' clause.")

```

This example prints:

```

LOG: Created my object!
LOG: In the body of the 'using' statement.
value of d.OpenFlag = true
LOG: Disposed of my object! Note that you must support multiple calls to dispose.
LOG: In the finally clause.
value of d.OpenFlag = false

java.lang.ArithmeticException: / by zero
[...]
```

Notice in the output:

- The `using` clause automatically cleaned up the disposable resource.
- The `finally` clause still runs.
- The `finally` clause can access the resource, which is already disposed.

Assert statements

An assert statement provides a concise syntax for asserting expectations and enforcing a programmatic contract with calling code. For this purpose, Gosu has an `assert` statement with the same semantics and syntax as in Java.

You can use the Gosu `assert` statement in two different forms:

```
assert expressionBoolean
assert expressionBoolean : expressionMessage
```

- *expressionBoolean* is an expression that returns a boolean result.
- *expressionMessage* is an expression that returns a value that becomes a detailed message.

For example:

```
assert i > 0 : i
```

By default, `assert` statements have no effect. Assertions are disabled.

To enable assertions, you must add the `-ea` flag on the JVM that hosts the application or Studio.

If assertions are enabled, Gosu evaluates the initial expression *expressionBoolean*:

- If the expression returns `true`, the `assert` statement has no effect.
- If the expression returns `false`, Gosu throws an `AssertionError` exception. If you use the version of the statement that has a second expression, Gosu uses that value as an exception detail message. Otherwise, there is no exception detail message.

Intervals

An interval is a sequence of values of the same type between a given pair of endpoint values. Gosu provides native support for intervals. For instance, the set of integers beginning with 0 and ending with 5 is an integer interval containing the values 0, 1, 2, 3, 4, 5. The Gosu syntax for this set is `0..5`. Intervals are useful for concise, readable for loops. Intervals can be of many types including numbers, dates, dimensions, and names. You can add custom interval types. In some programming languages, intervals are called *ranges*.

What are intervals?

An interval is a sequence of values of the same type between a given pair of endpoint values. Gosu provides native support for intervals. For instance, the set of integers beginning with 0 and ending with 10 is an integer interval. This interval contains the values 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10. The Gosu syntax for this set of integers is `0..10`. Intervals are particularly useful for concise, readable for loops.

For example, consider this concise, readable code:

```
for (i in 0..10) {  
    print("The value of i is " + i)  
}
```

This code prints the following:

```
The value of i is 0  
The value of i is 1  
The value of i is 2  
The value of i is 3  
The value of i is 4  
The value of i is 5  
The value of i is 6  
The value of i is 7  
The value of i is 8  
The value of i is 9  
The value of i is 10
```

This code replaces the more verbose and harder-to-read design pattern:

```
var i = 0  
while( i <= 10 ) {  
    print("The value of i is " + i)  
    i++  
}
```

Intervals do not need to be numbers. Intervals can be of many types including numbers, dates, dimensions, and names. Gosu includes built-in shorthand syntax with a double period (`..`) for intervals for dates and common number types, such as the previous `0..10` example. The built-in shortcut works with the types `Integer`, `Long`, `BigInteger`, `BigDecimal`, and `Date`. All decimal types map to the `BigDecimal` interval.

You can also add custom interval types that support any type that supports iterable comparable sequences. You can then use your new intervals in `for` loop declarations.

If you need to get a reference to the interval's iterator object (`java.lang.Iterator`), call the `iterate` method, which returns the iterator.

Omitting an initial or ending value

In the simplest case, a Gosu interval iterates from the beginning endpoint value to the ending endpoint value, including the values at both ends. For example, `0..5` represents the values 0, 1, 2, 3, 4, 5.

In some cases, you want to exclude the beginning value but you still want your code to show the beginning value for code legibility. In other cases, you want to exclude the endpoint value from the interval.

If omitting the initial or ending value results in an empty set of values, iterating over the interval does not occur.

To exclude an endpoint in Gosu, type the pipe `|` character:

- To omit the value of the starting endpoint, type the pipe character after the starting endpoint.
- To omit the value of the ending endpoint, type the pipe character before the ending endpoint.
- To do make both endpoints open, type the pipe character before and after the double period symbol.

Compare the following examples:

- `0..5` represents the values 0, 1, 2, 3, 4, 5.
- `0|.5` represents the values 1, 2, 3, 4, 5.
- `0..|5` represents the values 0, 1, 2, 3, 4.
- `0|..|5` represents the values 1, 2, 3, 4.
- `0..|0` does not iterate.
- `0|..|1` does not iterate.

See also

- “Writing your own interval type” on page 131

Reversing interval order

Sometimes you want a loop to iterate across elements in an interval in reverse order. To do this iteration, reverse the values of the beginning endpoint and ending endpoint for the double period symbol.

Compare the following examples:

- `0..5` represents the values 0, 1, 2, 3, 4, 5 in that order.
- `5..0` represents the values 5, 4, 3, 2, 1, 0 in that order.

Internally, these intervals are the same objects but Gosu marks `5..0` as being in reverse order.

For example, this code iterates from 10 to 1, including the end points:

```
for( i in 10..1) {
    print ( i )
}
```

If you have a reference to a reversed interval, you can force the interval to operate in its natural order. In other words, you can undo the flag that marks the interval as reversed. Use the following syntax:

```
var interv = 5..0
var leftIterator = interv.iterateFromLeft()
```

The result is that the `leftIterator` variable contains the interval for 0, 1, 2, 3, 4, 5.

The `iterate` method, which returns the iterator, always iterates across the items in the declared order, either regular order or reverse, depending on your definition of the interval.

Granularity of interval steps and units

You can customize the granularity with the `step` and `unit` builder-style methods on an interval. You use the `step` method to set the number of items to step or skip by. The `unit` method specifies the unit, which is necessary for

some types of intervals. For example, the granularity of a date interval is expressed in units of time: days, weeks, months, hours. You could iterate across a date interval in 2-week periods, 10-year periods, or 1-month periods. Each method returns the interval so you can chain the result of one method with the next method. For example:

```
// Simple int interval visits odd elements
var interv = (1..10).step( 2 )

// Date interval visits two week periods
var span = (date1..date2).step( 2 ).unit( WEEKS )
```

Notice the WEEKS value, which is an enumeration constant. You do not need to qualify WEEKS with the enumeration type. Gosu can infer the enumeration type so the code is always type-safe.

Writing your own interval type

You can add custom interval types of either of the two basic types of intervals:

- *Iterable intervals*, which are intervals that you can iterate across, such as in `for` loop declarations
- Non-iterable intervals

For typical code, iterable intervals are more useful because they can simplify common coding patterns in loops.

In some circumstances, you need to create a non-iterable interval. For example, suppose you want to encapsulate an inclusive range of real numbers between two endpoints. Such a set includes a theoretically infinite set of numbers between the values 1 and 1.001. Iterating across the set is meaningless.

The basic properties of an interval are as follows:

- The type of items in the interval must implement the Java interface `java.lang.Comparable`.
- The interval has left and right endpoints (the starting and ending values of the interval)
- Each endpoint can be closed (included) or open (excluded)

The main difference for iterable intervals is that they also implement the `java.lang.Iterable` interface.

Custom iterable intervals using sequenceable items

The following example demonstrates creating a custom iterable interval using sequenceable items. A *sequenceable* item is a type that implements the `ISequenceable` Gosu interface, which defines how to get the next and previous items in a sequence. If the item you want to iterate across implements that interface, you can use the `SequenceableInterval` Gosu class and you do not need to create your own interval class. Suppose you want to create a new iterable interval that can iterate across a list of predefined and ordered color names with a starting and ending color value. Define an enumeration containing the possible color values in their interval:

```
package example.pl.gosu.interval

enum Color {
    Red, Orange, Yellow, Green, Blue, Indigo, Violet
}
```

All Gosu enumerations automatically implement the `java.lang.Comparable` interface, which is a requirement for intervals. However, Gosu enumerations do not automatically implement the `ISequenceable` interface.

To determine an iterable interval dynamically, Gosu requires that a comparable endpoint also be sequenceable. To be sequenceable means that a class knows how to find the next and previous items in the sequence. Sequenceable and interval types have a lot in common. Both types both have the concept of granularity in terms of step amount and optionally a unit such as weeks, months, and so on.

The interface for `ISequenceable` is as follows. Implement these methods and declare your class to implement this interface.

```
public interface ISequenceable<E> extends ISequenceable<E, S, U>, S, U> extends Comparable<E> {
    E nextInSequence( S step, U unit );
    E nextNthInSequence( S step, U unit, int iIndex );
    E previousInSequence( S step, U unit );
    E previousNthInSequence( S step, U unit, int iIndex );
}
```

Method	Parameter	Description
nextInSequence		Returns <i>E element</i> , the next element in the sequence
	<i>S step</i>	An alias for the column name in the row query result
	<i>U unit</i>	A path expression that references the column to include in the row query result. For example, you can use a feature literal expression.
nextNthInSequence		Returns <i>E element</i> , the the element that is <i>int index</i> next in the sequence
	<i>S step</i>	An alias for the column name in the row query result
	<i>U unit</i>	
	<i>int index</i>	
previousInSequence		Returns <i>E element</i> , the previous element in the sequence
	<i>S step</i>	An alias for the column name in the row query result
	<i>U unit</i>	
previousNthInSequence		Returns <i>E element</i> , the element that is <i>int index</i> previous in the sequence
	<i>S step</i>	An alias for the column name in the row query result
	<i>U unit</i>	
	<i>int index</i>	

The syntax for the interface might look unusual because of the use of Gosu generics. This syntax indicates that the interface is parameterized across three dimensions:

- The type of each sequenceable element in the interval.
- The type of the step amount. For example, to skip every other item, the step is 2, which is an `Integer`. For typical use cases, pass `Integer` as the type of the step amount.
- The type of units for the interval. For example, for an integer (1, 2, 3), choose `Integer`. For a date interval, the type is `DateUnit`. That type contains values representing days, weeks, or months. For instance, `DateUnit.DAYS`. If you do not use units with the interval, type `java.lang.Void` for this dimension of the parameterization. Carefully note the capitalization of this type, because it is particularly important to access Java types, especially when using Gosu generics. In Gosu, as in Java, `java.lang.Void` is the special type of the value `null`.

The example later in this topic has a class that extends the type:

```
IterableInterval<Color, Integer, void, ColorInterval>
```

This *bidirectional* interface can fetch both next and previous elements. Gosu needs this capability to handle navigation from either endpoint in an interval to support the reverse mode. Gosu also requires that the class know how to jump to an element by its index in the series. This functionality can be achieved with the single step methods, some sequenceable objects can optimize this method without having to visit all elements in between. For example, if the step value is 100, Gosu does not need to call the `nextInSequence` method 100 times to get the next value.

The following example defines an enumeration class with additional methods that implement the required methods of `ISequenceable`.

```
package example.gosu.interval

enum ColorSequenceable
    implements gw.lang.reflect.interval.ISequenceable<ColorSequenceable, Integer, java.lang.Void>
{
    // Enumeration values....
    Red, Orange, Yellow, Green, Blue, Indigo, Violet

    // Required methods in ISequenceable interface...

    override function nextInSequence( stp : Integer, unit : java.lang.Void ): ColorSequenceable {
        return ColorSequenceable.AllValues[this.Ordinal + stp]
    }
}
```

```

    override function nextNthInSequence( stp : Integer, unit : java.lang.Void,
        iIndex : int) : ColorSequenceable {
        return ColorSequenceable.AllValues[this.Ordinal + stp * iIndex]
    }

    override function previousInSequence( stp : Integer, unit : java.lang.Void ) : ColorSequenceable {
        return ColorSequenceable.AllValues[this.Ordinal - stp]
    }

    override function previousNthInSequence( stp : Integer, unit : java.lang.Void,
        iIndex : int) : ColorSequenceable {
        return ColorSequenceable.AllValues[this.Ordinal - stp * iIndex]
    }
}

```

To use this class, run the following code in Gosu Scratchpad:

```

print("Red to Blue as a closed interval...")
var colorRange = new gw.lang.reflect.interval.SequenceableInterval(
    ColorSequenceable.Red, ColorSequenceable.Blue, 1, null, true, true, false )

for (i in colorRange) {
    print(i)
}

print("Red to Blue as an open interval...")
var colorRangeOpen = new gw.lang.reflect.interval.SequenceableInterval(
    ColorSequenceable.Red, ColorSequenceable.Blue, 1, null, false, false, false )

for (i in colorRangeOpen) {
    print(i)
}

```

This code prints:

```

Red to Blue as a closed interval...
Red
Orange
Yellow
Green
Blue
Red to Blue as an open interval...
Orange
Yellow
Green

```

To make your code look even more readable, you could create your own subclass of `SequenceableInterval` named for the sequenceable type that you plan to use. For example, `ColorSequenceInterval`.

[See also](#)

- “Enumerations” on page 169
- “Generics” on page 241

Custom iterable intervals using manually written iterators

If your items are not sequenceable, you can make an iterable interval class for those items, but you write more code to implement all necessary methods.

Create a custom iterable interval using manually written iterator classes

You perform the following general tasks to write a manual iterator class.

1. Confirm that the type of items in your interval implement the Java interface `java.lang.Comparable`.
2. Create a new class that extends (is a subclass of) the `IterableInterval` class parameterized using Gosu generics across four separate dimensions:
 - The type of each element in the interval
 - The type of the step amount. For example, to skip every other item, the step is 2, which is an `Integer`.
 - The type of units for the interval. For example, for an integer (1, 2, 3), choose `Integer`. For a date interval, the type is `DateUnit`. That type contains values representing days, weeks, or months. For instance,

`DateUnit.DAYS`. If the interval does not have units, use `java.lang.Void` for this dimension of the parameterization. Make sure that you use the capitalization of this type. In Gosu, as in Java, `java.lang.Void` is the special type of the value `null`.

- The type of your custom interval. This type is self-referential because some of the methods return an instance of the interval type itself.

The documentation provides an example of a class that extends the type:

```
IterableInterval<Color, Integer, void, ColorInterval>
```

3. Implement the interface methods for the `Interval` interface.
4. Implement the interface methods for the `Iterable` interface.

The most complex methods to implement correctly are methods that return iterators. A straightforward way to implement these methods is to define iterator classes as inner classes to your main class.

Your class must be able to return two types of iterators, one iterating forward, and one iterating in reverse. One way to do this is to implement a main iterator. Next, implement a class that extends your main iterator class, and which operates in reverse. On the class for the reverse iterator, to reverse the behavior you may need to override only the `hasNext` and `next` methods.

See also

- “Inner classes” on page 164
- “Generics” on page 241
- “Example: Color interval written with manual iterators” on page 134

Example: Color interval written with manual iterators

In some cases, the item you want to iterate across does not implement the `ISequenceable` interface. You cannot modify it to directly implement this interface because it is a Java class from a third-party library. Although you cannot use Gosu shortcuts for custom iterable intervals using sequenceable items, you can still implement an iterable interval.

The following example demonstrates creating a custom iterable interval. Suppose you want to create a new iterable interval that can iterate across a list of predefined and ordered color names with a starting and ending color value. Define an enumeration containing the possible color values in their interval:

```
package example.gosu.interval

enum Color {
    Red, Orange, Yellow, Green, Blue, Indigo, Violet
}
```

All Gosu enumerations automatically implement the `java.lang.Comparable` interface, which is a requirement for intervals.

Next, create a new class that extends the following type:

```
IterableInterval<Color, Integer, void, ColorInterval>
```

Next, implement the methods from the `IIterableInterval` interface. It is important to note that in this example the iterator classes are inner classes of the main `ColorInterval` class.

```
package example.gosu.interval

uses gw.lang.reflect.interval.IterableInterval

class ColorInterval extends IterableInterval<Color, Integer, java.lang.Void, ColorInterval> {
    construct(left : Color, right : Color, stp : Integer) {
        super(left, right, stp)
        //print("new ColorInterval, with 2 constructor args")
    }

    construct(left : Color, right : Color, stp : Integer, leftOpen : boolean,
        rightOpen : boolean, rev : boolean) {
        super(left, right, stp, null, leftOpen, rightOpen, rev)
    }
}
```

```

    //print("new ColorInterval, with 6 constructor args")
  }

  // Get the Nth item from the beginning (left) endpoint
  override function getFromLeft(i: int) : Color {
    return Color.AllValues[LeftEndpoint.Ordinal + i]
  }

  // Get the Nth item from the right endpoint
  override function getFromRight(i : int) : Color {
    return Color.AllValues[RightEndpoint.Ordinal - i]
  }

  // Return standard iterator
  override function iterateFromLeft() : Iterator<Color> {
    var startAt = LeftEndpoint.Ordinal
    if (!LeftClosed) {
      startAt++
    }
    return new ColorIterator(startAt)
  }

  // Return reverse order iterator
  override function iterateFromRight() : Iterator<Color> {
    var startAt = RightEndpoint.Ordinal
    if (!LeftClosed)
      startAt--
    return new ReverseColorIterator(startAt)
  }

  // DEFINE AN INNER CLASS TO ITERATE ACROSS COLORS -- NORMAL ORDER
  class ColorIterator implements Iterator<Color>{
    protected var _currentIndex : int;

    construct() {
      throw "required start at # -- use other constructor"
    }

    construct(startAt : int ) {
      _currentIndex = startAt
    }

    override function hasNext() : boolean {
      return ((_currentIndex) <= (RightEndpoint.Ordinal - (RightClosed ? 0 : 1)))
    }

    override function next() : Color {
      var i = _currentIndex
      _currentIndex++
      return Color.AllValues[i]
    }

    override function remove() {
      throw "does not support removing values"
    }
  }

  // DEFINE AN INNER CLASS TO ITERATE ACROSS COLORS -- REVERSE ORDER
  class ReverseColorIterator extends ColorIterator {

    construct(startAt : int ) {
      super(startAt)
    }

    override function hasNext() : boolean {
      return ((_currentIndex) >= (RightEndpoint.Ordinal + (LeftClosed ? 0 : 1)))
    }

    override function next() : Color {
      var i = _currentIndex
      _currentIndex--
      return Color.AllValues[i]
    }
  }
}

```

Note the parameterized element type using Gosu generics syntax. It enforces the property that elements in the interval are mutually comparable.

Finally, you can use your new intervals in for loop declarations:

```
uses example.gosu.interval.Color
uses example.gosu.interval.ColorInterval

print("Red to Blue as a closed interval...")
var colorRange = new ColorInterval(
    Color.Red, Color.Blue, 1, true, true, false )

for (i in colorRange) {
    print(i)
}

print("Red to Blue as an open interval...")
var colorRangeOpen = new ColorInterval(
    Color.Red, Color.Blue, 1, false, false, false )

for (i in colorRangeOpen) {
    print(i)
}
```

This code prints:

```
Red to Blue as a closed interval...
Red
Orange
Yellow
Green
Blue
Red to Blue as an open interval...
Orange
Yellow
Green
```

[See also](#)

- “Custom iterable intervals using sequenceable items” on page 131
- “Enumerations” on page 169

Custom non-iterable interval types

There are circumstances where a range of numbers is non-iterable. For example, suppose you want to encapsulate an inclusive range of real numbers between two endpoints. Such a set would be inclusive to a theoretically infinite set of numbers even between the values 1 and 1.001. Iterating across the set is meaningless.

To create a non-iterable interval type, create a new class that descends from the class `AbstractInterval`, parameterized using Gosu generics on the class of the object across which it iterates. For example, to iterate across `MyClass` objects, mark your class to extend `AbstractInterval<MyClass>`.

The class to iterate across must implement the `Comparable` interface.

A non-iterable interval cannot be used in for loop declarations or other types of iteration.

Calling Java from Gosu

You can write Gosu code that uses Java types. Gosu code can instantiate Java types, access properties of Java types, and call methods of Java types. Instead of writing Java code for your Gosu to call, if you write that code directly in Gosu, you can do everything that you can do in Java. For example, you can directly call existing Java classes and Java libraries. You can even write Gosu code enhancements that add properties and methods to Java types, and the new members are accessible from all Gosu code.

This topic describes how to write Gosu code that works with Java and how to write Java code that works with Gosu and works with Guidewire products.

Because Gosu is based on the Java Virtual Machine (JVM), Gosu code can directly use Java types. From Gosu, Java classes are first-class types and are used like native Gosu classes. In most ways, Java syntax also works in Gosu. For example, you can instantiate a Java class with the `new` keyword:

```
var bd = new java.math.BigDecimal(5, 25)
```

Gosu provides additional optional features and more concise syntax than Java, such as optional type inference and optional semicolons.

Gosu code can integrate with Java types in the following ways:

- Instantiate Java types.
- Manipulate Java classes as native Gosu objects.
- Manipulate Java objects as native Gosu objects.
- Manipulate Java primitives as native Gosu objects.

Gosu converts between primitives and the equivalent object types automatically in most cases.

- Call methods on Java types.

For methods that look like getters and setters, Gosu exposes the methods also as Gosu properties.

- Get instance variables and static variables from Java types.
- Add new methods to Java types by using Gosu enhancements.
- Add new properties to Java types by using Gosu enhancements.
- Add new properties to Java types automatically for methods that look like getters and setters.
- Create Gosu classes that extend Java classes.
- Create Gosu interfaces that extend Java interfaces.
- Use Java enumerations.
- Use Java annotations.
- Use Java properties files.

All these features work with built-in Java types as well as your own Java classes and libraries.

If you do not use Guidewire entity types, you do not need to do anything other than to set the class path so that it can access the files or to put compiled classes in a special directory.

If your Java code needs to get, set, or query Guidewire entity data, you must understand how PolicyCenter works with entity data.

See also

- “Gosu introduction” on page 17
- “Notable differences between Gosu and Java” on page 42
- *Integration Guide*

Locations for Java classes and libraries

Place your non-OSGi Java classes and libraries in the following locations. Subdirectories must match the package hierarchy.

If the class implements a single PolicyCenter plugin interface, in Guidewire Studio, you can define a separate plugin directory in the Plugins registry for that interface. In the following paths, *PLUGIN_DIR* represents the plugin directory that you define in the Plugins registry. If the **Plugin Directory** field in the Studio Plugins registry is empty, the default is the plugin directory name shared.

The following special values exist for *PLUGIN_DIR*.

- If your code uses entities:
 - To share code among multiple plugin interfaces, or to share between plugin code and non-plugin code, use the value `shared`.
 - If you are not implementing a plugin interface, use the value `Gosu`. Notice the capitalization of `Gosu`. For example, if the only code that calls your Java code is your own Gosu code, use the value `Gosu`.
- If your code does not use entities:
 - To share code among multiple plugin interfaces, or to share between plugin code and non-plugin code, use the value `shared/basic`.
 - If you are not implementing a plugin interface, use the value `Gosu/basic`. Notice the capitalization of `Gosu`. For example, if the only code that calls your Java code is your own Gosu code, use the value `Gosu/basic`.

The `PolicyCenter/modules/configuration/plugins/Gosu` directory has subdirectories called `gclasses`, `gresources`, and `idea-gclasses`. These subdirectories are for internal use as compilation output paths. Never use, modify, or rely upon the contents of the `gclasses`, `gresources`, and `idea-gclasses` directories.

Place non-OSGi Java classes in the following locations as the root directory of directories organized by package.

```
PolicyCenter/modules/configuration/plugins/PLUGIN_DIR/classes
```

The following example references a class file using the fully qualified name `mycompany.MyClass`.

```
PolicyCenter/modules/configuration/plugins/PLUGIN_DIR/classes/mycompany/MyClass.class
```

Place your Java libraries and any third-party libraries in the following locations.

```
PolicyCenter/modules/configuration/plugins/PLUGIN_DIR/lib
```

Core Java types in Gosu

Because Gosu is built on the JVM, many core Gosu classes are Java types. For example:

- The class `java.lang.String` is the core text object class for Gosu code.
- The basic collection types in Gosu reference the Java versions. For example:

```
java.util.ArrayList
print(list.get(0))
```

Java packages in scope

All types in the package `java.lang` and `java.util` are always in scope. Gosu code that uses types in those packages does not need fully qualified class names or explicit `uses` statements for those types.

For example, the following code is valid Gosu:

```
var f = new java.lang.Float(7.5)
```

The following simpler syntax produces more readable code:

```
var f = new Float(7.5)
```

See also

- “Importing types and package namespaces” on page 114

Using Java types in Gosu

Gosu can access all members of a Java type:

- Variables
- Functions
- Properties

You access these members on a Java class or object in the same way as you access members on a Gosu type, by using dot notation syntax.

For Java methods that look like getters and setters, Gosu exposes the methods also as Gosu properties. For example, you can access a getter method such as `getCurrentTime()` defined in class `MyClass` as Gosu property `MyClass.CurrentTime`.

You can also use reflection to inspect Java types.

See also

- “Java type reflection” on page 360

Using Java `get`, `set`, and `is` methods as Gosu properties

Gosu can call methods on Java types. For methods on Java types that look like getters and setters, Gosu exposes the methods instead as properties. Gosu uses the following rules for methods on Java types:

- If the method name starts with `set` and takes exactly one argument, Gosu exposes it as a setter for a property. The property name matches the original method but without the prefix `set`. For example, if the Java method signature is `setName(String thename)`, Gosu exposes this method as a property setter method for the property called `Name`.
- If the method name starts with `get`, takes no arguments, and returns a value, Gosu exposes it as a getter for a property. The property name matches the original method but without the prefix `get`. For example, if the Java method with signature `getName()` returns a `String` value, Gosu exposes this method as a property getter method for the property named `Name` of type `String`.
- Similar to the rules for `get`, if the method name starts with `is`, takes no arguments, and returns a Boolean value, Gosu exposes it as a property getter. The property name matches the original method but without the prefix `is`. For example, if the Java method with signature `isVisible()` returns a Boolean value, Gosu exposes the method as a property getter method for the Boolean property named `Visible`.
- Gosu makes these transformations on static methods as well as on instance methods.
- Even though Gosu provides a property that you can access for reading and writing information, Gosu does not prevent use of the getter and setter methods. For example, if a Java object has an `getName` method, you can use the expression `obj.Name` or `obj.getName()`.
- If the class has a setter and a getter, Gosu makes the property readable and writable. If the setter is absent, Gosu makes the property read-only. If the getter is absent, Gosu makes the property write-only.

Note: If the Java class has a public variable that has the same name as a property that Gosu converts from getter or setter methods, the class definition is ambiguous to Gosu. Gosu cannot determine whether to use the methods or the variable. For example, if your Java class has a public variable, `x`, and a public method, `setX` that sets the value of the private variable `_x`, the behavior of reading or writing the property, `x` is undefined. You must call the getter and setter methods directly instead of using property syntax. To access the ambiguous variable, use reflection.

Example: Using Java methods as properties in Gosu

This Java class defines multiple methods that Gosu can convert to properties:

```
package doc.example;

public class Circle {
    public static final double PI = Math.PI;
    private double _radius;

    // Constructor #1 - no arguments
    public Circle() {
    }

    // Constructor #2
    public Circle(int dRadius) {
        _radius = dRadius;
    }

    // From Java, these are METHODS that begin with get, set, is.
    // From Gosu, these are PROPERTY accessors.

    public double getRadius() {
        System.out.println("Running Java METHOD getRadius()");
        return _radius;
    }
    public void setRadius(double dRadius) {
        System.out.println("Running Java METHOD setRadius()");
        _radius = dRadius;
    }
    public double getArea() {
        System.out.println("Running Java METHOD getArea()");
        return PI * getRadius() * getRadius();
    }
    public double getCircumference() {
        System.out.println("Running Java METHOD getCircumference()");
        return 2 * PI * getRadius();
    }

    public boolean isRound() {
        System.out.println("Running Java METHOD isRound()");
        return true;
    }

    // The following methods remain methods, and do not convert to properties.

    // For GET/IS, the method must take 0 arguments and return a value
    public void isMethod1() {
        System.out.println("Running Java METHOD isMethod1()");
    }
    public double getMethod2(double a, double b) {
        System.out.println("Running Java METHOD isMethod2()");
        return 1;
    }

    // For SET, the method must take 1 argument and return void
    public void setMethod3() {
        System.out.println("Running Java METHOD setMethod3()");
    }
    public double setMethod4(double a, double b) {
        System.out.println("Running Java METHOD setMethod4()");
        return 1;
    }
}
```

The following Gosu code uses this Java class. Note which Java methods become property accessors and which ones do not.

```
// Instantiate the class with the constructor that takes an argument
var c = new doc.example.Circle(10)

// Use natural property syntax to SET GOSU PROPERTIES. In Java, this was a method.
c.Radius = 10

// Use natural property syntax to GET GOSU PROPERTIES
print("\tRadius: " + c.Radius)
print("\tArea: " + c.Area)
print("\tRound: " + c.Round) // boolean true coerces to String "true"
print("\tCircumference: " + c.Circumference)

// The following assignments cause syntax errors if you uncomment them.
// They are not writable—there is no setter method.
//
// c.Area = 3
// c.Circumference = 4
// c.Round = false

// These Java methods do not convert to properties because
// they have the wrong number of arguments or the wrong types.
c.isMethod1()
var temp2 = c.getMethod2(1,2)
c.setMethod3()
var temp4 = c.setMethod4(8,9)
```

This Gosu code prints the following:

```
Running Java METHOD setRadius()
Running Java METHOD getRadius()
    Radius: 10
Running Java METHOD getArea()
Running Java METHOD getRadius()
Running Java METHOD getRadius()
    Area: 314.1592653589793
Running Java METHOD isRound()
    Round: true
Running Java METHOD getCircumference()
Running Java METHOD getRadius()
    Circumference: 62.83185307179586
Running Java METHOD isMethod1()
Running Java METHOD isMethod2()
Running Java METHOD setMethod3()
Running Java METHOD setMethod4()
```

Using Java static members and static import in Gosu

Gosu supports static members on a type: variables, functions, and property declarations. A static member exists only on the type itself, not on instances of the type. You access static members on Java types just as you would on native Gosu types.

For Gosu code that accesses static members, you qualify the class that declares the static member or import the static members from the class. For example, the following line uses the cosine function and the static reference to value PI from the Math class without importing the static members:

```
var cosHalfPiDgrees = Math.cos(Math.PI * 0.5)
```

To use the static members without qualifying them with the class name, you use the syntax of the `uses` statement that imports static members. For example, the following lines are equivalent to the previous, single line of code:

```
uses java.lang.Math##*
var cosHalfPiDgrees = cos(PI * 0.5)
```

See also

- “Importing static members” on page 115

Using Java generics

Gosu can directly access types that use Java generics. In typical code, Gosu works identically to Java.

The runtime information about a type that uses generics is different for Java and Gosu generic types. You can access this information by using the `typeof` operator:

- The Java compiler erases the parameterization of the type during the compilation to Java byte code. At run time, the type `ArrayList<Integer>` for a Java type appears just as `ArrayList`. This Java behavior is known as type erasure.
- If the type is defined in Gosu, the runtime type retains the parameterization of the generic type. At run time, the type `ArrayList<Integer>` for a Gosu type appears as `ArrayList<Integer>`. This Gosu behavior is known as type reification.

[See also](#)

- “Compatibility of Gosu generics with Java generics” on page 242

Using other Java features in Gosu

You can use many other Java features in Gosu.

Implementing Java interfaces

Gosu classes can directly implement Java interfaces. For example:

```
class MyGosuClass implements Runnable {  
    function run() {  
        // your code here  
    }  
}
```

Gosu also supports its own native interface definitions.

[See also](#)

- “Interfaces” on page 173

Using Java enumerations

Gosu can directly use Java enumerations.

Gosu also supports its own native enumeration definitions.

[See also](#)

- “Enumerations” on page 169

Using Java annotations

Gosu can directly use Java annotations.

Gosu also supports its own native annotation definitions.

[See also](#)

- “Annotations” on page 227

Using Java primitives

Gosu supports the following primitive types: `int`, `char`, `byte`, `short`, `long`, `float`, `double`, `boolean`, and the special value `null` that means an empty object value. These primitives are the full set that Java supports. The Gosu versions of these primitives are fully compatible with the Java primitives, in both directions.

Additionally, every Gosu primitive type, other than the special value `null`, has an equivalent object type defined in Java. Java does the same for its primitive types. For example, for `int` there is the `java.lang.Integer` type that descends from the `Object` class.

- The object types that represent the equivalent of primitive types are called *boxed primitive* types.
- Primitive types are also called *unboxed primitives*.

The boxed primitive Java classes can be freely used in Gosu code because of Gosu's special relationship to the Java language.

In most cases, Gosu converts between boxed and unboxed primitives as needed. However, just as in Java, the differences between the boxed and unboxed types are important.

In both Gosu and Java, the language primitive types like `int` and `boolean` work differently from objects, which are descendents of the root `Object` class. For example:

- You can add objects to a collection, but you cannot add primitives.
- Variables of an object type can have the value `null`, but variables of a primitive type cannot.

Gosu provides *autoboxing*, which is to convert values from unboxed to Java-based boxed types as required by a method parameter or return value. Similarly, Gosu provides *unboxing*, which is to convert values from boxed to unboxed types.

In most cases, Gosu handles differences between boxed and unboxed types for you by automatically converting values as required. For example, Gosu implicitly converts between the native language primitive type called `boolean` and the Java-based object class `Boolean` (`java.lang.Boolean`). If you want explicit coercion, use the `as ...NEWTYPE` syntax, such as `myIntValue as Integer`.

If your code implicitly converts a variable's value from a boxed type to a unboxed type and the value is `null`, Gosu standard value type conversion rules apply. For example:

```
var bBoxed : Boolean
var bUnboxed : boolean

bBoxed = null           // bBoxed can genuinely be NULL
bUnboxed = bBoxed       // bUnboxed cannot be null, so is converted to FALSE
```

Using a Java properties file

Gosu provides a utilities class, `gw.util.PropertiesFileAccess`, that supports parsing a Java properties file. By using this class, you can access the values of the properties in a straightforward way.

You use the `getProperties` method of the `PropertiesFileAccess` class to load the properties from the file as a Java Properties object. This method takes the full path and name of the properties file as a `String` parameter. For example, if your properties file is `GlobalProperties.properties` in `com.mycompany.properties`, you pass `com/mycompany/properties/GlobalProperties.properties` to `getProperties`.

You then use the Java `getProperty` method on the Properties object to get each Java Property object. You can convert the Java Property object into a static Gosu property.

Example: Accessing Java properties in Gosu

Consider the following Java properties file, `MyProps.properties`, which is located in the `gsrc.doc.examples` folder in Guidewire Studio.

```
# doc.examples.MyProps

# The hash character as the first char means that the line is a comment
! The exclamation mark character as the first char means that the line is a comment

website = http://guidewire.com/
transport = bicycle

# The backslash below tells the application to continue reading
# the value onto the next line.
message = Welcome to \
        Gosu!
```

```
# Unicode support
tab : \u0009

# multiple levels of hierarchy for the key
gosu.example.properties.Key1 = Value1
```

The following Gosu class converts some of the Java properties into static Gosu properties.

```
package doc.examples

uses gw.util.PropertiesFileAccess

class MyPropsAccess {
    static property get MyProperties() : Properties {
        // Using getProperties reads the file only once. The Properties are cached as a static property.
        // You must use the fully qualified path to the properties file even though MyPropsAccess and
        // MyProps.properties are in the same folder.
        return PropertiesFileAccess.getProperties("doc/examples/MyProps.properties")
    }

    static property get Website() : String {
        return MyProperties.getProperty("website")
    }

    static property get Transport() : String {
        return MyProperties.getProperty("transport")
    }

    static property get Message() : String {
        return MyProperties.getProperty("message")
    }
}
```

The following code reads the static Gosu properties.

```
// Get static properties
print(" website: ${MyPropsAccess.Website}")
print("transport: ${MyPropsAccess.Transport}")
print(" message: ${MyPropsAccess.Message}")

// Use methods on the MyProperties static property to access other property values
print(" tab: before${MyPropsAccess.MyProperties.getProperty("tab")}after")
// Access a multi-level property
print("multi-level: ${MyPropsAccess.MyProperties.getProperty("gosu.example.properties.Key1")}")
```

Running this code produces the following output.

```
website: http://guidewire.com/
transport: bicycle
message: Welcome to Gosu!
tab: before after
multi-level: Value1
```


Classes

Gosu classes encapsulate data and code to perform a specific task. Typical use of a Gosu class is to encapsulate a set of Gosu functions and a set of properties to store within each class instance. A *class instance* is a new in-memory copy of the object of that class. If some Gosu code creates a new instance of the class, Gosu creates the instance in memory with the type matching the class you instantiated. You can manipulate each object instance by getting or setting properties. You can also invoke the Gosu functions of the class. Gosu functions defined in a class are also called *methods*.

You can *extend* an existing class, which means to make a subclass of the class with new methods or properties or different behaviors than existing implementations in the superclass.

Gosu classes are analogous to Java classes in that they have a package structure that defines the namespace of that class within a larger set of names. For example, if your company is called Smith Company and you were writing utility classes to manipulate addresses, you might create a new class called `NotifyUtils` in the namespace `smithco.utilities`. The fully qualified name of the class would be `smithco.utilities.NotifyUtils`.

You can write your own custom classes and call these classes from within Gosu, or call built-in classes. You create and reference Gosu classes by name just as you would in Java. For example, suppose you define a class called `Notification` in package `smithco.utilities` with a method (function) called `getName()`.

You can create an instance of the class and then call a method like this:

```
// Create an instance of the class
var myInstance = new smithco.utilities.Notification()

// Call methods on the instance
var name = myInstance.getName()
```

You can also define data and methods that belong to the class itself, rather than an instance of the class. You use this technique, for instance, to define a library of functions of similar purpose. The class encapsulates the functions but you never need to create an instance of the class. You can create static methods on a class independent of whether any code creates an instance of the class. Gosu does not force you to choose between the two design styles.

You can write Gosu classes that extend from Java classes. Your class can include Gosu generics features that reference or extend Java classes or subtypes of Java classes.

Guidewire Studio

To create and manage Gosu classes, Guidewire recommends that you always use Guidewire Studio.

See also

- “Static modifier” on page 162
- “Generics” on page 241

- *Configuration Guide*

Creating and instantiating classes

In Studio, create a class by right-clicking a package name on the **Project** pane, and then clicking **New→Gosu Class**. Studio creates a simple class in which you build the functionality that you need. Studio creates the package name and class definition.

After creating a new class, add constructors, class variables, properties, and functions to the class. All of these components are optional. Within a class, functions are also called *methods*. This term is standard object-oriented terminology. This documentation refers to functions as methods if the functions are part of classes.

Add variables to a class with the `var` keyword:

```
var myStringInstanceVariable : String
```

You can optionally initialize the variable:

```
var myStringInstanceVariable = "Butter"
```

Define methods with the keyword `function` followed by the method name and parentheses that enclose a parameter list, which is empty if the method has no arguments. The parameter list uses commas to separate multiple parameters. Each parameter has the format:

```
parameterName : typeName
```

If the method returns a value, after the parentheses, put a colon (`:`) and the return type. If the method does not return a value, specifying `void` as the return type is optional.

For example, the following line declares a method that has a single `String` parameter. The method does not return a value:

```
function doAction(arg1 : String)
```

A Gosu class with one instance variable and one public method looks like the following:

```
class MyClass {
    var myStringInstanceVariable : String

    public function doAction(arg1 : String) {
        print("Someone just called the doAction method with arg " + arg1)
    }
}
```

Extending another class to make a subclass

You can extend another class to inherit methods, fields, and properties from a parent class by using the `extends` keyword after your class name, followed by the parent class name. For example:

```
class Tree extends Plant {
}
```

Your class is a subclass and subtype of the parent class. The parent class is a superclass and supertype of your class. When subclassing a parent class or implementing an interface, you can override the methods defined in the parent class or interface.

To call the method that a parent class defines, use the `super` keyword using the following syntax:

```
super.method
```

For example, if a parent class defines a no-argument method `makeJourney`, the subclass can call the parent implementation like the following line:

```
super.makeJourney()
```

To call a default method that an interface defines, use the `super` keyword using the following syntax:

```
super[interface].method
```

For example, if an interface `Travel` defines a no-argument method `comeHome` that contains default code, the implementing class can call the default implementation like the following line:

```
super[Travel].comeHome()
```

Note: When overriding the `equals` method for objects, which is a common technique for overriding the `==` operator, you must also override the `hashCode` method. You must use logic ensuring that the return value for `a.equals(b)` always matches the return value for `a.hashCode() == b.hashCode()`.

In the following example, the `Plant` class is the parent of the `Tree` subclass:

```
class Plant {
    var _type = ""
    property get Type() : String {
        return _type
    }
    property set Type(value:String) : void {
        _type = value
    }
}

class Tree extends Plant {
    override property set Type(value:String) : void {
        _type = "Plant type: " + value
    }
}
```

In this example, if `myPlant` is an object of type `Plant`, running `myPlant.Type = "sunflower"` sets the private variable `_type` to `"sunflower"`.

If `myTree` is an object of type `Tree`, which has overridden the `Plant.Type` property setter, running `myPlant.Type = "sunflower"` sets the private variable `_type` to `"Plant type: sunflower"`.

Constructors

A Gosu class can have a *constructor*, which is a special method that Gosu calls after creating an instance of the class. For example, the Gosu code `new MyClass()` calls the `MyClass` constructor for initialization or other actions. To create a constructor, name the method `construct`. Do not use the function keyword. If you do not need a class constructor, you do not need to create the `construct` method.

Note: Constructors have public scope by default.

For example, the following class contains one constructor that takes no arguments:

```
package doc.example

class Tree {
    construct() {
        print("I just created a Tree object!")
    }
}
```

You can create constructors that take arguments, in the same way as for other methods. Any code that instantiates that object can pass arguments to the `new` expression. For example, the following class contains a constructor requiring a parameter:

```
package doc.example

class Tree {
    var _name : String // A private class var

    // A public constructor
    construct(name : String) {
        _name = name
        print("I just created a ${_name} Tree object!")
    }
}
```

The following code instantiates an object by using this constructor:

```
var t = new Tree("Ginkgo")
```

Running that code produce the following message:

```
I just created a Ginkgo Tree object!
```

If the class constructors all take one or more arguments, any code that instantiates the class must provide a parameter list. Failing to pass arguments to a new expression for this class produces a compilation error.

Accessing the constructor of a parent class

If your class extends another class, your constructor can call the constructor of its parent class by using the `super` keyword, followed by parentheses. This statement must be the first statement in the subclass constructor. For example, the following code calls the parent-class constructor, and then performs additional work.

```
package doc.example

class Tree extends Plant {
  construct() {
    super()
    print("I just created a Tree object!")
  }
}
```

If you call `super()` with no arguments between the parentheses, Gosu calls the no-argument constructor of the parent class. If you call `super(parameter_list)`, Gosu calls the parent-class constructor that matches the provided parameter list. You can call a parent-class constructor with different number of arguments or different types than the current constructor.

For every constructor that you write, Gosu always calls a version of the parent-class constructor, even if you omit an explicit reference to `super()`. If your class does not call the superclass constructor explicitly, Gosu attempts to use the no-argument superclass constructor:

- If a no-argument constructor exists in the parent class, Gosu calls that constructor, which is equivalent to the code `super()`.
- If a no-argument constructor does not exist in the parent class, Gosu throws a `no default constructor` compilation syntax error.

Static methods and variables

If you want to call a method directly on the class itself rather than an instance of the class, you can provide a static method. To declare a method as static, add the keyword `static` before the function declaration. You use similar syntax to make a variable static. For example, this Gosu code declares an instance method:

```
public function doAction(arg1 : String)
```

This Gosu code declares a static method:

```
static public function doAction(arg1 : String)
```

Gosu does not support a class having both an instance method and a static method that have the same name and argument list.

Gosu classes can also contain static values.

[See also](#)

- “Static modifier” on page 162

Using public properties instead of public variables

Although Gosu supports public variables for compatibility with other languages, best practice is to use public properties backed by private variables instead of using public variables. To define the public property name to use to access the data, you use the syntax as `PROPERTY_NAME` at the end of the declaration.

For example, to declare a public property, use this style of variable declaration:

```
private var _firstName : String as FirstName
```

Do not declare a public field:

```
public var FirstName : String // Do not do this. Public variables are not standard Gosu style
```

See also

- “Properties” on page 150

Creating a new instance of a class

Typically, you define a class, and then create one or more instances of that class. An instance of a class is also known as an *object*. Each instance has its own set of associated data. The process of constructing a new instance is called *instantiating* a class. To instantiate a class, use the new operator:

```
var e = new smithco.messaging.QueueUtils() // Instantiate a new instance of QueueUtils.
```

You can use object initializers to set properties on an object immediately after a new expression. Use object initializers for compact and clear object declarations. Object initializers are especially useful in combination with data structure syntax and nested objects. A simple use of an object initializer looks like the following line:

```
var sampleClaim = new Claim() {ClaimId = "TestID"} // Initialize the ClaimID on the new claim.
```

Note: You can use static methods, static variables, and static properties of a Gosu class without creating an instance of the class.

See also

- “Using the operator new in object expressions” on page 83
- “Static modifier” on page 162

Naming conventions for packages and types

The package name is the namespace for the class, interface, enhancement, enumeration, or other type. Defining a package prevents ambiguity about what class is accessed.

The following restrictions apply to package names.

- Package names use only lowercase characters.
- The root package name must not be a Gosu or Java strong keyword, such as `int`.
- Any subsequent part of the package name can be any name that conforms to Gosu naming conventions. You can use any Gosu or Java strong keyword. If you use a Java strong keyword, Studio displays a warning message that you will not be able to create classes in the package. This warning applies only to Java classes. You can create a Gosu class in the package.

Class names or other type names must always start with a capital letter. Type names can contain additional capital letters later in the name for clarity. If you write Gosu enhancements, the naming rules are slightly different from the conventions for other types.

Use the following standard package naming conventions:

Type of class	Package	Example of fully qualified class name
Classes you define	<i>companyname.subpackage</i>	smithco.messaging.QueueUtils
Guidewire Professional Services classes	gwservices.subpackage	gwservices.messaging.QueueUtils

WARNING Your types must never use a package name with the prefix `com.guidewire.` or the prefix `gw.` Those package namespaces are reserved for internal types.

See also

- “Importing types and package namespaces” on page 114
- “Enhancement naming conventions” on page 217

Using uses declarations to import a type

You can add `uses` declarations to your Gosu type to import types so that you can refer to them later without specifying the fully qualified name. For example, by including the following declaration, you can refer to the `FileInputStream` class as `FileInputStream`, rather than `java.io.FileInputStream`:

```
uses java.io.FileInputStream
```

All `uses` declarations must be at the top of the class, not within a Gosu method definition.

See also

- “Importing types and package namespaces” on page 114
- “Importing static members” on page 115

Properties

Properties are dynamic and virtual functions that behave like data. Gosu classes can define properties to provide access to a property value similarly to variables on the class. Other objects use the intuitive syntax of the period symbol (`.`) to access the properties. You can use Gosu code to implement get and set functionality for a property. The simplest code that gets or sets a property value uses an instance variable. You can also implement properties by writing getter and setter methods.

Getting and setting properties

To get and set properties on an object, use the period symbol just as for getting and setting standard variables. For example, the following code sets and gets properties on a `MyClass` object that has `Field1` and `Field2` properties:

```
// Create a new class instance
var a = new MyClass()

// Set a property value
a.Field1 = 5

// Get a property value
print(a.Field2)
```

Static properties

You use static properties directly on the class without creating an instance of the class.

For example, if the `MyClass` class has a static property, `BaseURL`, you access its value as shown in the following code:

```
var myURL = MyClass.BaseURL + "/destination"
```

The following lines are invalid:

```
// Create a new class instance
var b = new MyClass()

// Attempt to access a static property on the instance
myURL = b.BaseURL + "/destination" // This line does not compile
```

See also

- “Static modifier” on page 162

Defining properties

Standard Gosu style is to use public properties backed by private variables instead of using public variables.

Do not expose public variables in your classes, such as the following:

```
public var FirstName : String // Incorrect
```

Instead, write your Gosu classes to declare properties similar to the following line. This concise code defines the `FirstName` property and stores its value in the `_firstName` variable:

```
private var _firstName : String as FirstName
```

Gosu variable alias syntax provides this shortcut to declare an instance variable as a property. *Variable alias* syntax uses the `as` keyword followed by the property name to make simple getter and setter property methods backed by an class instance variable. Use this approach to specify a direct mapping between a publicly visible property and the private variable that contains the property's value. A property defined in this way is both readable and writable.

Although the data is stored in private variable `_firstName`, code that accesses this data does not access the private instance variable directly. Any code that needs to use the data uses the period symbol (`.`) and the property name:

```
var e = new Employee()
e.FirstName = "Pam" // Sets to "Pam" by calling the property setter
var g = e.FirstName // Calls the property getter
```

Code within a class does not need to use the property name to access a property on the same class. Classes can access their own private variables. In the previous example, other methods in that class could reference the `_firstName` variable rather than using the property accessor `FirstName`.

Writing property getter and setter methods explicitly

If you need to perform additional processing before storing or retrieving property values, you can write either or both of the getter and setter methods for the property. This form of a property definition is like a method that uses the keywords `property get` or `property set` before the method name instead of `function`. The property getter method must take zero parameters and the property setter method must take exactly one parameter.

Your property getter and setter methods can perform complex calculations or store the data in some other way than as a variable.

For example, the following code makes sure that the `FullName`, `FirstName`, and `LastName` properties are consistent in the setter methods for the `FirstName`, and `LastName` properties. The code also checks whether a user is authorized to access a social security number in the getter and setter methods for the `SSN` property.

```
package doc.example

uses gw.api.util.StringUtil

class Employee {
    var _fullName : String as readonly FullName
    var _firstName : String as FirstName
    var _lastName : String as LastName
    var _ssn : String as SSN

    construct(name : String) {
        setAllNames(name)
    }

    property get SSN() : String {
        return (isUserAuthorized()) ? _ssn : "Not authorized to see SSN"
    }

    property set SSN(ssn : String) {
        if (isUserAuthorized()) {
            _ssn = ssn
        } else {
            print("Not authorized to set SSN")
        }
    }
}
```

```

    }

    property set FirstName(fName : String) {
        setAllNames(fName + " " + _lastName)
    }

    property set LastName(lName : String) {
        setAllNames(_firstName + " " + lName)
    }

    private function setAllNames(name : String) {
        var allNames = StringUtil.splitWhitespace(name)

        _firstName = allNames[ 0]
        _lastName = allNames[allNames.Count - 1]
        _fullName = _firstName + " " + _lastName
    }

    private function isUserAuthorized() : boolean {
        // Dummy method
        return false
    }
}

```

The following code demonstrates the name and social security number properties:

```

var emp = new Employee("Pam Trujillo")
emp.SSN = "123456789"
print("First name:\t${emp.FirstName}")
print("Last name:\t${emp.LastName}")
print("Full name:\t${emp.FullName}")
print("SSN:\t${emp.SSN}")

```

This code produces the following output:

```

Not authorized to set SSN
First name:    Pam
Last name:     Trujillo
Full name:     Pam Trujillo
SSN:          Not authorized to see SSN

```

Read-only and write-only properties

By default, properties are both readable and writable. You can make a property read-only by adding the keyword `readonly` before the property name, as shown in the following code:

```

var _fullName : String as readonly FullName

```

Gosu does not provide a keyword to make a write-only property in this shortcut syntax.

If you need to use getter and setter methods to perform additional processing for the property, you make a read-only property by writing only the getter method. You use a different code path, such as the class constructor, to set the value of the property. Similarly, you can make a write-only property by writing only the setter method, as shown in the following code:

```

var _secretID : int

property set SecretID(max : int) {
    _secretID = (Math.random() * max) as int
}

```

Overriding a property definition in a subclass

In contrast to standard instance variables, `property get` and `property set` methods are virtual. You can override *virtual* property methods in subclasses and implement them from interfaces. You can even call the superclass's `property get` or `property set` methods.

Example

The following code is the superclass.


```
class TemperatureInCelsius {
    protected var _temp : float as Temperature

    property get FreezingText () : String {
        return (_temp <= 0 ? "freezing!" : "NOT freezing!")
    }

    public function howHot() {
        print ("In Celsius it is ${_temp} and it is ${FreezingText}")
    }
}
```

The following code shows how you override a property in a subclass.

```
// Subclass for Kelvin (which is Celsius - 273)
class TemperatureInKelvin extends TemperatureInCelsius {

    property get TemperatureInCelsius() : float {
        return super.Temperature
    }
    override property get Temperature() : float {
        return super.Temperature + 273 // Convert Celsius to Kelvin
    }
    override property set Temperature(t : float ) {
        super.Temperature = t - 273 // Convert Kelvin to Celsius
    }
}
```

The overridden property `get` method first calls the implicitly defined property `get` method from the superclass, which gets the class variable `_temp`, and then adds a value. This property `get` method does not change the value of the class variable `_temp`, but code that accesses the `Temperature` property from the subclass gets a different value.

To test overriding a property definition in a subclass, create the classes in the previous code. Next, run the following code in the Gosu Scratchpad:

```
var cTemp = new TemperatureInCelsius()
var kTemp = new TemperatureInKelvin()

cTemp.Temperature = 123
kTemp.Temperature = 123

cTemp.howHot()
kTemp.howHot()

print("Temperature in Kelvin: ${kTemp.Temperature}")
print("Temperature in Celsius: ${kTemp.TemperatureInCelsius}")
```

This code prints:

```
In Celsius it is 123 and it is NOT freezing!
In Celsius it is -150 and it is freezing!
Temperature in Kelvin: 123
Temperature in Celsius: -150
```

Property paths are null tolerant

One notable difference between Gosu and some other languages is that property accessor paths in Gosu are null-tolerant, also called null-safe. Only expressions separated by period (.) characters that access a series of instance variables or properties support null safety, such as the following form:

```
obj.PropertyA.PropertyB.PropertyC
```

In most cases, if any object to the left of the period character is null, Gosu does not throw a null-pointer exception (NPE) and the expression returns null. Gosu null-safe property paths tend to simplify real-world code. Often, a null expression result has the same meaning whether the final property access is null or earlier parts of the path are null. For such cases in Gosu, do not check for the null value at every level of the path. This conciseness makes your Gosu code easier to read and understand.

For example, suppose you have a variable called `house`, which contains a property called `walls`, and that object has a property called `windows`. The syntax to get the `windows` value is:

```
house.Walls.Windows
```

In some languages, you must be aware that if `house` is `null` or `house.Walls` is `null`, your code throws a null-pointer exception. The following common coding pattern avoids the exception:

```
// Initialize to null
var x : ArrayList<Windows> = null

// Check earlier parts of the path for null to avoid a null-pointer exceptions (NPE)
if (house != null and house.Walls != null) {
    x = house.Walls.Windows
}
```

In Gosu, if earlier parts of a pure property path are `null`, the expression is valid and returns `null`. The following Gosu code is equivalent to the previous example and a null-pointer exception never occurs:

```
var x = house.Walls.Windows
```

By default, method calls are not null-safe. If the right side of a period character is a method call, Gosu throws a null-pointer exception (NPE) if the value on the left side of the period is `null`.

For example:

```
house.myaction()
```

If `house` is `null`, Gosu throws an NPE. Gosu assumes that method calls might have side effects, so Gosu cannot skip the method call and return `null`.

Gosu provides a variant of the period operator that is always explicitly null-safe for both property access and method access. The null-safe period operator has a question mark before the period: `?.`

If the value on the left of the `?.` operator is `null`, the expression evaluates to `null`.

For example, the following expression evaluates left-to-right and contains three null-safe property operators:

```
obj?.PropertyA?.PropertyB?.PropertyC
```

Null-safe method calls

A *null-safe method call* does not throw an exception if the left side of the period character evaluates to `null`. Gosu just returns `null` from that expression. Using the `?.` operator calls the method with null safety:

```
house?.myaction()
```

If `house` is `null`, Gosu does not throw an exception. Gosu returns `null` from the expression.

Null-safe versions of other operators

Gosu provides null-safe versions of other common operators:

- The null-safe default operator (`? :`). You use this operator to specify an alternative value if the value to the left of the operator evaluates to `null`. For example:

```
var displayName = Book.Title ? : "(Unknown Title)" // Return "(Unknown Title)" if Book.Title is null
```

- The null-safe index operator (`?[ ]`). Use this operator with lists and arrays. The expression returns `null` if the list or array value is `null` at run time, rather than throwing an exception. For example:

```
var book = bookshelf?[bookNumber] // Return null if bookshelf is null
```

- The null-safe math operators (`?+`, `?-`, `?*`, `?/`, and `?%`). For example:

```
var displayName = cost ?* 2 // Multiply by 2, or return null if cost is null
```

Design code for null safety

Use null-safe operators where appropriate. These operators make code more concise and simplify the handling of edge cases.

You can design your code to take advantage of this special language feature. For example, expose data as properties in Gosu classes and interfaces rather than as setter and getter methods.

IMPORTANT Expose public data as properties rather than as getter functions. By using a property, you to take advantage of Gosu null-safe property accessor paths. Standard Gosu practice is to separate your implementation from your class's interaction with other code by using properties rather than public instance variables. Gosu provides the `as` keyword to expose an instance variable as a property.

See also

- “Handling null values in expressions” on page 94

Designing APIs around null-safe property paths

You may want to design your Gosu code logic around the null-safety feature. For example, Gosu uses the `java.lang.String` class as its native text class. This class includes a built-in method to check whether the `String` is empty. The method is called `isEmpty`, and Gosu exposes this method as the `Empty` property. Use of this property is difficult with Gosu property accessor paths. For example, consider the following `if` statement:

```
if (obj.StringProperty.Empty)
```

Because `null` coerces implicitly to the type `Boolean`, which is the type of the `Empty` property, the expression evaluates to `false` in either of the following cases:

- If `obj.StringProperty` has the value `null`.
- The value of `StringProperty` is non-null but its `Empty` property evaluates to `false`.

In typical code, distinguishing between these two very different cases is important. For example, if you need to use the value `obj.StringProperty` only if the value is non-empty, just checking the value `obj.StringProperty.Empty` is insufficient.

To work around this restriction, Gosu adds an enhancement property to `java.lang.String` called `HasContent`. This property effectively has the reverse of the logic of the `Empty` property. The `HasContent` property only returns `true` if the property has content, so you can use property accessor paths such as the following line:

```
if (obj.StringProperty.HasContent)
```

Because `null` coerces implicitly to the type `Boolean`, which is the type of the `Empty` property, the expression evaluates to `false` in either of the following cases:

- If `obj.StringProperty` is `null`.
- The `String` is non-null but the string has no content and so its `HasContent` property evaluates to `false`.

These cases are much more similar semantically than the variant that uses `Empty` (`obj.StringProperty.Empty`).

Be sure to consider null-safety of property paths as you design your code, particularly with `Boolean` properties.

More property examples

The following examples demonstrate how to create and use Gosu class properties and their getter and setter methods.

The examples use two classes, one of which extends the other.

The class `myFirstClass`:

```
package mypackage

class MyFirstClass {

    // Explicit property getter for Fred
    property get Fred() : String {
        return "myFirstClass"
    }
}
```

The class `mySecondClass`:

```
package mypackage
```

```
class MySecondClass extends MyFirstClass {

    // Exposes a public F0 property on _f0
    private var _f0 : String as F0

    // Exposes a public read-only F1 property on _f1
    private var _f1 : String as readonly F1

    // Simple variable with explicit property get/set methods
    private var _f2 : String

    // Explicit property getter for _f2
    property get F2() : String {
        return _f2
    }

    // Explicit property setter for _f2, visible only to classes in this package
    internal property set F2( value : String ) {
        _f2 = value
    }

    // A simple calculated property (not a simple accessor)
    property get Calculation() : Number {
        return 88
    }

    // Overrides MyFirstClass's Fred property getter
    property get Fred() : String {
        return super.Fred + " suffix"
    }
}
```

Create the classes, and then try the following lines in Gosu Scratchpad to test these classes.

First, create an instance of your class:

```
var test = new mypackage.MySecondClass()
```

Assign a property value. This code internally calls a hidden method to assign "hello" to variable `_f0`:

```
test.F0 = "hello"
```

The following line is invalid because `f1` is read-only:

```
// This line gives a compile error.
test.F1 = "hello"
```

Get a property value. This code indirectly calls the `mySecondClass` property getter function for `F2`:

```
print( test.F2 ) // Prints null because the property is not set yet
```

The following line is invalid because `F2` is not visible outside of the package namespace of `MySecondClass`. The `F2` property is publicly read-only.

```
// This line gives a compile error.
test.F2 = "hello"
```

Print the `Calculation` property:

```
print( test.Calculation ) // Prints 88
```

The following line is invalid because `Calculation` is read-only. This property does not have a setter function:

```
// This line gives a compiler error.
test.Calculation = 123
```

Demonstrate that properties can be overridden through inheritance because properties are virtual:

```
print( test.Fred ) // Prints "myFirstClass suffix"
```

Modifiers

In Gosu, a modifier is a specific keyword that you insert in front of a Gosu type, method, or variable to modify the behavior of that Gosu component. Gosu supports the following types of modifiers:

- Access modifiers
- Override modifiers
- Final modifiers
- Static modifiers

Access modifiers

You can use access modifier keywords to set the level of access to a Gosu class, interface, enumeration, or a type member, which is a function, variable, or property. The access level determines whether other classes can use a particular variable or invoke a particular function.

For example, methods and variables marked `public` are visible from other classes in the package. Additionally, because they are public, functions and variables also are visible to all subclasses of the class and to all classes outside the current package. For example, the following code uses the `public` access modifier on a class variable:

```
package com.mycompany.utils

class Test1 {
    public var Name : String
}
```

In contrast, the `internal` access modifier lets the variable be accessed only in the same package as the class:

```
package com.mycompany.utils

class Test2 {
    internal var Name : String
}
```

For example, another class with fully qualified name `com.mycompany.utils.Test2` could access the `Name` variable because it is in the same package. Another class `com.mycompany.integration.Test3` cannot see the `Test.Name` variable because it is not in the same package.

Similarly, modifiers can apply to an entire type, such as a Gosu class:

```
package com.mycompany.utils

internal class Test {
    var Name : String
}
```

Some access modifiers apply only to type members (functions, variables, properties, and inner types). Other access modifiers apply to both type members and top-level types (outer Gosu classes, interfaces, enumerations). The following table lists the Gosu access modifiers and their applicability and visibility:

Modifier	Description	Applies to top-level types	Applies to type members	Visible in class	Visible in package	Visible in subclass	Visible to all
<code>public</code>	Fully accessible with no restrictions	Yes	Yes	Yes	Yes	Yes	Yes
<code>protected</code>	Accessible only to types in the same package and subtypes	--	Yes	Yes	Yes	Yes	--
<code>internal</code>	Accessible only in same package	Yes	Yes	Yes	Yes	--	--
<code>private</code>	Accessible only in the declaring type, such as the Gosu class or interface that defines it	--	Yes	Yes	--	--	--

If you do not specify a modifier, Gosu uses the following default access levels:

Element	Default modifier
Types and classes	public
Variables	private
Functions	public
Properties	public

Coding style recommendations for variables

Always prefix private and protected class variables with an underscore character (`_`).

Avoid public variables. If you want to use public variables, convert the public variables to properties. This conversion separates the way code in other classes accesses the properties from the implementation of the storage and retrieval of the properties.

See also

- “Coding style” on page 219

Override modifier

Apply the override modifier to a function or property implementation to declare that the subtype overrides the implementation of an inherited function or property with the same signature.

For example, the following line might appear in a subtype overriding a `myFunction` method in its superclass:

```
override function myFunction(myParameter : String )
```

If Gosu detects that you are overriding an inherited function or method with the same name but you omit the override keyword, you get a compiler warning. Additionally, the Gosu editor offers to insert the modifier if the override keyword seems appropriate.

Abstract modifier

The abstract modifier indicates that a type is intended only to be a base type of other types. Typically, an abstract type does not provide implementations, which contain code to perform the function, for some or all of its functions and properties. This modifier applies to classes, interfaces, functions, and properties.

For example, the following code is a simple abstract class:

```
abstract class Vehicle {
}
```

If a type is specified as abstract, Gosu code cannot construct an instance of that type. For example, you cannot use code such as `new MyType()` for an abstract type. You can instantiate a subtype of the abstract type if the subtype fully implements all abstract members (functions and properties). A subtype that contains implementations for all abstract members of its supertype is called a *concrete type*.

For example, if class A is abstract and defines one method’s parameters and return value but does not provide code for the method, that method would be declared `abstract`. Another class B could extend A and provide code to implement that abstract method. Class A is the abstract class and class B is a concrete subclass of A.

For example, the following code defines an abstract Gosu class called `Vehicle`, which contains members but no abstract members:

```
package com.mycompany

abstract class Vehicle {
    var _name : String as Name
}
```

You cannot construct an instance of this class, but you can define another class that extends the class:

```
package com.mycompany
```

```
class Truck extends Vehicle {  
    // The subtype can add its own members.  
    var _TruckLength : int as TruckLength  
}
```

You can now use code such as the following to create an instance of `Truck`:

```
var t = new Truck()
```

Things work differently if the supertype, in this case, `Vehicle`, defines abstract members. If the supertype defines abstract methods or abstract properties, the subtype must define a concrete implementation of each abstract method or property to support instantiation of the subclass. A concrete method implementation must implement actual behavior, not just inherit the method signature. A concrete property implementation must implement actual behavior of getting and setting the property, not just inherit the property name.

The subtype must implement an abstract function or abstract property using the same name as the supertype. Use the `override` keyword to tell Gosu that the subtype overrides an inherited function or method of the same name. If you omit the `override` keyword, Gosu displays a compiler warning. Additionally, the Gosu editor offers to insert the `override` modifier if the `override` keyword seems appropriate.

For example, the following code expands the `Vehicle` class with abstract members:

```
package com.mycompany  
  
abstract class Vehicle {  
    // An abstract property -- every concrete subtype must implement this!  
    abstract property get Plate() : String  
    abstract property set Plate(newPlate : String)  
  
    // An abstract function/method -- every concrete subtype must implement this!  
    abstract function RegisterWithDMV(registrationURL : String)  
}
```

A concrete subtype of this `Vehicle` class looks like the following code:

```
package com.mycompany  
  
class Truck extends com.mycompany.Vehicle {  
    var _TruckLength : int as TruckLength  
  
    /* Create a class instance variable that uses the "as ..." syntax to define a property.  
     * This variable provides a concrete implementation of the abstract property "Plate".  
     */  
    var _licenseplate : String as Plate  
  
    /* Implement the function RegisterWithDMV, which is abstract in the supertype, which  
     * does not define how to implement the method, but does specify the method signature  
     * that you must implement to support instantiation with "new".  
     */  
    override function RegisterWithDMV(registrationURL : String ) {  
        // Here do whatever needs to be done  
        print("Simulation of registering " + _licenseplate + " to " + registrationURL)  
    }  
}
```

You can now construct an instance of the concrete subtype `Truck`, even though you cannot directly construct an instance of the supertype `Vehicle` because that type is abstract.

Create the classes, and then try the following lines in Gosu Scratchpad to test these classes.

```
var t = new com.mycompany.Truck()  
t.Plate = "ABCDEFGF"  
print("License plate = " + t.Plate)  
t.RegisterWithDMV( "http://dmv.ca.gov/register" )
```

This code prints the following lines:

```
License plate = ABCDEFG  
Simulation of registering ABCDEFG to http://dmv.ca.gov/register
```

Final modifier

The `final` modifier applies to types (including classes), type members (variables, properties, methods), local variables, and function parameters.

The `final` modifier specifies that the value of a property, local variable, or parameter cannot be modified after the initial value is assigned. The `final` modifier cannot be combined with the `abstract` modifier on anything. These modifiers are mutually exclusive. The `final` modifier implies that there is a concrete implementation and the `abstract` modifier implies that there is no concrete implementation.

Final types

If you use the `final` modifier on a type, the type cannot be inherited. For example, if a Gosu class is `final`, you cannot create a subclass of that class.

The `final` modifier is implicit with enumerations. An *enumeration* is an encapsulated list of enumerated constants. Enumerations are implemented like Gosu classes in most ways. No Gosu code can subclass an enumeration.

See also

- “Enumerations” on page 169

Final class variables

Using the `final` keyword on class variables specifies that the variable can be set only once, and only in the declared class. Subclasses cannot set the value of a final class variable.

For example:

```
class TestABC {
    final var _name : String = "John"
}
```

Optionally, you can use the `final` keyword on a class variable declaration without immediately initializing the variable. If you do not immediately initialize the variable in the same statement, all class constructors must initialize that variable in all possible code paths.

For example, the following syntax is valid because all constructors initialize the final variable once in each code path:

```
class TestABC {
    final var _name : String

    construct() {
        _name = "hello"
    }
    construct(b : boolean){
        _name = "there"
    }
}
```

The following code is invalid because one constructor does not initialize the final variable:

```
class TestABC {
    final var _name : String // INVALID CODE, ALL CONSTRUCTORS MUST INITIALIZE _name IN ALL CODE PATHS.

    construct() { // does not initialize the variable
    }

    construct(b : boolean){
        _name = "there"
    }
}
```


Final functions and properties

If you use the `final` modifier on a function or a property, the `final` modifier prevents a subtype from overriding that item. For example, a subclass of a Gosu class cannot reimplement a method that the superclass declares as `final`.

For example, the following code defines a class with a final function and final properties:

```
package com.mycompany

class Auto {

    // A final property -- no subtype can reimplement or override this property!
    final property get Plate() : String
    final property set Plate(newPlate : String)

    // A final function/method -- no concrete subtype can reimplement or override this function!
    final function RegisterWithDMV(registrationURL : String)
}
```

See also

- “Properties” on page 150

Final local variables

You can use the `final` modifier on a local variable to initialize the value and prevent the value from changing.

For example, the following code is valid:

```
class final1 {
    function PrintGreeting() {
        var f = "frozen"
        f = "dynamic"
    }
}
```

This code is not valid because it attempts to change the value of the final variable:

```
class final1 {
    function PrintGreeting() {
        final var f = "frozen"
        f = "dynamic" // Compile error because it attempts to change a final variable
    }
}
```

If you define a local variable as `final`, you can initialize the value of the variable immediately in the declaration. Alternatively, you can declare the variable as `final` but not immediately initialize the variable with a value. You must set the value in that function for all possible code paths. For example, any `if` statements or other flow control structures must set the value of the variable and not change a value that previous code set. The Gosu compiler verifies that all code paths initialize the final variable exactly once.

For example, the following code is valid:

```
function finalVar(a : boolean) {
    final var b : int
    if (a) {
        b = 0
    } else {
        b = 1
    }
}
```

If you remove the `else` branch, the code is not valid because the final variable is initialized only if `a` is true.

```
function finalVar(a : boolean) {
    final var b : int // INVALID CODE, UNINITIALIZED IF "a" IS FALSE
    if (a) {
        b = 0
    }
}
```

Final function parameters

You can use the `final` modifier on a function parameter to prevent the value of the parameter from changing within the function.

For example, the following code is valid:

```
package example

class FinalTest {
    function SuffixTest(greeting : String) {
        greeting = greeting + "fly"
        print(greeting)
    }
}
```

You can test this method with the code:

```
var f = new example.FinalTest()
var s = "Butter"
f.SuffixTest( s )
```

This code prints:

```
Butterfly
```

If you add the `final` modifier to the parameter, the code generates a compile error because the function attempts to modify the value of a final parameter:

```
class FinalTest {
    function SuffixTest(final greeting : String) {
        greeting = greeting + "fly"
        print(greeting)
    }
}
```

Static modifier

Gosu supports static variables, methods, properties, and inner types.

Static variables

Gosu classes can define a variable as static. A *static variable* is stored once for a Gosu class, rather than once for each instance of the class. The `static` modifier can be used with variables and properties.

For example, you can create a static variable called `_name` using this syntax:

```
private static var _name : String
```

This example creates a static method called `printedName`:

```
public static function printedName() : String {
    return _name
}
```

You can initialize static variables either with a value or the result of a method call:

```
// Initialize _name with a value
private static var _name : String = "John Doe"

// Initialize _copyOfName with result of static method call
private static var _copyOfName : String = printedName()
```

WARNING If you use static variables in a multi-threaded environment, you must take precautions to prevent simultaneous access from different threads. Use static variables sparingly if ever. If you use static variables, be sure you understand synchronized thread access fully.

For additional help on static variables and synchronized access, contact Customer Support.

To use a Gosu class variable, set its access level, such as `internal` or `public`, to restrict access to the set of classes that need to use the variable.

The `static` modifier cannot be combined with the `abstract` modifier.

Static constants

Gosu supports static constants within a class declaration, as shown in this example:

```
static final var DEFAULT_FIRST_NAME = "Default first name"
public static final var DEFAULT_LAST_NAME : String = "Default last name"
```

Static methods and properties

The `static` modifier can be used with methods and properties to indicate that the method or property belongs to the type itself rather than instances of the type.

The following example defines a static property and method:

```
class Greeting {
    private static var _name : String

    static property get Name() : String {
        return _name
    }

    static property set Name(str : String) {
        _name = str
    }

    static function PrintGreeting() {
        print("Hello World")
    }
}
```

The `Name` property get and set functions and the `PrintGreeting` method are part of the `Greeting` class itself because they are marked as `static`.

The following code accesses properties on the class itself, not an instance of the class. You can create the class, and then test this code in the Gosu Scratchpad:

```
Greeting.Name = "Initial value"
print(Greeting.Name)
Greeting.PrintGreeting()
```

This example does not use the `new` keyword to construct an instance of the `Greeting` class.

Static inner types

The `static` modifier can also be used with inner types, such as inner classes. The `static` modifier on the inner type indicates that the inner type belongs to the outer type itself rather than instances of the type.

An inner class must be declared `static` in order to define static members, such as static methods, static fields, or static properties. This Gosu requirement mirrors the requirement of Java inner classes.

The following example defines a static inner class called `FrenchGreeting` within the `Greeting` class:

```
package example

class Greeting {
    static class FrenchGreeting {
        static public function sayWhat() : String {
            return "Bonjour"
        }
    }

    static public property get Hello() : String {
        return FrenchGreeting.sayWhat()
    }
}
```

Create the classes, and then try the following lines in Gosu Scratchpad to test the inner class.

```
print(example.Greeting.Hello)
```

This code prints:

```
Bonjour
```

Static blocks and initialization

Gosu does not support static blocks. Guidewire recommends that you avoid static initialization and use lazy or on-demand initialization. If you need static blocks, use the following pattern:

```
class MyClass {
    static var _classInit = classInit()
    private static function classInit() : boolean {
        ...
    }
}
```

This pattern can also be expressed with the following notation:

```
class MyClass {
    static var _classInit = (-> {
        ...
    })()
}
```

See also

- “Access modifiers” on page 157
- “Abstract modifier” on page 158
- “Inner classes” on page 164
- “Concurrency and thread safety” on page 365

Inner classes

You can define inner classes in Gosu, similar to inner classes in Java. Inner classes are useful for encapsulating code even further within the same class as related code. Use named inner classes if you need to refer to the inner class from multiple related methods or multiple related classes. Use anonymous inner classes if you need a simple subclass that you can define in-line within a class method.

Inner classes can optionally include generics features.

The outer class that defines the inner class is also called the *enclosing class*.

See also

- “Generics” on page 241

Named inner classes

You can define a named class within another Gosu class. This inner class can be used within the outer class that contains the definition, or from classes that extend the outer class.

Example

The following code demonstrates a simple inner class.

```
package example

class Greeting {
    // function in the main (outer) class
    public function greet() {
        var x = new FrenchGreeting()
        print(x.sayWhat())
    }

    // INNER CLASS DECLARATION
    public class FrenchGreeting {
        public function sayWhat() : String {
```

```
        return "Bonjour"
    }
}
```

Create the classes, and then try the following lines in Gosu Scratchpad to test the inner class.

```
var f = new example.Greeting()
f.greet()
```

This code prints:

```
Bonjour
```

Referring to members of the outer class

Within the inner class, you refer to methods, fields, and properties on the outer class by using the `outer` keyword followed by a period, followed by the property name.

The following example has an inner class that refers to a field on the outer class:

```
package example

class Greeting {
    // Field in the main (outer) class
    public var intro : String = "I would like to say"

    // Function in the main (outer) class
    public function greet() {
        var x = new FrenchGreeting()
        print(x.sayWhat())
    }

    // INNER CLASS DECLARATION
    public class FrenchGreeting {
        public function sayWhat() : String {
            return outer.intro + " " + "bonjour"
        }
    }
}
```

If there is no ambiguity about the variable to which a name refers, the `outer` keyword and the following period are optional:

```
return intro + " " + "bonjour"
```

Static inner classes

A *static inner class* is an inner class that is a singleton, which is a predefined single instance within its parent class. You do not instantiate this class.

The following example defines a static inner class called `FrenchGreeting` within the `Greeting` class:

```
package example

class Greeting {
    public property get Hello() : String {
        return FrenchGreeting.sayWhat()
    }

    // STATIC INNER CLASS DECLARATION
    static class FrenchGreeting {
        static public function sayWhat() : String {
            return "Bonjour"
        }
    }
}
```

You can test this class in the Gosu Scratchpad using the code:

```
print(example.Greeting.Hello)
```

This code prints:

```
Bonjour
```

Notice that this example does not construct a new instance of the `Greeting` class or the `FrenchGreeting` class using the `new` keyword. The inner class in this example has the `static` modifier.

An inner class must be declared `static` in order to define static members, such as static methods, static fields, or static properties. This Gosu requirement mirrors the requirement of Java inner classes.

See also

- “Static modifier” on page 162

Accessing an inner class from a class that extends the outer class

Classes that derive from the outer class can use the inner class.

The following code defines the `Greeting` class and its inner class `FrenchGreeting`.

```
package example

class Greeting {
    // function in the main (outer) class
    public function greet() {
        var x = new FrenchGreeting()
        print(x.sayWhat())
    }

    // INNER CLASS DECLARATION
    public class FrenchGreeting {
        public function sayWhat() : String {
            return "Bonjour"
        }
    }
}
```

The following example subclasses the `Greeting` class and uses the method in the inner `FrenchGreeting` class:

```
package example

class OtherGreeting extends Greeting {
    public function greetme() {
        var f = new Greeting.FrenchGreeting()
        print(f.sayWhat())
    }
}
```

You can test this code by using the following code in the Gosu Scratchpad:

```
var t = new example.OtherGreeting()
t.greetme()
```

This code prints:

```
Bonjour
```

Anonymous inner classes

You can define anonymous inner classes in Gosu from within a class method, similar to usage in Java. The syntax for creating an anonymous inner class is very different from creating a named inner class. Constructing instances of anonymous inner classes is similar in many ways to using the `new` operator to create instances of a base class. However, you can extend the base class by following the class name with braces (`{}`) and then adding more variables or methods. If you do not have a more useful base class, use `Object`.

Example 1

The following class uses an anonymous inner class:

```
package example

class InnerTest {
```

```

static public function runme() {

    // Create instance of an anonymous inner class that derives from Object
    var counter = new Object() {

        // Anonymous inner classes can have variables (public, private, and so on)
        private var i = 0

        // Anonymous inner classes can have constructors
        construct() {
            print("Value is " + i + " at creation!")
        }

        // Anonymous inner classes can have methods
        public function incrementMe() {
            i = i + 1
            print("Value is " + i)
        }
    }

    // "counter" is a variable containing an instance of a class that has no name,
    // but derives from Object and adds a private variable and a method

    counter.incrementMe()
    counter.incrementMe()
    counter.incrementMe()
    counter.incrementMe()
    counter.incrementMe()
}

```

You can use the following code in the Gosu Scratchpad to test this class:

```
example.InnerTest.runme()
```

This code prints:

```

Value is 0 at creation!
Value is 1
Value is 2
Value is 3
Value is 4
Value is 5

```

Example 2

The following example demonstrates an advanced anonymous inner class. This anonymous inner class derives from a more functional class than `Object`. In this example, new inner class inherits the constructor and another method.

The following code defines a base class named `Vehicle` for the inner class:

```

package example

class Vehicle {
    construct() {
        print("A vehicle was just constructed!")
    }

    function actionOne(s : String) {
        print("actionOne was called with arg " + s)
    }
}

```

The following code creates a different class that uses `Vehicle` and defines an anonymous inner class that is based on `Vehicle`:

```

package example

class FancyVehicle {

    public function testInner() {

        // Create an inner anonymous class that extends Vehicle
        var test = new Vehicle() {
            public function actionTwo(s : String) {

```

```
        print("actionTwo was called with arg " + s)
    }
}
test.actionOne( "USA" )
test.actionTwo( "ABCDEFGG" )
}
```

The inner class that defines the `actionTwo` method uses the `new` operator and not the `class` operator. The `new` operator defines a new class with no name and then creates one instance of that class.

Create the classes, and then try the following lines in Gosu Scratchpad to test the `FancyVehicle` class:

```
var g = new example.FancyVehicle()
g.testInner()
```

This code prints:

```
A vehicle was just constructed!
actionOne was called with arg USA
actionTwo was called with arg ABCDEFG
```

Gosu block shortcut for anonymous classes implementing an interface

In certain cases, you can pass a block as a method argument instead of an instance of an anonymous class. If the method is part of an interface that contains exactly one method, you can pass a block instead of the anonymous class instance. This shortcut is especially helpful for APIs defined to take the type `BlockRunnable` or `Runnable`.

See also

- “Blocks as shortcuts for anonymous classes” on page 187

Enumerations

An enumeration is a list of named constants that are encapsulated into a special type of class. Gosu supports enumerations natively, as well as providing compatibility for using enumerations defined in Java. The sections in this topic refer to example enum classes, `TaskStatus` and `FruitType`. The source code for these classes is provided at the end of the topic.

Enum types

The Gosu enumeration type (*enum*) is a reference type and is used to declare distinct enumeration values. An enumeration type inherits from `java.lang.Enum`. An *enum type declaration* is a specialized form of class declaration beginning with a list of enumeration value declarations. The declaration of fields, method, nested types for enums are the same as the declarations for ordinary classes. Enumerations must have a comma-delimited list of enumeration constants, followed by the rest of the declarations, including fields and constructors and methods.

Note: The modifiers `abstract` and `final` cannot be used.

A nested enum type is implicitly static. The `static` modifier is allowed but is not required. A nested enum type cannot refer to instance fields of an enclosing type.

Enumeration values

The type of a declared enumeration value is the enclosing enum type. An enumeration value is similar to a `public static final` field. The ordinal value of an enumeration value is set by its position in the list of values, with the first value being zero. There are no predefined conversions between enumeration values and integers. A value of an enum type always equals a declared enumeration value. Numeric operations such as plus (+) are not supported for enumeration values. You can use comparison operations such as equal (==) or greater than (>) on enumeration values. Comparison operations are useful if there is a logical progression from one enumeration value in the order of definition to the next.

Outside an enum declaration, an enumeration value can be written in its qualified form, such as `TaskStatus.Assigned`. Alternatively, if the enum type can be inferred, you can use the unqualified form, such as `Assigned`, as shown in the following example:

```
uses doc.example.TaskStatus#*

...

if (firstTask.Status < Assigned) {
  myFirstTask = firstTask
  prevStatus = firstTask.Status
  firstTask.Status = Assigned
}
```

Enumeration constructors

Any constructors for an enum type are private. An enum variable instance cannot be explicitly created by using the `new` keyword. Any constructors must follow the list of enumeration value declarations.

Enumeration properties

All enum instances have the following properties:

Code

The name of the enum instance. For example: `Banana`

Name

The name of the enum instance. For example: `Banana`

Ordinal

The zero-based position of the enumeration value in the enum class definition. For example, the value of the `Ordinal` property for `FruitType.Banana` is `1`.

Value

The name of the enum instance. For example: `Banana`

DeclaringClass

The name of the class of the enum instance prefixed by `class`. For example: `class doc.example.FruitType`

You can create additional properties on enum classes, as shown in the `FruitType` enumeration example. The `FruitType` enum class defines `HasPeel`, `PeelEdible`, `BestEatingFormat`, and `LongName` properties.

You can access the properties on an enumeration in your code in the same way as properties on an instance any other Gosu class. For example:

```
print(Banana.HasPeel)
```

This line prints:

```
true
```

Enumeration functions

Enums support functions. The following example calls the `displayFruitTypeStuff` method on a `FruitType` enum instance.

```
var myFruitType = FruitType.Banana
myFruitType.displayFruitTypeStuff()
```

The following lines are printed:

```
Name: Banana
Code: Banana
Ordinal: 1
Has peel? true
Is peel edible? false
Eat raw or cooked? Raw
Long name: Banana: Peel before eating
```

Example ordered enumeration class `TaskStatus`

This enum class has enumeration values that have a meaningful order.

```
package doc.example

enum TaskStatus {
    Blocked,
    WaitingForCustomer,
    New,
    Assigned,
    InProgress,
    Complete
}
```

Example enumeration class with complex structure FruitType

This enum class demonstrates multiple constructors, properties, methods, and a nested enum definition.

```
package doc.example

enum FruitType {
    Apple(true, true, EatingFormat. Both),
    Banana(),
    Orange(),
    Strawberry(false, null, EatingFormat.Raw)

    // Properties
    var _hasPeel : Boolean as readonly HasPeel
    var _isPeelEdible : Boolean as readonly PeelEdible
    var _bestEatingFormat : EatingFormat as readonly BestEatingFormat

    // No-argument constructor sets default property values
    private construct() {
        setProperties(true, false, EatingFormat.Raw)
    }

    // Constructor with arguments sets explicit property values
    private construct(hasPeel : Boolean, isPeelEdible : Boolean, eatingFormat : EatingFormat) {
        setProperties(hasPeel, isPeelEdible, eatingFormat)
    }

    private function setProperties(hasPeel : Boolean, isPeelEdible : Boolean, eatingFormat : EatingFormat) {
        _hasPeel = hasPeel
        _isPeelEdible = isPeelEdible
        _bestEatingFormat = eatingFormat
        _longName = _hasPeel and not _isPeelEdible ? "${Name}: Peel before eating": Name
    }

    // Utility method
    function displayFruitTypeStuff() {
        print("Name: ${Name}")
        print("Code: ${Code}")
        print("Ordinal: ${Ordinal}")
        print("Has peel? ${HasPeel}")
        print("Is peel edible? ${PeelEdible}")
        print("Eat raw or cooked? ${BestEatingFormat}")
        print("Long name: ${LongName}")
    }

    // Nested enum
    enum EatingFormat {
        Raw, Cooked, Both
    }
}
```

Working with enumerations

Extracting information from an enumeration

To use an enumeration, use the period (.) operator to reference elements of the enumeration class:

```
enumerationClassName.enumerationConstantName
```

To use the name of the enumeration element as a String, get its `Name` property. To use the zero-based index of the enumeration element as an Integer, get its `Ordinal` property.

Comparing enumerations

You can compare two enumerations by using the `==` operator. For example:

```
if (myFruitType == FruitType.Apple) {
    print("An apple a day keeps the doctor away.")
}
if (myFruitType == FruitType.Banana) {
    print("Watch out for banana peels.")
}
```

Running this code produces the following line if `myFruitType` references `FruitType.Banana`:

Watch out for banana peels.

Create an enumeration

An enumeration provides a fixed set of values that you can use for a property or variable. For example, a `FruitType` enumeration provides names of fruits.

Procedure

1. Create an enumeration by using the same approach that you use to create a class.
 - a. In Studio, right-click a package folder and click the **New** submenu.
 - b. Click **Class** to create the enumeration class in that package.
 - c. Type a name for the enumeration.
 - d. In the drop-down list, click **Enum**.

Your enumeration looks like:

```
package doc.example

enum FruitType {

}
```

2. Add your named constants separated by commas:

```
enum FruitType {
    Apple, Banana, Orange, Passionfruit
}
```

3. Add constructors, properties, and methods, as required by your business logic.

Extract information from an enumeration

You can assign the name of a value from an enumeration to a variable. You use dot notation to access the name of the enumeration value.

Before you begin

To create the enumeration that this example uses, you must perform the steps in the following topic:

- “Create an enumeration” on page 172

Procedure

1. In Studio, in Scratchpad, create a variable that contains an enumeration constant:

```
uses doc.example.FruitType
var myFruitType = FruitType.Banana
```

2. Add the following lines to access the properties of the enumeration constant:

```
print(myFruitType.Name) // Prints "Banana"
print(myFruitType.Code) // Prints "Banana"
print(myFruitType.Ordinal) // Prints "2"
```

3. Run the program. The following lines appear in the console:

```
Banana
Banana
2
```

Interfaces

In Gosu, an *interface* defines a strict contract of interaction and expectation between two or more software elements. Conceptually, you can think of an interface as a list of method signatures grouped together. In practice, the interface provides a group of related method signatures that typically have empty bodies. The interface requires other code to implement those empty methods.

From a syntax perspective, an interface looks like a class definition. However, the interface is a specification of a set of required functions for any class that implements the interface. Thus, an interface requires that the implementing class provides the code in the set of interface methods.

If a class implements the interface, the class agrees to implement all these methods with the appropriate method signatures. The code implementing the interface agrees that each method appropriately performs the required task if external code calls those methods.

Gosu classes can implement interfaces defined in either Gosu or Java.

What is an interface?

Interfaces are a set of required functions necessary for a specific task. Interfaces define a strict contract of interaction and expectation between two or more software elements, but leave the implementation details to the code that implements the interface. In many cases, the person who writes the interface is different from the person who writes code to implement the interface.

If a Gosu class implements an interface, Gosu validates at compile time that all required methods are present and that the implementer class has the required method signatures.

PolicyCenter defines interfaces that PolicyCenter calls to perform various tasks or calculate values. For example, to integrate PolicyCenter with a document management system, implement a plugin interface that defines how the application interacts with a document management system. The implementation details of document management are separate from the contract that defines what actions your document management code must handle.

A physical example of an interface is a car stereo system. The buttons, such as for channel up and channel down, are the interface between you and the complex electrical circuits on the inside of the box. You press buttons to change the channel. However, you probably do not care about the implementation details of how the stereo performs those tasks behind the solid walls of the stereo. If you get a new stereo, it has equivalent buttons and matching behavior. Because you interact only with the buttons and the output audio, if the user interface is appropriate and outputs appropriate sounds, the internal details do not matter to you. You do not care about the details of how the stereo internally handles the button presses for channel up, channel down, volume up, and volume down.

[See also](#)

- *Integration Guide*

Defining and using an interface

In some ways, interfaces are similar to Gosu classes. There is no separate top-level folder in Studio for interfaces.

A Gosu interface can have the following content and properties:

- Empty methods, for which the implementing class must provide code
- Default methods, which include code and which the implementing class can use as is or can override
- Static variables and methods
- Properties
- Extending from a Java interface
- Extend from a Java interface that supports generics
- Including Gosu generics
- Abstracting an interface across a type defined in Java or a subtype of such a type

You use the `implements` keyword in a class declaration to specify the interfaces that the class implements. If a class implements more than one interface, separate the interface names by commas:

```
class MyRestaurant implements Restaurant, InitializablePlugin
```

The Gosu editor reveals compilation errors if your class does not properly implement the plugin interface. You must fix these issues.

When subclassing a parent class or implementing an interface, you can override the methods defined in the parent class or interface.

To call the method that a parent class defines, use the `super` keyword using the following syntax:

```
super.method
```

For example, if a parent class defines a no-argument method `makeJourney`, the subclass can call the parent implementation like the following line:

```
super.makeJourney()
```

To call a default method that an interface defines, use the `super` keyword using the following syntax:

```
super[interface].method
```

For example, if an interface `Travel` defines a no-argument method `comeHome` that contains default code, the implementing class can call the default implementation like the following line:

```
super[Travel].comeHome()
```

Note: When overriding the `equals` method for objects, which is a common technique for overriding the `==` operator, you must also override the `hashCode` method. You must use logic ensuring that the return value for `a.equals(b)` always matches the return value for `a.hashCode() == b.hashCode()`.

In the following example, `IMyInterface` defines a property called `Name`, and a default method called `speak`. The class `MyClass` implements the interface and calls the default method:

```
interface IMyInterface {
    property get Name() : String
    property set Name(n:String) : void
    function speak() : String {
        return "Hello World"
    }
}

class MyClass implements IMyInterface {
    var _name : String

    override property get Name() : String {
        return "name: " + _name
    }

    override property set Name(n:String) {
        _name = n
    }
}
```

```

    }

    override function speak() : String {
        return "Hello, " + _name
    }

    function shout() : String {
        return (super[IWithDefaultMethod].speak() +
            "\n" + gw.api.util.StringUtil.toUpperCase(super[IWithDefaultMethod].speak()))
    }
}

```

This code creates an instance of `MyClass` and tests the overridden `Name` property and the default method:

```

var myObject = new MyClass()
myObject.Name = "Andy Applegate"
print(myObject.Name)
print(d.speak())
print(d.shout())

```

In this example, `myObject` is an object of type `MyClass`, which has overridden the `IMyInterface` properties, and calling `myObject.Name("Andy Applegate")` sets the private variable `_type` to `"Andy Applegate"`. As the `print` function is called, the value returned by `myObject.Name` is `"name: Andy Applegate"`. The code also prints the default output from the overridden `speak` method and the output from the `shout` method that calls the default method in the interface. The output looks like the following lines:

```

name: Andy Applegate
Hello, Andy Applegate
Hello World
HELLO WORLD

```

See also

- “Generics” on page 241

Create and implement an interface

Procedure

1. To create an interface in Guidewire Studio, first create an appropriate package.
 - a. Right-click the package folder, and then click the **New** submenu.
 - b. Click **Class** to create the class in that package.
 - c. Type a name for the interface.
 - d. In the drop-down list, click **Interface**.

Studio creates a new interface with the appropriate syntax.
2. Write the rest of the interface like a Gosu class, except that methods are method signatures only with no method bodies.

For example, define an interface with the following code:

```

interface Restaurant {
    function retrieveMeals() : String[]
    function retrieveMealDetails(dishname : String) : String
}

```

3. To implement an interface, create a Gosu class and add `"implements MyInterfaceName"` after the class name. For example, if your class is called `MyRestaurant`, go to the line:

```

class MyRestaurant

```

4. Change that line to:

```

class MyRestaurant implements Restaurant

```

5. For the example `Restaurant` interface, implement the interface with a class.

For example, use code like the following lines:

```
class MyRestaurant implements Restaurant

    override function retrieveMeals() : String[]{
        return {"chicken", "beef", "fish"}
    }
    override function retrieveMealDetails(dishname : String) : String {
        return "Steaming hot " + dishname + " on rice, with a side of asparagus."
    }
}
```

Defining and using properties on interfaces

Interfaces created in Gosu can declare properties by specifying explicit property `get` or property `set` accessors in interfaces with the following syntax:

```
property get Description() : String
```

Classes can implement an interface property with the explicit property `get` or property `set` syntax.

For example, the following code defines an interface:

```
package example

interface MyInterface {
    property get VolumeLevel() : int
    property set VolumeLevel(vol : int) : void
}
```

A class could implement this interface with the following code:

```
class MyStereo implements MyInterface {
    var _volume : int

    property set VolumeLevel(vol : int) {
        _volume = vol
    }

    property get VolumeLevel() : int {
        return _volume
    }
}
```

After creating the class and interface, you can test the following code in the Gosu Scratchpad:

```
uses example.MyStereo

var v = new MyStereo()
v.VolumeLevel = 11
print("The volume goes to " + v.VolumeLevel)
```

This code prints:

```
The volume goes to 11
```

Alternatively, a class implementing a property can implement the property with the *variable alias* syntax by using the `as` keyword. Using this language feature, you make set methods that use a class instance variable to store property values, and get methods to get property values from the variable.

For example, the following, more concise code is functionally identical to the previous example implementation of `MyStereo`:

```
uses example.MyStereo

class MyStereo implements MyInterface {
    var _volume : int as VolumeLevel
}
```

If you run the same Gosu Scratchpad code as previously, the code prints the same results.

Interface methods that look like properties

If an interface's methods look like properties, a class that implements the interface must implement those methods in Gosu as a property by using property `get` or property `set` syntax. If the interface contains a method whose name

starts with "get" or "is" and takes no parameters, define the method by using the Gosu property syntax. See earlier in this topic for examples.

See also

- “Classes” on page 145
- “Properties” on page 150

Modifiers and interfaces

In many ways, interfaces are defined like classes. One way in which they are similar is the support for modifier keywords.

One notable difference for interfaces is that the `abstract` modifier is implicit for the interface itself and all methods defined on the interface. Consequently, Gosu does not support the `final` modifier on the interface or its members.

Implementing methods

Use of the `override` modifier is optional on an implementation of an interface method. If you do use the `override` modifier, your code in a class that implements an interface is more robust against changes in method signatures in the interface.

Superclass properties

When implementing an interface and referencing a property on a superclass, use the `super.PropertyName` syntax, such as:

```
property get WholeValue() : String {  
    _mySpecialPrivateVar = super.FirstHalf + super.SecondHalf  
}
```

See also

- “Modifiers” on page 157

Create an interface with getters, setters, and methods

About this task

To create an interface with getters, setters and methods, perform the following general steps:

Procedure

1. In the package for your new type, create a Gosu interface file with a `.gs` file extension.
2. Add property declarations and method signatures:
 - To provide data from a variable or property on an instance of this type, use property getter syntax:

```
property get PROPERTY_NAME() : TYPE
```

- To set the value of a variable or property on an instance of this type, use property setter syntax:

```
property set PROPERTY_NAME(TYPE)
```

- To provide a callable method on an instance of this type, use the syntax:

```
public function METHOD_NAME(ARG_LIST) : TYPE
```

Example

This example declares an interface called `DemoInterface` that has the following features:

- Name property of type `String`
- `count` method, which takes one `String` parameter and returns a `int` value

Start by creating a file defining the interface:

```
package examples.pl.gosu.intf

interface DemoInterface {
    property get Name() : String
    property set Name(n : String)
    public function count(s : String) : int
}
```

Create a class implementing the interface:

```
uses examples.pl.gosu.intf.DemoInterface

class MyClass implements DemoInterface {
    var _name : String
    override property get Name() : String {
        return "name: " + _name
    }
    override property set Name(n:String) {
        _name = n
    }
    override function count(s : String) : int {
        return s.length()
    }
}
```

The following code creates an instance of the class and tests its properties and method:

```
var myObject = new MyClass()
myObject.Name = "Andy Applegate"
print(myObject.Name)
print(myObject.count(myObject.Name))
```

Output:

```
name: Andy Applegate
20
```

Static properties and methods on interfaces

In both Gosu and Java, you can add static variables to types to declare data that exists at the type level. For Gosu classes, you add such data to a class by adding a declaration line with the `static` modifier:

```
public static var MyVariable
```

Adding a `var` declaration does not imply the ability to get or set a variable or property on an instance of the type. A `var` declaration on an interface defines a static constant variable on the type. A static constant variable is conceptually similar to a static variable in a Gosu class. However, for a variable on an interface, the `static` keyword is implicit and the value is read-only. The declaration does not specify the ability to get or set a variable or property on an instance of this type. Although you can use property getter and setter syntax to define data used with object instances, you do not use this syntax for static constant variables. This is why static constant variables do not require assignment compatibility between the interface and other types.

For example, the declaration of the `Age` variable in the following example will not compile because it implicitly creates a static constant variable, which must be initialized:

```
package examples.pl.gosu.intf

interface DemoInterface {
    property get Name() : String
    property set Name(n : String)
    public function count(s : String) : int
}
```

```
    public var Age : Integer // Compilation error. A static constant variable requires initialization.
}
```

Initializing the value of the static constant variable clears the error:

```
package examples.pl.gosu.intf

interface DemoInterface {
    property get Name() : String
    property set Name(n : String)
    public function count(s : String) : int

    public var Age : Integer = 18 // OK because the static constant variable is initialized.
}
```

The following code defines a class that implements the DemoInterface interface:

```
uses examples.pl.gosu.intf.DemoInterface

class MyClass implements DemoInterface {
    var _name : String
    override property get Name() : String {
        return "name: " + _name
    }
    override property set Name(n:String) {
        _name = n
    }
    override function count(s : String) : int {
        return s.length()
    }
}
```

The following code creates an instance of the class and tests its properties:

```
var myObject = new MyClass()
myObject.Name = "Andy Applegate"
print(myObject.Name)
print(myObject.count(myObject.Name))
print(myObject.Age)
```

Output:

```
name: Andy Applegate
20
18
```

Note: You can add the `static` modifier in the declaration of the age variable, but in interface declarations this modifier is implicit and therefore redundant.

To call a static interface method, do so using the interface name directly, rather than the name of the implementing class. This is required because static interface methods are not inherited.

```
package examples.pl.gosu.intf

interface DemoInterface {
    property get Name() : String
    property set Name(n : String)
    public function count(s : String) : int

    public var Age : Integer = 18

    static function nameWithLabel( person: String ) : String {
        return "Person: " + person
    }
}
```

The following code defines a class that implements the modified DemoInterface interface:

```
uses examples.pl.gosu.intf.DemoInterface

class MyClass implements DemoInterface {
    var _name : String
    override property get Name() : String {
        return "name: " + _name
    }
    override property set Name(n:String) {
        _name = n
    }
}
```

```

    }
    override function count(s : String) : int {
        return s.length()
    }
}

```

The following code creates and tests an instance of the class, calling the `DemoInterface.nameWithLabel` static interface method:

```

var myObject = new MyClass()
myObject.Name = "Andy Applegate"
print(myObject.Name)
// print(MyClass.nameWithLabel(myObject.Name)) // error: must use method directly on interface
print(DemoInterface.nameWithLabel( myObject.Name )) // OK

```

Output:

```

name: Andy Applegate
Person: name: Andy Applegate

```

Another reason static interface methods must be called directly from their interfaces is to prevent conflicts when a class implements multiple interfaces having the same static method names. In this example, a second interface, `DemoInterface2`, has an identically named static method to the one in `DemoInterface`:

```

interface DemoInterface2 {
    static function nameWithLabel( person: String ) : String {
        return "Person2: " + person
    }
}

```

`MyClass` is modified here to implement both interfaces:

```

class MyClass implements DemoInterface, DemoInterface2 {
    var _name : String
    override property get Name() : String {
        return "name: " + _name
    }
    override property set Name(n:String) {
        _name = n
    }
    override function count(s : String) : int {
        return s.length()
    }
}

```

The following code creates an instance of `MyClass`, calling the static methods defined on both interfaces:

```

var myObject = new MyClass()
myObject.Name = "Andy Applegate"
print(myObject.Name)
print(DemoInterface.nameWithLabel( myObject.Name )) // ok
print(DemoInterface2.nameWithLabel( myObject.Name )) // ok

```

Output:

```

name: Andy Applegate
Person: name: Andy Applegate
Person2: name: Andy Applegate

```

In this last example, both interfaces can be implemented without conflict because their static methods are not inherited.

Blocks

Gosu blocks are a special type of function that you can define in-line within another function. You can then pass that block of code to yet other functions to invoke as appropriate. Blocks are useful for generalizing algorithms and simplifying interfaces to certain APIs. For example, blocks can simplify tasks related to collections, such as finding items within or iterating across all items in a collection.

The following code illustrates the syntax of a Gosu block. It also illustrates the simple use of a block. The example code multiplies a number by itself, which is known as squaring a number:

```
var square = \ x : int-> x * x // No need for braces here
var myResult = square(10)    // Call the block
```

The value of `myResult` in this example is 100.

What are blocks?

Gosu blocks are functions without names, which you can define in-line within another function. In other programming languages, blocks are known as *anonymous functions*. You can then pass that block of code to yet other functions to invoke as appropriate. Blocks are useful for generalizing algorithms and simplifying interfaces to APIs. An API author can design most of an algorithm but let the API consumer contribute short blocks of code to complete the task. The API can use this block of code and call the block either once or possibly many times with different arguments.

For example, you might want to find items within a collection that meet particular criteria, or to sort a collection of objects by certain properties. If you can describe your find or sort criteria using a small amount of Gosu code, Gosu performs the general algorithm such as sorting the collection.

Some other programming languages have similar features to blocks and call them *closures* or *lambda expressions*. If you use the Java language, notice that Gosu blocks serve some of the most common uses of single-method anonymous classes in Java. However, Gosu blocks provide a concise and clear syntax that makes this feature more convenient in typical cases.

Blocks are particularly valuable for the following:

Collection manipulation

Using collection functions such as `map` and `each` with Gosu blocks allows concise, readable code with powerful and useful behaviors for real-world programming.

Callbacks

For APIs that wish to use callback functions after an action is complete, blocks provide a straightforward mechanism for triggering the callback code.

Resource control

Blocks can be useful for encapsulating code related to connection management or transaction management. Gosu code using blocks appropriately can simplify of your Gosu code. However, blocks can also cause confusion if your code uses them too frequently. If the previous list does not include your intended use, reconsider whether to use blocks. There may be a better and more conventional way to solve the problem. If you write a method that takes more than one argument that is a block, strongly consider redesigning or refactoring the code.

Basic block definition and invocation

To define a Gosu block, type use the backslash character (\) followed by a series of arguments. The arguments must be name/type pairs separated by a colon character (:) similar to the definitions of arguments to a method. Next, add a hyphen character (-) and a greater-than character (>) to form the appearance of an arrow: ->. Finally, add a Gosu expression or a statement list surrounded by braces ({}).

The syntax of a block definition is:

```
\ argumentList -> blockBody
```

The argument list (*argumentList*) is a standard function argument list, for example:

```
x : int, y : int
```

The argument list defines the parameters to pass to the block. The parameter list uses identical syntax as parameters to regular functions. In some cases, you can omit the types of the parameters, such as passing a block directly into a class method such that Gosu can infer the parameter type.

The block body (*blockBody*) can be either of the following:

- A simple expression. This expression includes anything legal on the right-hand side of an assignment statement. For example, the following line is a simple expression:

```
"a concatenated string " + "is a simple expression"
```

- A statement list of one or more statements surrounded by braces and separated by semi-colon characters, such as the following simple single-statement statement list:

```
\ x : int, y : int -> { return x + y }
```

For single-statement statement lists, you must explicitly include the brace characters. In particular, note that variable assignment operations are always statements, not expressions. Thus, the following block is invalid:

```
names.each( \ n -> myValue += n )
```

Changing the code to the following line produces a valid block:

```
names.each( \ n -> { myValue += n } )
```

For multiple statements, separate the statements with a semicolon character. For example:

```
\ x : int, y : int -> { var i = x + y; return i }
```

The following block multiplies a number by itself, which is known as squaring a number:

```
var square = \ x : int-> x * x // No need for braces here
var myResult = square(10)    // Call the block
```

The value of `myResult` in this example is 100.

IMPORTANT All parameter names in a block definition's argument list must not conflict with any existing in-scope variables, including but not limited to local variables.

Note: Gosu does not provide a syntax for declaring a variable type that is an array of blocks. To create an array of blocks, create an array of `Object` and assign blocks to the array elements.

Standard Gosu style is to omit all semicolon characters in Gosu at the ends of lines. Gosu code is more readable without these optional semicolons. However, if you provide statement lists on one line, such as within block definitions, use semicolons to separate statements.

See also

- “Argument type inference shortcut” on page 185
- “Coding style” on page 219

Block return types

A block definition does not need to explicitly declare a return type, which is the type of the return value of the block. The return value is statically typed even though the type is not explicitly visible in the code. Gosu infers the return type from either the expression, if you defined the block with an expression, or from the return statements in a statement list. This functionality makes the block appear shorter and more elegant.

For example, consider the following block:

```
var blockWithStatementBody = \ -> { return "hello blocks" }
```

Because the statement `return "hello blocks"` returns a `String` object, the Gosu compiler infers that the return type of the block is `String`.

The following block returns an array containing the square and the cube of the parameter. In this case the syntax `{return { expression, expression }}` is required because an array of values is returned:

```
var b1 : block(Integer):Integer[]
b1 = (\ i -> {return {(i * i), (i * i * i)}})
print(b1(2))
```

This code prints:

```
[4, 8]
```

Invoking blocks

You can invoke blocks just like normal functions by referencing a variable to which you previously assigned the block. To use a block, type the following items in the following order:

- The name of the block variable or an expression that resolves to a block
- An open parenthesis
- A series of argument expressions
- A closing parenthesis

For example, suppose you create a Gosu block with no arguments and a simple return statement:

```
var blockWithStatementBody = \-> { return "hello blocks" }
```

Because the statement list returns a `String`, Gosu infers that the block returns a `String`. The new block is assigned to a new variable `blockWithStatementBody`, and the variable has a type of `String` even though the code text does not show the type explicitly.

To call this block and assign the result to the variable `myresult`, use this code:

```
var myresult = blockWithStatementBody()
```

The value of the variable `myresult` is the `String` value `"hello blocks"` after this line executes.

The following example creates a simple block that has two numbers as parameters and returns the result of adding those numbers:

```
var adder = \ x : int, y : int -> { return x + y }
```

After defining this block, you can call it with code such as:

```
var mysum = adder(10, 20)
```

The variable `mysum` has the type `int` and has the value 30 after the line is executed.

You can also implement the same block behavior by using an expression rather than a statement list, which uses an even more concise syntax:

```
var adder = \ x : int, y : int -> x + y
```

Variable scope and capturing variables in blocks

A Gosu block maintains some context with respect to the enclosing statement that created the block. If code in the block refers to variables that are defined outside the block's definition but are in scope where the block is defined, the variable is *captured*. The variable is incorporated by reference into the block. Incorporating the variable *by reference* means that blocks do not capture the current value of the variable at the time its enclosing code creates the block. If the variable changes after the enclosing code creates the block, the block uses the most recent value in the original scope. This behavior continues even if the original scope exited or finished.

The following example adds 10 to a value. However, the value 10 was captured in a local variable, rather than included in an argument. The captured variable, called `captured` in this example, is used but not defined within the block:

```
var captured = 10
var addTo10 = \ x : int -> captured + x
var myresult = addTo10(10)
```

After the third line is executed, `myresult` contains the value 20.

A block captures the state of the stack at the point of its declaration, including all variables and the special symbol `this`, which represents the current object. For example, `this` can represent the current instance of a Gosu class running a method.

This capturing feature allows the block to access variables in scope at its definition. Capturing occurs even in the following cases:

- After passing the block as an argument to another function
- After the block returns to the function that defines it
- If code assigns the block to a variable and retains the variable indefinitely
- After the original scope exits or finishes

Each time the block runs, the block can access all variables declared in the original scope in which the block was defined. The block can get or set those variables. The values of captured variables are evaluated each time the block is executed, and can be read or set as needed. Captured variable values are not a static snapshot of the values at the time that the block was created.

The following example creates a block that captures a variable (`x`) from the surrounding scope. Next, the code that created the block changes the value of `x`. Code calls the block only after that change happens:

```
// Define a local variable, which is captured by a block
var x = 10

// Create the block
var captureX = \ y : int -> x + y

// Note: the variable "x" is now SHARED by the block and the surrounding context

// Now change the value of "x"
x = 20

// At the time the block runs, it uses the current value of "x".
```



```
// This value is NOT a snapshot of the value at the time the block was created
var z = captureX( 10 )

print(z) // Prints 30 --- Not 20!!!
```

The captured variable is shared by the original scope and the block that was created within that scope. The block references the variable itself, not merely its original value.

IMPORTANT Capturing variables in blocks is very powerful. Use this feature very carefully. Document your assumptions so that people who read your code can understand that code.

Argument type inference shortcut

The Gosu parser provides additional type inference in a common case. If a block is defined within a method call parameter list, Gosu can infer the type of the block's arguments from the parameter argument. You do not need to explicitly specify the argument type in this case.

In other words, if you pass a block to a method, in some cases Gosu can infer the type so you can omit the type for more concise code. This behavior is particularly relevant for using collection-related code that takes blocks as arguments.

For example, suppose you have this code:

```
var x = new ArrayList<String>(){ "a", "b", "c" }
var y = x.map( \ str : String -> str.length )
```

You can omit the argument type (`String`). The `map` method signature allows Gosu to infer the argument type in the block from the definition of the `map` method.

You can use the more concise code:

```
var x = new ArrayList<String>(){ "a", "b", "c" }
var y = x.map( \ str -> str.length )
```

The list method `map` is a built-in list enhancement method that takes a block with one argument. That argument is always the same as the type of the list. Therefore, Gosu infers that `str` must be of type `String`. You do not need to explicitly define either the type of the arguments or the return type.

The `map` method is implemented using a built-in Gosu enhancement of the Java language `List` interface.

[See also](#)

- See “Collections” on page 189

Block type literals

Block literals are a form of type literal. Use a block literal to reference a specific *block type*. The block literal specifies the kinds of arguments that the block takes and the type of the return value.

[Block types BNF notation](#)

The formal BNF notation of a block literal is:

```
blockliteral -> block_literal_1 | block_literal_2
block_literal_1 -> block ( type_list ) : type
block_literal_2 -> parameter_name ( type_list ) : return_type
type_list -> type | type_list , | null
```

[Block types in declarations](#)

To declare a variable to contain a block, the preferred syntax is:

```
var variableName( List_of_argument_types ) : return_type
```

For example, to declare that `strBlock` is a variable that can contain a block that takes a single `String` argument and returns a `String` value, use this code:

```
var strBlock( String ) : String
```

In declarations, you can also optionally use the `block` keyword, although this is discouraged in declarations:

```
var variableName : block( List_of_types ) : return_type
```

For example, this code declares the same block type as described earlier:

```
var x : block( String ) : String
```

Usage of block types not in declarations

For a block type literal that is not part of a declaration, the `block` keyword is strictly required:

```
block( List_of_types ) : return_type
```

For example:

```
return b as block( String ) : int
```

The `b` variable is assigned a value that is a block type. Because the block type literal is not directly part of the declaration, Gosu requires the `block` keyword.

Block types in argument lists

A function definition can specify a function argument as a block. As you define the block argument, provide a name for that block parameter so that you can use the block within the function. Use the following syntax for block types in argument lists:

```
parameter_variable_name( List_of_types ) : return_type
```

For example, suppose you want to declare a function that takes one argument, which is a block. Suppose the block takes a single `String` argument and returns no value. To refer to this block by name as `myCallback`, define the argument using the syntax:

```
myCallback( String ) : void
```

Example of a block type in an argument list

The following Gosu class includes a function that takes a callback block. The argument is called `myCallback`, which is a block that takes a single string argument and returns no value. The outer function calls that callback function with a `String` argument.

```
package mytest

class test1 {
    function myMethod( myCallback( String ) : void ) {

        // Call your callback block and pass it a String argument
        myCallback("Hello World")
    }
}
```

After creating the class, you can test the following code in the Gosu Scratchpad:

```
var a = new mytest.test1()
a.myMethod( \ s : String -> print("<contents>" + s + "</contents>") )
```

For more concise code, you can omit the argument type “`: String`” in the in-line block. The block is defined in-line as an argument to a method whose argument types are already defined. In other words, you can use the following code:

```
var a = new mytest.test1()
a.myMethod( \ s -> print("<contents>" + s + "</contents>") )
```

Both versions print the following:

```
<contents>Hello World</contents>
```

Usage of block types for recursion

Gosu blocks support recursion, in which the block calls itself until reaching an end condition. To use recursion in blocks, you must name and define the block signature before you define the block code. For example, the following code defines a block that returns the factorial of an integer:

```
var bFactorial: block(int): int
bFactorial = \ x -> x > 12 or x < 1 ? -1 : (x == 1 ? 1 : x * bFactorial(x-1))
```

This code has two end conditions. The first end condition ensures that the block terminates if the result would be greater than `Integer.MAX_VALUE` or if the parameter value is less than 1. The second end condition terminates the recursion when the argument value is 1. To test this block, use code like the following:

```
print(bFactorial(6))
```

This code prints:

```
720
```

Blocks and collections

Gosu blocks are particularly useful for working with collections of objects. Blocks support concise, readable code that loops across items, extracts information from every item in a collection, or sorts items. Common collection enhancement methods that use blocks are `map`, `each`, and `sortBy`.

For example, suppose you want to sort the following list of `String` objects:

```
var myStrings = new ArrayList<String>(){ "a", "abcd", "ab", "abc" }
```

You can use blocks to sort the list based on various attributes, such as the length of the strings. Create a block that takes a `String` and returns the sort key, which in this case is the `String` object's length. The built-in list `sortBy(...)` method handles the rest of the sorting algorithm and then returns the new sorted array:

```
var resortedStrings = myStrings.sortBy( \ str -> str.length() )
```

These block-based collection methods are implemented using a built-in Gosu enhancement of the Java language `List` class.

[See also](#)

- “Collections” on page 189

Blocks as shortcuts for anonymous classes

In some cases, you can pass a block as a method argument instead of an anonymous class instance. You can pass a block if the method argument is an implementation of a functional interface. A *functional interface* contains exactly one abstract method. For example, the Gosu interface type `BlockRunnable` or the Java interface type `Runnable` are both functional interfaces that contain a single method called `run`. Passing a block is much more concise than passing an anonymous class instance. This Gosu coding style works with interfaces that were originally implemented in either Gosu or Java. The parameters of the block must be the same number and type as the parameters to the single method of the interface. The return type of the block must be the same as the return type of that method.

For example, suppose a method signature looks like the following:

```
public function doAction(b : BlockRunnable)
```

You can call this method using a block:

```
obj.doAction(\ -> print("Do your action here"))
```

As a naming convention, if an API uses a type with a name that contains the word **Block**, you can probably use a block for that type.

This technique works with any interface, including interfaces defined as inner interfaces within another interface.

Example

This example demonstrates the use of a block argument for an inner functional interface that an outer interface defines. The `MyCallbackHandler` interface contains an inner functional interface called `MyCallbackHandler.CallbackBlock`. This functional interface implements a single method named `run`, similar to the `Runnable` interface. Instead of creating an anonymous class to use the inner interface, use a block that takes no arguments and has no return value.

This Java code defines the `MyCallbackHandler` interface:

```
public interface MyCallbackHandler {
    // A functional interface within this interface
    public interface CallbackBlock {
        public void run() throws Throwable;
    }
    // ...
    public void execute(CallbackBlock block) throws Throwable;
}
```

This Gosu code creates the anonymous class explicitly:

```
public function messageReceived(final messageId : int) : void {
    var _callbackHandler : MyCallbackHandler
    // Create an anonymous class that implements the inner interface
    var myBlock : _callbackHandler.CallbackBlock = new MyCallbackHandler.CallbackBlock() {
        // Implement the run() method in the interface
        public function run() : void { /* Your Gosu statements here */ }
    }
    // Pass the anonymous inner class with the one method
    _callbackHandler.execute(myBlock)
}
```

You can code the method more concisely by using a block:

```
public function messageReceived(messageId : int) {
    _callbackHandler.execute(\ -> { /* Your Gosu statements here */ })
}
```

Collections

Gosu collection and list classes rely on collection classes from the Java language. Gosu collections and lists provide significant built-in enhancements over the Java classes. By using the enhanced Gosu collection and list classes, you can loop through collection items to perform actions, extract item information, or sort items with single lines of code.

Gosu provides additional initializer syntax for both lists and maps similar to Gosu's compact initializer syntax for arrays.

Basic lists

Lists in Gosu inherit from the Java interface `java.util.List` and its common Java subclasses, such as `java.util.ArrayList`.

Creating a list

To create an empty list, you use generics notation to specify the type of object that the list contains, such as in this example of an `ArrayList` of `String` objects:

```
var myemptylist = new ArrayList<String>()
```

In many cases, you need to initialize the list with data. Gosu provides a natural syntax for initializing lists similar to initializing arrays.

For example, the following line is an array instantiation:

```
var s2 = new String[ ] { "This", "is", "a", "test." }
```

In comparison, the following line instantiates a new `ArrayList`:

```
var strs = new ArrayList<String>() { "a", "ab", "abc" }
```

The previous line is shorthand for the following code:

```
var strs = new ArrayList<String>()
strs.add("a")
strs.add("ab")
strs.add("abc")
```

See also

- “Generics” on page 241
- “Sorting lists or other comparable collections” on page 197

Type inference and list initialization

Because of Gosu type inference, you can use a very concise syntax to create and initialize a list.

```
var s3 = {"a", "ab", "abc"}
```

The compile-time type of the list is `java.util.ArrayList<String>`. The run-time type is different, `java.util.ArrayList<Object>`.

Gosu infers the type of the result list as the least upper bound of the components of the list. If you pass different types of objects, Gosu finds the most specific type that includes all of the items in the list.

If the types implement interfaces, Gosu attempts to preserve commonality of interface support in the list type. This behavior ensures that your list acts as expected with APIs that rely on support for the interface. In some cases, the resulting type is a compound type, which combines:

- Zero classes or one class, which might be `java.lang.Object`, if no other common class is found
- One or more interfaces

At compile time, the list and its elements use the compound type. You can use the list and its elements with APIs that expect those interfaces or the ancestor element.

For example, the following code creates classes that extend a parent class and implement an interface.

```
interface HasHello {
    function hello()
}

class TestParent {
    function unusedMethod() {}
}

class TestA extends TestParent implements HasHello{
    function hello() {print("Hi A!")}
}

class TestB extends TestParent implements HasHello{
    function hello() {print("Hi B!")}
}

var newList = { new TestA(), new TestB()}
print("Run-time type = " + typeof newList)
```

This code prints the following line.

```
Run-time type = java.util.ArrayList<java.lang.Object>
```

The compile-time type is a combination of:

- The class `TestParent`, which is the most specific type in common for `TestA` and `TestB`
- The interface `HasHello`, which both `TestA` and `TestB` implement

See also

- “Compound types” on page 356

Getting and setting list values

The following verbose code sets and gets `String` values from a list by using the native Java `ArrayList` class.

```
var strs = new ArrayList<String>(){ "a", "ab", "abc" }
strs.set(0, "b")
var firstStr = strs.get(0)
```

You can write this code in Gosu in more natural index syntax using Gosu shortcuts.

```
var strs = {"a", "ab", "abc"}
strs[0] = "b"
var firstStr = strs[0]
```

This Gosu syntax does not automatically resize lists. If a list has only three items, the following code fails at run time with an index out of bounds exception.

```
strs[3] = "b" // Index number 2 is the highest supported number
```

Basic hash maps

Maps in Gosu inherit from the Java class `java.util.HashMap`.

Creating a hash map

To create an empty map, you use generics notation to specify the type of object that the map contains. For example, define a `HashMap` that maps a `String` object to another `String` object:

```
var emptyMap = new HashMap<String, String>()
```

In many cases, you need to initialize the map with data. Gosu provides a natural syntax for initializing maps similar to initializing arrays and lists.

For example, the following code creates a new `HashMap` where "aKey" and "bKey" are keys, whose values are "aValue" and "bValue" respectively:

```
var strMap = new HashMap<String, String>(){ "aKey" -> "aValue", "bKey" -> "bValue" }
```

That code is shorthand for the following code:

```
var strMap = new HashMap<String, String>()  
strMap.put("aKey", "aValue")  
strMap.put("bKey", "bValue")
```

This Gosu syntax simplifies the declaration of static final data structures of this type, and produces more readable code than the equivalent Java code.

Types of the initialized map

For even more concise code, omit the new expression entirely, including the type:

```
var m = { "aKey" -> "aValue", "bKey" -> "bValue" }
```

Although the type name is omitted, the map is not untyped. Gosu infers that you want to create a `HashMap` by the way you use the hyphen and greater than symbol. Gosu infers the exact type of the map from the object types that you pass to the initialization.

The run-time type is always:

```
java.util.HashMap<java.lang.Object, java.lang.Object>
```

The compile-time type depends on the types of what you pass as keys and values. Gosu infers the type of the map to be the least upper bound of the components of the map. If you are consistent in the type of your keys and the types of your values, the compile-time type of the map is `HashMap<KEY_TYPE, VALUE_TYPE>`.

If you pass different types of objects to either the keys or the values, Gosu finds the most specific type that includes all of the items that you pass. Gosu finds a common ancestor class that all the objects support. If the types implement interfaces, Gosu attempts to preserve commonality of interface support in the map type. This behavior ensures that your map and its elements act as expected with APIs that rely on support for interfaces. In some cases, the resulting type at compile time is a compound type. A *compound type*, which combines zero or one classes and one or more interfaces into a single type. The purpose is for you to be able to use that list and its elements with APIs that require the interface as the argument type. At run-time, the type of the initialized map is `HashMap<Object, Object>`.

See also

- “Basic lists” on page 189
- “Generics” on page 241

Getting and setting values in a hash map

The following code sets and gets `String` values from a `HashMap`:

```
var strs = new HashMap<String, String>(){ "aKey" -> "aValue", "bKey" -> "bValue" }
strs.put("cKey", "cValue")
var valueForKey = strs.get("cKey")
```

You can write this code in the more natural index syntax using Gosu shortcuts:

```
var strs = new HashMap<String, String>(){ "aKey" -> "aValue", "bKey" -> "bValue" }
strs["cKey"] = "cValue"
var valueForKey = strs["cKey"]
```

Special enhancements on maps

Just as most methods for lists in Gosu are defined as part of Java's class `java.util.ArrayList`, most of the behavior of maps in Gosu is inherited from `java.util.Map`. Gosu provides additional enhancements to extend maps with additional features, some of which use Gosu blocks.

Map properties for keys and values

Enhancements to the `Map` class add two new read-only properties:

- `keys` – Calculates and returns the set of keys in the `Map`. This property is a wrapper for the `keySet` method.
- `values` – Returns the values of the `Map`.

Each key and value

Enhancements to the `Map` class add the `eachKeyAndValue` method, which takes a block that has two arguments: one of the key type and one of the value type. This method calls this block with each key/value pair in the `Map`, providing a more natural iteration over the `Map`.

For example:

```
var strMap = new HashMap<String, String>(){ "aKey" -> "aValue", "bKey" -> "bValue" }
strMap.eachKeyAndValue( \ key, value -> print("key : " + key + ", value : " + value ) )
```

Wrapped maps with default values

Gosu provides a class called `gw.util.AutoMap` that implements the Java interface `java.util.Map` and the Gosu interface `gw.lang.reflect.gs.IGosuObject`. This class provides an alternative to using the standard `java.util.HashMap` class.

The `AutoMap<K,V>` class provides a default value for the map if the key is not present. When you create an `AutoMap` object, you pass a Gosu block into the constructor. If your code calls `get` on the map and its value is `null`, Gosu runs the mapping block that you provide. Gosu takes the return value from the block, stores it in the map for that key, and returns the value to the original caller. All other methods delegate directly to the wrapped map.

The simplest constructor takes only one argument, which is a Gosu block that provides a default value. The block takes one argument, which is the key, and returns the default value to use. If you use this constructor, Gosu creates a new instance of `java.util.HashMap` as its wrapped map.

Another constructor takes a map (any object whose class implements `java.util.Map`) as the first argument and the Gosu block as the second argument. Use this alternative constructor if you have an existing map that you want to wrap rather than create a new map.

For example, the following code assigns values in a new map to entries that do not have values in the original map:

```
var origMap = { 1 -> "apple", 4 -> "orange" }
var newMap = origMap.toAutoMap( \ k -> "I want ${k} blueberries" )
for (i in 1..5) {
    print(newMap[i])
}
```

This code prints:


```
apple
I want 2 blueberries
I want 3 blueberries
orange
I want 5 blueberries
```

Conversion methods

The examples in the following table are based on the following code:

```
var mapper = new HashMap<String, String>()
mapper.put("aKey", "aValue")
mapper.put("bKey", "bValue")
var automap = mapper.toAutoMap(\k -> k)
```

Method	Returns	Description	Example
clear()	void	Clears the AutoMap.	<pre>automap.clear() print(automap.entrySet())</pre> Output: <pre>[]</pre>
entrySet()	java.util.Set<java.util.Map.Entry<K,V>>	Retrieves the key-value set for the AutoMap.	<pre>automap.entrySet() // Returns [bKey=bValue, aKey=aValue]</pre>
hashCode()	int	Retrieves the hash code value.	<pre>automap.hashCode() // Returns 1447807072</pre>
keySet()	java.util.Set<K>	Retrieves the set of keys for the AutoMap.	<pre>automap.keySet() // Returns [bKey, aKey]</pre>
put(java.lang.Object o, java.lang.Object o1)	java.lang.Object	Adds the new key value pair to the AutoMap.	<pre>automap.put("newKey", "newValue") print(automap.entrySet())</pre> Output: <pre>[bKey=bValue, aKey=aValue, newKey=newValue]</pre>
putAll(java.util.Map<K,V> map)	void	Adds the specified map information to the AutoMap.	<pre>var mapper2 = new HashMap<String, String>() mapper2.put("aKey2", "aValue2") mapper2.put("bKey2", "bValue2") automap.putAll(mapper2) print(automap.entrySet())</pre> Output: <pre>[bKey=bValue, aKey=aValue, bKey2=bValue2, aKey2=aValue2]</pre>
remove(java.lang.Object o)	java.lang.Object	Removes the specified object from the AutoMap.	<pre>automap.remove("aKey") print(automap.entrySet())</pre> Output:

Method	Returns	Description	Example
			[bKey=bValue]
values()	java.util.Collection<V>	Retrieves the set of values for the AutoMap.	<pre> automap.values() // Returns [bValue, aValue] </pre>

Evaluation methods

Method	Returns	Description
containsKey(java.lang.Object o)	boolean	Determines if the specified object is a key in the AutoMap.
containsValue(java.lang.Object o)	boolean	Determines if the specified object is a value in the AutoMap.
equals()	boolean	Evaluates if the specified object is in the AutoMap.
get()	java.lang.Object	Retrieves the specified object in the AutoMap.
isEmpty()	boolean	Determines if the AutoMap is empty.
size()	int	Retrieves the number of entries in the AutoMap.

List expansion (*.)

Gosu includes a special operator for array expansion and list expansion. The expansion operator is an asterisk followed by a period. Array expansion is valuable if you need a single one-dimensional array or list through which you can iterate.

The return value is as follows:

- If you use the expansion operator on an array, Gosu gets a property from every item in the array and returns all instances of that property in a new, single-dimensional, read-only array.
- If you use the expansion operator on a list, Gosu gets a property from every item in the list and returns all instances of that property in a new, single-dimensional, read-only list.

You can access elements in the new array or list individually with index notation, and you can iterate through the elements in a for loop. You cannot modify the elements of the new array or list.

The following code creates a String array. String objects in Gosu have a length property that contains the number of characters. The code uses the expansion operator on the array to create an array of string lengths for elements of the original array. Because the length property is of type int, the expansion array also is of type int.

```

var stringArray = new String[] {"one", "two", "three", "four"}
var lengthArray = stringArray*.length

for (element in lengthArray) {
    print(element)
}

```

The output from this code is:

```

3
3
5
4

```

Suppose you have an array of Book objects, each of which has a String property Name. You could use array expansion to extract the Name property from each item in the array. Array expansion creates a new array containing just the Name properties of all books in the array.

If a variable named `myArrayOfBooks` holds your array, use the following code to extract the `Name` properties:

```
var nameArray = myArrayOfBooks*.Name
```

The `nameArray` variable contains an array whose length is exactly the same as the length of `myArrayOfBooks`. The first item is the value `myArrayOfBooks[0].Name`, the second item is the value of `myArrayOfBooks[1].Name`, and so on.

Suppose you wanted to get a list of the groups to which a user belongs so that you can display the display name property of each group. Suppose a `User` object contains a `MemberGroups` property that returns a read-only array of groups to which the user belongs. In this case, the Gosu syntax `user.MemberGroups` returns an array of `Group` objects, each one of which has a `DisplayName` property. To get the display name property from each group, use the following Gosu code

```
user.MemberGroups*.DisplayName
```

Because `MemberGroups` is an array, Gosu expands the array by the `DisplayName` property on the `Group` component type. The result is an array of all the Groups to which the user belongs. The type is `String[]`.

The result might look like the following:

```
["GroupName1", "GroupName2", "GroupName14", "GroupName22"]
```

The expansion operator also applies to methods. Gosu uses the type on which the method runs to determine how to expand the type:

- If the original object is an array, Gosu creates an expanded array.
- If the original object is a list, Gosu creates an expanded list.

The following example calls a method on the `String` component of the `List` of `String` objects. Gosu generates a list of initials by extracting the first character in each `String`.

```
var s = {"Fred", "Garvin"}

// Generate the character list [F, G]
var charList = s*.charAt( 0 )
```

Important notes about the expansion operator:

- The generated array or list itself is always read-only from Gosu. You cannot assign values to elements of the array, such as setting `nameArray[0]`.
- The expansion operator `*` applies only to array expansion, not for standard property accessing.
- When using the `*` expansion operator, only component type properties are accessible.
- When using the `*` expansion operator, array properties are not accessible.
- The expansion operator applies to arrays and to any `Iterable` type and all `Iterator` types. The operator preserves the type of the array or list. For instance, if you apply the `*` operator to a `List`, the result is a `List`. All other expansion behavior is the same as for arrays.

See also

- “Enhancements on Gosu collections and related types” on page 201

Array flattening to a single dimensional array

If the property value on the original item returns an array of items, expansion behavior is slightly different. Gosu does not return an array of arrays, which is an array where every item is an array. Gosu returns an array containing all the individual elements of all the values in each array, which is known as *flattening* the array.

To demonstrate array flattening, create the following test Gosu class:

```
package test

class Family {
    var _members : String[] as Members
}
```

Next, paste the following lines into the Gosu Scratchpad:

```
uses java.util.Map
uses test.Family

// Create objects that each contain a Members property that is an array
var obj1 = new Family() { :Members = {"Peter", "Dave", "Scott"} }
var obj2 = new Family() { :Members = {"Carson", "Gus", "Maureen"} }

// Create a list of objects, each of which has an array property
var familyList : List<Family> = {obj1, obj2}

// List expansion, with FLATTENING of the arrays into a single-dimensional array
var allMembers = familyList*.Members

print(allMembers)
```

Running this program prints the following single-dimensional array:

```
["Peter", "Dave", "Scott", "Carson", "Gus", "Maureen"]
```

Examples

The following expression produces a name for each Form attached to a policy period.

```
PolicyPeriod.Forms*.DisplayName
```

The following expression returns a flattened array of display name values:

```
PolicyPeriod.ExposureUnits.Coverages*.DisplayName
```

Working with collections

Finding data in collections

Gosu provides the `firstWhere`, `where`, and `lastWhere` methods to enable you to find items in collections that match certain criteria. These functions can be very processor intensive, so be careful how you use them. Consider whether other approaches may be better, testing your code as appropriate.

The `where` method takes a block as a parameter. In turn, this block takes an element as a parameter. The block has a Boolean expression in its body that uses the element and evaluates this expression to `true` or `false`. In the case that the Boolean expression evaluates to `true`, the block returns the element.

When you call the `where` method, the block executes once for each element in a collection of elements. In executing, the block evaluates the Boolean expression for each such element. The block returns all elements for which the Boolean expression evaluates to `true`. In this way, the `where` method forms a new collection of elements. The method then returns this new collection. If the `where` method finds no matching items, it returns `null`.

The following example demonstrates how to find all items that match a criterion:

```
var strs = new ArrayList<String>(){ "a", "ab", "abc" }
var longerStrings = strs.where( \ str -> str.length >= 2 )
```

The value of `longerStrings` is `{"ab", "abc"}`. The expression `str.length >= 2` is true for both of them.

The `firstWhere` method takes a block as a parameter. As in the case of the `where` method, this block takes an element as a parameter, evaluates a Boolean expression body for the element, and returns the element in the case that the expression evaluates to `true`. The `firstWhere` method returns the first element that the block returns. If the `firstWhere` method finds no matching items, it returns `null`.

The following example demonstrates how to find the first item that matches a criterion:

```
var strs = new ArrayList<String>(){ "a", "ab", "abc" }
var firstLongerStr = strs.firstWhere( \ str -> str.length >= 2 )
```

The value of `firstLongerStr` is `"ab"` because `"ab"` is the first element in the list for which `str.length >= 2` evaluates to `true`.

The `lastWhere` method operates in a similar manner as the `where` and `firstWhere` methods. If the `lastWhere` method finds no matching items, it returns `null`.

Sorting lists or other comparable collections

Call the `sort` method on the list to sort a list in place.

```
myList.sort()
```

For descending order, use the `sortDescending` method.

By default, these methods do not use a localized sort collation order. For localized sorting, pass a comparator as a method argument.

Use `gw.api.util.LocaleUtil` to get the comparator for the current locale. For example:

```
myList.sort(LocaleUtil.getComparator())
```

If you need a comparator for a different locale, `LocaleUtil` provides methods to get a comparator by language or locale.

To retain the existing list and create a new list rather than sorting in place, use the `orderBy` and `orderByDescending` methods.

Sorting by a value other than the list element

The `sortBy` method on a list supports sorting on an element that you derive from the list element. For example, suppose you had an array list of `String` values that you want to sort by the text length. Create a block that takes a `String` and returns the sort key, which in this case is the number of characters of the parameter.

The `sortBy` method sorts the original list in place, and also returns a new list that contains the sorted values. To retain the existing list and return a new list, use the `orderBy` method.

The following example sorts strings by their length:

```
var myStrings = {"a", "abcd", "ab", "abc"}
var resortedStrings = myStrings.sortBy( \ str -> str.length )
```

The following line prints the contents of the collection of sorted strings:

```
resortedStrings.each( \ str -> print( str ) )
```

This line produces the output:

```
a
ab
abc
abcd
```

Similarly, you can use the `sortByDescending` method, which sorts in the opposite order.

For both of these methods, the block must return a comparable value. Comparable values include `Integer`, `String`, or any other values that can be compared with the greater than (`>`) or less than (`<`) operators.

By default, these methods do not use a localized sort collation order. For localized sorting, pass a comparator as a second argument to the method.

Advanced sort comparison

In some cases, comparison among your list objects is less straightforward. You might require more complex Gosu code to compare two items in the list. In such cases, use the sort method signature that takes a block that compares two list items.

The block must take two list elements and return `true` if the first element comes before the second, or otherwise return `false`. Using this method signature, the earlier sorting example looks like the following code:

```
var str1 = new ArrayList<String>(){ "a", "abc", "ab" }
var sortedStrs = str1.sort( \ str1, str2 -> str1.length < str2.length )
```

Although using the `sort` method is powerful, in most cases Gosu code is more readable if you use the `sortBy` or `sortByDescending` methods.

By default, these methods do not use a localized sort collation order. For localized sorting, pass a comparator as a second argument to the method.

Mapping data in collections

Use the `map` method to create an array list from a collection. A block expression transforms each element in the collection and stores the result as element in the new array list. For example, the following Gosu code declares and initializes an `ArrayList` of `String` objects. Then, it maps the lengths of the strings to a new array list.

```
// Declare and initialize an array list of strings.
var myStrings = new ArrayList<String>(){ "a", "b", "bb", "ab", "abc", "abcd" }

// Create a new array list with the lengths of the strings.
var lengthsOnly = myStrings.map(\ str -> str.length)
```

After the preceding code executes, the variable `lengthsOnly` is an array list, with elements 1, 1, 2, 2, 3, and 4.

Implicit type inference

In the preceding example, the `map` method takes a function that converts a `String` to its length. The code does not declare the type of the block argument `str` explicitly. At compile time, `str` is statically typed by inference as a `String` argument, from the declaration of the variable `myStrings`. That declaration uses the generics syntax `ArrayList<String>` to specify an `ArrayList` of `String` objects.

Similarly, the `lengthsOnly` variable is statically typed by inference, from the type returned by the `map` method. The block function passed to the `map` method returns an `Integer`, so the `map` method returns as an `ArrayList` of `Integer` objects.

Explicit type declaration

Although implicit type inference makes type declarations for you automatically, declare the types explicitly to increase the clarity of your code. For example, the following Gosu code declares the types for the block argument `str` and the type of object returned by the `map` method.

```
// Declare and initialize an array list of strings.
var myStrings = new ArrayList<String>(){ "a", "b", "bb", "ab", "abc", "abcd" }

// Create a new array list with the lengths of the strings.
var lengthsOnly : List<Integer> as ArrayList = myStrings.map(\ str:String -> str.length)
```

The type declaration for the argument to the block uses the declaration `:String`. The type declaration for the object that `map` method returns is more complex. The code declares that the `map` method returns an object that implements the `List` interface, with the type declaration `: List<Integer>`. The code downcasts the type of the returned object to an `ArrayList`, with the `as` keyword.

Note: The downcast to `ArrayList` is for clarity only. By convention, the Gosu compiler assumes that a type declaration of `List<T>` is a declaration of `ArrayList<T>`.

See also

- “Generics” on page 241

Iterating across collections

Suppose you want to print each element in a list. You could use the list method `each`, which can be used instead of a traditional Gosu `for` loop. The following example Gosu code chains the `map` and `each` functions to create a new list and process each element in the new list in a single Gosu statement.

```
// Declare and initialize a list of strings.
var myStrings = new ArrayList<String>(){ "a", "b", "bb", "ab", "abc", "abcd" }

// Create a new list with the lengths of the strings and
// iterate the new list to print each element
myStrings.map(\ str -> str.length).each(\ len -> print(len)) // Two methods chained in one statement
```

Use the `each` method to perform repeated actions with the elements in a collection.

You can achieve concise code by chaining the `map` and `each` methods in a single Gosu statement. Sometimes, the conciseness of method chaining makes your code hard for others to read and interpret. In these cases, you might instead assign the return value of the `map` method to a variable, and then call the `each` method on the variable. For example:

```
// Declare and initialize a list of strings.
var myStrings = new ArrayList<String>(){ "a", "b", "bb", "ab", "abc", "abcd" }

// Create a new list with the lengths of the string.
var strLengths = myStrings.map( \ str -> str.length )

// Iterate the new list to print each element.
strLengths.each( \ len -> print( len ) )
```

Method chaining discards the new list after executing the statement that includes the `map` method. If you need to access the new list returned by the `map` method later in your code, assign the return value to a variable.

Partitioning collections

Partitioning a collection creates a new `java.util.Map` of key/value pairs from values in a simple list. Gosu provides the `partition` and `partitionUniquely` methods to partition collections. Each method has a single parameter that is a Gosu block, which takes an element from the original list as the block argument. The value returned by the block becomes the key for that element in the new map. Gosu uses type inference to statically type the `Map` objects that the methods return.

The `partition` method

For example, suppose you have the following list of `String` values.

```
var myStrings = new ArrayList<String>(){ "a", "b", "bb", "ab", "abc", "abcd" }
```

The following line of code partitions the preceding list into a `Map` that uses the lengths of the various `String` values as the keys to those value.

```
var lengthsToStringsMap = myStrings.partition( \ str:String -> str.length )
```

Because some of the values in the original list have the same lengths, the created map contains four keys and six values, which the following example map definition illustrates:

```
Map { 1 -> ["a", "b"], 2 -> ["bb", "ab"], 3 -> ["abc"], 4 -> ["abcd"] }
```

If you partition a list with the `partition` method, subsequent code must handle the case where a map expression resolves to a list instead of a single value.

The `partitionUniquely` method

If you are certain that the list contains unique values, use the `partitionUniquely` method. For example, suppose you have a list with the following `String` values, each with a different length than the others.

```
var myStrings = new ArrayList<String>(){ "bb", "a", "abcd", "abc" }
```

The following line of code partitions the preceding list into a `Map` that uses the lengths of the various `String` values as the keys to those value.

```
var lengthsToStringsMap = myStrings.partitionUniquely( \ str:String -> str.length )
```

Because each value in the original list has a different length, the created map contains four keys and four values, which the following example map definition illustrates:

```
Map { 1 -> "a", 2 -> "bb", 3 -> "abc", 4 -> "abcd" }
```

If more than one list element has the same calculated value for its key, the `partitionUniquely` method throws an exception.

Example of partitioning a collection

The following example illustrates a typical case of partitioning a collection.

```

uses gw.api.database.Query
uses gw.api.database.DBFunction

// Query for new contacts from yesterday.
var contactQuery = Query.make(Contact).withLogSQL(true)
var contactResult =
    contactQuery.compare(DBFunction.DateFromTimestamp(contactQuery.getColumnRef("CreateTime")),
        Equals, java.util.Date.Yesterday ).select()

// Partition the result into a map by contact public ID.
var contactsById = contactResult.partitionUniquely( \ contact -> contact.PublicID )

```

The value of `contactsById` is a Map of contact `PublicID` values to contacts created yesterday. You can access a contact entity instance and its properties from the map by using the public ID as the key.

Converting lists, arrays, and sets

To convert arrays, lists, and sets to other types, use the following enhancement methods:

- Call the `toArray` method to convert a `Set` or a `List` to an `Array`.
- Call the `toList` method to convert a `Set` or an `Array` to a `List`.
- Call the `toSet` method to convert a `List` or an `Array` to a `Set`.
- Call the `join` method to combine the elements of an `Array`, a `List`, or a `Set` into a `String`, separated by a delimiter that you specify. For example, the following line combines all the items in the array into a `String`, separated by commas:

```
joinedString = array.join(",")
```

Converting an array or a list to a set removes duplicate values.

The following code shows the effects of various conversions:

```

// Declare and initialize an array list of strings.
var myStrings = new ArrayList<String>(){ "a", "b", "bb", "ab", "abc", "abcd" }

// Create a new array list with the lengths of the strings.
var lengthsOnly : List<Integer> as ArrayList = myStrings.map( \ str:String -> str.length )
var lengthsSet = lengthsOnly.toSet()
var lengthsArray = lengthsOnly.toArray()
var lengthsStr = lengthsOnly.join("|")
print(typeof lengthsOnly + " " + lengthsOnly.size() + " " + lengthsOnly)
print(typeof lengthsSet + " " + lengthsSet.size() + " " + lengthsSet)
print(typeof lengthsArray + " " + lengthsArray.length)
print(lengthsStr)

```

This code prints the following lines:

```

java.util.ArrayList<java.lang.Object> 6 [1, 1, 2, 2, 3, 4]
java.util.HashSet<java.lang.Object> 4 [1, 2, 3, 4]
java.lang.Object[] 6
1|1|2|2|3|4

```

Flat mapping a series of collections or arrays

Suppose you have a collection where each object has a property that is an array or collection. You can use the `flatMap` method to create a single `List` of elements from all the elements on the inner collections. You provide a block that takes the elements of the inner collection and returns them an array or collection. The `flatMap` method concatenates all the elements in the returned collections or arrays into a single `List`.

The `flatMap` method is similar to the array expansion feature of Gosu. However, the `flatMap` method is available on all collections and arrays. In addition, the `flatMap` method generates different extracted arrays dynamically using a Gosu block that you provide. Your Gosu block can perform any arbitrary and potentially complex calculation during the flat mapping process.

Example

Suppose your data has the following structure:

- A claim object has an `Exposures` property that contains an array of exposure objects.
- An exposure has a `Notes` property that contains a list of `Note` objects.

The `Claim.Exposures` property is the outer collection of exposures. The `Exposure.Notes` properties are the inner collections.

First, write a block that extracts the note objects from an exposure object:

```
\ e -> e.Notes
```

Next, pass this block to the `flatMap` method to generate a single list of all notes on the claim:

```
var allNotes = myClaim.Exposures.flatMap( \ e -> e.Notes )
```

This code generates a single list that contains all the notes on all the exposures on the claim. In generics notation, the `flatMap` method returns an instance of `List<Note>`.

See also

- “Gosu arrays and the expansion operator” on page 64

Sizes and length of collections and strings are equivalent

For collections and strings, `length` and `size` have the same meaning in Gosu. Gosu adds enhancements for the `Collection` and `String` classes to support both the `length` and `size` properties. The `length` property is deprecated on lists and sets. For these types, use the `size` property. The `size` property is deprecated on arrays. For array types, use the `length` property.

Collections and Gosu enhancements

Enhancements on Gosu collections and related types

Gosu collection and list classes rely on collection classes from the Java language. However, Gosu collections and lists have additional, built-in enhancements compared to Java. *Gosu enhancements* are Gosu methods and properties added to classes or other types without the subclassing that other languages require to make use of the new methods and properties.

See also

- “Enhancements” on page 215

List of enhancement methods and properties on collections

The following tables list the Gosu enhancements to collections and lists, all of which are contained in the `gw.lang.enhancements` package. The letter `T` refers to the type of the collection. The syntax `<T>` comes from the language feature known as generics. For example, for an argument that is listed as `conditionBlock(T):Boolean`, the argument is a block. That block must take exactly one argument of the list’s type (`T`) and return a `Boolean`.

Note: If a type letter such as `T` appears more than once in arguments or a return result, the letter represents the same type in each usage.

Most collection methods are implemented on `Iterable`, which is the parent interface of `Collection`, rather than on `Collection` or other types that extend `Iterable`.

Enhancement methods and properties on `Iterable<T>`

The following table lists the additional methods and properties for iterable objects, which are objects that implement `Iterable<T>`. The Gosu enhancements are defined in `gw.lang.enhancements.CoreIterableEnhancement`.

Note: There are also related type-specific enhancements such as `CoreIterableIntegerSumEnhancement`.

The following methods provide functionality to perform an action on every item in an iterable object. The code examples in the table are based on the following example:

```
var arrList = {1, 3, 9} // ArrayList<Integer> implements Iterable
```

Method	Description	Example
<code>each(action-block)</code>	Iterates each element of the iterable object.	<pre>var res = 0 arrList.each(\elt -> {res += elt}) print(res)</pre> The output is: <pre>13</pre>
<code>eachWithIndex(action-block)</code>	Iterates each element of the iterable object with an index.	<pre>var resList = {0, 0, 0, 0} // ArrayList<Integer> arrList.eachWithIndex(\elt, index -> {resList[index] = elt + 1}) print(resList)</pre> The output is: <pre>[2, 4, 10, 0]</pre>
<code>reverse()</code>	Returns a new <code>List<T></code> that contains all elements in this iterable object in reverse order.	<pre>var revList = arrList.reverse() print(revList)</pre> The output is: <pre>[9, 3, 1]</pre>

The following methods provide functionality to filter the elements in an iterable object. The code examples in the table are based on the following example:

```
var arrList = {1, 3, 9} // ArrayList<Integer> implements Iterable
```

Method	Description	Example
<code>removeWhere(condition-block)</code>	Removes all elements that satisfy the given criteria. This method returns no value, so it cannot be chained in series. The mutation takes place in the existing iterable object, rather than a new object being created with the specified elements removed.	<pre>arrList.removeWhere(\elt -> elt > 8) print(arrList)</pre> The output is: <pre>[1, 3]</pre>
<code>retainWhere(condition-block)</code>	Removes all elements that do not satisfy the given criteria. This method returns no value, so it cannot be chained in series. The mutation takes place in the existing iterable object, rather than a new object being created with only the specified elements.	<pre>arrList.retainWhere(\elt -> elt>8) print(arrList)</pre> The output is: <pre>[9]</pre>
<code>where(condition-block)</code>	Returns a new <code>List<T></code> that contains all elements in this iterable object that satisfy the given condition.	<pre>var resList = arrList.where(\elt -> elt>8) print(resList)</pre> The output is: <pre>[9]</pre>
<code>whereTypeIs(Type)</code>	Returns a new <code>List</code> of the given type.	<pre>var dList = {1, 3.2, 9} // ArrayList<Double> var resList = dList.whereTypeIs(Integer) //</pre>

Method	Description	Example
		<pre>List<Integer> print(resList)</pre> The output is: <pre>[1, 9]</pre>

The following methods and properties provide information about the collection. The code examples in the table are based on the following example:

```
var arrList = {1, 3, 9} // ArrayList<Integer> implements Iterable
```

Method or property	Description	Example
<code>allMatch(cond-block)</code>	Returns true if all elements in the Collection satisfy the condition.	<pre>arrList.allMatch(\elt1 -> elt1 < 10) //</pre> Evaluates to true
Count property	Returns the number of elements in the Iterable.	<pre>arrList.Count //</pre> Evaluates to 3
<code>countWhere(cond-block)</code>	Returns the number of elements in the Collection that match the given condition.	<pre>arrList.countWhere(\elt1 -> elt1 < 5) //</pre> Evaluates to 2
HasElements property	Returns true if this Collection has any elements. Use this method rather than the default collection method <code>isEmpty</code> because <code>HasElements</code> interacts better with null values. For example, the expression <code>coll.HasElements</code> does not return true if the expression <code>coll</code> is null, but <code>coll.isEmpty()</code> returns false.	<pre>arrList.HasElements //</pre> Evaluates to true
<code>hasMatch(cond-block)</code>	Returns true if this Collection contains any elements that match the given block.	<pre>arrList.hasMatch(\elt1 -> elt1 < 5) //</pre> Evaluates to true
<code>single()</code>	If there is only one element in the Iterable, returns that value. Otherwise, it throws an <code>IllegalStateException</code> exception.	<pre>var oneList = {9} // ArrayList<Integer></pre> <pre>oneList.single() //</pre> Evaluates to 9
<code>singleWhere(cond-block)</code>	If there is only one element in the collection that matches the given condition, this method returns that value. Otherwise, it throws an <code>IllegalStateException</code> exception.	<pre>print(arrList.singleWhere(\elt -> elt < 3))</pre> The output is: 1

The following methods provide functionality to get a particular element or value from a collection. The code examples in the table are based on the following example:

```
var arrList = {1, 3, 9} // ArrayList<Integer> implements Iterable
```

Method	Description	Example
<code>average()</code>	With no argument, returns the average of the numeric values in the collection.	<pre>print(arrList.average()) print(arrList.average{\elt -> (\elt % 3 == 0 ? \elt : 0)})</pre>
<code>average(block)</code>	With a block argument, returns the average of the numeric values derived from the collection.	The output is: <pre>4.333333333333333 4</pre>

Method	Description	Example
<code>first()</code>	Returns the first element in the Collection, or null if the collection is empty.	<pre>print(arrList.first())</pre> The output is: <pre>1</pre>
<code>firstWhere(cond-block)</code>	Returns the first element in the collection that satisfies the condition, or returns null if none do.	<pre>print(arrList.firstWhere{\elt -> elt < 5})</pre> The output is: <pre>3</pre>
<code>last()</code>	Returns the last element in the collection, or null if the list is empty.	<pre>print(arrList.last())</pre> The output is: <pre>9</pre>
<code>lastWhere(cond-block)</code>	Returns the last element in the collection that matches the given condition, or null if no elements match it.	<pre>print(arrList.lastWhere{\elt -> elt < 5})</pre> The output is: <pre>3</pre>
<code>max()</code> <code>max(proj-block)</code>	Returns the maximum of the selected values from the collection. This method excludes any null values. If there are no non-null values, this method throws an exception. With a block argument, returns the maximum of the values derived from the collection.	<pre>print(arrList.max())</pre> The output is: <pre>9</pre>
<code>maxBy(proj-block)</code>	Returns the maximum T of the collection based on the projection to a Comparable object.	<pre>print({"a", "ab", "abc"}.maxBy{\elt -> elt.length})</pre> The output is: <pre>abc</pre>
<code>min()</code> <code>min(proj-block)</code>	Returns the minimum of the selected values from the collection. This method excludes any null values. If there are no non-null values, this method throws an exception. With a block argument, returns the minimum of the values derived from the collection.	<pre>print(arrList.min())</pre> The output is: <pre>1</pre>
<code>minBy(proj-block)</code>	Returns the minimum T of the collection based on the projection to a Comparable object.	<pre>print({"a", "ab", "abc"}.minBy{\elt -> elt.length})</pre> The output is: <pre>a</pre>
<code>sum(proj-block)</code>	Returns the sum of the numeric values selected from the collection.	<pre>print({"a", "ab", "abc"}.sum{\elt -> elt.length})</pre> The output is: <pre>6</pre>

The following methods provide functionality to convert the collection to another class or format. The code examples in the table are based on the following example:

```
var arrList = {1, 3, 9} // ArrayList<Integer> implements Iterable
```

Method	Description	Example
<code>flatMap(proj-block)</code>	Maps each element of the collection to a Collection of values and flattens them into a single List.	Consider the following data structure: - A claim object has an Exposures property that contains an array of exposure objects. - An exposure has a Notes property that contains a list of Note objects. <code>myClaim</code> is a claim object <pre>var allNotes = myClaim.Exposures.flatMap(\ e -> e.Notes)</pre> The <code>allNotes</code> variable contains a single list that contains all the notes on all the exposures on the claim.
<code>fold(block)</code>	Accumulates the values of a <code>Collection<T></code> into a single <code>T</code> .	<pre>print(arrList.fold(\elt1, elt2 -> elt1 * elt2)) print({"a", "ab", "abc"}.fold(\elt1, elt2 - > elt1 + elt2))</pre> The output is: <pre>27 aababc</pre>
<code>join(delimiter)</code>	Joins all elements together as a string with a delimiter.	<pre>print(arrList.join("--"))</pre> The output is: <pre>[1--3--9]</pre>
<code>map(proj-block)</code>	Returns a List of each element of the <code>Collection<T></code> mapped to a new value.	<pre>print(arrList.map(\elt -> elt + 1))</pre> The output is: <pre>[2, 4, 10]</pre>
<code>partitionUniquely(proj-block)</code>	Partitions this Collection into a Map of keys to elements in this Collection. The block argument returns the keys in the map. The elements in the collection are the values in the map. An <code>IllegalStateException</code> exception is thrown if more than one element maps to the same key.	<pre>print(arrList.partitionUniquely(\elt -> elt + 1))</pre> The output is: <pre>{2=1, 4=3, 10=9}</pre>
<code>reduce(init, reducer-block)</code>	Accumulates the values of a <code>Collection<T></code> into a single <code>V</code> given an initial seed value.	<pre>print(arrList.reduce(100, \x, y -> x + y))</pre> The output is: <pre>113</pre>
<code>toCollection()</code>	If this iterable object is already a collection, this method returns the same Collection. Otherwise it creates a new Collection and copies this iterable object to it.	<pre>var coll = arrList.toCollection()</pre>
<code>toList()</code>	If this iterable object is already a list, this method returns the same List. Otherwise it creates a new List and copies this Iterable to it.	<pre>print(arrList.toList())</pre> The output is: <pre>[1,3,9]</pre>
<code>toSet()</code>	Converts the iterable object to a Set.	<pre>print(arrList.toSet())</pre>

Method	Description	Example
		The output is: <pre>[1,3,9]</pre>
<code>toArray()</code>	Converts the <code>Iterable<T></code> object into an array <code>T[]</code> .	<pre>var myTypedArray = arrList.toArray() print("myTypedArray type: \${typeof myTypedArray } ") for (i in myTypedArray) { print(i) }</pre> The output is: <pre>myTypedArray type: java.lang.Integer[] 1 3 9</pre>

The following methods provide functionality to act on two collections. The code examples in the table are based on the following example:

```
var arrList = {1, 3, 9} // ArrayList<Integer> implements Iterable
var arrList2 = {4, 9, 12, 12, 32}
```

Method	Description	Example
<code>concat(collection<T>)</code>	Returns a new <code>Collection<T></code> that is the concatenation of this iterable object and the other collection	<pre>var conclist = arrList.concat(arrList2) print(conclist)</pre> The output is: <pre>[1, 3, 9, 4, 9, 12, 12, 32]</pre>
<code>disjunction(collection<T>)</code>	Returns a new <code>Set<T></code> that is the symmetric difference of this collection and the other collection. This set contains the elements that exist in either one of the collections but not in both collections.	<pre>var sdList = arrList.disjunction(arrList2) print(sdList)</pre> The output is: <pre>[1, 3, 4, 12, 32]</pre>
<code>intersect(collection<T>)</code>	Returns a <code>Set<T></code> that is the intersection of the two collection objects. This set contains only the elements that exist in both of the collections.	<pre>var interList = arrList.intersect(arrList2) print(interList)</pre> The output is: <pre>[9]</pre>
<code>subtract(collection<T>)</code>	Returns a new <code>Set<T></code> that is the subtraction of the other collection from this collection. This set contains the elements that exist only in the collection that calls the method and do not exist in the collection that is the method argument.	<pre>var subList = arrList.subtract(arrList2) print(subList)</pre> The output is: <pre>[1, 3]</pre>
<code>union(collection<T>)</code>	Returns a new <code>Set<T></code> that is the union of the two collections. This set contains one instance of every element from both of the collections. If either collection has duplicated elements, the result contains only one instance of that element.	<pre>var unionList = arrList.union(arrList2) print(unionList)</pre> The output is: <pre>[1, 3, 9, 4, 12, 32]</pre>
<code>zip(iterable<R>)</code>	Returns a <code>List<T,R></code> of <code>gw.util.Pair</code> elements that contain corresponding values from the two iterable objects. If one iterable object contains	<pre>var zipList = arrList.zip(arrList2) print(zipList)</pre> The output is:

Method	Description	Example
	more elements than the other one, any extra elements are discarded.	<code>[(1, 4), (3, 9), (9, 12)]</code>

Enhancement methods and properties on `Collection<T>`

The Gosu enhancements are defined in `gw.lang.enhancements.CoreCollectionEnhancement.gsx`.

The following methods provide functionality to sort the elements in a collection.

Note: By default, these methods do not use a localized sort collation order. For localized sorting, pass a comparator as a second argument to the method.

The following table lists the available enhancement methods on `Collection`. The code examples in the table are based on the following example:

```
var strArrList = {"wolf cub", "puppy", "tiger cub", "kitten"} // ArrayList implements Collection
```

Method	Description	Example
<code>orderBy(proj-block)</code>	Returns a new <code>List<T></code> ordered by a block that you provide. This method is different from <code>sortBy</code> , which sorts in place.	<pre>var strArrListSorted = strArrList.orderBy{\elt -> elt.length} print(strArrListSorted)</pre> The output is: <pre>[puppy, kitten, wolf cub, tiger cub]</pre>
<code>orderByDescending(proj-block)</code>	Returns a new <code>List<T></code> reverse ordered by the given value. This method is different from <code>sortByDescending</code> , which sorts in place.	<pre>var strArrListSortedDesc = strArrList.orderByDescending{\elt -> elt.length} print(strArrListSortedDesc)</pre> The output is: <pre>[tiger cub, wolf cub, kitten, puppy]</pre>

Enhancement methods on `List<T>`

The following table lists the available enhancement methods on `List<T>`. The Gosu enhancements are defined in `gw.lang.enhancements.CoreListEnhancement.gsx`, `gw.lang.enhancements.CoreListOfComparablesEnhancement.gsx` and `gw.lang.enhancements.CoreIOrderedListEnhancement.gsx`. The code examples in the table are based on the following example:

```
var arrList = {1, 3, 9} // ArrayList implements List
```

Method	Description	Example
<code>cast(Type)</code>	Converts the <code>Collection<T></code> into a collection of <code>Type</code> . An <code>IllegalArgumentException</code> exception is thrown if the cast is invalid. For example, the expression <code>objList.cast(String)</code> throws an exception.	<pre>var objList : ArrayList<Object> // ArrayList implements List objList = {1, 3, 9} var intList = objList.cast(Integer) // ArrayList<Integer> print(intList)</pre> The output is: <pre>[1, 3, 9]</pre>
<code>copy()</code>	Creates a copy of the list as a <code>List<T></code> .	<pre>var newList = arrList.copy() print(newList)</pre> The output is:

Method	Description	Example
		[1, 3, 9]
freeze()	Returns a new unmodifiable version of the list as a List<T>. An attempt to modify the frozen list throws an UnsupportedOperationException exception.	<pre>var frozenList = arrList.freeze() print(frozenList)</pre> The output is: [1, 3, 9]
partition(proj-block)	Partitions this List object into a Map of keys to sub-lists of elements in this list. The block argument returns the keys in the map. The elements in the list are added to List objects, which are the values in the map.	<pre>var q = Query.make(Address).compare(Address#State, NotEquals, null) var qList = q.select().toList() var pMap = qList.partition(\elt -> elt.State) for (k in pMap.Keys) { print("\${k}:") for (v in pMap[k]) { print("\t\${v}") } }</pre> The output is similar to the following lines, which have been truncated: <pre>NV: 300 W Sahara Ave, Floor 0000, Developer Unit Habitation Cube #0000 ... NY: 3240 Riverdale Ave, Floor 0000, Developer Unit Habitation Cube #0000 ... Tokyo: 〒107-8556 Tokyo, 港区市, 1-1, 2-chome, Minami-Aoyama, Floor 0000, Japan ...</pre>
shuffle()	Returns a List<T> that contains the same elements in a random order.	<pre>var shuffList = arrList.shuffle() // ArrayList<Integer> print(shuffList)</pre> The output is similar to the following: [3, 1, 9]
sort() sort(comparator)	Sorts the list in place, with optional support for localized sorting. By default, this method does not use a localized sort collation order. For localized sorting, pass a comparator as an argument to the method. This method returns the original list, sorted in place. Note: Ensure the collection is cast to a sortable type.	<pre>var strArrList : ArrayList<String> // ArrayList implements List strArrList = {"wolf cub", "puppy", "tiger cub", "kitten"} var newArrList = strArrList.sort() print("strArrList: \${strArrList}") print("newArrList: \${newArrList}") print("\${strArrList == newArrList}")</pre> The output is: <pre>strArrList: [kitten, puppy, tiger cub, wolf cub] newArrList: [kitten, puppy, tiger cub, wolf cub] true</pre>

Method	Description	Example
<code>sortBy(proj-block)</code> <code>sortBy(proj-block, comparator)</code>	Sorts the list in place but instead of comparing the items directly like <code>sort</code> does, uses a block that returns the item to compare. By default, this method does not use a localized sort collation order. For localized sorting, pass a comparator as a second argument to the method. This method returns the original list, sorted in place. This method is different from <code>orderBy</code>, which returns a new <code>List<T></code> and does not sort the list in place. Note: Ensure the collection is cast to a sortable type.	<pre>var strArrList : ArrayList<String> // ArrayList implements List strArrList = {"wolf cub", "puppy", "tiger cub", "kitten"} var newArrList = strArrList.sortBy{\elt -> elt.length} print("strArrList: \${strArrList}") print("newArrList: \${newArrList}") print("\${strArrList} == newArrList")</pre> The output is: <pre>strArrList: [puppy, kitten, wolf cub, tiger cub] newArrList: [puppy, kitten, wolf cub, tiger cub] true</pre>
<code>sortByDescending(proj-block)</code> <code>sortByDescending(proj-block, comparator)</code>	Same as <code>sortBy</code>, but sorts the list in place in descending order. This method is different from <code>orderByDescending</code>, which returns a new <code>List<T></code>. This method returns the original list, sorted in place. Note: Ensure the collection is cast to a sortable type.	<pre>var strArrList : ArrayList<String> // ArrayList implements List strArrList = {"wolf cub", "puppy", "tiger cub", "kitten"} var newArrList = strArrList.sortByDescending{\elt -> elt.length} print("strArrList: \${strArrList}") print("newArrList: \${newArrList}") print("\${strArrList} == newArrList")</pre> The output is: <pre>strArrList: [tiger cub, wolf cub, kitten, puppy] newArrList: [tiger cub, wolf cub, kitten, puppy] true</pre>
<code>sortDescending()</code> <code>sortDescending(comparator)</code>	Same as <code>sort</code>, but sorts the list in place in descending order. This method returns the original list, sorted in place. Note: Ensure the collection is cast to a sortable type.	<pre>var strArrList : ArrayList<String> // ArrayList implements List strArrList = {"wolf cub", "puppy", "tiger cub", "kitten"} var newArrList = strArrList.sortDescending() print("strArrList: \${strArrList}") print("newArrList: \${newArrList}") print("\${strArrList} == newArrList")</pre> The output is: <pre>strArrList: [wolf cub, tiger cub, puppy, kitten] newArrList: [wolf cub, tiger cub, puppy, kitten] true</pre>

Enhancement methods on Set<T>

The following table lists the available methods and properties on `Set<T>`. The Gosu enhancements are defined in `gw.lang.enhancements.CoreSetEnhancement.gsx`.

The following table lists the available methods on `Set<T>`. The code examples in the table are based on the following example:

```
var set = new HashSet() // A set of type Object
set.add("String 1")
set.add("String 2")
```

Method	Description	Example
cast(Type)	Returns a copy of the set as a Set<Type> object.	<pre>var strSet = set.cast(String) print(strSet)</pre> The output is: <pre>[String 1, String 2]</pre>
copy()	Returns a copy of the set as a Set<T> object.	<pre>var newSet = set.copy() print(newSet)</pre> The output is: <pre>[String 1, String 2]</pre>
freeze()	Returns a new unmodifiable copy of the set as a Set<T> object.	<pre>var frozenSet = set.freeze() print(frozenSet)</pre> The output is: <pre>[String 1, String 2]</pre>
partition()	Partitions this Set object into a Map of keys to sub-sets of elements in this set. The block argument returns the keys in the map. The elements in this set are added to HashSet objects, which are the values in the map.	<pre>var q = Query.make(Address).compare(Address#State, NotEquals, null) var qSet = q.select().toSet() var pMap = qSet.partition{\elt -> elt.State} for (k in pMap.Keys) { print("\${k}:") for (v in pMap[k]) { print("\t\${v}") } }</pre> The output is similar to the following lines, which have been truncated: <pre>NV: 300 W Sahara Ave, Floor 0000, Developer Unit Habitation Cube #0000 ... NY: 3240 Riverdale Ave, Floor 0000, Developer Unit Habitation Cube #0000 ... Tokyo: 〒107-8556 Tokyo, 港区市, 1-1, 2-chome, Minami-Aoyama, Floor 0000, Japan ...</pre>
powerSet()	Returns the power set of the set as a Set<Set<T>> object.	<pre>var pwrSet = set.powerSet() print(pwrSet)</pre> The output is: <pre>[], [String 1, String 2], [String 1], [String 2]</pre>

See also

- “Enhancements” on page 215
- “Generics” on page 241
- “Sorting lists or other comparable collections” on page 197
- “Wrapped maps with default values” on page 192
- “Flat mapping a series of collections or arrays” on page 200

List of enhancement methods and properties on maps

The following tables list the available enhancement methods and properties on `Map<K,V>`. The symbol `K` represents the key class. The symbol `V` represents the value class. The Gosu enhancements are defined in `gw.lang.enhancements.CoreMapEnhancement.gsx`.

The following methods and properties provide functionality for general usage of a map. The code examples in the table are based on the following example:

```
var mapper = new HashMap<String, String>()
mapper.put("aKey", "aValue")
mapper.put("bKey", "bValue")
```

Method or property	Description	Example
<code>copy()</code>	Returns a <code>HashMap</code> object that is a copy of this map object. This method always returns a <code>HashMap</code> object, even if the original map was another class that implements the <code>Map</code> interface.	<pre>var newMap = mapper.copy() print(newMap)</pre> The output is: <pre>{bKey=bValue, aKey=aValue}</pre>
Count property	The number of keys in the map.	<pre>print(mapper.Count)</pre> The output is: <pre>2</pre>
Keys property	A collection of all keys in the map.	<pre>print(mapper.Keys)</pre> The output is: <pre>{bKey, aKey}</pre>
<code>toAutoMap(defaultValueBlock)</code>	Returns an <code>AutoMap</code> object that wraps the current map and provides handling of default values. The method takes a default value mapping block with the same meaning as in the constructor for the <code>AutoMap</code> object. The block takes one argument (the key) and returns a default value that is appropriate for that key.	<pre>var autoMap = new HashMap<String, String>().toAutoMap(\k -> " length \$ {mapper [k].length} mapped") autoMap["aKey"] = mapper["aKey"] + autoMap["aKey"] autoMap["new"] = "Hello World" + autoMap["new"] for (k in autoMap.Keys) { print("Key \${k} = \${autoMap[k]}") }</pre> The output is: <pre>Key new = Hello World length 0 mapped Key aKey = aValue length 6 mapped</pre>
Values property	A collection of all values in the map.	<pre>print(mapper.Values)</pre>

Method or property	Description	Example
		The output is:
		<code>{bValue, aValue}</code>

The following methods provide functionality to perform an action on every item in a map. The code examples in the table are based on the following example:

```
var mapper = new HashMap<String, String>()
mapper.put("aKey", "aValue")
mapper.put("bKey", "bValue")
```

Method	Description	Example
<code>eachKey(block)</code>	Runs a block for every key in the map. The block must take one argument (the key) and return no value.	<pre>mapper.eachKey(\k -> { /* Your method call here */ mapper[k] = mapper[k] + "Status: Reviewed"; print("Key \${k} = \${ mapper[k]}") })</pre> The output is: <pre>Key bKey = bValue Status: Reviewed Key aKey = aValue Status: Reviewed</pre>
<code>eachKeyAndValue(block)</code>	Runs a block for every key-value pair in the map. The block must take two arguments: a key and a value. The block must return no value.	<pre>var res = "Mapper" mapper.eachKeyAndValue(\k, val -> {res += " \${k}:\${val}"}) print(res)</pre> The output is: <pre>Mapper bKey:bValue aKey:aValue</pre>
<code>eachValue(block)</code>	Runs a block for every value in the map. The block must take one argument (the value) and return no value.	<pre>var res2 = "Values" mapper.eachValue(\value -> {res2 += " \${value}"}) print(res2)</pre> The output is: <pre>Values bValue aValue</pre>

The following methods provide functionality to filter and remap a map. The methods that have `retain` in the name are destructive. The methods with `filter` in the name create a new map and do not modify the original map. The code examples in the table are based on the following example:

```
var mapper = new HashMap<String, String>()
mapper.put("aKey", "aValue")
mapper.put("bKey", "bValue")
```

Method	Description	Example
<code>filterByKey(keyFilter)</code>	Returns a new map that is a clone of the original map but without entries whose keys do not satisfy the <code>keyFilter</code> expression. The key filter block must take one argument (a key) and return true or false.	<pre>var filterMap1 = mapper.filterByKey(\k -> k=="aKey") print(filterMap1)</pre>

Method	Description	Example
		The output is: <code>{aKey=aValue}</code>
<code>filterByValues(valueFilter)</code>	Returns a new map that is a clone of the original map but without entries whose values do not satisfy the <i>valueFilter</i> expression. The key filter block must take one argument (a value) and return true or false.	<pre>var filterMap2 = mapper.filterByValues(\value -> value=="bValue") print(filterMap2)</pre> The output is: <code>{bKey=bValue}</code>
<code>mapValues(mapperBlock)</code>	Returns a new map that contains the same keys as the original map but different values based on a block that you provide. The block must take one argument (the original value) and return a new value based on that value. The values can have an entirely different type. Gosu infers the return type of the block based on what you return and creates the appropriate type of map. For example, the Gosu code in the Example column takes a <code>Map<String, String></code> and maps it to a new object of type <code>Map<String, Integer></code>.	<pre>var remapper = mapper.mapValues(\value -> value.length) print(remapper)</pre> The output is: <code>{bKey=6, aKey=6}</code>
<code>retainWhereKeys(keyFilter)</code>	Destructively removes all entries whose keys do not satisfy the <i>keyFilter</i> expression. Returns true if and only if this map changed as a result of the block. The key filter block must take one argument (a key) and return true or false.	<pre>mapper.retainWhereKeys(\k -> k == "aKey") print(mapper)</pre> The output is: <code>{aKey=aValue}</code>
<code>retainWhereValues(valueFilter)</code>	Destructively removes all entries whose values do not satisfy the <i>valueFilter</i> expression. Return true if this map changed as a result of the call. The value filter block must take one argument (a value) and return true or false.	<pre>mapper.retainWhereValues(\value -> value == "bValue") print(mapper)</pre> The output is: <code>{bKey=bValue}</code>

The following methods provide functionality to use properties files for a map. The code examples in the table are based on the following example:

```
var mapper = new HashMap<String, String>()
mapper.put("aKey", "aValue")
mapper.put("bKey", "bValue")
```

The examples also use a properties file that contains the following lines:

```
name=Default values
maxAge=100
minAge=18
```

Method	Description	Example
<code>readFromPropertiesFile(file)</code>	Reads the contents of this map from a file in the Java properties format using APIs on <code>java.util.Properties</code> . The method takes a <code>java.io.File</code> object and returns a map of the keys and values. The return type is <code>Map<String,String></code> .	This example reads the text {bKey=bValue, aKey=aValue} from the file: <pre>var file = new java.io.File("c:\ \Guidewire\myprops.properties") var propsMapper : Map<String, String> propsMapper = propsMapper.readFromPropertiesFile(file) print(propsMapper)</pre> The output is: <pre>{maxAge=100, minAge=18, name=Default values}</pre>
<code>writeToPropertiesFile(file)</code> <code>writeToPropertiesFile(file, comments)</code>	Writes the contents of this map to a file in the Java properties format using APIs on <code>java.util.Properties</code> . The method takes a <code>java.io.File</code> object and returns nothing. The second method signature takes a <code>String</code> object to write comments in the properties file.	This example writes the text {bKey=bValue, aKey=aValue} to the file: <pre>var file = new java.io.File("C:\ \Guidewire\newprops.properties") mapper.writeToPropertiesFile(file)</pre> File contents: <pre># #Mon Oct 07 11:59:25 PDT 2019 bKey=bValue aKey=aValue</pre>

Enhancements

Gosu enhancements provide a way for you to augment classes and other types with additional concrete methods and properties. For example, use enhancements to define additional utility methods on a class or interface that cannot be directly modified. You can enhance Guidewire entity types and also classes defined in either Gosu or Java. The most valuable use of enhancements is to define additional utility methods on a Java class or interface that cannot be directly modified. This feature is most useful if the source code for a class is unavailable or class is marked final and so cannot be subclassed.

Comparing enhancements and subclasses

Enhancements differ from subclasses in important ways. Enhancement methods and properties on a class are available to all objects of the enhanced class. Methods and enhancements on a subclass are available only to objects of the subtype, not to objects of the superclass. Use enhancements to add functionality that the original authors of a class did not provide.

Enhancing a class makes the new methods and properties available to all instances of that class, not merely to subclass instances. For example, if you add an enhancement method to the `String` class, all `String` instances in Gosu have the additional method.

Additional differences between classes and enhancements include:

- Enhancements cannot save state information by allocating new variables or properties on the enhanced type.
- Enhancements can create new methods but cannot override existing methods.
- To reference methods on the enhanced class or type, use the symbol `this` to access the enhancements. For example, to call the enhanced object's `myAction` method, use the syntax `this.myAction()`. Do not use the keyword `super` in an enhancement.

Note: Enhancements are defined in terms of the external interface of the enhanced type. The keyword `super` implies a superclass rather than an interface, so `super` is inappropriate for enhancements.

Static dispatch of enhancements

Gosu enhancements are statically dispatched. Enhancement code is run at run time, but availability and the relevant code is determined at compile time. Availability of enhancement properties and methods is based on the compile-time type, not the run-time type. This behavior is different from regular properties and methods that are accessed at run time.

This distinction is important for two reasons:

Subtype/supertype distinctions might prevent access

Suppose a method argument is declared as the type `Object`. At compile time, the argument variable has type `Object`, even though at run time, it could be a much more specific type than `Object`. If at run time, the object type

is `java.util.ArrayList`, the enhancement method on `ArrayList` is unavailable. You can work around this limitation by downcasting (`myObject as ArrayList`) before accessing the enhancement. However, downcasting could fail at run time if the runtime type is not compatible with `ArrayList`, so you must understand the risks of that technique.

Subtype/supertype distinctions might dispatch in unexpected ways

If you have multiple enhancements that have the same names for properties and methods, there could be enhancements on more than one type in the hierarchy. For example, consider enhancements of both the `java.util.Map` interface and `java.util.HashMap` with different enhancement logic. At compile time, Gosu uses the most specific type for that programming context. If you use a variable declared to type `java.util.Map`, its enhancement methods are determined at compile time, even if at run time that object has subtype `java.util.HashMap`.

Using enhancements

You use Gosu enhancements to augment classes and other types with additional concrete methods and properties. You can also use enhancements to overcome the language shortcomings of Java or other languages defining a class or interface. For example, Java-based classes and interfaces can be used from Gosu, but they do not natively allow use of blocks, which are anonymous functions defined in-line within another function. Gosu includes many built-in enhancements to commonly used Java classes, so any Gosu code can use them.

For example, Gosu extends the Java class `java.util.ArrayList` so that you can use concise Gosu syntax to sort, find, and map members of a list. These list enhancements provide additional methods to Java lists that take Gosu blocks as parameters. The original Java class does not support blocks because the Java language does not support blocks. However, these enhancements add utilities without direct modifications to the class. Gosu makes these additional methods universally available wherever Gosu code uses `java.util.ArrayList`.

You can also enhance an interface. An enhancement does not add new methods to the interface itself, nor add new requirements for classes that implement the interface. Instead, enhancing an interface provides all classes that implement the interface with new methods and properties. For example, if you enhance the `java.util.Collection` interface with a new method, all collection types have your newly added method.

Although you can add custom properties to existing entities, those new properties do not appear in the generated Data Dictionary documentation that describes the application data model.

Syntax for using enhancements

Using an enhancement requires no special syntax. The new methods and properties are automatically available within the Gosu editor for all Gosu contexts.

For example, suppose there is an enhancement on the `String` type for an additional method called `calculateHash`. Use typical method syntax to call the method with any `String` object accessible from Gosu:

```
var s1 = "a string"
var r1 = s1.calculateHash()
```

You could even use the method on a value you provide at compile time:

```
"a string".calculateHash()
```

Similarly, if the enhancement adds a property called `MyProperty` to the `String` type, you could use code such as:

```
var p = "a string".MyProperty
```

The new methods and properties all appear in the list of methods that appears if you type a period (.) character in the Gosu editor. For example, suppose you are typing `s1.calculateHash()`. In typing this statement, after you type `s1.`, the list that appears displays the `calculateHash` method as a choice.

Creating a new enhancement

To create a new enhancement, put the file in your Gosu class file hierarchy in the package that represents the enhancement. It does not need to match the package of the enhanced type.

In Studio, right-click a package folder, and then click **New→Gosu Enhancement**. In the dialog that appears, you can enter the enhancement class name and the type, which is typically a class name or entity type name, to enhance. Studio creates a new enhancement with the appropriate syntax.

Enhancement methods can use properties already defined on the enhanced object or call other enhanced methods. Enhanced methods can also call native methods, as demonstrated by the `getPrettyLengthString` method, which calls the built-in `String` method `length()`.

You can add new properties as necessary and access those properties on the class or type within Gosu. However, you cannot save state information for the enhancement unless you can do so by using variables or properties that already exist on the enhanced type.

Syntax for defining enhancements

A special syntax is used to define new enhancements. Defining a new Gosu enhancement looks similar to defining a new Gosu class, with the following differences:

- To define an enhancement, use the keyword `enhancement` instead of `class`.
- To define what to enhance, use the syntax: `: TYPE TO EXTEND` instead of `extends CLASSTOEXTEND`.

For example, the following enhancement adds one standard method to the basic `String` class and one property:

```
package example

enhancement StringTestEnhancement : java.lang.String {

    public function myMethod(): String {
        return "Secret message!"
    }

    public property get myProperty() : String {
        return "length : " + this.length()
    }
}
```

Note: Although you can add custom properties to Guidewire entity types, these new properties do not appear in the generated *Data Dictionary* documentation that describes the application data model.

With this example, use code like the following to get values:

```
// Get an enhancement property:
print("This is my string".myProperty)

// Get an enhancement method:
print("This is my string".myMethod())
```

These lines print the following:

```
"length: 17"
"Secret message!"
```

Enhancements on arrays

To specify the enhanced type for an enhancement on an array type:

- For regular types, use standard array syntax, such as `String[]`.
- For generic types, use the syntax `T[]`, which means all arrays.

Enhancement naming conventions

The name of your enhancement must use the convention of the enhanced type name, then an optional functional description, and finally the word `Enhancement`, as shown in the following line:

```
[EnhancedTypeName][OptionalFunctionalDescriptor]Enhancement
```

For example, to enhance the `Report` class, you could call your enhancement:

```
ReportEnhancement
```

If the enhancement adds methods related to financials data, you can the enhancement's functional purpose by naming the enhancement:

```
ReportFinancialsEnhancement
```

Enhancement package conventions

Use your own company package to hierarchically group your own code and separate it from built-in types, in almost all cases. For example, you could define your enhancement with the fully qualified name `com.mycompany.ReportEnhancement`. Even if you are enhancing a built-in type, if at all possible use your own package for the enhancement class itself.

In extremely rare cases, you might need to enhance a built-in type and to use a protected property or method. If so, you might need to define your enhancement in a subpackage of the enhanced type. However, to avoid namespace conflicts with built-in types, avoid this approach if possible.

See also

- “Modifiers” on page 157
- “Naming conventions for packages and types” on page 149

Setting properties in enhancements

You can add new properties and add property set functions to set those properties. However, in contrast to a class, enhancements cannot define new variables on the type to store instance data for your enhancement. This restriction limits most types of state management if you cannot directly change the source code for the enhanced type to add more variables to the enhanced type. Enhancements cannot add new variables because different types have different property storage techniques, such as persistent database storage, Gosu memory storage, or file-based storage. Enhancements cannot transparently mirror these storage mechanisms.

Also, although enhancements can add properties, enhancements cannot override existing properties.

Within enhancement methods, your code can set other values as appropriate such as an existing class instance variable. You can also set properties with the `property set PROPERTYNAME()` syntax. For example, this enhancement creates a new settable property that appends an item to a list:

```
package doc.example

enhancement ListTestEnhancement<T> : java.util.ArrayList<T> {
    public property set LastItem(item : T) {
        this.add(item)
    }
}
```

Test this enhancement in the Gosu Scratchpad with this code:

```
uses java.util.ArrayList

var strlist = new ArrayList<String>() {"abc", "def", "ghi", "jkl"}

print(strlist)
strlist.LastItem = "hello"
print(strlist)
```

This code prints:

```
[abc, def, ghi, jkl]
[abc, def, ghi, jkl, hello]
```

Coding style

This topic lists some recommended coding practices for the Gosu language. These guidelines encourage good programming practices that improve Gosu readability and encourage code that is error-free, understandable, and maintainable by other people.

Use the following recommendations to improve the clarity and readability of your Gosu code.

Omit semicolons

Omit semicolons as line terminators, because they are unnecessary in almost all cases. Gosu code looks cleaner without semicolons.

Semicolons are only needed if you need to separate multiple Gosu statements all written on a single line in a one-line statement list. This construction is generally not recommended, but is sometimes appropriate for simple statement lists declared in-line in Gosu block definitions.

Type declarations

Omit the type declaration if you declare variables with an assignment. Instead, use `as TYPE` where appropriate. The type declaration is particularly redundant if a value must be cast to a type that is already included at the end of the Gosu statement.

The recommended type declaration style is:

```
var delplans = currentPage as DelinquencyPlans
```

Do not add the redundant type declaration:

```
var delplans : DelinquencyPlans = currentPage as DelinquencyPlans
```

The == and != operator recommendations and warnings

The Gosu `==` and `!=` operators are safe to use even if one side evaluates to `null`.

Use these operators where possible instead of using the `equals` method on objects.

The Gosu `==` operator is equivalent to the object method `equals(obj1.equals(obj2))` but the `==` operator is null-safe. The null-safety of the Gosu `==` operator is similar to the null-safety of the Java code `ObjectUtil.equals(...)`. In contrast, for both the Gosu and Java languages, the object method `myobject.equals(...)` is not null-safe.

Consider the following Gosu code that uses the `equals` method:

```
( planName.equals( row.Name.text ) )
```

Using the `==` operator is more readable, as shown in the following line:

```
( planName == row.Name.text )
```

Gosu prevents you from comparing incompatible array types for equality with the `==` and `!=` operators. For example, the following code generates a compile-time error because arrays of numbers and strings are incompatible:

```
new Number[] {1,2} == new String[] {"1","2"}
```

If the array types are comparable, Gosu does not recursively apply implicit coercion rules on the array's elements. For example, the following code evaluates to `false` because a `Number` is a subclass of `Object`, so Gosu compares the individual elements of the table:

```
new Number[] {1,2} == new Object[] {"1","2"}
```

WARNING Be careful if comparing arrays. Gosu performs recursive comparison of individual elements for compatible array types.

You can also compare objects by using the `===` and `!==` operators.

Capitalization conventions

The following table lists conventions for capitalization of various Gosu language elements:

Language element	Standard capitalization	Example
Gosu keywords	Always specify Gosu keywords correctly, typically lowercase.	<code>if</code>
Type names, including class names	Uppercase first character	<code>DateUtil</code> <code>Claim</code>
Local variable names	Lowercase first character	<code>myClaim</code>
Property names	Uppercase first character	<code>CarColor</code>
Method names	Lowercase first character	<code>printReport</code>
Package names	Lowercase all letters in packages and subpackages	<code>com.mycompany.*</code>

Because Gosu is case-sensitive, you must access existing types exactly as they are declared, including correct capitalization. Capitalization in the middle of a name is also important.

Use the Gosu editor's code completion feature to enter the names of types and properties correctly. This ensures standard capitalization.

Some entity and typelist APIs are case insensitive if they take a `String` value for the name of an entity, a property, or a typecode. However, best practice is to pass the name exactly as declared.

Booleans

Use Boolean-valued expressions. For example:

```
return ("A".equalsIgnoreCase(PR.cnaPaymentOption.Code))
```

Use the boolean data type. For example:

```
var isHoldIssuance = true, raiseEval = false // instead of isHoldIssuance = "Y", raiseEval = "N"
if (isHoldIssuance and raiseEval) {
    PolicyRevision.raiseEvalIssue( ... )
}
```

Class names

Apply uses declarations to avoid specifying fully qualified class names. For example:

```
uses com.cna.pc.webservices.plugin.AccountPlugin
...
var plugin: AccountPlugin = new AccountPlugin()
```

Array usage

Use the `length` property instead of `Libraries.Array.length()`. For example:

```
var len = baLine.AllBAVehicleEUs.length // instead of libraries.Array.length( baLine.AllBAVehicleEUs )
```

Class variable and class property recommendations

Always prefix private and protected class variables with an underscore character (`_`).

Avoid public variables. Convert public variables to properties, so that the property name, which is the interface to other code, is separated from the storage and retrieval of the value.

Although Gosu supports public variables for compatibility with other languages, the standard Gosu style is to use public properties backed by private variables rather than public variables. Gosu syntax supports creating the local variable and the public property on the same line by using the `as` keyword followed by the property name.

For example, in your Gosu classes that define class variables, use this variable declaration syntax:

```
private var _firstName : String as FirstName
```

This line declares a private variable called `_firstName`, which Gosu exposes as a public property called `FirstName`.

Do not create a public variable, like the following line:

```
public var FirstName : String // Do not do this. Public variable scope is not Gosu standard style.
```

Casting

Only use casts when necessary. For example, casting can be avoided here:

```
var searchResults = find (code in ProducerCode where ...)
for (code in searchResults) {
    if ((code).getUWBranch() == this) {} // instead of if ((code as ProducerCode).getUWBranch() ...)
}
```

Use typeis inference

To improve the readability of your Gosu code, Gosu automatically downcasts after a `typeis` expression if the type is a subtype of the original type. This behavior is particularly valuable for `if` statements and similar Gosu structures. In the Gosu code that the `if` statement bounds, you do not need to cast by using an `as TYPE` expression to the subtype. Because Gosu confirms that the object has the more specific subtype, Gosu implicitly considers that variable's type to be the subtype, within that block of code.

The structure of this type inference looks like the following code:

```
var VARIABLE_NAME : TYPE_NAME

// This comparison assumes SUBTYPE_NAME is a subtype of TYPE_NAME
if (VARIABLE_NAME typeis SUBTYPE_NAME) {

    // Use the VARIABLE_NAME as SUBTYPE_NAME in this block without casting
}
```

For example, the following example declares a variable as an `Object`, but downcasts the type to `String` in a block of code within an `if` statement.

Because of downcasting, the following code is valid:

```
var x : Object = "nice"
var strlen = 0

if ( x typeis String ) {
    strlen = x.length
}
```

This code works because the `typeis` inference is effective immediately and propagates to adjacent expressions.

Note that `length` is a property on `String`, not `Object`. By using `typeis` to downcast from `Object` to `String`, you do not need an additional cast around the variable `x`. The following code is equivalent but has an unnecessary cast:

```
var x : Object = "nice"
var strlen = 0

if ( x typeis String ) {
    strlen = (x as String).length // "length" is a property on String, not Object
}
```

Use automatic downcasting to write readable, concise Gosu code. Do not write Gosu code with unnecessary casts.

See also

- “Equality expressions” on page 78
- “Properties” on page 150
- “Basic type checking” on page 349

Indenting

The recommended base indentation level is two spaces, as shown in this example:

```
class BankAccount {
    private var _numberOfAccounts = 0 // object-level (instance) variables are preceded by an underscore
}
```

Wrapping

Ensure that wrapped lines and chained method calls have an extra indentation of four spaces from the previous line. In the following example, `verifyPolicyInvoiceItemMatching` and `returnToStatements` are indented four spaces from `yourAccountBalance.goToStatement()`, and `new BigDecimal` is indented four spaces from `verifyPolicyInvoiceItemMatching`:

```
yourAccountBalance.goToStatement( secondStatementDate )
    .verifyPolicyInvoiceItemMatching( policyPeriod, new BigDecimal( "250.00" ),
        new BigDecimal( "25.00" ), "10.0%", new BigDecimal( "225.00" ) )
    .returnToStatements()
```

Braces

Place opening curly brackets, or braces, on the same line as the function or block declaration. Place closing braces on their own line, lining up with the line the opening curly brace:

```
function giveMeMoney( yourAccount : Account, myAccount : Account, amount : BigDecimal ) : Boolean {
    var yourAccountBalance = TabBar.goToAccount( yourAccount ).balance()
}
```

Comment headers

Format comment blocks as follows:

- Use an initial `/**` and close with an `*/`.
- Precede comments with an `*`.
- Indent `/**` with two spaces and the content with three spaces, so that the content `*` lines up with the opening `/**`:

```
/**
 * Method-level comments go here, you should be answering the questions:
 * Why does this method exist?
 * Is there anything special about it?
 * Other comments, and so on.
 */
```

Arguments

When using argument lists, precede the first argument with a space and insert a space after the last argument:

```
someFunction( 10, "foo", "bar" )
```

Blocks

Short names are preferable for block parameter names. In this example, `e` is preferable to `event`:

```
var names = employees.map( \ e -> e.Name )
```

Always favor expression bodies over statement bodies. In this example, `e.Name` is favored over `{ return e.Name }`:

```
var names = employees.map( \ e -> e.Name )
var names = employees.map( \ e -> { return e.Name } )
```

Note: The return syntax is required if the lack of return causes syntactical ambiguity. For example, return syntax is required when the return type is an array.

When creating block argument lists, use:

- a space preceding the first argument
- a space after the last argument.
- a space after the right arrow
- a space after the block body

```
sampleFunction( \ arg1, arg2, arg3 -> print( arg1 + arg2 + arg3 ) )
```

Loops

Avoid loops if possible. For example, rather than testing within a `for` statement, apply the test without looping:

```
if (PR.WorkersCompLineExists and "NJ".equalsIgnoreCase(PR.BaseState)) {
    return false
}
```

Use a `return` or `break` statement when possible to exit a loop immediately. For example:

```
for (mSC in SCs) {
    if (checkAvailability(mSC, ST)) { return true }
}
return false
```

```
var bFound: Boolean = false
for (cov in covs) {
    if (cov.Selected == true) {
        bFound = true
        break // exit loop early after found
    }
}
if (!bFound) { ... }
```

Direct references

Avoid entity literals if possible. For example:

```
if (PolRev.Policy.Product.Code == "Connect") { ... } // instead of (PolRev.Policy.Product == Product( "Connect" ) )
```

Avoid unnecessary object graph traversals. For example:

```
var allCovs = PolicyLine.AllCoverages // instead of PolicyLine.PolicyRevision.BusinessAutoLine.AllCoverages
```

Searching

Use `getFirstResult()` and `getAtMostOneRow()` methods on `find()` results. For example:

```
var results = find(fp in FormPattern where ...)
return results.getFirstResult()
```

Use the `getCount()` method rather than iterating over all `find()` results to determine the number of results returned. For example:

```
var rs = find(...)
if (rs.getCount() > 100) {
```

```
...
}
```

Note: To determine whether `find()` results are empty, test using the expression `getFirstResult() == null` rather than `getCount() == 0`.

Add restrictions to `find()` method parameters to reduce the number of results returned. For example:

```
var rs = find(ps in cnaSicNaicsXref where ... and ps.naics_code == Location.IndustryCode.Code)
var xref = rs.getFirstResult()
if (xref != null) {
    print(...)
    actions.exitToNext()
}
```

Use a `HashSet` or `HashMap` to avoid $O(n^2)$ algorithms. For example:

```
var vins = new java.util.HashSet()
for (eu in baLine.AllBAVehicleEUs) {
    if (!vins.add(eu.Vin)) {
        PolicyLine.rejectWithFlowStep(...)
    }
}
```

Evaluate constant expressions first to avoid recomputation. For example:

```
if (Location.IndustryCode.Domain == "NAICS") {
    var rs = find(ps in cnaSicNaicsXref where ...)
    for (e in rs) {
        if (...) {
            print(...)
            actions.exitToNext()
        }
    }
}
```

Eliminate common sub-expressions. For example:

```
var baOwnedLiabCov = cov as BAOwnedLiabilityCov
var pubID = baOwnedLiabCov.BAOwnedLiabilityLiabTerm.PackageValue.PublicID
if (pubID == "ba:13" or pubID == "ba:14"){ ... }
```

See also

- *Best Practices Guide*

Advanced Gosu features

As well as the many features that Gosu shares with Java, Gosu includes the following additional features:

- *Static typing*, which helps you find errors at compile time.
- *Type inference*, which simplifies your code and preserves static typing.
- *Blocks*, which are in-line functions that you can pass around as objects. Some languages call these closures or lambda expressions.
- *Enhancements*, which add functions and properties to other types, even Java types. Gosu includes built-in enhancements to common Java classes, some of which add features that are unavailable in Java (such as blocks).
- *Generics*, which abstracts the behavior of a type to work with multiple types of objects. The Gosu generics implementation is 100% compatible with Java, and adds further powerful improvements.
- JSON support and dynamic typing.
- *Structural types*, which relate classes to one another based on shared characteristics rather than interfaces or inheritance.
- *Dimension types*, which include a unit as well as a value.
- *Composition*, which delegates the implementation of an interface to another class.
- Transactions for database integrity.
- XML/XSD support.
- Web service (SOAP) support.
- String templates.
- Concurrency.
- Checksums.

Annotations

Gosu annotations are a syntax to provide metadata about a Gosu class, constructor, method, property, field, or parameter. An annotation can control the behavior of the type, the documentation for the type, or the behavior of the code editor.

Annotating a class, method, type, class variable, or argument

Gosu annotations are a syntax to provide metadata about a Gosu class, constructor, method, property, field, or parameter. An annotation can control the behavior of the type, the documentation for the type, or the behavior of the code editor.

For example, annotations can indicate what a method returns or what kinds of exceptions the method might throw. You can add custom annotations and read this information at run time.

To add an annotation, type the at sign (@), followed by the annotation name immediately before the declaration of the item to annotate.

For example, the following example declares that a class is deprecated:

```
@Deprecated
class MyServiceAPI {
    public function myRemoteMethod() {}
}
```

You can use both annotations that are defined natively in Gosu and also Java annotations.

In your annotation usage, you can use either the fully qualified name of the annotation class or just the name. If you do not use the fully qualified name, add a `uses` line for that annotation class at the top of the file. For example, the following line shows the `uses` line for the `@Deprecated` annotation:

```
uses java.lang.Deprecated
```

In some cases, you follow the annotation name with an argument list within parentheses. The following example uses arguments to the annotation to specify that a function might throw a specific exception:

```
class MyClass{
    @Throws(java.text.ParseException, "If text is invalid format, throws ParseException")
    public function myMethod() {}
}
```

The annotation may not require any arguments, or the arguments may be optional. If you do not provide arguments to an annotation, you can omit the parentheses. For example, suppose you add an annotation called `MyAnnotation` that takes no arguments. You could use `MyAnnotation` in the following, verbose syntax:

```
@MyAnnotation()
```

Because there are no arguments, you can optionally omit the parentheses:

```
@MyAnnotation
```

Gosu requires argument lists to be in the same format as regular function or method argument lists:

```
// Standard Gosu argument lists
@param("str", "The String value to parse")
```

Gosu annotations optionally support the named arguments calling convention, which includes a colon before the argument name, and commas between arguments:

```
@Param(:FieldName = "str", :FieldDescription = "The String value to parse")
```

To get parameter annotations at run time for a Gosu class, get the Java backing class for the Gosu class and use the native Java reflection APIs. For example, the following class defines an add method:

```
package doc.example

uses doc.example.annotations.MyAnnotation

class MyClass {
    function add (@MyAnnotation a : Integer, b : Integer) : Integer {
        return a + b
    }
}
```

The following example retrieves the first annotation of the first parameter of the add method:

```
// Get a reference to an object of that class
var c = new MyClass()

// Get the Java backing class for the Gosu class, using reflection (not type safe)
var cls = c.getClass()

// Get a reference to the method using reflection (not type safe)
var refToMethod = cls.getMethod( "add", {Integer, Integer} )

// Using the native Java API for annotations, get the annotations list
var paramAnnotations = refToMethod.getParameterAnnotations()

// Get the first annotation of the first parameter
var firstAnnotationOnFirstParameter = paramAnnotations[0][0]
print("First annotation on first parameter: " + firstAnnotationOnFirstParameter)
```

See also

- “Named arguments and argument defaults” on page 112
- “Importing types and package namespaces” on page 114

Function argument annotations

The Gosu language supports annotations on method parameters, including Gosu block declarations. In some cases, you need to explicitly add `uses` lines to declare the annotation class to use.

For example:

```
package doc.example

uses doc.example.annotations.MyAnnotation

class TestABC {

    function add(@MyAnnotation a : Integer, b : Integer) : Integer {
        return a + b
    }

    function addAndLog(@Deprecated a : Integer, b : Integer) : Integer {
        return a + b
    }
}
```

Note: Gosu does not support special compiler or IDE behavior for Java 8 Type annotations such as `@m`, `@NonNull`, `@ReadOnly`, `@Regex`, `@Tainted`, and `@Untainted`.

See also

- “Importing types and package namespaces” on page 114

Annotations provided by Gosu

The Gosu language provides annotations that are defined in the `gw.lang.*` package. This package is always in scope, so the fully qualified annotation name is not required.

The information in annotations that provide documentation for a type appears in the Gosudoc HTML page for the type. For example, web service endpoints use the `@Param`, `@Returns`, and `@Throws` annotations to document their methods. Running the `gwb gosudoc` command generates a Gosudoc HTML page that contains the information from the annotations.

The following table lists the general annotations:

Annotation	Description	Usage limits	Parameters
<code>@AutoCreate</code>	Annotates a property to indicate that if the property is null and referenced on the left hand side of an assignment statement, a non-null value is automatically created for it before continuing the evaluation of the left hand side of the assignment. In the following example, if <code>Address</code> is null, a new <code>Address</code> entity instance is created and <code>CA</code> is assigned to its <code>State</code> property. <pre>person.Address.State = CA</pre>	Methods only	
<code>@AutoInsert</code>	Annotates a <code>List</code> property to indicate that if the property is null and referenced on the left hand side of an assignment statement, a non-null value is automatically added for it in the <code>List</code> before continuing the evaluation of the left hand side of the assignment. In the following example, <code>Hobbies</code> is a <code>List</code> property on <code>Person</code> and annotated with <code>@AutoInsert</code> . If the item at index 0 does not exist, an entity instance of the appropriate type is created and added to the list before <code>Name</code> is assigned. <pre>person.Hobbies[0].Name = "Cycling"</pre>	Methods only	
<code>@DataType</code>	Annotates properties on objects so that appropriate field validators can be added. For example, the maximum number of characters for a first name can be specified.	Gosu objects and entities.	In Studio, see the type and parameter definitions described in the <code>.dti</code> files contained in the configuration→config→datatypes folder.
<code>@Deprecated</code>	Specifies that a class, method, constructor, or property will no longer be available in a future release. Begin rewriting code to avoid using this class,	Can appear anywhere, but only once for any	A String warning to display if this deprecated class, method, or constructor is used.

Annotation	Description	Usage limits	Parameters
	method, constructor, property, or function parameter.	specific class, method, constructor, property, or function argument.	Note: Guidewire also provides a <code>@Deprecated</code> annotation that has the following parameters: <code>version:String</code> <code>value:String</code>
<code>@DoNotPublish</code>	Does not publicly publish the method to the external web service.		
<code>@Exportable</code>	A platform-level annotation indicating that the tagged property is to be published as a public member of the declaring class.		
<code>@Param</code>	Specifies the documentation of a parameter.	Methods only	<code>FieldName:String</code>: The name of the parameter. <code>FieldDescription:String</code>: Documentation in Javadoc format for the method's parameter.
<code>@Params</code>	Adds an array of <code>@Param</code> annotations to a method. This annotation does not need to be explicitly declared because declaring a <code>@Param</code> annotation automatically creates a <code>@Params</code> annotation containing it.	Methods only	
<code>@PublishedName</code>	Provides an alternative name to replace the declared name in code. Note: This annotation is deprecated		
<code>@PublishInGosu</code>	Publishes the class in Gosu.	Classes only	
<code>@Returns</code>	Specifies the documentation for the return result of the method.	Methods only, and only once for any specific method	<code>value:String</code> : Documentation in Javadoc format for the method's return value.
<code>@ShortCircuitingProperty</code>	If a property getter has this annotation, member access expressions having null values will short-circuit to zero. Boolean properties currently always short-circuit to false. For certain properties, such as <code>CoreIterableEnhancement.Count</code>, null values that were permitted in previous versions of Java might now result in a <code>NullPointerException</code> being thrown.		
<code>@SuppressWarnings</code>	Tells the compiler to suppress warnings. Gosu provides limited support for the Java annotation <code>@SuppressWarnings</code> .	Declarations of a type, function, property, constructor, field, or parameter.	<code>value:String</code> : Indicates the warnings to suppress. - Pass the argument "all" to suppress all warnings. - Pass the argument "deprecation" to suppress deprecation warnings.

Annotation	Description	Usage limits	Parameters
		Note: Local variables do not support this annotation.	For example, to suppress deprecation warnings in a Gosu class, on the line before the class declaration, add: <code>@SuppressWarnings("deprecation")</code>
@Throws	Specifies what exceptions might be thrown by this method.	Methods only	<code>ExceptionType:Class</code>: An exception type <code>ExceptionDescription:String</code>: A description in Javadoc format of the circumstances that cause the exception to be thrown, and how to interpret that exception.
@UnstableAPI	Marks a class, method, or parameter as unstable, indicating that the API might change in a future version and its use is to be avoided if possible.		

Internal annotations

Some Gosu annotations are reserved for internal use. You might see internal annotations in Gosu code that is provided in the base configuration. These annotations are not supported for your use. The following table lists these internal annotations so you can understand how they are used.

Internal annotation	Description	Usage limits
@Export	Lets a class be visible and editable in Studio.	Classes only
@ReadOnly	Lets a class be visible in Studio, but not editable. Studio does not permit copying a read-only class into the configuration directory for modification.	Classes only

No support for Java 8 type annotations

The Gosu compiler permits method parameter annotations. However, Gosu does not support compiler or IDE behavior for the Java 8 Type annotations such as:

- @m
- @NonNull
- @ReadOnly
- @Regex
- @Tainted
- @Untainted

Web service annotations

Several annotations related to publishing web services are available.

Examples

The following code uses several annotations:

```
package com.mycompany
uses java.lang.Exception

@WsiWebService
class Test {
    @Param("Name", "The user's name. Must not be an empty string.")
    @Returns("A friendly greeting with the user's name")
    @Throws(Exception, "General exception if the string passed to us is empty or null")
    public function FriendlyGreeting(Name : String) : String {
```

```
    if (Name == null or Name.length == 0) throw "Requires a non-empty string!"
    return "Hello, " + Name + "!"
  }
}
```

The following example specifies that a method is deprecated. A *deprecated* API component is temporarily available, but a future release will remove that component. Refactor any code that uses deprecated APIs to ensure that your code is compatible with future releases to simplify future upgrades.

```
package com.mycompany

class MyClass {

    @Deprecated("Don't use MyClass.myMethod(). Instead, use betterMethod().")
    public function myMethod() {print("Hello")}

    public function betterMethod() {print("Hello, World!")}
}
```

The annotation class that you are implicitly using must be in the current Gosu scope. You can ensure that it is in scope by fully qualifying the annotation. For example, if the `SomeAnnotation` annotation is defined within the package `com.mycompany.some.package`, specify the annotation as:

```
@com.mycompany.some.package.SomeAnnotation
class SomeClass {
    ...
}
```

Alternatively, import the package by using the Gosu `uses` statement and then use the annotation more naturally and concisely by using only its name:

```
uses com.mycompany.some.package.SomeAnnotation.*

@SomeAnnotation
class SomeClass {
    ...
}
```

[See also](#)

- *Integration Guide*

Getting annotations at run time

You can get annotation information from a class either directly or by getting the type from an object at run time. You can get the type of an object at run time by using the `typeof` operator, such as: `typeof TYPE`.

Annotation access at run time

You can get annotation information from a type, a constructor, a method, or a property by accessing the metadata information objects that are attached to the type.

To get a single instance of a specific annotation, call the `getAnnotation` method on the type.

For a list of multiple annotation instances of one type, call the `getAnnotationsOfType` for the specified type.

To get an annotation from targets such as constructors, methods, or properties, call the `getAnnotation` method on the item, not on the type itself. The following table shows how to get an annotation from each target type. In the examples in the table, the variable `i` represents the index in the list. In practice, you would likely search for the required target by name using List methods like `list.firstWhere(\ s -> s.Name = "MethodName")`.

Get annotations on this object Example Gosu to access the example <code>@MyAnnotation</code> annotation	
Type	<code>(typeof obj).TypeInfo.getAnnotation(MyAnnotation)</code>
Constructor for a type	<code>(typeof obj).TypeInfo.Constructors[i].getAnnotation(MyAnnotation)</code>

Get annotations on this object	Example Gosu to access the example <code>@MyAnnotation</code> annotation
Method of a type	<code>(typeof obj).TypeInfo.Methods[i].getAnnotation(MyAnnotation)</code>
Property of a type	<code>(typeof obj).TypeInfo.Properties[i].getAnnotation(MyAnnotation)</code>

Getting multiple annotations of multiple types

You can get all annotations of any annotation type using the two properties `Annotations` and `DeclaredAnnotations`. These two properties resemble the Java versions of annotations of the same names.

On types and interfaces, `Annotations` returns all annotations on this type or interface and on all its supertypes or superinterfaces. `DeclaredAnnotations` returns annotations only on the given types, ignoring supertypes or superinterfaces.

In constructors, properties, and methods, the `Annotations` and `DeclaredAnnotations` properties return the same thing: all annotations including supertypes/superinterfaces. In the examples in the table, the variable `i` represents the index in the list. In practice, you would probably search for the required target by name using `List` methods like `list.firstWhere(\ s -> s.Name = "MethodName")`.

Get all annotations on...	Example
Type	<code>(typeof obj).TypeInfo.Annotations</code>
Constructor	<code>(typeof obj).TypeInfo.Constructors[i].Annotations</code>
Method	<code>(typeof obj).TypeInfo.Methods[i].Annotations</code>
Property	<code>(typeof obj).TypeInfo.Properties[i].Annotations</code>

The return result of these methods is typed as a list of the expected type. Using the examples in the previous table, the result is of type `List<MyAnnotation>`. This type is shown using generics syntax, which means “a list of instances of the `MyAnnotation` annotation class”.

See also

- “Getting annotations at run time” on page 232
- “Generics” on page 241

Getting annotation data from a type at run time

You perform the following general tasks to get annotation data from a type at run time.

1. Get a reference to a type. If you have a reference to an object instead of its type, get the type at run time using the `typeof` operator.
2. Get the result's `TypeInfo` property.
3. To access annotations on the type, call the `getAnnotation` method, passing your annotation class name directly as a parameter. If you want to get annotations from the constructor, method, or property, get the subobject as described in the previous table, then call its `getAnnotation` method.
4. On the result, get its `Instance` property or call its `getInstance` method. The `Instance` property throws an exception if there is more than one instance of that annotation on the type in that context.
5. Cast the result to the desired annotation type using the `as` keyword. For example, suppose the result of the previous step is a variable called `res` and your annotation is called `@MyAnnotation`. Use the syntax:
`as MyAnnotation`.
6. From the result, get the data from the annotation instance with any annotation arguments defined as methods. For example, if an annotation has constructor arguments `purpose` and `importance`, get those values using methods `purpose` and `importance`.

Example

The classes `example.MyClass` and `example.MyAnnotation` demonstrate the prior steps:

```
package example

annotation MyAnnotation {
    function purpose() : String
    function importance() : int = 0
}

package example

@MyAnnotation(:purpose="to do the right thing", :importance=44)
class MyClass {
}
```

The following code uses the `example.MyClass` and `example.MyAnnotation` classes:

```
uses example.MyAnnotation
uses example.MyClass

// Get a reference to an object of that class
var c = new MyClass()

// Get the type info
var ti = (typeof c).TypeInfo

// Get the annotation of our specific annotation type
var ann = ti.getAnnotation(MyAnnotation).Instance as MyAnnotation

print("Purpose is " + ann.purpose())
print("Importance is " + ann.importance())
```

This example prints the following lines:

```
Purpose is to do the right thing
Importance is 44
```

Defining your own annotations

You can define annotations to add entirely new metadata annotations, apply them to programming declarations, and optionally retrieve information at run time. You can also get information at run time about objects annotated with built-in annotations. For example, you could mark a Gosu class with metadata and retrieve it at run time.

You can define new annotation types that can be used in other Gosu code. Annotations are defined in `.gs` files like Gosu classes, but they use a different syntax for the declaration. Your annotation can take arguments, which can set metadata that can be queried by IDEs or dynamically at run time.

The declaration syntax:

- Use the `annotation` keyword instead of using the `class` keyword.
- Define each annotation argument as if it were a class method, ensuring you declare the return type
- Append an equal sign and then the default value to specify a default value to an annotation argument.

For example, the following code defines an annotation with one `String` argument and one integer primitive, with a default value for the integer:

```
package example

annotation MyAnnotation {
    function purpose() : String
    function importance() : int = 0
}
```

You can now use this annotation on a new class:

```
package example

@MyAnnotation(:purpose="do the right thing", :importance=44)
class MyClass {
}
```

By default, this annotation can be used on any method, type, property, or constructor, and as many times as necessary.

Customizing annotation context and behavior

Gosu supports special annotations on an annotation declaration to customize its behavior. Annotations on an annotation declaration are called *meta-annotations*.

Setting inheritance of an annotation

Gosu supports the native Java meta-annotation `@Inherited`. It takes no arguments. Use it to indicate that this annotation is inherited by subtypes.

Setting documented status of an annotation

Gosu supports the native Java meta-annotation `@Documented`. It takes no arguments. This annotation indicates that an annotation is to be documented by automatic documentation tools.

See also

- “Getting annotations at run time” on page 232

Limiting annotation usage to specific code contexts

Usage of each annotation can be customized, such as allowing the annotation only under certain conditions. For example, the built-in annotation `@Returns` can appear only on methods, not on classes. To restrict usage in this way, use the native Java `java.lang.annotation.Target` annotation on the declaration of your annotation and pass a list of targets. The *target* defines where the annotation can be used.

Declare targets with a list of instances of the `java.lang.annotation.ElementType` enumeration:

- `METHOD` – This annotation can be used on a method. Gosu properties are dynamic and similar to methods, so use this target for Gosu properties.
- `TYPE` – This annotation can be used on a type, including classes
- `FIELD` – This annotation can be used on a field or enumeration constant. Because Gosu properties are dynamic and similar to methods, instead use `METHOD` for Gosu properties.
- `CONSTRUCTOR` – This annotation can be used on a constructor
- `PARAMETER` – This annotation can be used on a function argument (a parameter)
- `ANNOTATION_TYPE` – This annotation can be used on annotation type declarations

Unlike Java, Gosu does not support the value `LOCAL_VARIABLE` for annotations on local variable declarations.

If an annotation uses no `@Target` annotation, the Gosu default is to allow the annotation on all parts of a type.

You can use the compact Gosu syntax to define lists of items by surrounding the values with braces (`{}`).

Additionally, you can omit `ElementType` before the static values for the enumerations.

For example, the following annotation declaration defines valid usage to be types, including Gosu classes, and methods:

```
package example

uses java.lang.annotation.ElementType
uses java.lang.annotation.Target

@Target({ METHOD, TYPE })
annotation MyAnnotation {
    function purpose() : String
    function importance() : int = 0
}
```

Setting retention of an annotation

Gosu supports the native Java meta-annotation `@Retention`. Pass one of the following `RetentionPolicy` enumeration constants:

- `CLASS` – Annotations are recorded in the class file but need not be retained by the VM at run time.
- `RUNTIME` – Annotations are recorded in the class file and retained by the VM at run time. These annotations can be read reflectively.
- `SOURCE` – Annotations are discarded by the compiler.

For example:

```
uses java.lang.annotation.Retention

@Retention( RUNTIME)
annotation MyAnnotation {
    function purpose() : String
    function importance() : int = 0
}
```

Dimensions

Dimensions represent a quantifiable amount that can be measured in one or more units. Dimensions can include any measurable physical or abstract dimension, for example:

- Length, such as 1 mm
- Time, such as 1 day or 1 year
- Weight, such as 1 kg
- Money, such as 1 US Dollar
- Amount of computer storage space, such as 1 GB

Dimensions can reference complex combinations of dimensions. Speed can be represented by a unit that combines distance and time, such as 60 miles per hour. Acceleration can be represented as speed per time unit, such as 1 meter per second per second.

Each Gosu dimension is represented by a class that implements the interface `gw.lang.IDimension`. You create custom dimensions by implementing the `IDimension` interface. In your class declaration, parameterize the interface as follows:

```
final class YOURCLASSNAME implements IDimension<YOURCLASSNAME, UNITNAME>
```

Replace `YOURCLASSNAME` with your class name. Replace `UNITNAME` with a unit. The unit type might be a built-in numeric type like `int` or `BigDecimal`, but could also be any arbitrary type including custom types.

The default syntax for creating a dimension object is the standard `new` operator with constructor arguments. A typical dimension class uses at least one numeric constructor argument and optionally a unit type if the context requires the unit type. For example, a new monetary amount object representing 3 US Dollars might require a numeric value (3) and the unit (US Dollars):

```
var xx = new MonetaryAmount(3, Currency.TC_USD)
```

Create as many constructors as typical usage of your custom dimension type requires.

Gosu natively supports basic arithmetic functions on dimensions as part of the `IDimension` interface. The Gosu parser looks for special methods for each operator and checks the argument types to see what types of operations are supported. For example, a `Velocity` dimension can define a `multiply` method that takes a `Time` argument and returns a `Length` dimension. Gosu uses this information to let you use the `*` operator to multiply a `Velocity` and `Time` dimension, which generates a `Length` object:

```
// Multiple Velocity and Time to get a Length object representing 150 miles
var distance = velocity * time
```

The following methods implement arithmetic on a custom dimension. The argument must be the type of the object on the right hand side of the arithmetic operator. The return type can be any type.

- `add` – Addition using the `+` operator
- `subtract` – Subtraction using the `-` operator
- `multiply` – Multiplication using the `*` operator
- `division` – Division using the `/` operator
- `modulo` – Modulo (remainder after division) using the `%` operator

Additionally, you can support the unary negation operator (`-`) by implementing the `negate` method, which takes no arguments.

Dimensions expressions example (Score)

The following example creates a custom dimension called `Score` that represents a score in a game. The score measures progress for each player in a point value stored as a `BigDecimal` value.

The following is the `Score` class:

```
package example

uses java.math.BigDecimal

final class Score implements IDimension<Score, BigDecimal> {

    // Declare a Points property
    var _points : BigDecimal as Points

    /* REQUIRED BY IDimension */
    construct( p : BigDecimal) {
        _points = p
    }

    construct() {
        _points = 0 // If used with a no argument constructor, just set points to 0
    }

    override function toNumber(): BigDecimal {
        return _points
    }

    override function fromNumber(bigDecimal: BigDecimal): Score {
        return new Score(bigDecimal)
    }

    override function numberType(): Class<BigDecimal> {
        return BigDecimal
    }

    /* REQUIRED BY Comparable */
    override function compareTo(o: Score): int {
        return (_points.compareTo(o.Points))
    }

    /* Improve printing... */
    public function toString() : String {
        return "Score is " + _points + " points"
    }

    // ADDITIONAL conversions

    function fromNumber(i: int): Score {
        return new Score(i as BigDecimal)
    }

    function fromNumber(i: Integer): Score {
        return new Score(i as BigDecimal)
    }

    /* ENABLE arithmetic for Dimensions */

    function add( td: Score) : Score {
```

```

    return new Score(td.Points + _points)
}

function subtract( td: Score) : Score {
    return new Score(td.Points + _points)
}

function multiply( bd : BigDecimal) : Score {
    return new Score(bd * _points)
}

function multiply( td : Score) : Score {
    return new Score(td.Points * _points)
}

function divide( bd : BigDecimal) : Score {
    return new Score(_points / bd)
}

function divide( td: Score) : Score {
    return new Score(_points / td.Points)
}
}

```

Consider the following code that you can run in the Gosu Scratchpad:

```

uses example.Score

var x = new Score()
var y = new Score()
var addme = new Score(2)

print ("x = " + x + " ..... y = " + y)

x = x + addme
y = y + addme + addme
print ("x = " + x + " ..... y = " + y)

var z = x * y.Points
print ("z = " + z )

```

This example prints the following:

```

x = Score is 0 points ..... y = Score is 0 points
x = Score is 2 points ..... y = Score is 4 points
z = Score is 8 points

```

Built-in dimensions

The built-in dimensions are as follows:

- `gw.pl.currency.MonetaryAmount`
- `gw.api.financials.CurrencyAmount`

Both classes have a numeric component and a currency from the Currency typelist. For example, you can instantiate each of them as follows:

```

var ma = new MonetaryAmount(3, Currency.TC_USD)
var ca = new CurrencyAmount(3, Currency.TC_USD)

```


Generics

Generics is a language feature that you use to define a class or function that applies to many types by abstracting its behavior across multiple types of objects. Gosu generics work in a very similar manner to generics in Java.

You use generics to write statically typed code that can be abstracted to work with multiple types. By designing an API to apply to different types of objects, you write the code only once, and the code supports those different types. In this way, you can generalize classes or methods to work with multiple types and retain compile-time type safety.

Generics make the relationships explicit between the types of parameters and return values of a function. Generics are especially useful if any of the parameters or return value are collections. For example, you can require two arguments to a function to be homogeneous collections of the same type of object, and that the function returns the same type of collection. Designing functions to be abstract in that way enables your code and the Gosu language to infer other relationships. For example, a function that returns the first item in a collection of `String` objects is always typed as a `String`. You do not need to write coercion code with the syntax as *TYPE* with functions that use generics. Because generics increase how often Gosu can use type inference, your collection-related code can be readable, concise, and type-safe.

[See also](#)

- “More about the Gosu type system” on page 35

Overview of Gosu generics

The greatest power of Gosu generics is defining an API that works with multiple types, not just a single type. To specify that an item is not restricted to a single type requires a syntax called *parameterization*. Functions defined with generics clarify the relationships among the types of parameters and return values. For example, even though the type of a parameter is generic, the return value can be strongly typed to match the exact type of that parameter. Generics are particularly useful in specifying the type of object that a collection parameter or return value contains. For example, the method returns a “list of `MyClass` objects” or “a `MyClass` object”, rather than a “list of `Object` objects” or just “an `Object` object”.

If a method uses Gosu generics to define arguments or a return value, the API is straightforward to use for an API consumer. The syntax for defining the type of item is angle bracket notation such as *CLASS<USING_TYPE>*, *FUNCTION<USING_TYPE>*, or *COLLECTION_CLASS<OF_TYPE>*. The parameterized definition of a method or class uses a generics wildcard type inside the angle brackets. By convention, the symbol for the wildcard type is `T`, so the generics syntax is `<T>`. To use a collection that is an argument to or return value from a method that uses generics, you define the collection as a specific type. For example, the following line defines a list of `String` objects, which you can use as an argument to a generic method.

```
var myStrs = new ArrayList<String>() { address1, address2, address3, address4 }
```

Note: In practice, you sometimes do not need to define the type because of type inference or special object constructors. For example, `var myStrs = { "a", "ab", "abc" }` is equivalent to `var myStrs = new ArrayList<String>() { "a", "ab", "abc" }`.

You can create a method that parameterizes an argument as a specific type of list, in this case a list of strings. This argument has an explicit type and does not use generics to support multiple types of array list. The code in the method uses methods that are specific to the `String` type, so does not support other list types.

```
function printStrings( strs : ArrayList<String> ) {
    for ( s in strs ) {
        print( "${s} ${s.length}" )
    }
}
```

To call a method that uses a parameterized argument type, call the method in the usual way:

```
printStrings( myStrs )
```

A compile-time error occurs if you use an argument that does not match the parameterized specification on a method that uses Gosu generics to specify the argument type:

```
printStrings( {1, 2, 3, 4} ) // Array list of Integer is not an array list of String
```

Parameterizing collections and other containers is a typical use of generics, but Gosu does not limit generics support to only container types. You can define any class to use Gosu generics to generalize what the class supports or how the class works with various types.

Gosu provides two ways to use generic types:

- You can *parameterize a class*, which adds generic types to a class definition.
- You can *parameterize a method*, which adds generic types to a method definition.

Compatibility of Gosu generics with Java generics

Gosu generics are compatible with generics implemented in Java version 1.8. You can use Java utility classes designed for Java 1.8 generics and even extend them in Gosu.

No support for super syntax in Gosu generics

There is one exception in syntax for Java-Gosu compatibility, which is that Gosu does not support the syntax `<? super TYPE>`.

Reification of Gosu generics

One important difference between Gosu and Java is that Gosu generics are reified. *Reified* Gosu generics retain the actual specific type at run time. For example, at run time, you can check whether an object is an instance of `PetGroup<Cat>` or `PetGroup<Dog>` including the information in the angle brackets.

Java generics have *type erasure*, in which generics lose the generic parameter information at run time. Java generics maximize compatibility with older Java code that did not support generics.

Gosu does not reify the type of a generic Java object. If you need the reification behavior for a Java class that Gosu provides for Gosu classes, wrap the Java class in a Gosu class.

Example of reifying a Java type

The following Gosu class uses a Java `HashMap` object. This object is not reified.

```
class MyTester {
    // Return a map from two lists
    // This signature returns a Java object that has its type erased
    public reified function mapArray<K,V>(kArray : ArrayList<K>, vArray : ArrayList<V>) : HashMap<K,V> {

        // This signature returns a reified Gosu object
        // public reified function mapArray<K,V>(kArray : ArrayList<K>, vArray : ArrayList<V>) : MyMapper<K,V> {

        var theMap : HashMap<K,V>
        for (v in vArray index i) {
```

```

        theMap.put(kArray[i], v)
    }
    return theMap
}
}

```

In the `mapArray` method, the symbols `K` and `V` are used as types and Gosu matches `K` and `V` to the types of the collections passed into the method.

The following code uses this class:

```

var myKeys = new ArrayList<Integer>(){1, 2, 3, 4}
var myStrings = new ArrayList<String>(){ "a", "abcd", "ab", "123"}
var t = new MyTester()
var myMap = t.mapArray(myKeys, myStrings)

print("My map is a ${typeof myMap}")

```

The variable `myMap` is typed as `HashMap<java.lang.Object, java.lang.Object>`.

To cause reification of the `myMap` variable, define a Gosu class that wraps `HashMap`.

```

class MyMapper<K,V> extends HashMap {
}

```

Comment out the method signature in the `MyTester` class that returns a `HashMap` object and uncomment the signature that returns a `MyMapper` object. In the code that uses the `MyTester` class, the variable `myMap` is now typed as `MyMapper<java.lang.Integer, java.lang.String>`.

[See also](#)

- “Restricting generics” on page 248

Variance and Gosu generics

Variance is the assignability of one object of a generic type to another. Three types of variance exist:

Covariance

You can assign values of type `T` or any subtype of `T` to an item of type `T`.

Contravariance

You can assign values of type `T` or any supertype of `T` to an item of type `T`.

Invariance

You can assign values only of type `T` to an item of type `T`.

The context of an object's usage implies the variance of that object. Typically, Gosu can infer the appropriate variance. Gosu can determine variance even for an object that is used by a Java class rather than a Gosu class. This ability to infer variance is important for passing blocks as arguments to methods. Gosu uses the block type to infer variance, and therefore can determine that values passed to a block and assigned from the block's return value are valid. Java's lambdas must use function interfaces to supply the variance because Java does not infer variance. Gosu does not require this complexity and therefore Gosu generics are less complicated to use than Java generics. If you use Java lambdas from Gosu code, Gosu inspects the underlying Java methods to infer the correct variance.

In Java, calling a generic method or using a generic type requires explicit specification of variance by using the wildcard syntax. The following Java example shows a use of the `Function` type as a method parameter in the `Stream` interface:

```

<R> Stream<R> map(Function<? super T, ? extends R> mapper)

```

The wildcard syntax enforces `T` as contravariant and `R` as covariant. The equivalent method declaration in Gosu does not require the wildcard syntax because Gosu correctly infers the variance:

```

function map <R>(mapper(elt : T) : R) : List<R>

```

This method definition is in the `CoreIterableEnhancement` that Gosu provides for `java.lang.Iterable`. Gosu can also directly return a `List` object because Gosu does not require the intermediate `Stream` object that Java uses to overload the usage of the `map` method onto a collection type.

About covariance

Covariance means that an item of type *T* can be assigned values of type *T* or any subtype of *T*. For example, return values from a method are typically covariant from the point of view of the method. Your Gosu code stores the result in a variable that is assignable from the return type of the method.

Similarly, Gosu arrays are covariant in the same way as Java arrays. The following code shows that you can assign a value of any type that extends *Number* to an array of *Number*:

```
var numbers = new Number[3]
numbers[0] = new Integer(10)
numbers[1] = new Double(3.14)
numbers[2] = new Byte(0)
```

Covariance also allows the following example because an array *Integer[]* is a subtype of an array *Number[]*:

```
var myInts = new Integer[] {1,2,3,4}
var myNumber : Number[] = myInts
```

The following code compiles but generates an *ArrayStoreException* exception at run time:

```
myNumber[0] = 3.14 // Attempted heap pollution
```

The array store exception occurs because an *Integer* array has been assigned to *myNumber* and 3.14 is a double, which cannot be cast to a subtype of *Integer*. The exception occurs because the array is an array of *Integer* objects, not because the array access occurs through a *Number* reference.

Gosu generics are covariant by default. For example, this Gosu assignment works as expected.

```
var list : List<Object> = new ArrayList<String>()
```

Because Gosu applies covariance to this assignment, the wildcard syntax that Java requires is not necessary. Gosu does not use the *?* wildcard syntax anywhere.

In Java, the equivalent assignment is illegal because the type arguments are invariant without wildcard qualification:

```
// Does not compile because List<Object> and ArrayList<String> are not related
List<Object> list = new ArrayList<String>();

// This line compiles because the wildcard qualification explicitly accepts subclasses
List<? extends Object> list = new ArrayList<String>();
```

About contravariance

Contravariance means that an item of type *T* can be assigned values of type *T* or any supertype of *T*. For example, input parameters to a method are typically contravariant from the point of view of the method. Your Gosu code provides an argument that is assignable to the type of the parameter.

About invariance

Invariance means that an item of type *T* can be assigned values of only type *T* and cannot contain either subclasses or superclasses of type *T*. Java generics are invariant by default.

For example, in Java, a *List<T>* is invariant on the type *T* of the list. You can assign only an item of type *ArrayList<Number>* to a variable of type *ArrayList<Number>*. You cannot assign an item of type *ArrayList<Integer>* to a variable of type *ArrayList<Number>*, nor can you assign an item of type *ArrayList<Number>* to a variable of type *ArrayList<Integer>*. Java does not treat the two types as being related.

Gosu is not invariant in this case, but is covariant. In Gosu, you can assign an item of type *ArrayList<Integer>* to a variable of type *ArrayList<Number>*.

Parameterizing a class

To specify a class that operates with a generic type, define the class with the angle bracket notation *CLASSNAME<GENERIC_TYPE_WILDCARD>*. By convention, for *GENERIC_TYPE_WILDCARD*, use a one-letter name, preferably *T*. For example, you could define a class *MyClass* as *MyClass<T>*.

In the following example, the class *Genericstest* has one method that returns the first item in a list. Gosu strongly types the return value to match the type of the items in the collection:

```
class GenericsTest<T> {
    // Print out (for debugging) and then return the first item in the list, strongly typed
    public function printAndReturnFirst(aList : ArrayList<T>) : T {
        print(aList[0])
        return aList[0]
    }
}
```

Other code can use this class and pass an array list of any type to the method:

```
var myStrings = new ArrayList<String>(){ "a", "abcd", "ab", "abc" }

var t = new GenericsTest<String>()
var first = t.printAndReturnFirst(myStrings)
```

The variable `first` is strongly typed as `String` at compile time because the code uses a method that was defined with generics.

Because the code is strongly typed at compile time, the reliability of the code improves at run time.

Instantiating a generic class

You can instantiate a generic class in the following ways:

- Using the type token for the type parameter requires information about the type at run time. If you instantiate a generic class in this way in a parameterized method, you must use the `reified` keyword in the method declaration. This instantiation creates a strongly typed object that uses the explicit type that is reified by the method. For example, in the following lines of code, the type of the variable `x` is `GenericsTest<T>` where `T` is the type of the method parameter `t`:

```
public reified function makeANewGenericOfSpecifiedType<T>(t : T) {
    var x = new GenericsTest<T>()
}
```

- Specifying the type of the type parameter creates a strongly typed object that uses the explicit type. For example, in the following line of code, the type of the variable `x` is `GenericsTest<Integer>`:

```
public function makeANewGenericOfSpecificType() {
    var x = new GenericsTest<Integer>()
}
```

You can include this instantiation of `GenericsTest` in a parameterized method without using the `reified` keyword because you have provided Gosu with the necessary information for the type parameter.

- Omitting the type parameter creates a strongly typed object that uses the least restrictive type of the bound. For example, in the following line of code, the type of the variable `x` is `GenericsTest<Object>`:

```
public function makeANewGenericOfNonspecificType() {
    var x = new GenericsTest()
}
```

You can include this instantiation of `GenericsTest` in a parameterized method without using the `reified` keyword because Gosu substitutes the least restrictive type that is available for the type parameter.

Enhancements of generic classes

If you create an enhancement of a generic class, you must use the correct syntax:

- For a generic enhancement, you parameterize both the enhancement name and the class name. For example, the declaration of a generic enhancement to the `GenericsTest` class looks like the following line.

```
enhancement GenericsTest_Enh<T> : GenericsTest<T>
```

- For an enhancement that applies to a specific type on the generic class, you specify the type only on the class name. For example, the declaration of an enhancement to the `String` type of the `GenericsTest` class looks like the following line.

```
enhancement GenericsTest_StringEnh : GenericsTest<String>
```

See also

- “Enhancements” on page 215

Parameterizing a method

You can add a finer granularity of type usage by adding generic type modifiers to a method, immediately after the method name. In Gosu, this syntax of adding the generic type modifiers is called *parameterizing* the method. In other languages, this syntax of adding the generic type modifiers is known as making a *polymorphic method with a generic type*.

If the parameterized method needs information about the runtime type of the type parameter, you must add the modifier `reified` to the method declaration. Some parameterized methods do not require information about the runtime type of the type parameter. For these methods, the modifier `reified` is optional.

For example, in the following code, the class is not parameterized but one method is parameterized and does not require information about the type of `T` at run time:

```
class GenericTypesTester<T> {
    // Return whether the argument is a String
    public function isStringType<T>(t : T) : boolean {
    // public reified function isStringType<T>(t : T) : boolean { // This declaration is also valid
        return t.type.is String
    }
}
```

In the method’s Gosu code, the symbol `T` can be used as a type and Gosu matches `T` to the type of the argument passed into the method.

The following code uses this class and the parameterized method:

```
var t = new ReificationTester()
print("'abc' is a String: " + t.isStringType("abc"))
print("123 is a String: " + t.isStringType(123))
```

Running this code prints the following messages:

```
'abc' is a String: true
123 is a String: false
```

Generic methods that reify a type

To support parameterized methods that reify the generic type, Gosu provides the keyword `reified`. This keyword instructs Gosu to make information about the runtime type of the type parameter available. Gosu requires the keyword `reified` as a modifier on generic functions that are equivalent to parameterized Java functions that have type erasure. Use the modifier `reified` on a parameterized function that has any of the following characteristics, where `T` is the token for the type parameter of a generic type:

- The method uses the new keyword to create an instance of `T`.
- The method uses the new keyword to create an instance of a generic type and includes the `<T>` qualifier on the type name.
- An expression in the method uses the generic type with the `typeis` or `typeof` operator.
- The method contains code that uses `as T` to cast a value to the generic type.
- The method overrides a reified method in a parent class or implements a reified method in an interface.
- The method calls another reified method and passes an argument of a generic type to that method.

The modifier `reified` is optional on a parameterized method that does not have any of these characteristics.

Gosu forbids the modifier `reified` on methods that are not generic.

For a function that requires this modifier, the signature looks like the following examples:

```
reified function cast<N>(type : Type<N>) : List<N>
reified function partition<Q>( partitioner(elt : T):Q ) : Map<Q, List<T>>
static reified function returnLast<T>(a : ArrayList<T>) : T
```

Although some parameterized functions do not require the `reified` keyword, good practice is to make such methods reified. Using the `reified` keyword prevents error conditions if future changes to the code in the method or code that overrides the method cause reification of the parameterized type.

Using a generic collection in a generic method

A generic method that uses `new` to create a generic collection requires the `reified` keyword only if the method instantiates an object of the generic type. The `reified` keyword is optional if the `new` collection is empty. In this case, Gosu does not require the type of the collection. The `reified` keyword is also optional if the method populates the collection with existing objects. In this case, Gosu infers the type of the collection from the types of the objects. For example:

```
// reified is optional because the ArrayList is empty
reified function emptyListE<E>() : List<E> {
    return new ArrayList<E>()
}

// reified is optional because the code populates the ArrayList with an existing object
function oneListFromE<E>(e : E) : List<E> {
    return new ArrayList<E>({e})
}

// reified is required because the code creates a new object of type E to populate the ArrayList
reified function oneListE<E>() : List<E> {
    return new ArrayList<E>({new E()})
}
```

Example: Basic use of a reified method

In the following code, the class is not parameterized but one method is parameterized. Casting this to `T` requires the method to be reified:

```
class BasicReificationTester {
    public reified function returnAsT<T>(a : ArrayList<T>) : T {
        return this as T
    }
}
```

In the method's Gosu code, the symbol `T` is used as a type. Gosu matches `T` to the type of the collection passed into the method.

The following code uses this class and the reified method:

```
var myStrings = new ArrayList<String>(){ "a", "abcd", "ab", "123" }
var t = new BasicReificationTester()
var stringT = t.returnAsT(myStrings)

print("Returned item type: " + typeof stringT)
```

The variable `stringT` is strongly typed as `String`, not `Object`.

Example: Reification by accessing type properties

Accessing Gosu properties of the parameterized type requires reification. The following method accesses a Gosu property of the `T` type and requires the `reified` keyword:

```
reified function typeofClassT<T>(t : T) {
    var typ = T.TypeInfo
}
```

Example: Parameterized method that does not require the reified keyword

The following method is parameterized but does not require reification, so the `reified` keyword is optional:

```
// Return whether the argument is a String
public function isStringType<T>(t : T) : boolean {
    return t.typeis String
}
```

If future code changes to the code in this method or code that overrides this method cause reification to be necessary, the methods would not compile. Good practice is to use the `reified` keyword.

```
// Return whether the argument is a String
public reified function isStringType<T>(t : T) : boolean {
    return t.type.is String
}
```

Example: Another parameterized method that does not require the `reified` keyword

In the following example, the class `Genericstest` is parameterized:

```
class Genericstest<T> {
    // Print out (for debugging) and then return the first item in the list, strongly typed
    public function printAndReturnFirst(aList : ArrayList<T>) : T {
        print(aList[0])
        return aList[0]
    }
}
```

The following method is parameterized but does not require reification, so the `reified` keyword is optional:

```
// Create an unspecified type of Genericstest
public function makeANewGenericOfUnspecifiedType<T>(aList : ArrayList<T>) {
    var x = new Genericstest().printAndReturnFirst(aList)
}
```

Because the code does not specify the type of `Genericstest`, Gosu uses the the lowest possible bound of `Genericstest` for the type of the variable `x`, which is `Object`.

If future modifications to the code in this method or code that overrides this method cause reification to be necessary, the methods would not compile. Good practice is to use the `reified` keyword.

Restricting generics

You can use restrictions on generics to define a parameter that supports a specific set of types. For example, you might want to support homogeneous collections or only instances of a class and its subclasses or subinterfaces. A *homogeneous collection* is one in which all items are of the same type.

Consider a custom class `Shape`. You might need a method to support collections of circle shapes or collections of line shapes, where both `Circle` and `Line` classes extend the `Shape` class. For this example, assume the collections are always homogeneous and never contain a mix of both types. You might attempt to define a method like this:

```
public function DrawMe(shapeArray : ArrayList<Shape>)
```

The method accepts an argument of only type `ArrayList<Shape>`. Your code would not compile if you tried to pass an `ArrayList<Circle>` to the method, even though `Circle` is a subclass of `Shape`.

You can specify support of multiple types but limit support to certain types and types that extend those types. Use the syntax `extends TYPE` after the type wildcard character. In Java, this restriction is known as *bounded generics*.

For example:

```
public function DrawMe(shapeArray : ArrayList<T extends Shape>)
```

A valid argument for the parameter `shapeArray` is an `ArrayList` containing objects that are all of the same type, and that type extends the `Shape` class.

Example: Generic class using a restricted bound for its type parameter

The following class restricts the generic type to types that extend `Number`:

```
class BoundedGenericsTest<T extends Number> {
    // Print out (for debugging) and then return the first item in the list, strongly typed
    public function printAndReturnFirst(aList : ArrayList<T>) : T {
        print(aList[0])
        return aList[0]
    }
}
```


The following sections use this class in code examples.

Instantiating a generic class that is restricted

Instantiating a generic class that is restricted is very similar to instantiating a generic class that is not restricted. The only difference is that if you specify a type parameter for the type token, the restriction must apply to that type.

You can instantiate a generic class that is restricted in the following ways:

- Using the type token for the type parameter requires information about the type at run time. If you instantiate a generic class in this way in a parameterized method, you must use the `reified` keyword in the method declaration. This instantiation creates a strongly typed object that uses the explicit type that is reified by the method. For example, in the following lines of code, the type of the variable `x` is `BoundedGenericsTest<T>` where `T` is the type of the method parameter `t`:

```
public reified function makeANewGenericOfSpecifiedType<T extends Integer>(t : T) {
    var x = new BoundedGenericsTest<T>()
}
```

- Specifying the type of the type parameter creates a strongly typed object that uses the explicit type. For example, in the following line of code, the type of the variable `x` is `BoundedGenericsTest<Integer>`:

```
public function makeANewGenericOfSpecificType() {
    var x = new BoundedGenericsTest<Integer>()
}
```

You can include this instantiation of `BoundedGenericsTest` in a parameterized method without using the `reified` keyword because you have provided Gosu with the necessary information for the type parameter.

- Omitting the type parameter creates a strongly typed object that uses the least restrictive type of the bound. For example, in the following line of code, the type of the variable `x` is `BoundedGenericsTest<Number>`:

```
public function makeANewGenericOfNonspecificType() {
    var x = new BoundedGenericsTest()
}
```

You can include this instantiation of `BoundedGenericsTest` in a parameterized method without using the `reified` keyword because Gosu substitutes the least restrictive type of the specified bound for the type parameter.

See also

- “Instantiating a generic class” on page 245

Arithmetic operations on type parameter restricted to Number types

Gosu does not support arithmetic operations directly on objects of a type that is restricted to a subclass of `Number`. You must use casting if you need to perform these operations. For example, the following code does not compile:

```
function doArithmetic<T extends java.lang.Number>(d1 : double, d2 : T) : T {
    var result = d1 + d2
    return result
}
```

You must cast an input parameter to a valid numeric type. Similarly, you must cast a result to the generic type. The following code demonstrates this technique:

```
function doArithmetic(d1 : double, d2 : T) : T {
    var d3 = d2 as Double
    var result = d1 + d3
    return result as T
}
```

As always for code that uses casting, you must be certain that the values you use can be cast in this way at run time.

Restricting parameterized enhancements of classes

If you create an enhancement of a parameterized class that applies to a restricted set of types, you must use the correct syntax. For example, consider a parameterized class, `GenericsTest` that has the following declaration:

```
class GenericsTest<T>
```

For an enhancement that applies to a restricted set of type on the generic class, you specify the restrictions on the enhancement name and the type wildcard character on the class name. For example, the declaration of an enhancement to types that extend `CharSequence` types of the `GenericsTest` class looks like the following line:

```
enhancement GenericsTest_CharSeqEnh<T extends CharSequence> : GenericsTest<T>
```

See also

- “Enhancements” on page 215

Using generics with collections and blocks

Common uses of generics apply them to collections and to block functions. You can combine these uses of generics to create API methods that are relevant for many purposes. Blocks are anonymous functions that can be passed as parameter objects to other methods.

If a method takes a block as a parameter, you can use generics to describe the set of blocks that the method supports. The parameter definition specifies the number of arguments, the argument types, and the return type of the block. For example, consider a block that takes a `String` parameter and returns another `String`. The definition of the block itself indicates one argument, the parameter type `String`, and the return type `String`. In the definition of the block parameter on the method, you can use generics to specify how the parameter to the block and its return type match the type on the parameterized class.

For example, Gosu provides an enhancement to the Java `List` class that provides additional functionality to any collection. This enhancement provides multiple methods that rely on generics to determine the type of the collection. A typical example of this use of generics is the list enhancement method `sortBy`, which takes a block parameter. That block takes exactly one argument, which must be the same type as the items in the list. For example, if the list type is `ArrayList<String>`, the block must take a `String` parameter. The following code shows the method definition in the enhancement.

```
enhancement CoreListEnhancement<T> : List<T> {
  ...
  function sortBy( value(elt : T):Comparable, comparator : Comparator = null) : List<T>
  {
    ...
  }
}
```

In the definition of the enhancement, the method, and the block parameter, the symbol `T` is treated as the type of the collection. Note the use of the letter `T` in the block signature:

```
value(elt : T):Comparable
```

That signature specifies that the block parameter takes one argument of type `T` and returns a `Comparable` value, such as an `Integer` or `String`.

Consider the following array list of strings:

```
var myStrings = new ArrayList<String>(){ "a", "abcd", "ab", "abc" }
```

You can re-sort the list based on the length of the strings by using a block. Create a block that takes a `String` parameter and returns the sort key, in this case the text length. The `List.sortBy(...)` method implements the sorting algorithm to sort the original list in place, and does not return an entirely new list.

The following example uses the `sortBy` method to sort a list of `String` objects by the length of the `String`, and then prints the sorted list values.

```
var resortedStrings = myStrings.sortBy( \ str -> str.length() )
resortedStrings.each( \ str -> print( str ) )
```

This code prints:

```
a
ab
```

```
abc
abcd
```

Using generics for type inference in cases like this example produces readable and concise Gosu code.

This example omitted the types of the block argument `str` and the sort key. You do not have to specify these types explicitly. Although the following code is valid, Gosu uses type inference to determine the types.

```
var resortedStrings = myStrings.sortBy( \ str : String -> str.length() as Integer )
```

See also

- “Blocks” on page 181
- “Collections” on page 189

Multiple dimensionality generics

Typical use of generics is with one-dimensional objects, such as lists of a certain type of object. For example, a list of `String` objects or a collection of `Address` objects are one-dimensional.

Gosu generics are flexible and support multiple dimensions, just like Java generics. Generics that specify multiple dimensions can define the use of a `Map`, which stores a set of key-value pairs and is like a two-dimensional collection.

By using generics, you can define behavior for multiple types of maps. For example, you can use two-dimensional generics to define a method signature:

```
public function getHighestValue( theMap : Map<K,V> ) : V
```

The parameter `theMap` has type `Map`. Gosu generics uses two wildcard types as single capital letters separated by commas to parameterize the types in the map. By convention, the first wildcard type (`K`) represents the type of the key and the second wildcard type (`V`) represents the type of the value. Because the method signature uses the `V` again in the return value type, the Gosu compiler makes assumptions about relationships between the type of map and the return value. The two uses of `V` in the method signature specify the same type for both the map value in the parameter and the return value.

At run time, the symbols `K` and `V` are assigned to specific types. The code that calls the method creates new instances of the parameterized class with specific types. The compiler can determine which method to call and the types that `K` and `V` represent.

A specific type of map that has keys of type `Long` and values of type `String` has a definition like the following line of code.

```
Map<Long, String> contacts = new HashMap<Long, String>()
```

If you pass the variable `contacts` to this `getHighestValue` method, the compiler knows that this call to the method returns a `String` value.

You can also define a class that uses multiple dimensions of types. For example, to work with key-value maps, you can use generics to define a class:

```
class Mymapping<K,V> {
  public function put( key : K, value : V ) {...}
  public function get( key : K ) : V {...}
}
```

In the method definitions, the values in angle brackets have the same meanings as the type names in the parameterized class definition. In this example, the `K` and `V` symbols specify the types. Use these symbols in method signatures in the class to represent types in arguments, return values, and Gosu code inside the method. You can use this class to provide strongly typed results.

In the following example, the concrete types are `String` for `K` and `Integer` for `V`.

```
var myMap = new Mymapping<String, Integer>()
myMap.put("ABC", 29)

var theValue = myMap.get("ABC")
```

The `theValue` variable is strongly typed at compile time as `Integer`.

Because the code is strongly typed at compile time, the reliability of the code improves at run time.

Generics with custom containers

Although Gosu generics are often used with collections and lists, there is no requirement to use generics only with `Collection` and `List` interfaces. Any class that represents a container for other objects can use Gosu generics to define the type of items in the container.

Abstract example

Suppose you need to store key-value maps. You can define a class that uses the `Object` class to define two types of object:

```
class Mymapping {
  function put( key : Object, value : Object) {...}
  function get( key : Object) : Object {...}
}
```

Alternatively, you can use generics to define the class:

```
class Mymapping<K,V> {
  function put( key : K, value : V) {...}
  function get( key : K) : V {...}
}
```

This class enforces strongly typed values at compile time. In the following code, the `theValue` variable is strongly typed at compile time as `Integer`.

```
var myMap = new Mymapping<String, Integer>
myMap.put("ABC", 29)

var theValue = myMap.get("ABC")
```

Real-world example

Consider an application that tracks the construction of vehicles in multiple factories. You can represent cars with `Car` objects, trucks with `Truck` objects, and vans with `Van` objects. All these classes derive from a root class `Vehicle`.

Your application uses a custom container object of type `FactoryGroup` that does not extend a collection class. For this example, assume that each factory only constructs one type of vehicle. A `FactoryGroup` object can contain multiple `Car` objects, or multiple `Truck` objects, or multiple `Van` objects.

You need an API to work with all of the following types:

- A `FactoryGroup` containing one or more `Car` objects
- A `FactoryGroup` containing one or more `Truck` objects
- A `FactoryGroup` containing one or more `Van` objects

You can use generics to parameterize the definition of the class by using the following syntax:

```
public class FactoryGroup<T>
```

You can then represent the specific factory groups by using the syntax:

- `FactoryGroup<Car>`
- `FactoryGroup<Truck>`
- `FactoryGroup<Van>`

Consider a method in your application that returns all vehicles in the last step of a multistep manufacturing process. The method definition is like the following line:

```
public function getLastStepVehicles(groupOfVehicles : FactoryGroup<T>) : FactoryGroup<T>
```

By using generics, the method supports all types of `FactoryGroup` objects. Because the letter `T` appears more than once in the method signature, this syntax defines relationships between the parameter and the return value. The

method `getLastStepVehicles` takes one argument that is a factory group containing any one vehicle type. The method returns another factory group that is guaranteed to contain the same type of vehicle.

Alternatively, you could define your method with a bounded wildcard specification for the type:

```
public function getLastStepVehicles(groupofvehicles : FactoryGroup<T extends Vehicle>) : FactoryGroup<T>
```

By using this approach, your code can make more assumptions about the type of object in the factory group. This approach also prevents some coding errors, such as accidentally passing `FactoryGroup<String>` or `FactoryGroup<Integer>`, which fail at compile time. You can discover your coding errors quickly.

Dynamic types and expando objects

Gosu supports dynamic language features that permit coding styles similar to non-statically typed languages. The `dynamic.Dynamic` type allows dynamic assignment as well as dynamic dispatch of property access and method invocation. Gosu also provides support for *expando objects*, which simplify dynamic property and method access for typical code contexts.

Overview of dynamic language features

Gosu is a statically typed language employing a nominal type system. A *nominal type system* is one in which type assignability is based on declared type names, type ancestry, and declared interfaces. Because Gosu is statically typed, in general programming, access to properties is dependent on the property definitions of the declared class. In typical code, these restrictions encourage coding styles that catch most types of programming errors at compile time, and support refactoring code safely.

In rare cases, dynamic coding styles can be useful. The Gosu type system supports two features that expand support for dynamic type access:

- **Dynamic type declaration** – Declare a variable or function parameter as the type `dynamic.Dynamic` to allow assigning any reference type. This type declaration enables dynamic dispatch of property and method invocation by the name of the property or method.
- **Expando object support** – Expando objects are objects that use the dynamic declaration and store properties as key-value mappings in a `javax.script.Bindings` object. That class extends the `java.util.Map` class.

You can use these features to use coding styles that are common in languages that are not statically typed.

WARNING Use dynamic language features judiciously because misuse can hide errors that the Gosu editor cannot catch at compile time. For example, if you misspell a property or method name, there is no compiler error but run time errors can occur.

Note that these two features are separate from structural types. *Structural types* are similar to interfaces but can be used with objects with no shared ancestry or interfaces.

See also

- “Structural types” on page 263

Dynamic type

In a variable declaration or function parameter, you can declare the type as the type `Dynamic`. The `Dynamic` type permits any type of object to be assigned to that variable. The fully qualified name is `dynamic.Dynamic`. At first

glance, this might be similar to using the `Object` type, which is the base class of all reference types. However, there are important differences.

By declaring the `Dynamic` type for the variable or function argument, you can programmatically access any property or method without requiring compile-time confirmation of its existence. For example, get the `Name` or `Age` property from an object even if it does not exist at compile time. In contrast, if you use the `Object` type, the Gosu compiler validates any property or method on the `Object` class itself, which does not have many properties or methods.

On its own, the dynamic feature can be used for dynamic programming styles where property names and method names may not be known at compile type. To dynamically get properties, set properties, and invoke methods, you can optionally implement special methods on the target class directly on the declaring class or with a Gosu enhancement.

WARNING Use dynamic language features judiciously because misuse can hide errors that the Gosu editor cannot catch at compile type. For example, if you misspell a property or method name, there is no compiler error but run time errors can occur.

The following table lists the special methods you may implement on classes who objects are accessed by variables declared with the `dynamic.Dynamic` type. In the following table, the term `UNHANDLED` refers to value `gw.lang.reflect.IPlaceholder.UNHANDLED`.

Dynamic type method name	Description
<code>\$getProperty</code>	Gets a property by its name specified as an argument. If the property is unknown, return the value <code>UNHANDLED</code> . If you return <code>UNHANDLED</code> , Gosu calls the <code>\$getMissingProperty</code> method, which must exist or Gosu throws an exception.
<code>\$getMissingProperty</code>	Similar to <code>\$getProperty</code> , but Gosu only calls this method if the type does not explicitly declare the property. Gosu calls this method if some code sets a property and either of the following is true: - The <code>\$getProperty</code> method is absent and the property is not defined explicitly on the class - The <code>\$getProperty</code> method exists and it returned <code>UNHANDLED</code>.
<code>\$setProperty</code>	Sets a property by its name specified as an argument, with the value in the second argument. If the property is unknown, return the value <code>UNHANDLED</code> . If you return <code>UNHANDLED</code> , Gosu calls the <code>\$setMissingProperty</code> method, which must exist or Gosu throws an exception.
<code>\$setMissingProperty</code>	Similar to <code>\$setProperty</code> , but Gosu only calls this method if the type does not explicitly declare the property. Gosu calls this method if some code sets a property and either of the following is true: - The <code>\$setProperty</code> method is absent and the property is not defined explicitly on the class - The <code>\$setProperty</code> method exists and it returned <code>UNHANDLED</code>.
<code>\$invokeMethod</code>	Invokes a method by its name specified as an argument, with the parameter list in the second argument as an array of <code>Object</code> instances. If the method is unknown or the arguments have the wrong number or types, return the value <code>UNHANDLED</code> . If you return <code>UNHANDLED</code> , Gosu calls the <code>\$invokeMissingMethod</code> method, which must exist or Gosu throws an exception.
<code>\$invokeMissingMethod</code>	Similar to <code>\$invokeMethod</code> , but Gosu only calls this method if the type does not explicitly declare the method. Gosu calls this method if some code invokes a method and either of the following is true: - The <code>\$invokeMethod</code> method is absent and the method with matching argument number and types is not defined explicitly on the class - The <code>\$invokeMethod</code> method exists and it returned <code>UNHANDLED</code>.

The following example defines a simple object with a dynamic property getter method. In the example, the `$getProperty` method handles two property names explicitly, and then returns `UNHANDLED` for any other property names. Note that for the unhandled property name, Gosu automatically calls the `$getMissingProperty` method.

In real-world code, the `$getProperty` method might get the value from a `java.util.Map`, from a related subobject, from a calculation, or even from an external system:


```
package doc.example

uses gw.lang.reflect.IPlaceholder
uses dynamic.Dynamic

class DynamicGetter {

    // Handle any property name. Dynamically return
    // values for two values, and return UNHANDLED for others
    public function $getProperty(fieldName: String): Object {
        print("$getProperty called with field: " + fieldName)

        if (fieldName == "StreetAddress") {
            return "123 Main Street"
        } else if (fieldName == "FirstName") {
            return "John"
        } else {
            return IPlaceholder.UNHANDLED
        }
    }

    // Handle any unhandled requests for missing property names
    public function $getMissingProperty(fieldName: String): Object {
        print("$getMissingProperty called with field: " + fieldName)
        return ("fakeValue")
    }
}
```

The following code uses the dynamic object and gets a property from it:

```
uses doc.example.DynamicGetter
uses dynamic.Dynamic

var obj : Dynamic = new DynamicGetter()

print("---")
print("Dynamically get a known property...")
print("RESULT = " + obj.StreetAddress)

print("---")
print("Dynamically get a missing property...")
print("RESULT = " + obj.OtherField)
```

This code prints the following:

```
---
Dynamically get a known property...
$getProperty called with field: StreetAddress
RESULT = 123 Main Street
---
Dynamically get a missing property...
$getProperty called with field: OtherField
$getMissingProperty called with field: OtherField
RESULT = fakeValue
```

The dynamic type works well with the IExpando interface and Expando objects. Use these objects together to create dynamic property getters and setters and store data in a `java.util.Map` object.

[See also](#)

- “Expando objects” on page 257
- “Structural types” on page 263
- “Enhancements” on page 215

Expando objects

An expando object is an object whose properties, and potentially methods, are created dynamically on assignment. Most dynamic languages provide what are called *expando objects*. For object oriented programming, some languages provide only expando objects. An expando object behaves a lot like a map, in that you associate named

keys with values. In contrast to a map, with expando objects the keys appear as natural properties directly on the object.

IExpando interface

To enable dynamic programming with a map-based implementation, Gosu provides the `IExpando` interface, which contains the following method and property declarations. Any class that implements this interface can behave as an expando object in the context of a variable declared as the `Dynamic` type.

The base configuration provides the `Expando` class, which is an implementation of this interface.

Get field value

```
override function getField(s: String): Object
```

The `getFieldValue` method gets a field and returns an `Object`.

Set field value

```
override function setField(s: String, o: Object)
```

The `setFieldValue` method sets a field with an `Object`.

Invoke method

```
override function invoke(s: String, objects: Object[]): Object
```

The `invoke` method invokes a method with zero, one, or more arguments passed as an array.

Set default field value

```
override function setDefaultFieldValue(s: String)
```

The `setDefaultFieldValue` method sets a field default value. Gosu calls this method in certain cases where Gosu code tries to get an uninitialized field value. The only situation where Gosu calls the method is during automatic creation of intermediary objects in an object path. For example:

```
var dyn = new Dynamic()
dyn.abc.def = "hi"
print( dyn.abc.def ) // prints "hi"
```

In this example, note the middle line. Because the `dyn.abc` property does not exist yet, and it is necessary to set the `abc.def` property, Gosu creates the intermediate object. During this automatic creation process, Gosu calls the `setDefaultFieldValue` to create the default value for `abc`. In the built-in `Expando` implementation, this method always creates another instance of the `Expando` class.

See also

- “Expando objects” on page 257
- “The Expando class” on page 258
- “Property assignment triggering instantiation of intermediate objects” on page 51

The Expando class

Gosu provides a default implementation of the `IExpando` interface called `gw.lang.reflect.Expando`. This default implementation delegates to a `java.util.HashMap` for data storage.

The property name is always a `String` value, the value is an `Object`. If you get an uninitialized property, the value is `null`.

For any methods, the value must be a Gosu block. If you try to call a method that does not exist, the method returns the special value `IPlaceholder.UNHANDLED`.

The following example creates an Expando instance in a variable declared to type Dynamic.

```
uses gw.lang.reflect.Expando
uses dynamic.Dynamic

// Create new Expando object in a Dynamic variable
var villain : Dynamic = new Expando()

// Set dynamic properties and methods, which are stored internally in a java.util.HashMap
villain.Health = 10
villain.punch = \-> { if ( villain.Health > 0 ) villain.Health-- }
villain.isDead = \-> villain.Health <= 0

// Use the Expando object with a natural, readable syntax:
print("Health = " + villain.Health)

while ( !villain.isDead() ) {
    villain.punch()
    print("After punch, health = " + villain.Health)
}

print("Health = " + villain.Health)
```

This example prints:

```
Health = 10
After punch, health = 9
After punch, health = 8
After punch, health = 7
After punch, health = 6
After punch, health = 5
After punch, health = 4
After punch, health = 3
After punch, health = 2
After punch, health = 1
After punch, health = 0
Health = 0
```

See also

- “Blocks” on page 181
- “Dynamic type” on page 255
- “Structural types” on page 263
- “Enhancements” on page 215

Concise syntax for dynamic object creation

Gosu supports a concise object creation syntax similar to JSON directly in its grammar. The following Gosu features provide this functionality:

- You can omit the type name in a new expression if Gosu can infer the type. For example, instead of typing `new TestClass(arg1, arg2)`, you can type `new(arg1, arg2)`. The programming context already assumes the compile-time type called `TestClass`.
- You can initialize fields during instantiation using a special syntax with curly braces around a comma-delimited list of property assignments, with a colon before the property name. For example:

```
new TestClass() { :Name = "John Smith" }
```

- Gosu supports dynamic non-type-safe access to properties and methods for variables of the type called `dynamic.Dynamic`. No declaration of specific property names or method names is required in advance. For programming contexts of type `dynamic.Dynamic`, a newly instantiated object assumes the default run-time type of `Expando`. The `Expando` class implements the `Bindings` interface. Gosu adds enhancement methods to this class to convert to JSON (`toJson`), XML (`toXml`), or Gosu object initialization code (`toGosu`).

When you combine these features, you can create objects with multiple types of data without declaring any specific structure in advance, and can convert to other formats as needed.

The following Gosu code creates a dynamic complex object and prints it in JSON syntax, XML syntax, and Gosu object initialization syntax:

```
var person: dynamic.Dynamic

person = new() {
  :Name = "John Smith",
  :Age = 39,
  :Address = new() {
    :Number = 123,
    :Street = "Main St.",
    :City = "Foster City",
    :State = "CA"
  },
  :Hobby = {
    new() {
      :Category = "Sport",
      :Name = "Swimming"
    },
    new() {
      :Category = "Recreation",
      :Name = "Hiking"
    }
  }
}

print("**** To JSON:")
print(person.toJson())

print("**** To XML:")
print(person.toXml())

print("**** To Gosu:")
print(person.toGosu())
```

This example prints the following:

```
**** To JSON:
{
  "Name": "John Smith",
  "Age": 39,
  "Address": {
    "Number": 123,
    "Street": "Main St.",
    "City": "Foster City",
    "State": "CA"
  },
  "Hobby": [
    {
      "Category": "Sport",
      "Name": "Swimming"
    },
    {
      "Category": "Recreation",
      "Name": "Hiking"
    }
  ]
}
**** To XML:
<object>
  <Name>John Smith</Name>
  <Age>39</Age>
  <Address>
    <Number>123</Number>
    <Street>Main St.</Street>
    <City>Foster City</City>
    <State>CA</State>
  </Address>
  <Hobby>
    <li>
      <Category>Sport</Category>
      <Name>Swimming</Name>
    </li>
    <li>
      <Category>Recreation</Category>
      <Name>Hiking</Name>
    </li>
  </Hobby>
```

```
</object>
**** To Gosu:
new Dynamic() {
  :Name = "John Smith",
  :Age = 39,
  :Address = new() {
    :Number = 123,
    :Street = "Main St.",
    :City = "Foster City",
    :State = "CA"
  },
  :Hobby = {
    new() {
      :Category = "Sport",
      :Name = "Swimming"
    },
    new() {
      :Category = "Recreation",
      :Name = "Hiking"
    }
  }
}
```

Note that for the Gosu initializer syntax serialization, you can evaluate the `String` data at run time to create a Gosu object:

```
var clone = eval( person.toGosu() )
```

See also

- “Optional omission of type name with the new keyword” on page 85
- “Object initializer syntax” on page 86
- “Dynamic types and expando objects” on page 255

Structural types

Use structural typing to write code that works with objects with similar features but no common inheritance and interface declarations. Define structural types similar to defining interfaces, by specifying the common properties and method signatures. However, use the `structure` keyword, not the `interface` keyword.

Structural types are weaker than interfaces with respect to the amount of enforced type information they have in them. However, their flexibility supports situations where interfaces are ineffective or impossible. Structural types extend static typing to include a broader set of real-world situations but still support concise code that catches common coding problems at compile time.

Gosu is a statically typed language employing a nominal type system. A *nominal type system* is one in which type assignability is based on declared type names, type ancestry, and declared interfaces. For instance, in Gosu a type called `Test1` is assignable to interface `Interface1` only if `Test1` declares `Interface1` in its hierarchy explicitly.

Another form of static typing called *structural typing* determines assignability based on declared type features, such as methods and properties. The type called `B` is structurally assignable to a structural type `StructA` if `B` has compatible versions of all the methods and properties of `StructB`. The type `B` does not have to formally declare that it implements `StructA`. Structural typing is more flexible than nominal typing because it measures a type based on its capability, not its name or inheritance declarations.

Overcoming the limitations of nominal typing

Strictness with nominal typing is not always desirable. Consider the following three nominally unrelated types that might be part of a third-party Java library whose code you do not control:

```
// Three different classes that have X and Y properties
// but they have no shared inheritance nor shared interfaces that define X and Y

class UIObject {
    public var X : double
    public var Y : double
    public var H : double
    public var W : double
}
class Rectangle {
    private var _x : double as X
    private var _y : double as Y
    private var _w : double as Width
    private var _h : double as Height
}
class Point {
    private var _x : double as X
    private var _y : double as Y
}
```

Suppose you want to write code that just gets the X and Y properties and you want your code to work with instances of any of the three classes.

Although each of these classes defines methods for obtaining the X and Y properties, these properties do not appear as part of a common interface or ancestor class. Remember that in this example, the classes are defined in a third-party library, so you cannot control the class declarations. Their only base class in common is `Object`, but `Object` does not have the properties X or Y. To write code that reads the X and Y properties for these three different classes, you must write three distinct yet nearly identical versions of the code.

Situations like this are common. Typically programmers resolve the problem by duplicating potentially large amounts of code for each related use cases, which makes it more difficult to maintain the code.

Structural typing helps these types of situations. Use structural typing in Gosu to unite the behavior of the three classes based on the presence of being able to read the X and Y properties.

Just like you would declare an interface, define the method signatures. However, use the `structure` keyword:

```
package docs.example

structure Coordinate {
    property get X() : double // get the "X" public property or field
    property get Y() : double // get the "Y" public property or field
}
```

You can now create code that uses the new structural type `docs.example.Coordinate`.

A variable declared as the structural type can contain any object with compatible qualities defined by the structural type. For example, a variable defined to the structural type `Coordinate` can be assigned with a `Rectangle` instance. This works because `Rectangle` has the X and Y properties of type `double`:

```
var coord : Coordinate
coord = new Rectangle()
print("X value is " + coord.X)
```

To illustrate the flexibility of structural typing, suppose you want to write a function that gets the X and Y properties and prints them and performs arithmetic with the values. Using structural typing, implementing this behavior once works with any class instance that satisfies the requirements of the structural type. For example, the following function takes the structural type as a parameter:

```
public function printCoordinate(myStruct : Coordinate) {
    print("X value is " + myStruct.X)
    print("Y value is " + myStruct.Y)
    print("Sum of X and Y value is " + (myStruct.X + myStruct.Y) )
}
```

In many ways this coding style is similar to using an interface defined by the `interface` keyword. You can call this function and pass any object that satisfies the requirements of the structural type. However, unlike interfaces, this code works independent of inheritance or interface declarations for the classes.

Use the following example to see how this function can be used by multiple concrete classes that are unrelated. Paste the following into the Gosu Scratchpad:

```
// Three different classes that have X and Y properties
// but have no shared inheritance nor shared interfaces that define X and Y

class Rectangle {
    private var _x : double as X
    private var _y : double as Y
    private var _w : double as Width
    private var _h : double as Height
}

class Point {
    private var _x : double as X
    private var _y : double as Y
}

class UIObject {
    public var X : double
    public var Y : double
    public var H : double
    public var W : double
}
```



```
// DEFINE A STRUCTURAL TYPE
structure Coordinate {
    property get X() : double
    property get Y() : double
}

// Define a function that works with any class that is compatible with the structural type
public function printCoordinate(myStruct : Coordinate) {
    print("X value is " + myStruct.X)
    print("Y value is " + myStruct.Y)
    print("Sum of X and Y value is " + (myStruct.X + myStruct.Y) )
}

// Use that function with three different classes that all contain X and Y public fields
// or properties, but are unrelated

var p = new Point()
p.X = 2
p.Y = 3
printCoordinate(p)

var r = new Rectangle()
r.X = 4
r.Y = 5
printCoordinate(r)

var ui = new UIObject()
ui.X = 6
ui.Y = 7
printCoordinate(ui)
```

Running this code prints the following lines:

```
X value is 2.0
Y value is 3.0
Sum of X and Y value is 5.0
X value is 4.0
Y value is 5.0
Sum of X and Y value is 9.0
X value is 6.0
Y value is 7.0
Sum of X and Y value is 13.0
```

These examples demonstrate that structural types are statically weaker than interfaces regarding the amount of enforced type information, but their flexibility supports situations where interfaces are ineffective or impossible. Structural types extend static typing to include a broader set of real-world situations but still support concise code that catches common coding problems at compile time.

Create a structural type with getters, setters, and methods

About this task

To create a structural type, you perform the following general steps:

Procedure

1. In the package for your new type, create a Gosu class file with a `.gs` file extension.
2. In the Gosu editor, change the word `class` to `structure`.
3. Just as you would declare on an interface, add property declarations and method signatures:

- To provide data from a variable or property on an instance of this type, use property getter syntax:

```
property get PROPERTY_NAME() : TYPE
```

- To set the value of a variable or property on an instance of this type, use property setter syntax:

```
property set PROPERTY_NAME(TYPE)
```

- To provide a callable method on an instance of this type, use the syntax:

```
public function METHOD_NAME(ARG_LSIT) : TYPE
```

Example

```
package docs.example

structure DemoStructure{
    property get Name() : String
    property set Name(n : String)
    public function count(s : String) : double
}
```

This example declares a structural type called `DemoStructure` that has the following features:

- Name property of type `String`
- count method, which takes one `String` parameter and returns a double value

See also

- “Interfaces” on page 173
- “Adding static constant variables to a structural type” on page 266

Adding static constant variables to a structural type

In both Gosu and Java, you can add static variables to types to declare data that exists at the type level. For Gosu classes, you add such data to a class by adding a declaration line with the `static` modifier:

```
public static var MyVariable
```

For structural type definitions, any variables that store state are implicitly static and constant so you can omit the word `static`:

```
public var MyVariable
```

Adding a `var` declaration does not imply the ability to get or set a variable or property on an instance of the type. A `var` declaration on a structural type defines a static constant variable on the type. A static constant variable is conceptually similar to a static variable on a Gosu class. However, as for a variable on an interface, the `static` keyword is implicit and the value is read-only. The declaration does not specify the ability to get or set a variable or property on an instance of this type.

For example, note the `Age` variable in the following example:

```
package docs.example

structure DemoStructure2 {
    property get Name() : String
    property set Name(n : String)
    public var Age : Integer    // Compilation error. A static constant variable requires initialization
}
```

The `var` declaration on the structural type creates a static constant variable, which must be initialized, so the previous code causes a compilation error. Initializing the value of the static constant variable clears the error:

```
package docs.example

structure DemoStructure2 {
    property get Name() : String
    property set Name(n : String)
    public var AGE : Integer = 18 // OK because the static constant variable is initialized
}
```

You can add the `static` modifier in the declaration of the `AGE` variable, but in structural type declarations, this modifier is implicit and therefore redundant.

Although you can use property getter and setter syntax to define data to get and set on instances, you do not use this syntax for static constant variables. Therefore, static constant variables do not require assignment compatibility between the structural type and other types.

The DemoStructure2 class from the previous example is compatible with the following class:

```
package docs.examples

class MyClass {
    public var Name: String
    public var UnusedField1: String
    public var UnusedField2: String
}
```

Notice the following characteristics of MyClass:

- The Gosu class MyClass has a Name variable, which is the only requirement to be compatible for assignment to a variable of the structural type DemoStructure2. Because MyClass is a Gosu class and not a structural type, the Name variable is a natural instance variable and is not static. Instance variables and properties on a concrete type are compatible with getters and setters on the structural type, so these are compatible names and types.
- The Gosu class MyClass does not have an AGE variable or property. Because the AGE variable on the structural type is a static constant, it exists on the structural type only. For a static constant variable, the concrete type does not require compatibility for assignment between the types.

For example, you can now run the following code in the Gosu Scratchpad:

```
uses docs.examples.DemoStructure2

uses docs.examples.MyClass

var obj : MyClass
var struct : DemoStructure2

// Create a new instances of the concrete type
obj = new MyClass()

// Assign the MyClass object to a variable declared as the structural type
struct = obj

// Set the shared field on the variable of the structural type, and it affects both
struct.Name = "John"
print("struct.Name is " + struct.Name)
print("obj.Name is " + obj.Name)

// Set the shared field on variable of type MyClass, and it affects both
obj.Name = "Eric"
print("struct.Name is " + struct.Name)
print("obj.Name is " + obj.Name)

// struct.AGE is a STATIC CONSTANT and is not shared, and is read-only
print("struct.AGE (a STATIC CONSTANT) is " + struct.AGE)

// The following line is not valid if you uncomment it. Static constant fields are READ-ONLY.
// struct.AGE = 40
```

This code prints the following:

```
struct.Name is John
obj.Name is John
struct.Name is Eric
obj.Name is Eric
struct.AGE (a STATIC CONSTANT) is 18
```

See also

- “Create a structural type with getters, setters, and methods” on page 265

Assignability of an object to a structural type

If a variable is declared as a structural type, any object whose type satisfies the structural type can be assigned to that variable. The participating members of a structural type are:

- Property getters
- Property setters
- Methods

Adding a `var` declaration to a structural type does not specify the ability to get or set a variable or property on an instance of this type. Instead, a `var` declaration defines a static constant variable on the type. To get or set data from a variable and properties on an instance of the structural type, you must use the property getter and setter syntax in its declaration.

Property assignment compatibility

For properties, assignment compatibility extends to primitive types. For example, if a structure defines a property as type `double`, a compatible property can define it as a class that defines it as an `int`. Because an `int` can coerce to a `double` with no loss of precision, the property is call-compatible.

Method assignment compatibility

For methods, given type `T` and structure `S`, a `T` method is structurally assignable to an identically named method in `S` if all of the following are true:

- The number of parameters in both methods is the same.
- The parameter types of the method on `T` are assignable from the parameter types of the method on `S`.
- The return type of the method on `T` is assignable to the return type of the method on `S`.

In other words, the parameter types are contravariant and the return type is covariant.

Method signature variance on structure methods supports primitive types as well as reference types.

Interface assignment compatibility

If you define a structure that extends an interface, the structure gets the properties and method signatures of the interface. The structural assignability rules stated earlier apply, rather than interface assignability rules.

See also

- “Adding static constant variables to a structural type” on page 266
- “Create a structural type with getters, setters, and methods” on page 265

Assignability of a structural type to another type

A structure is assignable to variables and programming contexts of the following types:

- A structure of the same type
- Other structures with compatible assignable properties and compatible method signatures
- `Object`

Using enhancements to make types compatible with structures

Gosu enhancements are a language feature to add methods and properties to other types even if you do not control the implementation classes for those types. Programming access to enhancements methods and properties are dispatched statically at compile time based on the programming context. For example, if you add the `toPublicName`

method to the type `MyClass`, every programming context for an instance of `MyClass` now supports the `toPublicName` method.

You can use Gosu enhancements to add methods and features to types that make the types compatible with structural types. For example, suppose you want a structural type to work with multiple types that have a `toPublicName` method that returns a `String` value. You might create a structure such as:

```
structure HasPublicName {  
    public function toPublicName() : String  
}
```

Suppose a relevant class does not provide the `toPublicName` method but exposes the appropriate data using other methods or properties. You can add a Gosu enhancement to add the desired property on target types, in this case instances of the `com.example.MyClass` class:

```
enhancement AddDisplayNameToString: com.example.MyClass {  
    public function toPublicName() : String {  
        return this.toString()  
    }  
}
```

Any instance of `com.example.MyClass` has the `toPublicName` function and is compatible with the structure that requires that method:

```
uses com.example.MyClass  
  
var s = new MyClass()    // Create a new String  
  
var named : HasPublicName // Variable declared to a structural type  
  
// Assign a MyClass to the structural type. This code works because of the Gosu enhancement method.  
named = s
```

See also

- “Enhancements” on page 215

Composition

Gosu provides the language feature called composition by using the `delegate` keyword in variable definitions. Composition enables a class to delegate responsibility for implementing an interface to a different object. This compositional model supports implementation of objects that are proxies for other objects, or encapsulation of shared code independent of the type inheritance hierarchy.

The language feature *composition* enables a class to delegate responsibility for implementing an interface to a different object. This feature supports code reuse for projects with complex requirements for shared code. With composition, you do not rely on class inheritance hierarchies to choose where to implement reusable shared code.

Class inheritance is useful for some types of programming problems. However, it can make complex code dependencies fragile. Class inheritance tightly couples a base class and all subclasses and can cause changes to a base class to break all subclasses. Languages that support multiple inheritance, which allows a type to extend from multiple supertypes, can increase such fragility. For this reason, Gosu does not support multiple inheritance.

What if you have shared behavior that applies to multiple unrelated classes? Because the classes are unrelated, class inheritance does not naturally apply. Classes with a shared behavior or capability might not share a common type inheritance ancestor other than `Object`. No natural place exists to implement code that applies to both classes.

Let us consider a general example to illustrate this situation. Suppose you have a window class and a clipboard-support class. Suppose you have a user interface system with different types of objects and capabilities. However, some of the capabilities might not correspond directly to the class inheritance. For example, suppose you have classes for visual items like windows and buttons and scroll bars. However, only some of these items might interact with the clipboard copy and paste commands.

If not all user interface items support the clipboard, you might not want to implement your clipboard-supporting code in the root class for your user interface items. However, where do you put the clipboard-related code if you want to write a window-handling class that is also a clipboard part? One way to do this is to define a new interface that describes what methods each class must implement to support clipboard behavior. Each class that uses this interface implements the interface with behavior uniquely appropriate to each class. This behavior is an example of sharing a behavioral contract defined by the interface. However, each implementation is different within each class implementation.

What if the actual implementation code for the clipboard part is identical for each class that uses this shared behavior? Ideally, you write shared code only once so you have maximum encapsulation and minimal duplication of code. In some cases there does not exist a shared root class other than `Object`, so it might not be an option to put the code there. If Gosu supported multiple inheritance, you could encapsulate the shared code in its own class and classes could inherit from that class in addition to any other supertype.

Fortunately, you can get many of the benefits of multiple inheritance using another design pattern called composition. Composition encapsulates implementation code for shared behavior such that calling a method on the main object forwards method invocations to a subobject to handle the methods required by the interface.

Let us use our previous example with clipboard parts and windows. Let us suppose you want to create a subclass of window but that implements the behaviors associated with a clipboard part. First, create an interface that describes the required methods that you expect a clipboard-supporting object to support, and call it `IClipboardPart`. Next, create an implementation class that implements that interface, and call it `ClipboardPart`. Next, create a window subclass that implements the interface and delegates the actual work to a `ClipboardPart` instance associated with your window subclass.

See also

- “Interfaces” on page 173
- “Classes” on page 145

Delegation syntax

Delegation requires the Gosu keyword `delegate` within your class variable definitions. Declaring a delegate is like declaring a special type of class variable.

The `delegate` keyword has the following syntax:

```
delegate PRIVATE_VARIABLE_NAME represents INTERFACE_LIST
```

Optionally, you can use the following syntax that specifies the type:

```
delegate PRIVATE_VARIABLE_NAME : TYPE represents INTERFACE_LIST
```

The `INTERFACE_LIST` is a list of one or more interface names, with commas separating multiple interfaces.

For example:

```
delegate _clipboardPart represents IClipboardPart
```

In the class constructor, create an instance of an object that implements the interface. For example:

```
construct() {
    _clipboardPart = new ClipboardPart( this )
}
```

After the constructor runs, Gosu intercepts any method invocations on the object for that interface and forwards the method invocation to the delegated object.

Example

Let us look at complete code for this example.

The interface:

```
package test

interface IClipboardPart {
    function canCopy() : boolean
    function copy() : void
    function canPaste() : boolean
    function paste() : void
}
```

The delegate implementation class:

```
package test

class ClipboardPart implements IClipboardPart {
    var _myOwner : Object

    construct(owner : Object) {
        _myOwner = owner
    }

    // This is an implementation of these methods...
    override function canCopy() : boolean { return true }
    override function copy() : void { print("Copied!") }
```



```
    override function canPaste() : boolean { return true }  
    override function paste() : void { print("Pasted!") }  
}
```

The class that delegates the `IClipboardPart` implementation to another class:

```
package test  
  
class MyWindow implements IClipboardPart {  
    delegate _clipboardPart represents IClipboardPart  
  
    construct() {  
        _clipboardPart = new ClipboardPart( this )  
    }  
}
```

Finally, enter the following code into the Gosu Scratchpad:

```
uses test.MyWindow  
  
var a = new MyWindow()  
  
// Call a method handled on the delegate  
a.paste()
```

This code prints:

```
Pasted!
```

Composition and delegates

Overriding methods independent of the delegate class

You can override any of the interface methods that you delegated. Using the previous example, if the `canCopy` method is in the delegate interface, your `MyWindow` class can choose to override the `canCopy` method to specially handle it. For example, you could trigger different code or choose whether to delegate that method call.

For example, your `MyWindow` class can override a method implementation using the `override` keyword, and calling the private variable for your delegate if desired:

```
override function canCopy() : boolean {  
    return someCondition && _clipboardPart.canCopy();  
}
```

Declaring delegate implementation type in the variable definition

You can declare a delegate with an explicit type for the implementation class. This is particularly valuable if any of your code accessing the delegate directly in terms of the implementation class. For example, by declaring the type explicitly, you can avoid casting before calling methods on the implementation class that you know are not defined in the interface it implements.

To declare the type directly, add the implementation type name followed by the keyword `represents` before the interface name. In other words, use the following syntax:

```
private delegate PRIVATE_VARIABLE_NAME : IMPLEMENTATION_CLASS represents INTERFACE_NAME
```

For example, in the following line, `_clipboardPart` implements the `IClipboardPart` interface for the `ClipboardPart` class:

```
private delegate _clipboardPart : ClipboardPart represents IClipboardPart
```

Using one delegate for multiple interfaces

You can use a delegate to represent multiple interfaces for the enclosing class by implementing the methods for those interfaces. Instead of providing a single interface name, specify a comma-separated list of interfaces. For example:

```
private delegate _employee represents ISalariedEmployee, IOfficer
```

You might notice that in this example the line does not specify an explicit type for `_employee`, which represents two different types, which in this case are interface types. You might wonder about the compile-time type of the `_employee` variable. Because the variable must satisfy all requirements of both types, Gosu uses a special type called a compound type. A *literal of a compound type* is expressed in Gosu as a list separated by the ampersand symbol (&). For example:

```
ISalariedEmployee & IOfficer
```

Typical code does not need to mention a compound type explicitly. However, remember this syntax in case you see it during debugging code that uses the `delegate` keyword with multiple interfaces.

[See also](#)

- “Compound types” on page 356

Using composition with built-in interfaces

You can use composition with any interfaces, including built-in interfaces. For example, you could give a custom object all the methods of `java.util.List` and delegate the implementation to an instance of `java.util.ArrayList` or another `List` implementation.

For example:

```
class MyStringList implements List<String> {  
    delegate _internallist represents List<String> = new ArrayList<String>()  
}
```

You could now use this class and call any method defined on the `List` interface:

```
var x = new MyStringList()  
x.add( "TestString" )
```

Working with XML in Gosu

XML files describe complex structured data in a text-based format with strict syntax for data interchange. Gosu can read or write any XML document. If you have an associated XML Schema Definition (XSD) to define the document structure, Gosu parses the XML, which it validates using the schema, to produce a statically typed tree of XML elements with structured data. You can manipulate XML or generate XML without an XSD file, but Guidewire recommends that you use XSDs whenever possible in order to take advantage of programming shortcuts such as Gosu properties on each element and intelligent static typing.

Manipulating XML

To manipulate XML in Gosu, Gosu creates an in-memory representation of a graph of XML elements. The main Gosu class to handle an XML element is the class called `XmlElement`. Instead of manipulating XML by modifying text data in an XML file, your Gosu code can manipulate `XmlElement` objects. You can read in XML data from a file or other sources and parse it into a graph of XML elements. You can export a graph of XML elements as standard XML. For example, you can export an array of bytes containing XML data.

Gosu can manipulate structured XML documents in two ways:

Untyped nodes

Any XML can be created, manipulated, or searched as a tree of untyped nodes. This approach is similar to manipulating Document Object Model (DOM) untyped nodes. Gosu treats attribute and node values as strings.

Strongly typed nodes using an XSD

If the XML has an XSD file, you can create, manipulate, or search data with statically typed nodes that correspond to legal attributes and child elements. If you can provide an XSD file, the XSD approach is much safer. It dramatically reduces errors due to incorrect types or incorrect structure.

See also

- “Introduction to the XML element in Gosu” on page 275
- [W3C Extensible Markup Language \(XML\) 1.0 \(Fifth Edition\)](#)

Introduction to the XML element in Gosu

The `XmlElement` class represents an XML element and encapsulates the following components:

- The element name as a qualified name, a `QName`
- A backing type instance
- The nilness of the element

Element qualified name

The element's name is a `String` value that is a fully qualified name called a *QName*. A *QName* represents a more advanced definition of a name than a simple `String` value. To define a *QName*, use the `javax.xml.namespace.QName` class. A *QName* object contains the following components:

- A `String` value that represents the local part, also called the `localPart`.
- A `String` value that represents the namespace URI in which the local part of the name is defined. For example, a namespace can have the value `http://www.w3.org/2001/XMLSchema-instance`.
- A suggested prefix name to use if Gosu serializes this element. This prefix is not guaranteed upon serialization because there might be conflicts.

For example, an XML file may contain an element name with the syntax of two parts separated by a colon, such as `veh:childelement`. The `childelement` part of the name is the local part. The prefix `veh` is a convenient shortcut representing the full URI of a namespace previously declared in the XML document, as shown in the following example:

```
<?xml version="1.0"?>
<veh:root xmlns:veh="http://mycompany.com/schema/vehiclexsd">
  <veh:childelement/>
</veh:root>
```

This XML code specifies the following properties of the XML document:

- The root element of the document has the name `root` within the namespace `http://mycompany.com/schema/vehiclexsd`.
- The text `xmlns:veh`, followed by the URI, means that elements can use the prefix `veh:` to represent the longer URI (`http://mycompany.com/schema/vehiclexsd`).
- The root element has one child element that has the name `childelement` and is within the namespace `http://mycompany.com/schema/vehiclexsd`.

There are three constructors for *QName*:

- A constructor specifying the namespace URI, local part, and suggested prefix:

```
QName(String namespaceURI, String localPart, String prefix)
```

- A constructor specifying the namespace URI and local part:

```
QName(String namespaceURI, String localPart)
```

- A constructor specifying the local part only:

```
QName(String localPart)
```

You can set the namespace in the *QName* object to the empty namespace, which is the constant `javax.xml.XMLConstants.NULL_NS_URI`. The recommended approach for creating *QName* objects in the empty namespace is to use the `QName(String localPart)` constructor.

To create multiple *QName* objects in the same namespace, you can use the optional utility class called `XmlNamespace`.

Whenever you construct an `XmlElement`, you must always specify a non-empty *QName*.

Other Gosu XML APIs use *QName* objects for other purposes. For example, attributes on an element are names defined within a namespace, even if it is the default namespace for the XML document or the empty namespace. Gosu natively represents both attribute names and element names as *QName* objects.

Backing type instance

Each element contains a reference to a Gosu type that represents this specific element. To get the backing type instance, use the element's `TypeInstance` property. For XML elements that Gosu created based on an XSD, Gosu sets this backing type information automatically so it can be used in a type-safe manner.

Whenever you construct an `XmlElement`, an explicit backing type is optional. If you are constructing the element from an XSD, Gosu automatically sets the backing type based on the `XmlElement` subclass.

You can use `XmlElement` as untyped nodes without providing an XSD for your data format. If you are not using an XSD and do not provide a backing type, Gosu uses the default backing type `gw.xml.xsd.w3c.xmlschema.types.complex.AnyType`. All valid backing types are subclasses of `AnyType`.

The backing type instance of an XML element is responsible for most of the element's behavior but does not contain the element's name. It is important to be aware of the functionality that a particular element's backing type instance supports, especially when using an XSD.

Nilness of an element

An XML element that has no child elements but has attributes is `nil`. Additionally, an XSD can define whether an element is nillable, meaning it is allowed to be `nil`.

Note: This state is not exactly the same as being `null`.

XmlElement constructor

The `XmlElement` constructor that accepts a `String` parameter is convenient, because it is equivalent to passing a new `QName` object with that `String` as a single argument to a `QName` constructor. In this case, the namespace and prefix in the `QName` are `null`.

The following code creates an in-memory Gosu object that represents an XML element `<Root>` in the empty namespace:

```
var el = new XmlElement( "Root" )
```

In this case, the `el.TypeInstance` property returns an instance of the default type `gw.xsd.w3c.xmlschema.types.complex.AnyType`. If you instantiate a type instance, Guidewire recommends that you use a more specific subclass of `AnyType`, specifying either an XSD-based type or a simple type.

See also

- “Creating many qualified names in the same namespace” on page 285
- “Getting data from an XML element” on page 295
- “Accessing the nilness of an element” on page 302

XmlElement core property reference

The `XmlElement` instance contains the following core properties.

XmlElement property	Description	Type
QName	A read-only property that returns the element's QName.	QName
Namespace	Returns an <code>XmlNamespace</code> object that represents the element's namespace. • If this element was created with one of the method signatures with a <code>QName</code>, this property returns the namespace of the <code>QName</code>. • If this element is XSD-based, this property returns the namespace as defined by the XSD.	XmlNamespace
TypeInstance	Returns the element's backing type instance.	<code>gw.xsd.w3c.xmlschema.types.complex.AnyType</code> or any subclass of that class
Nilness	Specifies whether this element is nil, which is an XML concept that is not the same as being <code>null</code> .	boolean

IMPORTANT If you get these properties on an XSD-based element, you must use a dollar sign prefix for the property name.

See also

- “Accessing XSD type properties” on page 279
- “Accessing the nilness of an element” on page 302

Contents of an XML element

As well as XML attributes and metadata, an XML element can have either of these two basic types of content:

- Child elements
- A simple value, which can represent simple types such as numbers or dates

Gosu exposes properties and methods on the XML type instances so that you can access or manipulate child elements or text contents. For example, if an element is not an XSD-based element, access the properties directly, such as in `element.Children`.

An element can contain either child elements or simple values, but never both. This distinction is critical for XSD-based types. Gosu handles properties on an element differently depending on whether the element contains a simple value or is a type that can contain child elements.

The Gosu object that represents the element does not directly contain the child elements or the text content. It is the backing type instance for each element that contains the text content. This structure does not affect the way that Gosu provides access to the element or its contents.

See also

- “XSD-based properties and types” on page 286

Create an XmlElement

Procedure

1. To create a basic `XmlElement`, decide the element name that you want to use.
2. In a new expression, pass the element name to the constructor as either a `QName` object or a `String`. For example, the following line passes the name as a `String`:

```
var el = new XmlElement("Root")
```

In this case, the `el.TypeInstance` property returns an instance of the default type `gw.xsd.w3c.xmlschema.types.complex.AnyType`. If you instantiate a type instance, typically you would use a more specific subclass of `AnyType`, either an XSD-based type or a simple type.

3. Optionally set attributes.
4. Optionally add child elements by calling the `addChild` method with a reference to a child element.

Example

The following Gosu code creates a new `XmlElement` without an XSD, and then adds child elements:

```
uses gw.xml.XmlElement
uses javax.xml.namespace.QName

// Create parent element
var xe = new XmlElement(new QName("http://mycompany.com/schema/vehiclexsd", "root", "veh"))

// Create child elements
var xe2 = new XmlElement(new QName("http://mycompany.com/schema/vehiclexsd", "childelement", "veh"))
xe.addChild(xe2)
var xe3 = new XmlElement("childelementWithNoNamespace") // no namespace for this element
xe.addChild(xe3)
```

```
// Print element
xe.print()
print("Namespace of xe = " + xe.$Namespace)
print("Namespace of xe2 = " + xe2.$Namespace)
print("Namespace of xe3 = " + xe3.$Namespace)
```

This code prints the following lines:

```
<?xml version="1.0"?>
<veh:root xmlns:veh="http://mycompany.com/schema/vehiclexsd">
  <veh:childelement/>
  <childelementWithNoNamespace/>
</veh:root>
Namespace of ex = {http://mycompany.com/schema/vehiclexsd}
Namespace of e2 = {http://mycompany.com/schema/vehiclexsd}
Namespace of e3 = {}
```

See also

- “XML attributes” on page 298
- “Getting data from an XML element” on page 295

Accessing XSD type properties

For some XML element properties, Gosu provides access directly from the XML element even though the actual implementation internally is on the backing type instance. If an element is not an XSD-based element, you can access the properties directly, such as `element.Children`.

However, if you use an XSD type, you must prefix the property name with a dollar sign (\$). This convention prevents ambiguity between properties defined on the XSD type and on the type instance that backs that type. For example, suppose the XSD defines an element’s child element named `Children`. There would unfortunately be two similar properties with the same name. Gosu prevents ambiguity by requiring properties to have a dollar-sign prefix if and only if the element is XSD-based:

- To access the child elements of an XSD-based element, use the syntax `element.$Children`.
- To access a custom child element named `Children` that the XSD defines, use the syntax `element.Children`. This name is not recommended because of the ambiguity, but Gosu can access this element correctly. You may not have control over the XSD format that you are using, so Gosu must disambiguate them.

Notes about this convention:

- This convention only applies to properties defined on XSD-based types.
- It does not apply to methods.
- It does not apply to non-XSD-based XML elements.

For example, suppose you use the root class `XmlElement` directly with no XSD to manipulate an untyped graph of XML nodes. In that case, you can omit the dollar sign because the property names are not ambiguous. There are no XSD types, so there is no overlap in namespace.

This requirement affects the following type instance property names that appear on an XML element, listed with their dollar sign prefix:

- `$Attributes`
- `$Class`
- `$Children`
- `$Namespace`
- `$NamespaceContext`
- `$Comment`
- `$QName`
- `$Text`
- `$TypeInstance`
- `$SimpleValue`

- `$Value` – Only for elements with an XSD-defined simple content
- `$Nil` – Only for XSD-defined nillable elements

Note: If you create an `XmlElement` element directly, not a subclass, the object is not an XSD type. The new object is an untyped node that uses the default type instance, which is an instance of the type `AnyType`. In such cases, there is no dollar sign prefix because there is no ambiguity between properties that are part of the type instance and properties on the XSD type.

See also

- “Accessing the nullness of an element” on page 302

Exporting XML data

The `XmlElement` class includes methods and properties that export XML data. All of these methods have alternative method signatures that take a serialization options object (`XmlSerializationOptions`).

Export-related methods on an XML element

Each XML element provides several methods to serialize its value.

Note: Nillable elements (`nillable="true"`) with a value of `null` are skipped and are not exported.

This behavior occurs even if the element is required (`minOccurs="1"`).

To ensure the correct serialization of XML elements that contain characters with high Unicode code points, best practice is to test serialized values that contain non-English characters.

Exporting bytes

The `bytes` method returns an array of bytes (the type `byte[]`) that contains the UTF-8-encoded bytes in the XML. Generally speaking, the `bytes` method is the best approach for serializing the XML.

```
var ba = element.bytes()
```

The method has no required arguments, but can be customized by passing an optional argument of an `XmlSerializationOptions` object.

Code that sends XML with a transport that understands only character data and not byte data must always base-64 encode the bytes to compactly and safely encode binary data.

```
var base64String = gw.util.Base64Util.encode(element.bytes())
```

Base-64 encoded bytes can be decoded with the `decode` method.

```
var bytes = gw.util.Base64Util.decode(base64String)
```

Note: For example, for PolicyCenter messaging, the payload field in a `Message` entity is type `String`.

Printing to standard output stream

The `print` method serializes the element to the standard output stream (`System.out`).

```
element.print()
```

The method has no required arguments, but can be customized by passing an optional argument of an `XmlSerializationOptions` object.

Writing to a specific output stream

The `writeTo` method writes to an output stream (`java.io.OutputStream`). The output stream remains open and is not closed.

The method accepts a single required argument of an `OutputStream` object. Serialization can be customized by passing an optional second argument of an `XmlSerializationOptions` object.

Output to string with XML header

The `asUTFString` method serializes the element to a `String` object, with a header line that is compatible with UTF-8 encoding.

```
var s = element.asUTFString()
```

IMPORTANT Although the `asUTFString` method is helpful for debugging use, the `asUTFString` method is not the best way to export XML safely to external systems. Instead, it is usually best to use the XML element's `bytes` method to produce an array of bytes.

The `asUTFString` method outputs the node as a `String` value that contains XML, with a header suitable for later export to UTF-8 or UTF-16 encoding. The generated XML header does not specify the encoding. In the absence of a specified encoding, all XML parsers must detect the encoding (UTF-8 or UTF-16). The existence of a byte order mark at the beginning of the document tells the parser what encoding to use.

The method has no required arguments. However, it can be customized by passing an optional argument for an `XmlSerializationOptions` object.

WARNING Although the `asUTFString` method adds an XML header that can specify the encoding, there is no automatic encoding transformation into bytes. To write this `String` object to another system or a file, you must be careful to do the correct encoding transformation on export. Also be aware that if you use the optional `XmlSerializationOptions` argument, you could choose another encoding. However, just like for the default `asUTFString` method signature, which assumes UTF-8 encoding, the encoding choice only affects the serialized header, not the encoding of the output bytes.

XML serialization options and methods

The `XmlSerializationOptions` type exposes properties and also special methods for each one of these properties. Each method has the prefix `with` followed by the property name. For example, `withSort`. The method takes one argument of the type of that property.

These methods return the `XmlSerializationOptions` object again, so the methods are chainable. This behavior supports combining multiple method invocations in a single chained statement.

```
var opts = new gw.xml.XmlSerializationOptions().withSort(false).withValidate(false)
```

In addition, the `withEncoding` property has a secondary method signature that takes the Java short name for the encoding. You can set the encoding by either of the following operations.

- Use the `withEncoding` method and pass a standard Java encoding name as a `String`, such as "Big5".
- Set the `Encoding` property to a raw character set object for the encoding. You can use the static method `Charset.forName(ENCODING_NAME)` to get the desired static instance of the character set object. For example, pass "Big5".

See also

- “XML serialization options reference and examples” on page 281
- <http://tools.ietf.org/html/rfc3629>

XML serialization options reference and examples

The following table lists properties on a serialization options object of type `gw.xml.XmlSerializationOptions`. These properties provide serialization options:

Serialization option	Type	Description	Default value
Comments	Boolean	If true, exports each element's comments.	true

Serialization option	Type	Description	Default value
Sort	Boolean	If true, ensures that the order of children elements of each element match the XSD. This is particularly important for sequences. This feature only has an effect on an element if it is based on an XSD type. If the entire graph of <code>XmlElement</code> objects contains no XSD-based elements, this property has no effect. If a graph of XML objects contains a mix of XSD and non-XSD-based elements, this feature only applies to the XSD-based elements. This is true independent of whether the root node is an XSD-based element. WARNING For large XML objects with many nested layers, sorting requires significant computer resources.	true
XmlDeclaration	Boolean	If true, writes the XML declaration at the top of the XML document.	true
Validate	Boolean	If true, validates the XML document against the associated XSD. This feature only has an effect on an element if it is based on an XSD type. If the entire graph of <code>XmlElement</code> objects contains no XSD-based elements, this property has no effect. GX models created in Studio cannot be invalid. As a result, they do not need to be validated when serialized to XML.	false
Encoding	Charset	The character encoding of the resulting XML data as a <code>java.nio.charset.Charset</code> object. See discussion after this table for tips for setting this property.	UTF-8 encoding
Pretty	Boolean	If true, Gosu attempts to improve visual layout of the XML with indenting and line separators. If you set this to false, then Gosu ignores the values of the properties: <code>Indent</code> , <code>LineSeparator</code> , <code>AttributeNewLine</code> , <code>AttributeIndent</code> .	true

The following properties provide options that take effect only if the `Pretty` property is true:

Serialization option	Type	Description	Default value
Indent	String	The String to export for each indent level to make the hierarchy clearer.	Two spaces
LineSeparator	String	The line separator as a String.	The new line character (ASCII 10).
AttributeNewLine	Boolean	If true, puts each attribute on a new line.	false
AttributeIndent	int	The number of additional indents beyond its original indent for an attribute.	2

XML serialization examples

The following example creates an element, then adds an element comment. Next, it demonstrates printing the element with the default settings that include comments and how to customize the output to omit comments.

```
uses gw.xml.XmlElement
uses javax.xml.namespace.QName
uses gw.xml.XmlSerializationOptions

// Create an element
var a = new XmlElement(new QName("http://mycompany.com/schema/vehiclexsd", "root", "veh"))

// Add a comment
a.$Comment = "Hello I am a comment"

print("print element with default settings...")
a.print()

print("print element with no comments...")
a.print(new XmlSerializationOptions() { Comments = false })
```

For Guidewire application messaging, follow the pattern of the following PolicyCenter Event Fired rule code. It creates a new message that contains the XML for a contact entity as a `String` to work with the standard message payload property. The messaging system requires a `String`, not an array of bytes. To properly and safely encode XML into a `String`, use the syntax:

```
if (MessageContext.EventName == "ContactChanged") {
    var xml = new mycompany.messaging.ContactModel.Contact(MessageContext.Root as Contact)
    var strContent = gw.util.Base64Util.encode(xml.bytes())

    var msg = MessageContext.createMessage(strContent)

    print("Message payload of my changed contact for debugging:")
    print(msg)
}
```

Your messaging transport code takes the payload `String` and exports it:

```
override function send(message : Message, transformedPayload : String) : void {

    // Decode the Base64 encoded bytes stored in a String.
    var bytes = Base64Util.decode(message.Payload)

    // Send the byte array to a foreign system.
    ...

    message.reportAck();
}
```

All serialization APIs generate XML data for the entire XML hierarchy with that element at the root.

Serialization performance and element sorting

Sorting the set of serialized objects ensures that the order of children elements of each element matches the XSD. This behavior is particularly important for sequences. For large XML objects with many nested layers, sorting requires a lot of computer resources. If you create your XML objects in the order that their sequence definitions specify, you can safely disable sorting during serialization.

The element serialization sorting property (`XmlSerializationOptions.Sort`) controls whether PolicyCenter sorts the serialized XML objects.

For example, the following line of code creates an anonymous XML serialization object that performs no sorting.

```
myXMLElement.print(new XmlSerializationOptions() { :Sort = false } )
```

Parsing XML data into an XML element

The `XmlElement` class contains static methods for parsing XML data into a graph of `XmlElement` objects. *Parsing* means to convert serialized XML data into a more complex in-memory representation of the document. All these methods have the name `parse`. There are multiple methods because Gosu supports parsing from several different sources of XML data.

IMPORTANT For each source of data, there is an optional method variant that modifies the way Gosu parses the XML. Gosu encapsulates these options in an instance of the type `XmlParseOptions`. The `XmlParseOptions` specifies additional schemas that resolve schema components for the input instance XML document. Typical code does not need this. Use this if your XML data contains references to schema components that are neither directly nor indirectly imported by the schema of the context type.

For example, the following example parses XML contained in a `String` into an `XmlElement` object, and then prints the parsed XML data:

```
var a = XmlElement.parse("<Test123/>")
a.print()
```

If you are using an XSD, call the `parse` method directly on your XSD-based node, which is a subclass of `XmlElement`. For example:

```
var a = docexamples.gosu.xml.demoattributes.Element1.parse(xmlDataString)
```

The following table lists the signatures of the parse method.

Parameters	Description
byte[] byte[], XmlParseOptions	Parses XML from a byte array, with optional parsing options.
java.io.File java.io.File, XmlParseOptions	Parses XML from a file, with optional parsing options.
java.io.InputStream java.io.InputStream, XmlParseOptions	Parses XML from an InputStream, with optional parsing options.
java.io.Reader java.io.Reader, XmlParseOptions	Parses XML from a reader, which is an object for reading character streams. Optionally, add parsing options. WARNING Because this uses character data, not bytes, the character encoding is irrelevant. Any encoding header at the top of the file has no effect. It is strongly recommended to treat XML as binary data, not as String data. If your code needs to send XML with a transport that only understands character (not byte) data, always Base64 encode the bytes. (From Gosu, use the syntax: <code>Base64Util.encode(element.bytes())</code>)
String String, XmlParseOptions	Parses XML from a String, with optional parsing options. IMPORTANT Because this uses character data, not bytes, the character encoding is irrelevant. Any encoding header at the top of the file has no effect. It is strongly recommended to treat XML as binary data, not as String data. If your code needs to send XML with a transport that only understands character (not byte) data, always Base64 encode the bytes. From Gosu, create the syntax: <code>Base64Util.encode(element.bytes())</code>

Checking XML well-formedness and validation during parsing

For XSD-based XML elements, Gosu has the following behavior:

- Gosu checks for well-formedness. For example, Gosu checks that the XML contains no unclosed tags or other structural errors.
- Gosu always validates the XML against the XSD.

For non-XSD-based XML elements:

- Gosu checks for well-formedness.
- If the XML parse options object includes references to other schemas, Gosu validates the XML against those schemas.

If the XML document fails any of these tests, Gosu throws an exception.

See also

- “Referencing additional schemas during parsing” on page 284

Referencing additional schemas during parsing

In some advanced parsing situations, you might need to reference additional schemas as well as your main schema during parsing.

To specify additional schemas, set the `XmlParseOptions.AdditionalSchemas` to a specific `SchemaAccess` object. This `SchemaAccess` object represents the XSD. To access an additional schema from an XSD, use the syntax:

```
package_for_the_schema.util.SchemaAccess
```

To see how and why you would specify additional schemas, suppose you have the following two schemas:

ImportXSD1.xsd:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema" targetNamespace="urn:ImportXSD1"
  xmlns:ImportXSD1="urn:ImportXSD1">
  <xsd:element name="ElementFromSchema1" type="ImportXSD1:TypeFromSchema1"/>
  <xsd:complexType name="TypeFromSchema1"/>
</xsd:schema>
```

ImportXSD2.xsd:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema" targetNamespace="urn:ImportXSD2"
  xmlns:ImportXSD1="urn:ImportXSD1" xmlns:ImportXSD2="urn:ImportXSD2"
  elementFormDefault="qualified">
  <xsd:import schemaLocation="ImportXSD1.xsd" namespace="urn:ImportXSD1"/>
  <xsd:complexType name="TypeFromSchema2">
    <xsd:complexContent> <!-- the TypeFromSchema2 type extends the TypeFromSchema1 type! -->
      <xsd:extension base="ImportXSD1:TypeFromSchema1"/>
    </xsd:complexContent>
  </xsd:complexType>
</xsd:schema>
```

Notice that the ImportXSD2 XSD extends a type that the ImportXSD1 defines. This definition is analogous to saying the ImportXSD2 type called TypeFromSchema2 is a subclass of the ImportXSD1 type called TypeFromSchema1.

The following code fails by throwing an exception because the ImportXSD1 references the schema type ImportXSD2:TypeFromSchema2 and Gosu cannot find that type in the current schema.

```
var schema2 = docexamples.gosu.xml.importxsd2.util.SchemaAccess

var xsdtext = "<ElementFromSchema1 xmlns=\"urn:ImportXSD1\" xmlns:ImportXSD2=\"urn:ImportXSD2\" +
  \" xmlns:xsi=\"http://www.w3.org/2001/XMLSchema-instance\" xsi:type=\"ImportXSD2:TypeFromSchema2\"/>"

// Parse an element defined in the the first schema, but pass an extension to that
// that type that the second schema defines. THIS FAILS without using the AdditionalSchemas feature.
var element = ElementFromSchema1.parse(xsdtext)
```

The ImportXSD1 XSD cannot locate the schema called ImportXSD2 even though the specification extends a type that ImportXSD2 defines.

To provide access to the additional schema, set the AdditionalSchemas property of the XmlParseOptions object to a list containing one or more SchemaAccess objects. The following code parses XML successfully:

```
var schema2 = docexamples.gosu.xml.importxsd2.util.SchemaAccess
var options = new gw.xml.XmlParseOptions() { :AdditionalSchemas = { schema2 } }

var xsdtext = "<ElementFromSchema1 xmlns=\"urn:ImportXSD1\" xmlns:ImportXSD2=\"urn:ImportXSD2\" +
  \" xmlns:xsi=\"http://www.w3.org/2001/XMLSchema-instance\" xsi:type=\"ImportXSD2:TypeFromSchema2\"/>"

// Parse an element defined in the the first schema, but pass an extension to that
// type that the second schema defines by using the XmlParseOptions.
var element = ElementFromSchema1.parse(xsdtext, options)
```

You can remap an external namespace or XSD URL to a local XSD using the gwxmlmodule.xml file.

[See also](#)

- “Using a local XSD for an external namespace or XSD location” on page 309

Creating many qualified names in the same namespace

The name of each element is a qualified name, which is an object of type `javax.xml.namespace.QName`.

A `QName` object contains the following parts:

- A namespace URI
- A local part
- A suggested prefix for this namespace.

On serialization of an `XmlElement`, Gosu tries to use the prefix to generate the name, such as

"wsdl:definitions". In some cases however, it might not be possible to use this name. For example, if an

XML element defines two attributes with different namespaces but the same prefix. On serialization, Gosu auto-creates a prefix for one of them to prevent conflicts.

Typical code repetitively creates many `QName` objects in the same namespace. One way to create many such objects is to store the namespace URI in a `String` variable, then create `QName` instances with new local parts.

To simplify this process, Gosu includes a utility class called `gw.xml.XmlNamespace`. It represents a namespace URI and a suggested prefix. In other words, it is like a `QName` but without the local part.

There are two ways to use `XmlNamespace`:

- Create an `XmlNamespace` directly and call its `qualify` method and pass the local part `String`. For example:

```
uses gw.xml.XmlNamespace
var ns = new XmlNamespace("namespaceURI", "prefix")
var ex = new XmlElement(ns.qualify("localPartName"))
```

- Reuse the namespace of an already-created XML element. To get the namespace from an XML element instance, get its `Namespace` property. Then, call the `qualify` method and pass the local part `String`:

```
// Create a new XML element.
var xml = new XmlElement(new QName("namespaceURI", "localPart", "prefix"))

// Reuse the namespaceURI and prefix from the previously created element.
var xml2 = new XmlElement(xml.Namespace.qualify("localPart2"))
```

See also

- “Introduction to the XML element in Gosu” on page 275

XSD-based properties and types

The most powerful way to use XML in Gosu is to use an XSD that describes what is valid in your XML format. If you can use or generate an XSD for your XML data, it is strongly recommended that you use an XSD file. The name of each XSD file must include an `.xsd` extension in lowercase.

To tell Gosu to load your XSD, put your XSD files in the same file hierarchy in the configuration module as Gosu classes, organized in subdirectories by package. At the file system level, this is the hierarchy of files starting at:

```
PolicyCenter/modules/configuration/gsrc
```

For example, to load an XSD in the package `mycompany.schemas`, put your XSD file at the path:

```
PolicyCenter/modules/configuration/gsrc/mycompany/schemas
```

Gosu creates new types in the type system for element declarations in the XSD. Where appropriate, Gosu creates properties on these types based on the names and structure within the XSD. By using an XSD and the generated types and properties, your XML-related code is significantly more readable. You can use natural Gosu syntax to access attributes and child elements using the period operator. For example, access a child element named `ChildName` on an element with code like `element.ChildName`.

The new types are defined in a subpackage that matches the name of the XSD, except in lowercase and with no `.xsd` suffix. For example, if the XSD is at the path `gsrc/example/MyTest.xsd` and contains an element called `Address`, the new element type is `example.mytest.Address`.

If you cannot use an XSD, you can use the basic properties and methods of `XmlElement` like `element.Children` and `element.getChild("ChildName")`. Getting and setting values in this way is harder to understand than using XML types and properties. This type of XML-related code that does not use XSD types is not type-safe, so Gosu cannot perform type checking, which can lead to run-time errors.

Important concepts in XSD properties and types

There are some important distinctions to make in terminology when understanding how Gosu creates types from XSDs. In the following table, note how every definition in the XSD has a corresponding instance in an XML document (although in some cases might be optional).

Definitions in the XSD	Instances in an XML document
A schema (an XSD)	XML document
Element definition	Element instance
Complex type definition	Complex type instance
Simple type definition	Simple type instance
Attribute definition	Attribute instance

For every element definition in the XSD:

- There is an associated type definition.
- The type definition is either a complex type definition or simple type definition.
- The element definition has one of the following qualities:
 - It references a top-level type definition, such as a top-level complex type.
 - It embeds a type definition inside the element definition, such as an embedded simple type.
 - It includes no type, which implicitly refers to the built-in complex type `<xsd:anyType>`.

In an XSD, various definitions cause Gosu to create new types:

- An element definition causes Gosu to create a type that describes the element.
- A type definition causes Gosu to create a type that describes the type (for example, a new complex type).
- An attribute definition causes Gosu to create a type that describes the attribute.

For example, suppose an XSD declares a new top-level simple type that represents a phone number. Suppose there are three element definitions that reference this new simple type in different contexts for phone numbers, such as work number, home numbers, and cell number. In this example, Gosu creates:

- One type that represents the phone-number simple type.
- Three types that represent the individual element definitions that reference the phone number.

From Gosu, whenever you create objects or set properties on elements, you need to know which type to use. In some cases, you can do what you need in more than one way, so consider using the most readable technique.

If you have a reference to the element, you can always reference the backing type. For example, for an element, you can reference the backing type instance using the `$TypeInstance` property.

See also

- “XSD generated type examples” on page 290

Reference of XSD properties and types

The following table lists the types and properties that Gosu creates from an XSD. For this topic, *schema* represents the fully qualified path to the schema, *eElementName* represents an element name, and *parentName* and *childName* represent names of parent and child elements.

For a property, the rightmost column indicates whether the property can appear more than once, in which case the property becomes a list property. If the rightmost column contains “Yes”, the property has type `java.util.List` parameterized on the type that the property has when it is singular. For example, suppose a child element is declared in the XSD with the type `xsd:int`:

- If its `maxOccurs` is 1, the property’s type is `Integer`.
- If its `maxOccurs` is greater than 1, the property’s type is `List<Integer>`, which is a list of integers.

There are other circumstances in which a property becomes a list. For example, suppose there is a XSD choice (`<xsd:choice>`) in an XSD that has `maxOccurs` attribute value greater than 1. Any child elements become list properties. For example, if the choice defines child elements with names “elementA” and “elementB”, Gosu creates properties called `ElementA` and `ElementB`, both declared as lists. Be aware that Gosu exposes shortcuts for inserting items.

Notes about generated types containing the text `anonymous` in the fully qualified type name:

- Although the package includes the word `anonymous`, this does not imply that these elements have no defined names. The important quality that distinguishes these types is that the object is defined at a lower level than the top level of the schema. By analogy, this is similar to how Gosu and Java define inner classes within the namespace of another class.
- Several rows contain a reference to the path from root as the placeholder text *PathFromRoot*. The path from root is a generated name that embeds the path from the root of the XSD, with names separated by underscore characters. The intermediate layers may be element names or group names. See each row for examples.

The following table lists XML definitions and the syntax for accessing an item that the definition describes.

Definition	New item	Syntax...
An element at the top level	Type	<code>schema.ElementName</code> IMPORTANT However, Gosu behaves slightly differently if the top-level element is declared in a web service definition language (WSDL) document. Instead, Gosu creates the type name as <code>schema.elements.ElementName</code> .
An element lower than the top level	Type	<code>schema.anonymous.elements.PathFromRoot_ElementName</code> For example, suppose the top level group A that contains an element called B, which contains an element called C. The <i>PathFromRoot</i> is A_B and the fully qualified type is <code>schema.anonymous.elements.A_B_C</code> .
A complex type at the top level	Type	<code>schema.types.complex.TypeName</code>
A complex type lower than the top level	Type	<code>schema.anonymous.types.complex.PathFromRoot</code> For example, suppose a top level element A contains an embedded complex type. The <i>PathFromRoot</i> is A. Note that complex types defined at a level lower than the top level never have names.
A simple type at the top level	Type	<code>schema.types.simple.TypeName</code>
A simple type lower than the top level	Type	<code>schema.anonymous.types.simple.PathFromRoot</code> For example, suppose a top level element A contains element B, which contains an embedded simple type. The path from root is A_B. Simple types defined at a level lower than the top level never have names.
An attribute at the top level	Type	<code>schema.attributes.AttributeName</code>
An attribute	Type	<code>schema.anonymous.attributes.PathFromRoot</code> For example, suppose a top level element A contains element B, which has the attribute C. The <i>pathPathFromRoot</i> is A_B and the fully qualified type is <code>schema.anonymous.attributes.A_B_C</code> .
An attribute lower than the top level	Property	<code>element.AttributeName</code> Unlike most other generated properties on XSD types, an attribute property never transforms into a list property.

A common pattern converts a `simpleType` to `simpleContent` to add attributes to an element with a simple type. To support this pattern, Gosu provides the `ChildName` and `ChildName_elem` properties for every child element that has either a simple type or both a complex type and simple content. The property with the `_elem` suffix on its name contains the element object instance. The property without the `_elem` suffix contains the element value. Because of this design, if you later decide to add attributes to a `simpleType` element, the change does not require modification of your existing XML code.

The following table lists XML definitions for child elements and the syntax for accessing an item that the definition describes. These items can occur anywhere in the schema.

Definition	New item	Syntax...
A child element with either of: - A simple type - A complex type and a simple content	Property	<code>element.ChildName_eLem</code> The property type is as follows: - If element is defined at top-level, <code>schema.ElementName</code> - If element is defined at lower levels, <code>schema.anonymous.elements.PathFromRoot_ElementName</code>. IMPORTANT If this property can appear more than once, it transforms into a list type.
The value of a child element with either of: - A simple type - A complex type and a simple content	Property	<code>element.ChildName</code> The property type is as follows: - If element is defined at top-level, <code>schema.ElementName</code> - If element is defined at lower levels, <code>schema.anonymous.elements.PathFromRoot_ElementName</code>. IMPORTANT If this property can appear more than once, it transforms into a list type.
A child element with a complex type and no simple content	Property	<code>element.ChildName</code> The property type is as follows: - If element is defined at top-level, <code>schema.ElementName</code> - If element is defined at lower levels, <code>schema.anonymous.elements.PathFromRoot_ElementName</code>. IMPORTANT If this property can appear more than once, it transforms into a list type.

The following table lists the XML definition and the syntax for accessing an XML schema.

Definition	New item	Syntax...
Schema definition	Schema access object	<code>schema.util.SchemaAccess</code> This special utility object provides access to the original schema that produced this type hierarchy. This object represents this schema. Use this object if you need one schema to include another schema.

See also

- “Automatic insertion into lists” on page 291
- “Referencing additional schemas during parsing” on page 284

Normalization of Gosu generated XSD-based names

In cases where Gosu creates type names and element names, Gosu performs slight normalization of the names:

- One prominent aspect of normalization is capitalization to conform to Gosu naming standards for packages, properties, and types. For example, Gosu packages become all lowercase. Types must start with initial capitals. Properties must start with initial capitals.
- If the type or property names contains invalid characters for Gosu for that context, Gosu changes them. For example, hyphens are disallowed and removed.
 - If Gosu finds an invalid character and the following character is lowercase, Gosu removes the invalid character and uppercases the following letter.
 - If Gosu finds an invalid character and the following character is uppercase, Gosu converts the invalid character to an underscore and does not change the following character.
 - If the first character is invalid as a first character but otherwise valid, such as a numeric digit, Gosu adds an underscore prefix. If the first character is entirely invalid in a name in that context, such as a hyphen, Gosu removes the character. If after removing multiple start characters, no characters remain, Gosu uses an underscore character as the name.
- If names would be duplicates, Gosu appends numbers to remove ambiguity. For example, duplicates of the name `MyProp` become `MyProp2`, `MyProp3`, and so on.

XSD generated type examples

These examples demonstrate how to use generated types.

XSD generated type examples 1

Create `myschema.xsd` in the package `examples.pl.gosu.xml`:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Element1">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="Child1" /> <!-- default type is xsd:anyType -->
        <xsd:element name="Child2" type="xsd:int"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```

Review the following Gosu code:

```
uses examples.pl.gosu.xml.myschema.Element1
var xml = Element1()
var child1 = xml.Child1           // Child1 has type anyType (the default when a type is not specified).
var child2 = xml.Child2           // Child2 has type java.lang.Integer.
xml.Child1.Text = "a string"      // Set the XML property as a text string
xml.Child2 = 5                    // Set the XML property with a simple type.
var child2Elem = xml.Child2_elem  // Get this lower-level element as
                                   // myschema.anonymous.elements.Element1_Child2.
xml.print()                       // Display the entire element
child2Elem.print()                // Display the Child2 lower-level element
```

Note the following:

- The `Child1` property is of type `schema.anonymous.elements.Element1_Child1`, which is a subclass of `XmlElement`.
- The `Child2` property is of type `java.lang.Integer`. When a child element has a simple type, its natural property name gets the object's value, rather than the child element object. If you wish to access the element object (the `XmlElement` instance) for that child, instead use the property with the `_elem` suffix. In this case, for the child named `Child2`, you use the `element.Child2_elem` property, which is of type `schema.anonymous.elements.Element1_Child2`.

XSD generated types: element type and backing type instances

The following XSD defines a **phone** complex type, as well as multiple elements using that complex type:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="person">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="cell" type="phone"/>
        <xsd:element name="work" type="phone"/>
        <xsd:element name="home" type="phone"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>

  <xsd:complexType name="phone">
    <xsd:sequence>
      <xsd:element name="areaCode" type="xsd:string"/>
      <xsd:element name="mainNumber" type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:schema>
```

This XSD supports multiple ways to create and assign the phone numbers.

The following code creates three different phone numbers:

```
var ex = new schema.Person()

ex.Cell.AreaCode = "415"
ex.Cell.MainNumber = "555-1213"

ex.Work.AreaCode = "416"
ex.Work.MainNumber = "555-1214"

ex.Home.AreaCode = "417"
ex.Home.MainNumber = "555-1215"
```

The following code creates one phone number used in multiple elements:

```
uses examples.pl.gosu.xml.myschema.Person
uses examples.pl.gosu.xml.myschema.types.complex.Phone

var ex = new Person()

var c = new Phone()
var w = new Phone()
var h = new Phone()

c.AreaCode = "415"
c.MainNumber = "555-1212"
w.AreaCode = "510"
w.MainNumber = "555-1213"
h.AreaCode = "408"
h.MainNumber = "555-1214"

ex.Cell.$TypeInstance = c // use the $TypeInstance property for assignment
ex.Work.$TypeInstance = w
ex.Home.$TypeInstance = h
```

Note: An element's `$TypeInstance` property is required to access the element's backing type instance.

For example, you cannot create the complex type and directly assign it to the element type:

```
ex.Cell = c // SYNTAX ERROR: cannot assign complex type instance to element type instance
ex.Work = w // SYNTAX ERROR: cannot assign complex type instance to element type instance
ex.Home = h // SYNTAX ERROR: cannot assign complex type instance to element type instance
```

Additionally, different element-based types can be mutually incompatible for assignment, even if they are associated with the XSD type definition. For example:

```
var ex = new Person()
ex.Cell = ex.Work // SYNTAX ERROR: cannot assign one element type to a different element type
```

Automatic insertion into lists

If you are using XSDs, for properties that represent child elements that can appear more than once, Gosu exposes that property as a list. For properties that Gosu exposes as list properties, Gosu provides a special shorthand syntax

for inserting items into the list. If you assign to the list index equal to the size of the list, the index assignment becomes an insertion.

This behavior is also true if the size of the list is zero: use the `[0]` array/list index notation and set the property. This syntax inserts the value into the list, which is equivalent to dynamically adding an element to the list, meaning that you do not have to check whether the list already exists. If you are creating XML objects in Gosu, by default the lists do not yet exist and are initialized to `null`.

You use this syntax to add an element:

```
element.PropertyName[0] = childElement
```

If the list does not yet exist for a list property, Gosu creates the list when you perform the first insertion.

Note: If you use XSDs, Gosu creates intermediate XML elements as needed. Use this feature to improve the readability of your XML-related Gosu code.

In the following example, `List1` contains an unbounded number of instances of the `Child1` child element. Because the XSD declares that there can be more than one instance of the child element, the `List1.Child1` property is a list of integers.

Example XSD

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Element1">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="Child1" type="xsd:int" maxOccurs="unbounded"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```

Code

```
uses examples.pl.gosu.xml.myschema.List1

var xml = new List1()

print("Before insertion: ${xml.Child1.Count}")

xml.Child1[0] = 0
xml.Child1[1] = 1
xml.Child1[2] = 2

print("After insertion: ${xml.Child1.Count}")

xml.print()
```

Output

```
Before insertion: 0
After insertion: 3
<?xml version="1.0"?>
<list1>
  <Child1>0</Child1>
  <Child1>1</Child1>
  <Child1>2</Child1>
</list1>
```

See also

- “XSD-based properties and types” on page 286

XSD list property example

If the possibility exists for a child element name to appear multiple times, the property becomes a list-based property.

Example XSD

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Element1">
    <xsd:complexType>
      <xsd:choice>
        <xsd:element name="Child1" type="xsd:int"/>
        <xsd:sequence maxOccurs="unbounded">
          <xsd:element name="Child2" type="xsd:int"/>
        </xsd:sequence>
      </xsd:choice>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```

Code

```
var xml = new schema.Element1()
xml.Child1 = 1
xml.print()

print( "-----" )

xml.Child1 = null
xml.Child2 = {1, 2, 3, 4}
xml.print()
```

Output

```
<?xml version="1.0"?>
<Element1>
  <Child1>1</Child1>
</Element1>
-----
<?xml version="1.0"?>
<Element1>
  <Child2>1</Child2>
  <Child2>2</Child2>
  <Child2>3</Child2>
  <Child2>4</Child2>
</Element1>
```

Customizing XSD type code generation to exclude types

You can customize the way PolicyCenter converts XSDs to internal bytecode, which is a process called *code generation*. By default, code generation happens for every XSD type. However, for better Studio performance, you can optionally omit code generation for some XSDs in some contexts or all contexts.

To customize XSD code generation, you can modify the following file:

```
PolicyCenter/modules/configuration/res/gwxm1module.xml
```

This one file controls code generation for all XSDs. The file contents look similar to the following:

```
<?xml version="1.0"?>
<module xmlns="http://guidewire.com/xml/module" name="ab-customer-build">
  <dependencies>
    <dependency name="pl"/>
    <dependency name="ab"/>
  </dependencies>
  <settings>
    <setting name="gx.product" value="ab"/>
    <setting name="gx.use-gosu-enhancements" value="true"/>
  </settings>
  <excludes>
    <!-- <pattern>path/to/schema.xsd</pattern> -->
  </excludes>
  <included-only>
    <!-- <pattern>path/to/schema.xsd</pattern> -->
  </included-only>
</module>
```

Gosu supports modifying the `<excludes>` and `<included-only>` elements. All other changes are unsupported.

Exclude XSD types from code generation

Procedure

1. To find and edit the XSD code generation configuration file, in Studio, type `Ctrl+Shift+N`, and then type:

```
gwxm1module.xml
```

2. For any XSDs that do not need code generation in any context, create a pattern in the `<excludes>` element:

- a. Add a `<pattern>` element to the `<excludes>` element.
- b. As content for the element, type the relative path to the XSD in the source directory including the package.

The source directory is your `gsrsc` directory for production code or `gtest` for XSDs that you need only for GUnit tests. Use forward slashes (/) rather than periods to separate levels in the package hierarchy.

For example, suppose your XSD is at:

```
PolicyCenter/modules/configuration/gsrc/com/mycompany/schemas/integration/rare.xsd
```

Add the following `<pattern>` element to the `<excludes>` element:

```
<excludes>
  <pattern>com/mycompany/schemas/integration/rare.xsd</pattern>
</excludes>
```

3. For any XSDs that need code generation but not repeated code generation for XSD `<include>` elements, create a pattern in the `<included-only>` element:

- a. Add a `<pattern>` element to the `<included-only>` element.
- b. As content for the element, type the relative path to the XSD in the source directory including the package. Use forward slashes (/) rather than periods to separate levels in the package hierarchy. For example, suppose your XSD is at:

For example, suppose your XSD is at:

```
PolicyCenter/modules/configuration/gsrc/com/mycompany/schemas/integration/common.xsd
```

Add the following `<pattern>` element to the `<included-only>` element:

```
<include-only>
  <pattern>com/mycompany/schemas/integration/common.xsd</pattern>
</include-only>
```

4. Make no other changes to the file.
5. Save the file.

Special handling of very large XSD enumerations

Due to limitations in the Java language for some types of code generation, very large enumerations are handled differently from most enumerations. If an XSD defines an enumeration (an `<xs:enumeration>` element) with more than 2000 entries, PolicyCenter converts the enumeration to a `String` value (an `<xs:string>` element).

This conversion affects your Gosu code that uses the enumeration, as well as the overall type safety of related code. In addition to changing the type of any relevant properties, the Gosu compiler cannot perform compile-time verification of the individual enumeration values. For example, if you set a property of that enumeration type, be careful to spell the value correctly. If any changes to the XSD remove an enumeration value, setting that value does not cause an automatic compilation error.

During code generation on the command line or within Studio, you will see messages that identify substitutions due to this issue:

```
*****
Java enum limit exceeded. Treating xs:enumeration as String
```

```
As a result there will not be static typing for this enumeration.
[gw/accelerators/test/common/xsd/test/TestLocationService_CodeExt_2013-10-17.xsd]
[gw/accelerators/test/common/xsd/test/TestLocationService_CodeExt_2013-10-17
/types/simple/LineItemMotorcycleCategoryClosedEnumType.java]
3434 entries found
*****
```

Getting data from an XML element

The main work of an XML element happens in the type instance associated with each XML element. The type instance of an XML element is responsible for nearly all of the element behavior but does not contain the element's name. You can usually ignore the division of labor between an `XmlElement` and its backing type instance. If you are using an XSD, this distinction is useful.

If you instantiate a type instance, typically you use a more specific subclass of `gw.xsd.w3c.xmlschema.types.complex.AnyType`.

Gosu exposes properties and methods on the XML type instances for you to get child elements or simple values.

Note that XML elements contain two basic types of content:

- Child elements
- Simple values

An element can contain either child elements or a simple value, but not both at the same time.

See also

- “Introduction to the XML element in Gosu” on page 275

Manipulating elements and values

To get the child elements of an element, use the `Children` property, which contains a list (`java.util.List<XmlElement>`) of elements. If this XML element is an XSD-based type, you must prefix the property name with a `$` (`$Children` in this case).

If the element has no child elements, there are two possibilities:

- If the element has no text content, the `Children` property contains an empty list.
- If the element has text content, the `Children` property is `null`.

To add a child element, call the parent element's `addChild` method and pass the child element as a parameter.

For example, in `myschema.xsd` (see “XSD generated type examples” on page 290):

- The element named `Child1` has the default type (`xsd:anyType`).
- The element named `Child2` has the simple type `xsd:int`. If the integer value is empty or `null`, the XML will not validate.

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Element1">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="Child1"/> <!-- default type is xsd:anyType -->
        <xsd:element name="Child2" type="xsd:int"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```

If you have a reference to an XML element of a simple type, you can set its value by assigning its `SimpleValue` property to a subclass of `XmlSimpleValue`. If you are using an XSD, add the dollar sign prefix: `$SimpleValue`. For example, the code used in “XSD generated type examples” on page 290 can be modified so an integer value is assigned to `Child1` using the static `XmlSimpleValue.makeIntegerInstance` method:

```
uses examples.pl.gosu.xml.myschema.Element1
var xml = Element1()
var child1 = xml.Child1                                     // Child1 has type anyType (the default when a type is
```

```

not specified).
var child2 = xml.Child2
xml.Child1.$SimpleValue = XmlSimpleValue.makeIntegerInstance(55) // Set the XML property as an integer type.
xml.Child2 = 5
var child2Elem = xml.Child2_elem
xml.print() // Display the entire element
child2Elem.print() // Display the Child2 lower-level element

```

To set a simple value such as an integer value for an element, use one of the following approaches:

- Set the value in the `SimpleValue` property to a subclass of `XmlSimpleValue`. This allows you to directly create the simple value that Gosu stores in the pre-serialized graph of XML elements. If the content is on an XSD type, specify the property name with the dollar sign prefix: `$SimpleValue`. To create an instance of the `XmlSimpleValue` of the appropriate type, call one of the static `make` methods belonging to the `XmlSimpleValue` class. For example, call the `makeIntInstance` method and pass it an `Integer`. It returns an `XmlSimpleValue` instance that contains an integer. In memory, Gosu stores this information as a non-serialized value. When exporting to a byte array or using the debugging `print` method, Gosu serializes the `XmlSimpleValue` into bytes or encoded text.
- To create simple text content, which is a text simple value, set the element's `Text` property to a `String` value. If the content is on an XSD type, specify the property name with the dollar sign prefix: `$Text` (as shown in “XSD generated type examples” on page 290).
- If you are using an XSD, you can set the natural value in the `Value` property. If the content is on an XSD type, specify the property name with the dollar sign prefix (`$Value`). For example, use natural-looking code like `xml.Child2.$Value = 5`. If you are using an XSD and have non-text content, this approach tends to result in more natural-looking Gosu code than creating instances of `XmlSimpleValue`.
- If you are using an XSD, Gosu provides a syntax to get and set child values with simple types. For example, set numbers and dates from an element's parent element using natural syntax using the child element name as a property accessor. Using this syntax, you access the child element's simple value with readable code. For example, `xml.Child2 = 5`.

Example

The following Gosu code adds two child elements, sets the value of an element using the `Value` property and the `SimpleValue` property, and then prints the results. In this example, we use XSD types, so we must specify the special property names with the dollar sign prefix: `$Value` and `$SimpleValue`.

```

uses examples.pl.gosu.xml.myschema.Element1
uses examples.pl.gosu.xml.myschema.anonymous.elements.Element1_Child1
uses examples.pl.gosu.xml.myschema.anonymous.elements.Element1_Child2
uses gw.xml.XmlSimpleValue

// Create a new element whose type is in the namespace of the XSD.
var xml = new Element1()
var c = xml.$Children // returns an empty list of type List<XmlElement>
print("Children " + c.Count + c)
print("")

// Create a new CHILD element that is legal in the XSD, and add it as child.
var c1 = new Element1_Child1()
xml.addChild(c1)

// Create a new CHILD element that is legal in the XSD, and add it as child.
var c2 = new Element1_Child2()
xml.addChild(c2) // Add the child element.
print("before set: " + c2.$Value) // prints "null" -- it is uninitialized

c2.$SimpleValue = XmlSimpleValue.makeIntegerInstance(5) // set value for c2
print("after set with $SimpleValue: " + c2.$SimpleValue)

c2.$SimpleValue = XmlSimpleValue.makeIntegerInstance(7) // update value for c2
print("after update with $SimpleValue: " + c2.$SimpleValue)
print("")

c = xml.$Children // Return a list of two child elements
print("Children " + c.Count + c)
xml.print()

print("")

```


This code prints the following:

```
Children 0[]

before set: null
after set with $SimpleValue: Simple value (java.math.BigInteger): 5
after update with $SimpleValue: Simple value (java.math.BigInteger): 7

Children 2[examples.pl.gosu.xml.myschema.anonymous.elements.Element1_Child1 instance,
examples.pl.gosu.xml.myschema.anonymous.elements.Element1_Child2 instance]

<?xml version="1.0"?>
<Element1>
  <Child1/>
  <Child2>7</Child2>
</Element1>
```

Note that the Child2 element contains the integer as text data in the serialized XML export since Gosu does not serialize the simple types to bytes (or a `String`) until serialization. In this example, the final print statement serializes the element and all its subelements.

Getting child elements by name

To iterate through a `List` of child elements to find data, use the `Children` property described earlier in this topic. In some cases you can use the Gosu enhancements on lists to locate the items you want.

To search for a child element by its name, use the XML element's `getChild()` method if you expect only one match, or its `getChildren` method if you expect multiple matches:

- `getChild(QName)` – Searches the content list for a single child with the specified `QName`. An alternative method signature takes a `String` value for the local part name. For that method signature, Gosu internally creates a `QName` with an empty namespace and the specified local part name. This method requires there to be exactly one child with this name. If there are multiple matches, the method throws an exception. In such cases use the `getChildren` method instead.
- `getChildren(QName) : List` – Searches the content list for all children with the specified `QName` name. An alternative method signature takes a `String` value for the local part name. For that method signature, Gosu internally creates a `QName` with an empty namespace and the specified local part name.

Reusing the code from the previous example, add the following lines to get the child elements by name:

```
// Get a child using the empty namespace by passing a String.
var getChild1 = xml.getChild("Child1");

// Get a child using a QName, and "reuse" the namespace of a previous node.
var getChild2 = xml.getChild(getChild1, $Namespace.qualify("Child2"))

print("getChild1: " + getChild1)
print("getChild2: " + getChild2)

print("UTF String: " + getChild2.asUTFString())
```

This code prints the following:

```
getChild1: examples.pl.gosu.xml.myschema.anonymous.elements.Element1_Child1 instance
getChild2: examples.pl.gosu.xml.myschema.anonymous.elements.Element1_Child2 instance
UTF String: <?xml version="1.0"?>
<Child2>5</Child2>
```

Removing child elements by name

To remove child elements, use the XML element's `removeChild` method if you expect only one match, or its `removeChildren` method if you expect multiple matches:

`removeChild(QName) : XmlElement`

Removes the child with the specified `QName`. An alternative method signature takes a `String` value for the local part name. For that method signature, Gosu internally creates a `QName` with an empty namespace and the specified local part name.

```
removeChildren(QName) : List<XmlElement>
```

Removes the child with the specified `QName` name. An alternative method signature takes a `String` value for the local part name. For that method signature, Gosu internally creates a `QName` with an empty namespace and the specified local part name.

See also

- “XML simple values” on page 298
- “XSD-based properties and types” on page 286
- “Collections” on page 189

XML attributes

Attributes are additional metadata on an element. For example, in the following example an element has the `color` and `size` attributes:

```
<myelement color="blue" size="huge">
```

Every type instance contains its attributes, which are `XmlSimpleValue` instances specified by a name (a `QName`).

Each `XmlElement` object contains the following methods and properties related to attributes of the element:

- `AttributeNames` property – Gets a set of `QName` objects. The property type is `java.util.Set<QName>`.
- `getAttributeSimpleValue(QName)` – Get an attribute simple value by its name, specified as a `QName`. Returns a `XmlSimpleValue` object. An alternative method signature takes a `String` instead of a `QName`. This method assumes an empty namespace.
- `getAttributeValue(QName) : String` – Get attribute value by its name, specified as a `QName`. Returns a `String` object. An alternative method signature takes a `String` instead of a `QName`. This method assumes an empty namespace.
- `setAttributeSimpleValue(QName, XmlSimpleValue)` – Set an attribute simple value by its name (as a `QName`) and its value (as a `XmlSimpleValue` object). An alternative method signature takes a `String` instead of a `QName`. This method assumes an empty namespace.
- `setAttributeValue(QName, String)` – Set attribute value by its name (as a `QName`) and its value (as a `XmlSimpleValue` object). An alternative method signature takes a `String` instead of a `QName`. This method assumes an empty namespace.

Using the previous example, the following code gets and sets the attributes:

```
myelement.setAttributeValue("color", XmlSimpleValue.makeStringInstance("blue"))
var s = myelement.getAttributeValue("size")
```

If you use XSDs for your elements, for typical use, do not use these methods. Instead, use the shortcuts that Gosu provides. These shortcuts provide a natural and concise syntax for getting and setting attributes.

See also

- “XML simple values” on page 298
- “XSD-based properties and types” on page 286

XML simple values

Gosu uses the `gw.xml.XmlSimpleValue` type to represent the XML format of simple values. An `XmlSimpleValue` object stores a value and provides the logic to serialize that value into XML. Until serialization occurs, Gosu may internally store the value in a format other than `java.lang.String`.

For example, XML represents hexadecimal-encoded binary data using the `xsd:hexBinary` type, representing such values with an `XmlSimpleValue` object whose backing storage is a `byte[]` array. Gosu converts the byte array to hexadecimal digits when it serializes the XML element.

The following properties are provided by `XmlSimpleValue`:

GosuValueType

The IType of the Gosu value

GosuValue

The type-specific Gosu value, such as a `javax.xml.namespace.QName` for an `xsd:QName`

StringValue

A `String` representation of the simple value, such as for a `QName`, which might not be the string that is actually serialized

See also

- “Getting data from an XML element” on page 295

Methods to create XML simple values

The following table lists static methods on the `XmlSimpleValue` type that create `XmlSimpleValue` instances of various types.

Method signature	Description
<code>makeAnyURIInstance(java.net.URI)</code>	Creates a URI instance
<code>makeBase64BinaryInstance(byte[])</code>	Creates a base 64 binary instance from byte array
<code>makeBase64BinaryInstance(gw.xml.BinaryDataProvider)</code>	Creates a base 64 binary instance from binary data provider
<code>makeBooleanInstance(java.lang.Boolean)</code>	Creates a Boolean instance
<code>makeByteInstance(java.lang.Byte)</code>	Creates a byte instance
<code>makeDateInstance(gw.xml.date.XmlDate)</code>	Creates a date-time instance from an <code>XmlDate</code>
<code>makeDateTimeInstance(gw.xml.date.XmlDateTime)</code>	Creates a date instance from an <code>XmlDateTime</code>
<code>makeDecimalInstance(java.math.BigDecimal)</code>	Creates a decimal instance from a <code>BigDecimal</code>
<code>makeDoubleInstance(java.lang.Double)</code>	Creates a decimal instance from a <code>Double</code>
<code>makeDurationInstance(gw.xml.date.XmlDuration)</code>	Creates a duration instance
<code>makeFloatInstance(java.lang.Float)</code>	Creates a float instance
<code>makeGDayInstance(gw.xml.date.XmlDay)</code>	Creates a <code>GDay</code> instance (the numerical day of the month)
<code>makeGMonthDayInstance(gw.xml.date.XmlMonthDay)</code>	Creates a <code>GMonthDay</code> duration instance (the numerical month and day)
<code>makeGMonthInstance(gw.xml.date.XmlMonth)</code>	Creates a <code>GMonth</code> instance (the numerical month)
<code>makeGYearInstance(gw.xml.date.XmlYear)</code>	Creates a <code>GYear</code> instance (the numerical year)
<code>makeGYearMonthInstance(gw.xml.date.XmlYearMonth)</code>	Creates a <code>GYearMonth</code> instance (the numerical year and month)
<code>makeHexBinaryInstance(byte[])</code>	Creates a hex binary instance from byte array
<code>makeIDInstance(java.lang.String)</code>	Creates an <code>IDInstance</code> instance from a <code>String</code>
<code>makeIDREFInstance(gw.xml.XmlElement)</code>	Creates an <code>IDREF</code> instance
<code>makeIntegerInstance(java.math.BigInteger)</code>	Creates a big integer instance
<code>makeIntInstance(java.lang.Integer)</code>	Creates an integer instance
<code>makeLongInstance(java.lang.Long)</code>	Creates a long integer instance
<code>makeQNameInstance(javax.xml.namespace.QName)</code>	Creates a <code>QName</code> instance

Method signature	Description
<code>makeQNameInstance(java.lang.String, javax.xml.namespace.NamespaceContext)</code>	Creates a QName instance from a standard Java namespace context. A namespace context object encapsulates a mapping of XML namespace prefixes and their definitions (namespace URIs). You can get an instance of NamespaceContext from an XmlElement its NamespaceContext property. The String argument is the qualified local name (including the prefix) for the new QName.
<code>makeShortInstance(java.lang.Short)</code>	Creates a duration instance
<code>makeStringInstance(java.lang.String)</code>	Creates a String instance
<code>makeTimeInstance(gw.xml.date.XmlTime)</code>	Creates a duration instance
<code>makeUnsignedByteInstance(java.lang.Short)</code>	Creates an unsigned byte instance
<code>makeUnsignedIntInstance(java.lang.Long)</code>	Creates an unsigned integer instance
<code>makeUnsignedLongInstance(java.math.BigInteger)</code>	Creates an unsigned long integer instance
<code>makeUnsignedShortInstance(java.lang.Integer)</code>	Creates an unsigned short integer instance

The following example creates instances of GDay, GMonthDay, GMonth, GYear and GYearMonth:

```
uses gw.xml.date.XmlDate
uses gw.xml.date.XmlDay
uses gw.xml.date.XmlMonth
uses gw.xml.date.XmlMonthDay
uses gw.xml.date.XmlYear
uses gw.xml.date.XmlYearMonth

var mydate = new XmlDate("2018-12-11")
var cal    = java.util.Calendar.getInstance()

var gd     = new XmlDay(cal, false)
var gmd    = new XmlMonthDay(cal, false)
var gm     = new XmlMonth(cal, false)
var gy     = new XmlYear(cal, false)
var gym    = new XmlYearMonth(cal, false)

print('Gday: ' + XmlSimpleValue.makeGDayInstance(gd))
print('GMonthDay: ' + XmlSimpleValue.makeGMonthDayInstance(gmd))
print('GMonth: ' + XmlSimpleValue.makeGMonthInstance(gm))
print('GYear: ' + XmlSimpleValue.makeGYearInstance(gy))
print('GYearMonth: ' + XmlSimpleValue.makeGYearMonthInstance(gym))
```

This code prints the following:

```
Gday: Simple value (gw.xml.date.XmlDay): ---11
GMonthDay: Simple value (gw.xml.date.XmlMonthDay): --12-11
GMonth: Simple value (gw.xml.date.XmlMonth): --12
GYear: Simple value (gw.xml.date.XmlYear): 2018
GYearMonth: Simple value (gw.xml.date.XmlYearMonth): 2018-12
```

The following examples create similar instances using strings, including the creation of a duration instance:

```
var mydate = new XmlDate("2018-12-11")
var gd     = new XmlDay("---11")
var gmd    = new XmlMonthDay("--12-11")
var gm     = new XmlMonth("--12")
var gy     = new XmlYear("2018")
var gym    = new XmlYearMonth("1900-03")
var dt     = new XmlDateTime("2000-01-12T12:13:14-07:00");
var dur    = new XmlDuration("P50MT50M50")
```

See also

- [W3C XML Schema Part 2](#)

XSD to Gosu simple type mappings

For all elements that have simple types, and for all attributes in an XSD, Gosu creates properties based on the simple schema type of that item. The following table describes how Gosu maps XSD schema types to Gosu types. For schema types that are not listed in the table, Gosu uses the schema type's supertype. For example, the schema type `String` is not listed, so Gosu uses the supertype `anySimpleType`.

Schema type	Gosu type
<code>anySimpleType</code>	<code>java.lang.String</code>
<code>anyURI</code>	<code>java.net.URI</code>
<code>base64Binary</code>	<code>gw.xml.BinaryData</code>
<code>boolean</code>	<code>java.lang.Boolean</code>
<code>byte</code>	<code>java.lang.Byte</code>
<code>date</code>	<code>gw.xml.date.XmlDate</code>
<code>dateTime</code>	<code>gw.xml.date.XmlDateTime</code>
<code>decimal</code>	<code>java.math.BigDecimal</code>
<code>double</code>	<code>java.lang.Double</code>
<code>duration</code>	<code>gw.xml.date.XmlDuration</code>
<code>float</code>	<code>java.lang.Float</code>
<code>int</code>	<code>java.lang.Integer</code>
<code>gDay</code>	<code>gw.xml.date.XmlDay</code>
<code>gMonth</code>	<code>gw.xml.date.XmlMonth</code>
<code>gMonthDay</code>	<code>gw.xml.date.XmlMonthDay</code>
<code>gYearMonth</code>	<code>gw.xml.date.XmlYearMonth</code>
<code>gYear</code>	<code>gw.xml.date.XmlYear</code>
<code>hexBinary</code>	<code>byte[]</code>
<code>IDREF</code>	<code>gw.xml.XmlElement</code>
<code>integer</code>	<code>java.math.BigInteger</code>
<code>long</code>	<code>java.lang.Long</code>
<code>QName</code>	<code>javax.xml.namespace.QName</code>
<code>short</code>	<code>java.lang.Short</code>
<code>time</code>	<code>gw.xml.date.XmlTime</code>
<code>unsignedByte</code>	<code>java.lang.Short</code>
<code>unsignedInt</code>	<code>java.lang.Long</code>
<code>unsignedLong</code>	<code>java.math.BigInteger</code>
<code>unsignedShort</code>	<code>java.lang.Integer</code>
Any type with enumeration facets	Schema-specific enumeration type
Any type derived by list of T	<code>java.util.List<T></code>
Any type derived by union of (T1, T2,... Tn)	Greatest common supertype of (T1, T2,... Tn)

Facet validation

A facet is a characteristic of a data type that restricts possible values. For example, setting a minimum value or matching a specific regular expression.

Gosu represents each facet as an element. Each facet element has a fixed attribute that is a Boolean value. All the facets for a simple type collectively define the set of legal values for that simple type.

Most schema facets are validated at property setting time. A few facets are not validated until serialization time to allow incremental construction of lists at runtime. This mostly affects facets relating to lengths of lists, and those that validate QName objects. Gosu cannot validate QName objects at property setting time because there is not enough information available. Guidewire recommends that you do not apply facets to QName objects.

Example

The following XSD contains defines an attribute that has restricted values.

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Element1">
    <xsd:complexType>
      <xsd:attribute name="Attr1" type="AttrType"/>
    </xsd:complexType>
  </xsd:element>
  <xsd:simpleType name="AttrType">
    <xsd:restriction base="xsd:int">
      <xsd:minInclusive value="0"/>
      <xsd:maxInclusive value="5"/>
    </xsd:restriction>
  </xsd:simpleType>
</xsd:schema>
```

The following code demonstrates the facet validation of the attribute that has restricted values.

```
uses examples.pl.gosu.xml.myschema.Element1
var xml = new Element1()
xml.Attr1 = 3 // Works!
xml.Attr1 = 6 // Fails with an exception.
```

This code prints the following:

```
gw.xml.XmlSimpleValueException: Value '6' violated one or more facet constraints
of simple type definition: value must be no greater than 5
```

Accessing the nilness of an element

XML elements can be nil, which is not the same as being null. An element that is nil must have no child elements, but can have attributes. Additionally, an XSD can define whether an element is nillable, meaning it is allowed to be nil.

If an XSD-based element is nillable, the XmlElement object exposes a property with the name \$Nil. All non-XSD elements also have this property, but in such cases it is called Nil (with no \$ prefix).

Note: For XSD-based elements not marked as nillable, this property is unsupported. In the Gosu editor, if you attempt to use the \$Nil property, Gosu generates a deprecation warning.

Setting this property on an element to true affects whether Gosu adds an xsi:nil attribute to the element upon serialization. Getting this property returns the state of that flag (true or false).

Note: Nillability is an aspect of XSD-based elements, not an aspect of the XSD type itself.

Example

The following XSD contains defines an element that is nillable.

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Element1" type="xsd:int" nillable="true"/>
</xsd:schema>
```

The following code demonstrates how to set the \$Nil property of the element.

```
uses examples.pl.gosu.xml.myschema.Element1
var xml = new Element1()
xml.$Nil = true
xml.print()
```

This code prints the following:

```
<?xml version="1.0"?>
<Element1 xsi:nil="true" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"/>
```

See also

- “Introduction to the XML element in Gosu” on page 275

Automatic creation of intermediary elements

If you use XSDs, whenever a property path appears in the left side of an assignment statement, Gosu creates intermediary elements to ensure that the assignment succeeds. This behavior provides a useful shortcut for typical XML coding. Use this feature to make your Gosu code more understandable.

Example

The following XSD contains defines multiple nested elements.

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Element1">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="Child1">
          <xsd:complexType>
            <xsd:sequence>
              <xsd:element name="Child2" type="xsd:int"/>
            </xsd:sequence>
          </xsd:complexType>
        </xsd:element>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```

The following code demonstrates assignment to a nested element, in which an instance of the intermediary element is automatically created.

```
uses examples.pl.gosu.xml.myschema.Element1
uses examples.pl.gosu.xml.myschema.anonymous.elements.Element1_Child1
uses examples.pl.gosu.xml.myschema.anonymous.elements.Element1_Child1_Child2

var xml = new schema.Element1()
print("Before assignment: ${xml.Child1}")
xml.Child1.Child2 = 5 // Assignment of a value to Child2 automatically creates Child1.
print("After assignment: ${xml.Child1}")
```

This code prints the following:

```
Before assignment: null
After assignment: schema.anonymous.elements.Element1_Child1 instance
```

Default and fixed attribute values

The default values for `default` and `fixed` attributes and elements come from the statically typed property getters. These default values are not stored in the attribute map or content list for an XML type. Gosu adds default or fixed values to attributes and elements in the XML output stream when it serializes its representation of an XML document.

Example

The following example XSD defines an XML element named `person`. The element definition includes an attribute definition named `os` with a default value of “Windows” and an attribute definition named `location` with a fixed value of “San Mateo”.

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="root">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="person" minOccurs="0" maxOccurs="unbounded">
          <xsd:complexType>
            <xsd:sequence>
              <xsd:element name="name" type="xsd:string"/>
            </xsd:sequence>
            <xsd:attribute name="os" type="ostype" default="Windows"/>
            <xsd:attribute name="location" type="xsd:string" fixed="San Mateo"/>
          </xsd:complexType>
        </xsd:element>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>

  <xsd:simpleType name="ostype">
    <xsd:restriction base="xsd:string">
      <xsd:enumeration value="Windows"/>
      <xsd:enumeration value="MacOSX"/>
      <xsd:enumeration value="Linux"/>
    </xsd:restriction>
  </xsd:simpleType>
</xsd:schema>
```

The following sample Gosu code creates a new Gosu representation of an XML document based on the preceding XSD. The code adds two person elements, one for `jsmith` and one for `aanderson`. For `jsmith`, the code adds an `os` attribute having the value `Linux`. The code does not add an `os` attribute to `aapplegate`, nor does the code add the `location` attribute to either person. Instead, the code relies on the default and fixed values defined in the XSD.

```
uses examples.pl.gosu.xml.myschema.Root

var xml = new Root()
xml.Person[0].Name = "jsmith"
xml.Person[0].Os = Linux
xml.Person[1].Name = "aapplegate"
// Gosu adds default and fixed values to the XML document at the time Gosu serializes XML for print.
for (person in xml.Person) {
  print("${person.Name} (${person.Location}) -> ${person.Os}")
}
xml.print()
```

When the preceding Gosu code serializes its representation of an XML document, Gosu adds the fixed and default values to the XML output stream. The printed output shows that the Gosu representation of the XML document does not contain the values “San Mateo” or “Windows”.

```
jsmith (San Mateo) -> Linux
aanderson (San Mateo) -> Windows
<?xml version="1.0"?>
<root>
  <person os="Linux">
    <name>jsmith</name>
  </person>
  <person>
    <name>aanderson</name>
  </person>
</root>
```

Substitution group hierarchies

Just as Gosu reproduces XSD-defined type hierarchies in its type system, Gosu also exposes XSD-defined substitution group hierarchies (*substitution groups*). You can define an XSD `substitutionGroup` attribute on any top-level element to indicate the QName of another top-level element for which it can substitute..

To create a substitution group member, set the XML attribute `substitutionGroup` on an element to the element name (QName) of the substitution group head. The typical usage is to create a substitution group head (the group main element) with an abstract name (for example, "Address").

If an XML element uses a substitution group member QName in place of the head's QName, the Gosu XML processor knows that the substitution happened. However, to cast the type to the expected type for the substitution, call the `cast` method on the result with no arguments. For example:

```
xml.Address = new schema.UKAddress().cast()
```

Example

Create `myschema.xsd` in the package `examples.pl.gosu.xml`:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema" elementFormDefault="qualified">
  <xsd:element name="Customer">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element ref="Address"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
  <xsd:element name="Address"/>
  <xsd:element name="USAddress" substitutionGroup="Address"/>
  <xsd:element name="UKAddress" substitutionGroup="Address"/>
</xsd:schema>
```

The following code creates an `Address` property and uses the substitution group hierarchy to cast that property to a `UKAddress`:

```
uses examples.pl.gosu.xml.myschema.Customer
uses examples.pl.gosu.xml.myschema.UKAddress

var xml = new Customer()
xml.Address = new UKAddress().cast()
xml.print()
```

This code prints the following:

```
<?xml version="1.0"?>
<Customer>
  <UKAddress/>
</Customer>
```

The XML Schema specification requires that the XSD type of a substitution group member be a subtype of the XSD type of its substitution group head. In the example above, `UKAddress`, `USAddress` and `Address` are all of the type `xsd:anyType`, which is the default when there is no explicit type.

Element sorting for XSD-based elements

An XSD can define the strict order of children of an element. For non-XSD elements, element order is undefined.

Each `XmlElement` exposes a `Children` property. For XSD-based elements, the property name is `$Children`.

If the list of child elements is out of order according to the XSD, Gosu sorts the element list during serialization to match the schema. This sorting does not affect the original order of the elements in the content list.

If you use APIs to directly add child elements, such as adding to the child element list or using an `addChild` method, you can add child elements out of order. Similarly, some APIs indirectly add child elements. For example, this occurs when intermediary elements are automatically created. In all of these cases, Gosu permits the children to be out of order in the `XmlElement` object graph.

During serialization, Gosu sorts the elements to ensure that the elements conform to the XSD.

Note that if you use an XSD when you parse XML into an `XmlElement`, the elements must be in the correct order according to the XSD. If the child order violates the XSD, Gosu throws an exception.

Example 1

The following XSD defines three ordered child elements for a complex XML type.

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Element1">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="Child1" type="xsd:int"/>
        <xsd:element name="Child2" type="xsd:int"/>
        <xsd:element name="Child3" type="xsd:int"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```

The following code assigns values to the child elements of the parent element:

```
var xml = new schema.Element1()
xml.Child2 uses examples.pl.gosu.xml.myschema.Element1

var xml = new Element1()
xml.Child2 = 2
xml.Child1 = 1
xml.Child3 = 3
xml.print()
```

This code prints the following:

```
<?xml version="1.0"?>
<Element1>
  <Child1>1</Child1>
  <Child2>2</Child2>
  <Child3>3</Child3>
</Element1>
```

Example 2

The following XSD defines two sets of ordered child elements for a complex XML type. Both sets include some of the same child elements, but use a different ordering.

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Element1">
    <xsd:complexType>
      <xsd:choice>
        <xsd:sequence>
          <xsd:element name="A" type="xsd:int"/>
          <xsd:element name="B" type="xsd:int"/>
          <xsd:element name="C" type="xsd:int"/>
        </xsd:sequence>
        <xsd:sequence>
          <xsd:element name="B" type="xsd:int"/>
          <xsd:element name="C" type="xsd:int"/>
          <xsd:element name="A" type="xsd:int"/>
          <xsd:element name="Q" type="xsd:int"/>
        </xsd:sequence>
      </xsd:choice>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```

The following code assigns values to the child elements of the parent element:

```
uses examples.pl.gosu.xml.myschema.Element1

var xml = new Element1()
xml.A = 5
xml.B = 5
xml.C = 5
xml.print()
print( "-----" )
xml.Q = 5
xml.print()
```

This code prints the following:

```
<?xml version="1.0"?>
<Element1>
  <A>5</A>
  <B>5</B>
```

```

    <C>5</C>
  </Element1>
  -----
  <?xml version="1.0"?>
  <Element1>
    <B>5</B>
    <C>5</C>
    <A>5</A>
    <Q>5</Q>
  </Element1>

```

See also

- “Automatic creation of intermediary elements” on page 303

Sorting of XSD-based elements already in the correct order

If the children of an element are in an order that matches the XSD, Gosu does not sort the element list. This is important if there is more than one sorted order that conforms to the XSD and you expect a particular order.

Example

The following XSD defines two distinct strict orderings of the same elements:

```

<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Element1">
    <xsd:complexType>
      <xsd:choice>
        <xsd:sequence>
          <xsd:element name="A" type="xsd:int"/>
          <xsd:element name="B" type="xsd:int"/>
          <xsd:element name="C" type="xsd:int"/>
        </xsd:sequence>
        <xsd:sequence>
          <xsd:element name="C" type="xsd:int"/>
          <xsd:element name="B" type="xsd:int"/>
          <xsd:element name="A" type="xsd:int"/>
        </xsd:sequence>
      </xsd:choice>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>

```

The following code assigns values to the child elements of the parent element in two different orders that are both valid:

```

uses examples.pl.gosu.xml.myschema.Element1

var xml = new Element1()
xml.A = 5
xml.B = 5
xml.C = 5
xml.print()
print( "-----" )
xml = new Element1()
xml.C = 5
xml.B = 5
xml.A = 5
xml.print()

```

This code prints the following:

```

<?xml version="1.0"?>
<Element1>
  <A>5</A>
  <B>5</B>
  <C>5</C>
</Element1>
-----
<?xml version="1.0"?>
<Element1>
  <C>5</C>
  <B>5</B>

```

```
<A>5</A>
</Element1>
```

Sorting of XSD-based elements matching multiple correct orders

If the children of an element are out of order but multiple correct orderings exist, Gosu uses the first correct ordering defined in the schema.

Example

The following XSD defines two distinct strict orderings of the same elements:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Element1">
    <xsd:complexType>
      <xsd:choice>
        <xsd:sequence>
          <xsd:element name="A" type="xsd:int"/>
          <xsd:element name="B" type="xsd:int"/>
          <xsd:element name="C" type="xsd:int"/>
        </xsd:sequence>
        <xsd:sequence>
          <xsd:element name="C" type="xsd:int"/>
          <xsd:element name="B" type="xsd:int"/>
          <xsd:element name="A" type="xsd:int"/>
        </xsd:sequence>
      </xsd:choice>
    </xsd:complexType>
  </xsd:element>
</xsd:schema>
```

The following code assigns values to the child elements of the parent element:

```
uses examples.pl.gosu.xml.myschema.Element1

var xml = new Element1()
xml.C = 5
xml.A = 5
xml.B = 5
xml.print()
```

This code prints the following:

```
<?xml version="1.0"?>
<Element1>
  <A>5</A>
  <B>5</B>
  <C>5</C>
</Element1>
```

Built-in schemas

Gosu includes several XSDs in the `gw.xsd.*` package. The following table lists the built-in XSDs.

Description of the XSD	Fully qualified XSD package name
The SOAP XSD	<code>gw.xsd.w3c.soap</code>
SOAP envelope XSD	<code>gw.xsd.w3c.soap_envelope</code>
WSDL XSD	<code>gw.xsd.w3c.wsdl</code>
XLink XSD for linking constructs	<code>gw.xsd.w3c.xlink</code>
The XML XSD that defines the attributes beginning with the <code>xml:</code> prefix, such as <code>xml:lang</code>	<code>gw.xsd.w3c.xml</code>
The XML Schema XSD, which is the XSD that defines the format of an XSD	<code>gw.xsd.w3c.xmlschema.Schema</code>

See also

- “The metaschema XSD that defines an XSD” on page 309

The metaschema XSD that defines an XSD

The definition of an XSD is itself an XML file. The *XML Schema XSD* is the XSD that defines the XSD format. The XML Schema XSD is also known as the *metaschema*. This schema is in the Gosu package `gw.xsd.w3c.xmlschema`. You can use the metaschema for building or parsing schemas.

Example

The following code parses the metaschema.

```
var schema = new gw.xsd.w3c.xmlschema.Schema()
schema.Element[0].Name = "Element1"
schema.Element[0].ComplexType.Sequence.Element[0].Name = "Child"
schema.Element[0].ComplexType.Sequence.Element[0].Type = new javax.xml.namespace.QName( "Type1" )
schema.ComplexType[0].Name = "Type1"
schema.print()
```

This code prints the following:

```
<?xml version="1.0"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Element1">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="Child" type="Type1"/>
      </xsd:sequence>
    </xsd:complexType>
  </xsd:element>
  <xsd:complexType name="Type1"/>
</xsd:schema>
```

Gosu does not provide a way to inject a schema into the type system at run time.

Using a local XSD for an external namespace or XSD location

Sometimes an XSD refers to another XSD that is external. Requiring XSD processing to connect across the internet to access that XSD is strongly discouraged. Network connections have low performance and are unreliable in a production system, and the external XSD might not be reachable. A best practice is to copy the XSD locally to the resource tree and specify how to map the external XSD namespace to a local XSD.

In some situations it is common for an XSD to define an external namespace URI without specifying the external download location for an XSD.

In both cases, Gosu provides a registry file, `gwxmlmodule.xml`, that lets you use a local XSD instead. The file is located in the **configuration→res** module.

The base configuration includes multiple `gwxmlmodule.xml` files located in various modules. The **configuration→res→gwxmlmodule.xml** instance is the application-level file. The other instances of the file are for Guidewire internal use only and are not visible in Studio. Their purpose is to remap common W3C XSD files to local locations to avoid unnecessarily connecting to the W3C web site.

See also

- “Referencing additional schemas during parsing” on page 284

Use a local XSD for an external namespace URI or XSD location URL

To remap an external namespace to a local location, edit the `gwxmlmodule.xml` file.

Procedure

1. Download a copy of the target external XSD.
2. Place that XSD in the local resources hierarchy.
3. In Studio, open the `gwxm1module.xml` file. The file is located in the **configuration**→**res** module. Add a `schema-locations` element with an `xmlns` attribute that specifies the local resource location.
4. Add a child schema element to the `schema-locations` element.
5. Set the element's `resourcePath` attribute to the fully qualified path to the local XSD. You must use a forward slash (/), not a period (.), to indicate any subpackage.
6. To direct Gosu to find and remap an external namespace declaration in parsed XSD, set the `xmlNamespace` attribute to the external XSD namespace URI. The value starts with `http://` so it looks like a quoted URL, but it is actually a unique ID and not a URL address.
7. To direct Gosu to find and remap an external XSD download location in parsed XSD, set the `externalLocations` attribute to the external XSD URL address. This value may also be useful for your own reference, in case you need to download the latest version of the file.

Example

Sample `schema-locations` and schema elements are defined in the following example.

```
<!-- XML namespaces and external schema locations can be registered in this file to
      remap to local resources.
      The remappings are used when either an <xsd:import> is encountered without a
      <schemalocation> element, or when an external schema location is specified. The
      "externalLocations" attribute is a space-delimited list of external references to
      recognize for a particular schema.
-->
<schema-locations xmlns="http://guidewire.com/xml/module">
  <schema xmlNamespace="http://guidewire.com/archiving"
    resourcePath="gw/plugin/archiving/archiving.xsd"
    externalLocations=""/>
  <schema xmlNamespace="http://guidewire.com/importing"
    resourcePath="gw/api/importing/importing.xsd"
    externalLocations=""/>
</schemalocations>
```

Schema access type

For each XSD that Gosu loads, it creates a `SchemaAccess` object that represents the loaded XSD. The most important reason to load XSDs is to provide Gosu with additional schemas during XML parsing. `SchemaAccess` objects have a `Schema` property, which is the Gosu XML representation of the XSD. The `Schema` property contains the `gw.xsd.w3c.xmlschema.Schema` object that represents the XSD.

Example

Suppose you have this XSD loaded as `schema.util.SchemaAccess.Schema`:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="Element1"/>
  <xsd:element name="Element2"/>
  <xsd:element name="Element3"/>
</xsd:schema>
```

The following code prints the name of each element in the schema.

```
var schema = examples.pl.gosu.xml.myschema.util.SchemaAccess.Schema
schema.Element.each(\ el ->print(el.Name))
```

This code prints the following:

```
Element1
Element2
Element3
```

The following example uses the XSD of XSDs to print a list of primitive schema types:

```
var schema = gw.xsd.w3c.xmlschema.util.SchemaAccess.Schema
print(schema.SimpleType.map(\s -> s.Name))
```

This code prints the following:

```
[formChoice, reducedDerivationControl, derivationSet, typeDerivationControl, fullDerivationSet,
allNNI, blockSet, namespaceList, public, string, boolean, float, double, decimal, duration, dateTime,
time, date, gYearMonth, gYear, gMonthDay, gDay, gMonth, hexBinary, base64Binary, anyURI, QName,
NOTATION, normalizedString, token, language, IDREFS, ENTITIES, NMTOKEN, NMTOKENS, Name, NCName,
ID, IDREF, ENTITY, integer, nonPositiveInteger, negativeInteger, long, int, short, byte, nonNegativeInteger,
unsignedLong, unsignedInt, unsignedShort, unsignedByte, positiveInteger, derivationControl, simpleDerivationSet]
```

See also

- “Parsing XML data into an XML element” on page 283
- “The metaschema XSD that defines an XSD” on page 309

Conversions from XSD types to Gosu types

The following table lists conversions that Gosu uses to convert from XSD types to Gosu types.

XSD type in custom XSD	Maps to this Gosu type
xsd:anyType	null
xsd:anyURI	java.net.URI
xsd:base64Binary	byte[]
xsd:boolean	java.lang.Boolean
xsd:byte	java.lang.Byte
xsd:date	gw.xml.xsd.types.XSDDate
xsd:dateTime	gw.xml.xsd.types.XSDDateTime
xsd:decimal	java.math.BigDecimal
xsd:double	java.lang.Double
xsd:duration	gw.xml.xsd.types.XSDDuration
xsd:ENTITIES	java.lang.String
xsd:ENTITY	java.lang.String
xsd:ID	java.lang.String
xsd:float	java.lang.Float
xsd:gDay	gw.xml.xsd.types.XSDGDay
xsd:gMonth	gw.xml.xsd.types.XSDGMonth
xsd:gMonthDay	gw.xml.xsd.types.XSDGMonthDay
xsd:gYear	gw.xml.xsd.types.XSDGYear
xsd:gYearMonth	gw.xml.xsd.types.XSDGYearMonth
xsd:hexBinary	byte[]
xsd:IDREF	gw.xml.xsd.IXMLNodeWithID<gw.xml.IReadOnlyXMLNode>
xsd:IDREFS	java.util.List<gw.xml.xsd.IXMLNodeWithID<gw.xml.IReadOnlyXMLNode>>
xsd:int	java.lang.Integer

XSD type in custom XSD	Maps to this Gosu type
xsd:integer	java.math.BigInteger
xsd:language	java.lang.String
xsd:long	java.lang.Long
xsd:Name	java.lang.String
xsd:NCName	java.lang.String
xsd:negativeInteger	java.math.BigInteger
xsd:NMTOKEN	java.lang.String
xsd:NMTOKENS	java.util.List<java.lang.String>
xsd:nonNegativeInteger	java.math.BigInteger
xsd:nonPositiveInteger	java.math.BigInteger
xsd:normalizedString	java.lang.String
xsd:NOTATION	java.lang.String
xsd:positiveInteger	java.math.BigInteger
xsd:QName	javax.xml.namespace.QName
xsd:short	java.lang.Short
xsd:string	java.lang.String
xsd:time	gw.xml.xsd.types.XSDTime
xsd:token	java.lang.String
xsd:unsignedByte	java.lang.Byte
xsd:unsignedInt	java.lang.Integer
xsd:unsignedLong	java.lang.Long
xsd:unsignedShort	java.lang.Short

Working with JSON in Gosu

Gosu provides native support for the JavaScript Object Notation (JSON) data format.

Overview of JSON support

Gosu has native support for JavaScript Object Notation (JSON) data format. JSON is an open-standard format that uses human-readable text to transmit data objects consisting of attribute-value pairs, hierarchical data structures, and arrays. Web sites often send or receive small amounts of JSON data as a lightweight alternative to the XML standard. Creation of JSON data and parsing can be implemented in any language or operating system, but is popular due to efficient web browser client-side parsing of JSON in JavaScript.

The JSON format is a binding of name-value pairs where a value is one of the following:

- A simple type, like a `String` or a number
- An object that contains name/value pairs
- An array of values

JSON itself has no inherent ability to provide type information other than those previously mentioned built-in types. For example, there is no built-in way to encode a record as a specific complex type that represents a person, a car, or a mailing address. Despite widespread use of JSON in web development, there is no standard schema for JSON. There is no XSD or DTD equivalent for JSON such that the publisher of a format can specify the correct data format to expect or validate against. As a consequence, most development environments do minimal validation and treat JSON code as untyped attribute-value pairs.

The native Gosu features called *expando* objects and dynamic types support the general metaphor of general purpose attribute-value pairs. You can use those Gosu features to generate JSON-like structures with a natural lightweight syntax:

```
var person: Dynamic = new() {
    :Name = "Andy Applegate",
    :Age = 39
}
```

This dynamic access can be used to get and set data in a non-type-safe way with little effort.

However, you may want to generate actual JSON data from this type of structure, or parse complex JSON data into a structure that you can naturally navigate from Gosu. Gosu provides a way of generating a type-safe class called a *structural type* from JSON data, which can assist type-safe Gosu coding with JSON data.

See also

- “Structural types” on page 263
- “Dynamic types and expando objects” on page 255
- “Dynamic access to JSON objects” on page 314
- “Structural type-safe access to JSON objects” on page 317
- “Concise dynamic object creation syntax for JSON coding” on page 319

Dynamic access to JSON objects

You can convert a JSON document as `String` data to a dynamic Gosu object. You can navigate the result using property names and array index syntax in a natural way in Gosu.

WARNING Dynamic access to JSON data is powerful and convenient but is not type safe. For example, if you misspell a property name, the Gosu editor does not catch your error at compile time.

Parsing JSON from a `String` object

To convert JSON data to a dynamic object with Gosu property access, use the `gw.lang.reflect.json.Json` class. Call its `fromJson` method and pass the `String` data as an argument. The method is declared to return a Java object that implements the standard interface `javax.script.Bindings`. At run time, the return type `javax.script.SimpleBindings`, which implements the `Bindings` interface.

If you declare a variable of type `Bindings` or `SimpleBindings`, you can get properties from the object. However, you must use the awkward syntax of the `get` method and passing an attribute name as a quoted `String` object.

```
var jsonParsed = Json.fromJson(jsonText)
print("Name = " + jsonParsed.get("Name"))
```

This syntax is neither convenient nor natural Gosu syntax.

Instead, Gosu natively provides natural attribute-value programming access to the `Bindings` type if you declare a variable as the type `dynamic.Dynamic`. Use this approach to get properties by name with natural syntax. Any JSON property that contains an array becomes a standard Java list, which you can use with array index syntax, collection-related Gosu enhancements, or loop syntax.

For example, run the following code in the Gosu Scratchpad:

```
uses dynamic.Dynamic
uses gw.lang.reflect.json.Json

// For this example, note that a String literal must use \" to escape quote symbols
//
// { "Name": "Andy Applegate",
//   "Age": 29,
//   "Hobbies": ["Swimming", "Hiking"] }
//
var jsonText = "{ \"Name\": \"Andy Applegate\", \"Age\": 29, \"Hobbies\": [\"Swimming\", \"Hiking\" ] }"
var j : Dynamic = Json.fromJson(jsonText)

print("Name = " + j.Name)
print("Age = " + j.Age)
print ("Number of hobbies... " + j.Hobbies.size)
for (h in j.Hobbies) {
    print(" one hobby = " + h)
}
```

This code prints the following

```
Name = Andy Applegate
Age = 29
Number of hobbies... 2
one hobby = Swimming
one hobby = Hiking
```

See also

- “Dynamic types and expando objects” on page 255

JSON APIs for URLs

Because JSON data often comes from internet requests, Gosu adds an enhancement property named `JsonContent` to the standard Java class `java.net.URL` to simplify your JSON integrations. If the URL object is an HTTP URL, accessing the `JsonContent` property fetches the JSON content by running the HTTP GET method. If the content of the URL object is a JSON document, this property provides a JSON object that reflects the structure and data in that document.

Parsing JSON from a URL

To get JSON data from a URL, you first create an instance of `java.net.URL` using the URL as the argument. Next, you get the `JsonContent` property on an instance of URL. The `JsonContent` property getter performs several complex tasks:

1. The getter makes the HTTP or other URL request from the internet and gets the result as text.
2. The getter parses the JSON text into a `Bindings` object using the static method `gw.lang.reflect.json.Json.fromJson(urlRequestString)`.
3. The getter coerces this `Bindings` object to the type `dynamic.Dynamic` before returning the object. Because the type is dynamic, you can use type inference when assigning the result to a variable.

For example, run the following code in the Gosu Scratchpad:

```
// Create a URL
var personUrl = new URL( "http://gosu-lang.github.io/data/person.json" )

// Access the URL, parse the JSON text content to a Bindings object, return as dynamic.Dynamic.
var person = personUrl.JsonContent

// Parse the dynamic object using a natural Gosu syntax
print( person.Name )
```

Note that any JSON property that contains an array becomes a standard Java list, which you can use with array index syntax or loops.

See also

- “Useful JSON API methods” on page 321

Generating URLs from JSON data

You can use JSON data to create a standard URL query. The result is a root URL, followed by a question mark, followed by a list of URL argument assignments from JSON data, separated by ampersands. The syntax is similar to:

```
http://example.com/page?arg1=val1&arg2=val2
```

Create a URL of this type using the static Gosu enhancement method `makeUrl` on the `java.net.URL` class. Pass a base URL as the first argument, and the JSON data as the second argument. The JSON data must be a bindings object like `javax.script.SimpleBindings` or anything that implements interface `javax.script.Bindings`. A Gosu expando object, such as the Gosu class `Expando`, satisfies the `Bindings` requirement.

The following example Gosu code creates JSON-like data in two ways and generates a URL that uses the JSON data for query arguments with `String` and number values. Numbers are coerced to a `String` to serialize them into text.

```
uses gw.lang.reflect.Expando
uses java.net.URL

var rootUrl = "http://example.com/en"

// ** Using Dynamic type and initialization syntax
```

```
var d: dynamic.Dynamic = new() {
    :Name = "Andy Applegate",
    :Age = 39
}
var url = URL.makeUrl(rootUrl, d)
print("Using dynamic bindings object = " + url)

// ** Using explicit Expando instantiation
var e : dynamic.Dynamic = new Expando()
e.Name = "John Smith"
e.Age = 44
url = URL.makeUrl(rootUrl, e)
print("Using dynamic expando object = " + url)
```

This code prints:

```
Using dynamic bindings object = http://example.com/en?Name=Andy+Applegate&Age=39
Using dynamic expando object = http://example.com/en?Name=John+Smith&Age=44
```

Note that space characters became + characters. This transformation is part of the process called URL encoding. Be aware that some special characters may become multiple characters as part of URL encoding.

The previous example used Bindings values that were String objects or numbers. The makeUrl method also supports values of other complex types:

- If the value is a `javax.script.Bindings` object, a Gosu dynamic expando object, or a list, Gosu transforms the object to JSON String data.
- Any other value is coerced to a String.

Finally, in all cases, the values are URL encoded before becoming part of the final URL.

The following example creates a new URL from a Bindings object with values that include one Bindings object and one object of list type `List<String>`:

```
uses javax.script.Bindings
uses java.net.URL

var rootUrl = "http://example.com/en"

var d: dynamic.Dynamic = new() {
    :Who = new() { :FirstName="Andy", :LastName="Applegate" },
    :Likes = {"Hiking", "Football"}
}

print("JSON of :Who = " + d.Who.toJson())
print("JSON of :Likes = " + Bindings.listToJson(d.Likes))

var url = URL.makeUrl(rootUrl, d)
print ("URL encoding = " + url)
```

The URL output for this example is harder to read because of URL encoding of special characters such as braces, comma characters, and new-line characters. Also, multiple + characters represent space characters for each line of indented JSON.

This example prints the following, which is shown formatted with added new lines in the URL for clarity:

```
JSON of :Who = {
  "FirstName": "Andy",
  "LastName": "Applegate"
}
JSON of :Likes = [
  "Hiking",
  "Football"
]
URL encoding = http://example.com/en
?Who=%7B%0A+++%22FirstName%22%3A+%22Andy%22%2C%0A+++%22LastName%22%3A+%22Applegate%22%0A%7D
&Likes=%5B%0A+++%22Hiking%22%2C%0A+++%22Football%22%0A+++%5D
```

See also

- “Dynamic type” on page 255
- “Concise dynamic object creation syntax for JSON coding” on page 319.
- “Expando objects” on page 257
- “Useful JSON API methods” on page 321

Structural type-safe access to JSON objects

You can use dynamic property access with parsed JSON data, which is powerful and convenient, but not type safe. For example, if you misspell a property name, the Gosu editor cannot catch the error at compile time.

Gosu provides APIs that allow you to generate type-safe code to access JSON data. These APIs protect your code against misspelling a property name. More errors can be caught at compile time. Additionally, this approach lets you create meaningful types that you can use in the rest of your code. For example, you can create a type that represents a person, a group, or complex hierarchical business data types.

Although JSON structural types are type safe and client code validates at compile time, JSON structural types do not perform the same validations as XML schemas such as XSDs. For example, Gosu JSON structural types do not check for non-nullable fields or enforce minimum or maximum numbers of list elements. You may need to write your own validation logic to enforce your application’s unique business logic and field validation.

WARNING If property names contain unusual characters or reserved Gosu keywords, the name is changed automatically and preserved in the `@ActualName` annotation. However, the built-in APIs that convert Gosu data to JSON do not use the data that the annotation preserves.

See also

- “Dynamic access to JSON objects” on page 314

Create a JSON structural type from an example object

Procedure

1. Generate an example JSON object that contains every possible property. Every property must be non-empty and have the correct JSON type: a String, a number, an array, or another object.

For example:

```
{
  "Name": "Andy Applegate",
  "Age": 29,
  "Address": {
    "Number": 123,
    "Street": "Main St",
    "City": "Foster City",
    "State": "CA"
  },
  "Hobby": [
    {
      "Category": "Sport",
      "Name": "Baseball"
    },
    {
      "Category": "Recreation",
      "Name": "Hiking"
    }
  ]
}
```

2. Upload that data as a file on a web server.
3. In the Gosu Scratchpad, use the `java.net.URL` Gosu enhancement property `JsonContent`.

```
var personUrl = new URL( "http://URL-path/file-name.json" )
```

4. In the Gosu Scratchpad, take the result of the previous step and call the `toStructure` method and pass the following two arguments:
 - A name of your new structural type, as a `String`
 - A Boolean that represents whether the structural type is mutable. If you want only property getters but not setters, pass the value `false`. If you want property getters and setters, pass the value `true`.

For example:

```
var personUrl = new URL( "http://gosu-lang.github.io/data/person.json" )
var person: Dynamic = personUrl.JsonContent
var sj = person.toStructure( "Person", false )
```

5. Print the result, which contains Gosu code for a new structural type based on the structure of your example JSON data.

For example:

```
print(sj)
```

The output looks like the following:

```
structure Person {
  static function fromJson( jsonText: String ): Person {
    return gw.lang.reflect.json.Json.fromJson( jsonText ) as Person
  }
  static function fromJsonUrl( url: String ): Person {
    return new java.net.URL( url ).JsonContent
  }
  static function fromJsonUrl( url: java.net.URL ): Person {
    return url.JsonContent
  }
  static function fromJsonFile( file: java.io.File ) : Person {
    return fromJsonUrl( file.toURI().toURL() )
  }
  property get Address(): Address
  property get Hobby(): List<Hobby>
  property get Age(): Integer
  property get Name(): String
  structure Address {
    property get Number(): Integer
    property get State(): String
    property get Street(): String
    property get City(): String
  }
  structure Hobby {
    property get Category(): String
    property get Name(): String
  }
}
```

6. To use the structural type in other types, you can do either of the following:
 - To make a top-level type, copy and paste the structural type declaration into a new `.gs` file within your desired package.
 - To make an inner type, similar to an inner class in Gosu or Java, copy and paste the structural type declaration into a class declaration.

See also

- “JSON APIs for URLs” on page 315
- “Structural type-safe access to JSON objects” on page 317
- “Problematic property names for JSON structural types” on page 318
- “Useful JSON API methods” on page 321

Problematic property names for JSON structural types

The JSON object syntax supports a wider range of characters in the name part of the name-value pairs than Gosu supports for property names. For example, space characters are permissible in JSON object name fields but not Gosu

property names. This makes it problematic to represent the structural type representation of this name as a Gosu property name. JSON also supports object name fields that would be Gosu reserved keywords such as `new` or `class`, and would cause syntax issues in type declaration or API usage.

For example, the following is legal JSON that requires special handling in Gosu structural types:

```
{
  "Space Available": "8",
  "new": "true",
  "class": 1984
}
```

For problematic property names in generated structural types, Gosu generates alternative names that satisfy Gosu property name syntax as follows:

- For illegal characters, Gosu substitutes the underscore character (`_`).
- For reserved keywords, Gosu reverses the case of the first character. Gosu keywords are case-sensitive, so this removes the conflict.

In both cases, Gosu preserves the original names in the annotation `@gw.lang.reflect.ActualName`.

For the previous example JSON, the generated structural type would include the following property getters and setters, with the `@ActualName` annotation as needed:

```
structure TestStructuralType {
  ...
  @gw.lang.reflect.ActualName( "new" )
  property get New(): String
  @gw.lang.reflect.ActualName( "class" )
  property get Class(): Integer
  @gw.lang.reflect.ActualName( "Space Available" )
  property get Space_Available(): String
}
```

WARNING If there were problematic property names due to unusual characters or reserved Gosu keywords, the name is changed automatically and preserved in the `@ActualName` annotation. However, the data preserved by the annotation is unused by the built-in APIs to convert Gosu data to JSON.

Concise dynamic object creation syntax for JSON coding

Gosu supports a concise object creation syntax similar to JSON directly in its grammar. The following Gosu features provide this functionality:

- You can omit the type name in a new expression if Gosu can infer the type. For example, instead of typing `new TestClass(arg1, arg2)`, you can type `new(arg1, arg2)`. The programming context already assumes the compile-time type called `TestClass`.
- You can initialize fields during instantiation using a special syntax with curly braces around a comma-delimited list of property assignments, with a colon before the property name. For example:

```
new TestClass() { :Name = "John Smith" }
```

- Gosu supports dynamic non-type-safe access to properties and methods for variables of the type called `dynamic.Dynamic`. No declaration of specific property names or method names is required in advance. For programming contexts of type `dynamic.Dynamic`, a newly instantiated object assumes the default run-time type of `Expando`. The `Expando` class implements the `Bindings` interface. Gosu adds enhancement methods to this class to convert to JSON (`toJson`), XML (`toXml`), or Gosu object initialization code (`toGosu`).

When you combine these features, you can create objects with multiple types of data without declaring any specific structure in advance, and can convert to other formats as needed.

The following Gosu code creates a dynamic complex object and prints it in JSON syntax, XML syntax, and Gosu object initialization syntax:

```
var person: dynamic.Dynamic
```

```

person = new() {
  :Name = "John Smith",
  :Age = 39,
  :Address = new() {
    :Number = 123,
    :Street = "Main St.",
    :City = "Foster City",
    :State = "CA"
  },
  :Hobby = {
    new() {
      :Category = "Sport",
      :Name = "Swimming"
    },
    new() {
      :Category = "Recreation",
      :Name = "Hiking"
    }
  }
}

print("**** To JSON:")
print(person.toJson())

print("**** To XML:")
print(person.toXml())

print("**** To Gosu:")
print(person.toGosu())

```

This example prints the following:

```

**** To JSON:
{
  "Name": "John Smith",
  "Age": 39,
  "Address": {
    "Number": 123,
    "Street": "Main St.",
    "City": "Foster City",
    "State": "CA"
  },
  "Hobby": [
    {
      "Category": "Sport",
      "Name": "Swimming"
    },
    {
      "Category": "Recreation",
      "Name": "Hiking"
    }
  ]
}
**** To XML:
<object>
  <Name>John Smith</Name>
  <Age>39</Age>
  <Address>
    <Number>123</Number>
    <Street>Main St.</Street>
    <City>Foster City</City>
    <State>CA</State>
  </Address>
  <Hobby>
    <li>
      <Category>Sport</Category>
      <Name>Swimming</Name>
    </li>
    <li>
      <Category>Recreation</Category>
      <Name>Hiking</Name>
    </li>
  </Hobby>
</object>
**** To Gosu:
new Dynamic() {
  :Name = "John Smith",
  :Age = 39,
  :Address = new() {

```



```

    :Number = 123,
    :Street = "Main St.",
    :City = "Foster City",
    :State = "CA"
  },
  :Hobby = {
    new() {
      :Category = "Sport",
      :Name = "Swimming"
    },
    new() {
      :Category = "Recreation",
      :Name = "Hiking"
    }
  }
}

```

Note that for the Gosu initializer syntax serialization, you can evaluate the String data at run time to create a Gosu object:

```
var clone = eval( person.toGosu() )
```

See also

- “Optional omission of type name with the new keyword” on page 85
- “Object initializer syntax” on page 86
- “Dynamic types and expando objects” on page 255

Useful JSON API methods

The following table lists and describes some useful JSON API methods and properties.

Class	Method or property	Arguments	Description
java.net.URL	JsonContent	Not applicable	The getter for this property: • Makes the HTTP or other URL request from the internet and gets the result as text. • Parses the JSON text into a <code>javax.script.Bindings</code> object. • Coerces this Bindings object to the type <code>dynamic.Dynamic</code> before returning the object. For this dynamic type, Gosu uses type inference when assigning the result to a variable. For example: <pre>var person: Dynamic = personUrl.JsonContent</pre>
gw.lang.reflect.json.Json	fromJson	String	Parses JSON data as a String and converts it to the structural type. To use natural Gosu syntax for properties of the result, assign the result to a variable of type <code>dynamic.Dynamic</code>. For example: <pre>var j : Dynamic = Json.fromJson(jsonText)</pre>
	fromJsonFile	java.io.File	Get JSON data from a File object and convert it to the structural type. To use natural Gosu syntax for properties of the result, assign the result to a variable of type <code>dynamic.Dynamic</code>.
	fromJsonUrl	String or java.net.URL	Gets JSON data from a URL and convert it to the structural type. One method signature takes a URL as a String, and one method signature takes the URL as a <code>java.net.URL</code> object. To use natural Gosu syntax for properties of the result, assign the result to a variable of type <code>dynamic.Dynamic</code>.

Class	Method or property	Arguments	Description
dynamic.Dynamic	toGosu	No arguments	Converts an instance of the structural type to Gosu code that creates that instance. The return value is a <code>String</code> object containing Gosu code. Escaped Unicode characters in the structural type remain unchanged in the Gosu code.
	toJson	No arguments	Converts an instance of the structural type to JSON. The return value is a <code>String</code> object containing JSON data. Escaped Unicode characters in the structural type remain unchanged in the JSON data.
	toXml	No arguments	Converts an instance of the structural type to XML. The return value is a <code>String</code> object containing XML data. Escaped Unicode characters in the structural type are converted to unescaped Unicode characters in the XML data.

Templates

Gosu includes a native template system. Templates are text with embedded Gosu code within a larger block of text. The embedded Gosu code optionally calculates and generates text at the location of the code in the template text. It is also possible to store a Gosu template as a separate file. You can design your template files in either HTML or XML format, as long your HTML or XML conforms to the applicable standards.

Gosu provides the following mechanisms to use a Gosu template:

Template syntax inside a text literal

Inside your Gosu code, use template syntax for an in-line `String` literal value containing an embedded Gosu expression. Gosu template syntax combines static text that you provide with dynamic Gosu code that executes at run time and returns a result. Gosu uses the result of the Gosu expression to produce dynamic output at run time as a `String` value.

Scriptlet syntax inside a text literal

Inside your Gosu code, use scriptlet syntax for in-line Gosu statements.

Separate template file

Define a Gosu template as a separate file that you can execute from other code to perform an action and generate output. If you use a separate template file, you can use additional features such as passing custom parameters to your template.

Template expressions

Use the following syntax to embed a Gosu expression in `String` text:

```
${ EXPRESSION }
```

For example, suppose you want to display text with some calculation in the middle of the text:

```
var mycalc = 1 + 1
var myVariable = "One plus one equals " + mycalc + "."
```

Instead of this multiple-line code, embed the calculation directly in the `String` as a template:

```
var myVariable = "One plus one equals ${ 1 + 1 }."
```

If you print this variable, Gosu prints:

```
One plus one equals 2.
```

Gosu evaluates your template expression at run time. The expression can include variables or dynamic calculations that return a value:

```
var s1 = "One plus one equals ${ myVariable }."
var s2 = "The total is ${ myVariable.calculateMyTotal() }."
```

At compile time, Gosu uses the built-in type checking system to ensure the embedded expression is valid and type safe.

If the expression does not return a value of type `String`, Gosu attempts to coerce the result to the type `String`.

Alternative template expression syntax `<%= ... %>`

The syntax `${ EXPRESSION }` is the preferred style for template expressions.

Gosu also provides an alternative template style. Use the three-character text `<%=` to begin the expression. Use the two-character text `%>` to end the expression. For example, you can rewrite the previous example as the following concise code:

```
var myVariable = "One plus one equals <%= 1 + 1 %>."
```

Running Gosu statements in a template scriptlet

Text enclosed with the text `<%` and `%>` evaluates at run time in the order the parser encounters the text but generates no output from the result. These character sequences are called scriptlet tags. Note that this type of tag has no equal sign in the opening tag. *Scriptlet tags* enclose Gosu statements, not expressions. Gosu evaluates all plain text between scriptlet tags within the scope and the logic of the scriptlet code.

The scriptlet tags are available in template files and in `String` literals that use template syntax.

The following template uses a scriptlet tag to run code to assign a variable and uses the result later:

```
<% var MyCalc = 1 + 2 %>One plus two is ${ MyCalc }
```

The result of this template is the following:

```
One plus two is 3
```

The result of the scriptlet tag at the beginning of the template does not generate any output. The statement inside the scriptlet tags creates a variable `MyCalc`, and assigns the result of a calculation to that variable. The subsequent expression uses the expression delimiters `${` and `}` to cause Gosu to retrieve the value of the variable `MyCalc`, which is 3.

The scope of the Gosu code continues across scriptlet tags. Use this feature to write advanced logic that uses Gosu code that you spread across multiple scriptlet tags. For example, the result of the following template code is “x is 5” if the variable `MyCalc` has the value 5, otherwise the result is “MyCalc is not 5”:

```
<% if (MyCalc == 5) { %>
MyCalc is 5
<% } else { %>
MyCalc is not 5
<% } %>
```

The `if` statement controls the flow of execution of later elements in the template. You use this feature to control the export of static text in the template as well as template expressions.

Scriptlet tags are particularly useful when used with template parameters because you can define conditional logic as shown in the previous example.

Use this syntax to iterate across lists, arrays, and other iterable objects. You can combine this syntax with the expression syntax to generate output from the inner part of your loop. Remember that the scriptlet syntax does not itself support generating output text.

For example, suppose you set a variable called `MyList` that contains a `List` of objects with a `Name` property. The following template iterates across the list:

```
<% for (var obj in MyList) {
  var theName = obj.Name %>
  Name: ${ theName }
<% } %>
```

This template might generate output such as:

```
Name: John Smith
Name: Wendy Wheathers
Name: Alice Applegate
```

The following code shows the use of scriptlets in a `String` value:

```
var str = "<% for (i in 1..5) { var odd = (i % 2 == 1) %>${i} is ${ (odd?"odd":"even") } \n<% } %>"
print(str)
```

This code prints:

```
1 is odd
2 is even
3 is odd
4 is even
5 is odd
```

IMPORTANT Gosu has no supported API to generate template output from within a template scriptlet. Instead, design your template to combine template scriptlets and template expressions using the code pattern in this topic.

See also

- “Template parameters” on page 329

Escaping special characters for templates

Gosu templates use standard characters to indicate the beginning of a special block of Gosu code or other template structures. In some cases, to avoid ambiguity for the Gosu parser you must escape special characters.

For non-template-tag use, escape `${` or `<%`

Gosu templates use the following two-character sequences to begin a template expression

- `${`
- `<%`

In a `String` literal in your code, if you want to use these sequences to indicate template tags, you do not need to escape these special characters.

If you require either of those two special two-character sequences in your `String`, rather than as a template tag, escape the first character of that sequence. To escape a character, add a backslash character immediately before the character to escape. For example:

- To define a variable containing the non-template text `"Hello${There}"`:

```
var s = "Hello\${There}"
```

- To define a variable containing the non-template text `"Hello<%There"`:

```
var s = "Hello\<%There"
```

If your `String` uses the initial character of the template sequence, but the next character does not complete that sequence, you do not need to escape the character. For example:

- To define a variable containing the non-template text `"Hello$There"`, just use:

```
var s = "Hello$There"
```

- To define a variable containing the non-template text `"Hello<There"`, just use:

```
var s = "Hello<There"
```

Within template tag blocks, use standard Gosu escaping rules

In typical use, if you defined a `String`, you must escape any quotation marks with the syntax `\` to avoid ambiguity about the end of the `String`. For example:

```
var quotedString = "\"This string has quotation marks around it\", is that correct?"
```

This code creates a `String` with the following value, including quotation marks:

```
"This string has quotation marks around it", is that correct?
```

If you use a template, this rule does not apply between your template-specific opening and closing tags that contain Gosu code. Instead, use standard Gosu syntax for the code between those open and closing tags.

For example, the following two lines are valid Gosu code:

```
var s1 = "${ "1" }"
var s2 = "${ "1" } \"and\" ${ "1" }"
```

Note that the first character within the template's Gosu block is an unescaped quotation mark.

The following code is invalid because of incorrect escaping of internal quotation marks:

```
var s = "${ \"1\" }"
```

In this invalid case, the first character in the template's Gosu block is an escaped quotation mark.

In the rare case that your Gosu code requires creating a `String` literal containing a quotation mark character, remember that the standard Gosu syntax rules apply. You need to escape any quotation marks inside the `String` literal. For example, the following is valid Gosu:

```
var quotedString = "${ "\"This string has quotation marks around it\", is that correct?" }"
```

Note that the first character in the template's Gosu block is an unescaped quotation mark. This template generates a `String` with the value:

```
"This string has quotation marks around it", is that correct?
```

IMPORTANT Be careful with how you escape quotation mark characters within your embedded Gosu code or other special template blocks.

Using template files

Instead of defining your templates in inline text, you can store a Gosu template as a separate file. Template files support all the features that inline templates support. In addition, template files provide additional advantages and features:

Separate your template definition from code that uses the template

For example, define a template that generates a report or a notification email. You can then call this template from many places but define the template only once.

Encapsulate your template definition for better change control

By defining the template in a separate file, your teams can edit and track template changes over time separate from code that uses the template.

Run Gosu statements and return no value using scriptlet syntax

You can define one or more Gosu statements as a statement list embedded in the template. Contrast this syntax with the template expression syntax, which requires Gosu expressions rather than Gosu statements. The results of scriptlet tags generate no output.

Define template parameters

Template files can define parameters that you pass to the template at run time.

Extend a template from a class to simplify static method calls

If you call static methods on one main class in your template, you can simplify your template code by using the `extends` feature.

See also

- “Running Gosu statements in a template scriptlet” on page 324

Creating and running a template file

Gosu template files have the extension `.gst`. Create template files within the package hierarchy in the file system just as you create Gosu classes. Choose the package hierarchy carefully because you use this package name to access and run your template. The template is a first-class object in the Gosu type system within its package namespace.

In your template file, include the template body with no surrounding quotation marks. The following line is a simple template:

```
One plus one equals ${ 1 + 1 }.
```

To create a new template within Studio, in the **Project** pane within the **gsrc** section, right-click a package. Next, click **New→Gosu Template**. In the **New Gosu Template** dialog, type a meaningful name, and then click **OK**.

Rendering to a String

To run a template, get a reference to your template and call the `renderToString` method of the template. The `renderToString` method returns the template results as a `String` value.

For example, suppose you create a template file `NotifyAdminTemplate.gst` within the package `mycompany.templates`. The fully qualified name of the template is `mycompany.templates.NotifyAdminTemplate`.

Use the following code to render your template:

```
var x = mycompany.templates.NotifyAdminTemplate.renderToString()
```

The variable `x` contains the `String` output of your template.

If you need to pass template parameters to your template, provide arguments to the `renderToString` method.

Rendering to a writer

Optionally, you can render the template directly to a Java writer object. Your writer must be an instance of `java.io.Writer`. Get a reference to the template and call its `render` method. Pass the writer as an argument to the `render` method.

For example, suppose you create a template file `NotifyAdminTemplate.gst` within the package `mycompany.templates`. If the variable `myWriter` contains an instance of `java.io.Writer`, the following Gosu statement renders the template to the writer:

```
mycompany.templates.NotifyAdminTemplate.render(myWriter)
```

If you use template parameters in your template, put your additional parameters after the writer argument.

See also

- “Template parameters” on page 329

Create and use a template file

About this task

This example shows a common design pattern for templates that need to combine complex logic in scriptlet syntax with generated text into the template within a loop. This example produces static text. A typical template uses one or more parameters to produce dynamic output.

Procedure

1. Using Studio, create a new template. In the **Project** pane within the **gsrc** section, right-click a package. Next, click **New→Gosu Template**. In the **New Gosu Template** dialog, type a meaningful name, and then click **OK**.

For example, the fully qualified name of your template could be `mycompany.templates.IntegerStringsInArray`.

2. Use a template scriptlet to create a static array or other iterable object:
 - a. Use `<%` to begin your definition and type normal Gosu code to define the object.

```
<% var MyArray : ArrayList<String> = {
    "1",
    "Two",
    "3.0"
}
```

- b. End the scriptlet, using `%>`.
Alternatively, define a parameter that specifies an array or other iterable object.

3. Optionally, generate some static text.

For example:

```
Testing strings for number content:
```

4. Use a template scriptlet to iterate over your object. For example:

- a. Use `<%` to begin your loop. For example:

```
<% for (var obj in MyArray) {
```

- b. Before ending the scriptlet, set up a variable with your data to export and optionally perform application logic.

```
    var string = obj.toString()
    try {
        obj.toDouble()
```

- c. End the scriptlet, using `%>`.

5. Insert a template expression to export the value of your variable, by enclosing a Gosu expression with `$` { and } tags. Use your variable and, optionally, static text to produce data to export. Use more scriptlet tags as necessary to produce valid Gosu code.

```
    ${ string } is a number<%> catch (e : Exception) { %>
    "${ string }" is not a number
```

6. Optionally, generate some static text.
7. Add another template scriptlet by using `<%` and `%>` to contain code that closes your loop. Remember that scriptlets share scope across all scriptlets in that file, so you can reference other variables or close loops or other Gosu structural elements.

```
    <% }
    }%>
```

8. If Studio is running a server, you must load the changed file by remaking the project. From the menu, click **Build→Make Project**.
9. In Gosu Scratchpad, run the template by using code like the following:

```
print(mycompany.templates.IntegerStringsInArray.renderToString())
```

Result

Running this code produces text like the following:

```
1 is a number
"Two" is not a number
3.0 is a number
```

Example

The complete code for this template file example looks like:


```
<% var MyArray : ArrayList<String> = {  
    "1",  
    "Two",  
    "3.0"  
}  
>%  
Testing strings for number content:  
<% for (var obj in MyArray) {  
    var string = obj.toString()  
    try{  
        obj.toDouble() >%  
        ${ string } is a number<%} catch (e : Exception) { %>  
        "${ string }" is not a number  
    }  
<% }  
>%>
```

See also

- “Template parameters” on page 329

Template parameters

You can pass parameters of any type to your self-contained Gosu template files. The syntax for defining parameters for a template is:

```
<%@ params(ARGLIST) %>
```

ARGLIST is an argument list such as for a standard Gosu function.

You can use template parameters in template files, but not in `String` literals that use template syntax. In a template file, you can use a parameter in either the template expression syntax (`${ }` and `}`) or template scriptlet syntax (`<%` and `%>`). The expression syntax always returns a result and generates additional text. The scriptlet syntax executes Gosu statements.

For example, suppose you create a template file `NotifyAdminTemplate.gst` within the package `mycompany.templates`. Edit the file to contain the following lines of code:

```
<%@ params(personName : String, contactHR: boolean) %>  
The person ${ personName } must update their contact information in the company directory.  
  
<% if (contactHR) { %>  
Call the human resources department immediately.  
<% } %>
```

In this example, the `if` statement, including its trailing brace, is within scriptlet tags. The `if` statement uses the parameter value at run time to conditionally run elements that appear later in the template. This template exports the warning to call the human resources department only if the `contactHR` parameter is `true`. Use `if` statements and other control statements to control the export of static text in the template as well as template expressions.

To run your template, you use the following code:

```
var x : String = mycompany.templates.NotifyAdminTemplate.renderToString("hello", true)
```

If you want to export to a character writer, use code like the following:

```
var x : String = mycompany.templates.NotifyAdminTemplate.render(myWriter, "hello", true)
```

IMPORTANT Gosu has no supported API to generate template output from within a template scriptlet. Instead, design your template to combine template scriptlets and template expressions using the code pattern in this topic.

Support and use parameters in a template file

Procedure

1. Create a template file, for example, `mycompany.templates.NotifyAdminTemplate.gst`.

2. At the top of the template file, create a parameter definition.

For example, the following argument list includes a `String` argument and a boolean argument:

```
<%@ params(personName : String,
contactHR: boolean) %>
```

3. Later in the template, use template tags that use the values of those parameters.

For example:

```
The person ${ personName } must update their contact information in the company directory.

<% if (contactHR) { %>
    Call the human resources department immediately.
<% } %>
```

4. If Studio is running a server, you must load the changed file by remaking the project. From the menu, click **Build→Make Project**.
5. To run the template, add your parameters to the call to the `renderToString` method or after the writer parameter to the `render` method.

For example, in the Gosu Scratchpad:

```
print(mycompany.templates.NotifyAdminTemplate.renderToString("Pamela Castillo", true))
```

Maximum file-based template size

The Gosu compiler converts a Gosu template file into generated Java class files. The Java compiler has a maximum size of 65535 bytes for any class method. Sufficiently large Gosu templates can result in templates that fail at run time due to this JVM limitation.

If you have very large templates, break them into nested templates. For example, suppose you have a large template that generates three separate sections of a large page. Create three additional templates that each generate one part of the content. The original template could contain code that calls the other three templates. This design practice prevents breaching the size limit. Additionally, this style produces more readable and more manageable code.

Extending a template from a class

Gosu provides a special syntax to simplify calling static methods on a class. The metaphor for this template shortcut is that your template can extend from a type that you define. Technically, templates are not instantiated as objects. However, your template can call static methods on the specified class without fully qualifying the class. Static methods are methods defined directly on a class, rather than on instances of the class.

To use this feature, at the top of the template file, add a line with the following syntax:

```
<%@ extends CLASSNAME %>
```

CLASSNAME must be a fully qualified class name. You cannot use a package name or hierarchy.

You can use the `extends` syntax in template files, but not in `String` literals that use template syntax.

Example

Suppose your template needs to clean up the email address with the `sanitizeEmailAddress` static method on the class `gw.api.email.EmailTemplate`. The following template takes one argument that is an email address:

```
<%@ params(address : String) %>
<%@ extends gw.api.email.mailTemplate %>
Hello! The email address is ${sanitizeEmailAddress(address)}
```

Notice that the class name does not appear immediately before the call to the static method.

See also

- “Modifiers” on page 157

Template comments

You can add comments within your template. Template comments do not affect template output.

The syntax of a template comments is the following:

```
<%-- your comment here --%>
```

You can use template comments in template files, but not in `String` literals that use template syntax.

For example:

```
My name is <%-- this is a comment --%>John.
```

Rendering this template file produces:

```
My name is John.
```

Template export formats

Because HTML and XML are text-based formats, there is no fundamental difference between designing a template for HTML or XML export compared to a plain text file. The only difference is that the text file must conform to HTML and XML specifications.

HTML results must be a well-formed HTML, ensuring that all properties contain no characters that might invalidate the HTML specification, such as unescaped “<” or “&” characters. This is particularly relevant for especially user-entered text such as descriptions and notes.

Systems that process XML typically are very strict about syntax and well-formedness. Be careful not to generate text that invalidates the XML or confuses the recipient of the XML. For instance, beware of unescaped “<” or “&” characters in a notes field. If possible, export data within an XML `<CDATA>` tag, which allows more types of characters and character strings without problems of unescaped characters.

Bundles and database transactions

Gosu provides APIs to change how changes to Guidewire entity data save to the database. For many programming tasks in Gosu, such as typical rule set code, you may not need to know how database transactions work. For some situations, however, you must understand database transactions, for example:

- Adding entities to the current database transaction
- Moving entities from one database transaction to another
- Explicitly saving data to the database (committing a bundle)
- Undo the current database transaction by throwing Gosu exceptions.

WARNING Only commit entity changes at appropriate times or you could cause serious data integrity issues. In many cases, such as typical Rule sets and PCF code, it is best to rely on default behavior and not explicitly commit entity data manually.

When to use database transaction APIs

PolicyCenter groups all related entity instance changes together in a group called a *bundle*. Gosu provides APIs to manipulate entity data in bundles. These APIs also provide the ability to change how to save business data object modifications to the database. Saving to the database all related changes to current objects is called *committing* the bundle.

For many programming tasks in Gosu, you do not need to know details of how database transactions work. For example, after rule set execution, the application commits the bundle.

If errors prevent the entire commit action from safely completing, Gosu commits no data changes to the database. In certain cases, after an error occurs, Gosu retains the bundle. Depending on application context, the bundle is preserved, for example to let a user fix validation errors. In contrast, in a web service implementation, a commit failure typically results in discarding the temporary bundle and returning an error to the web service client.

The most important types of database transaction APIs provide the functionality to:

- Add an entity instance to a bundle
- Run code in a new bundle
- Commit a bundle explicitly, if necessary

WARNING Using database transaction APIs has significant effects on application logic and data integrity. Using APIs incorrectly can adversely affect other application logic that tracks when to commit changes or undo recent data changes.

The following table lists typical requirements for using the main database transaction APIs.

Code context	Create a new bundle with <code>runWithNewBundle</code> ?	Add entity instances to bundles before changing data?	Commit bundle explicitly for data changes?	Notes
Code that only reads properties	No	No	No	If you do not change entity data, you do not need to use any database transaction APIs.
Rule sets	No	Yes, for database query results. See the Note column.	No	IMPORTANT It is dangerous and unsupported for rule set code to explicitly commit any bundle. Some rule sets, such as validation rules or pre-setup rules, have a main object that is already in the current writable bundle. If you change only that main entity and its subobjects, your Gosu rule set code does not typically require special database transactions.
PCF code in: - List views - View screens - Edit screens	No	Yes, for database query results.	No	IMPORTANT In most PCF code, it is dangerous to create additional bundles or explicitly commit bundles. Typical PCF widgets handle database transactions automatically. For example, after you enter Edit mode, the application creates a new bundle. After you click the Save button, the application commits the bundle with any data updates. In general, use the default PCF behaviors. In wizard user interface flow, you can optionally configure steps to commit the latest changes. Even in those cases your Gosu code does not explicitly commit the bundle.
PCF pages with asynchronous actions	Yes	Yes, for database query results.	No. The <code>runWithNewBundle</code> API method commits the bundle.	In unusual cases in PCF files, you might need more advanced bundle management. For example, rare self-contained code must change the database independent of success of the primary user action. Use the <code>runWithNewBundle</code> API method to create a new bundle for your entity data changes.
Plugin implementation code	No, in most cases. Plugin method arguments that are intended to be modified are in a writable current bundle. Startable plugins require creating a new bundle. Message reply plugins use a different mechanism for using bundles.	Yes, for database query results. Plugin method arguments that are intended to be modified are in a current bundle that is writable.	No, in most cases. For startable plugins, the <code>runWithNewBundle</code> API method commits the bundle.	If the application expects changes to plugin method arguments, the application sets up a current bundle.

Code context	Create a new bundle with <code>runWithNewBundle</code> ?	Add entity instances to bundles before changing data?	Commit bundle explicitly for data changes?	Notes
Workflow code	No.	Yes, for database query results	No	The bundle for workflow actions is the bundle that the application uses to load the Workflow entity instance.
Custom batch processes	Yes	Yes, for database query results	No. The <code>runWithNewBundle</code> API method commits the bundle.	Use the <code>runWithNewBundle</code> API method to create a new bundle for your entity data changes.
Custom work queues	Yes	Yes, for database query results	No. The <code>runWithNewBundle</code> API method commits the bundle.	Use the <code>runWithNewBundle</code> API method to create a new bundle for your entity data changes. WARNING Custom work item code has access to a current bundle. However, using this bundle for your own business data changes is unsupported.
Web services	Yes, if the operation modifies data.	Yes, for database query results	No. The <code>runWithNewBundle</code> API method commits the bundle.	Use the <code>runWithNewBundle</code> API method to create a new bundle for your entity data changes.

See also

- “Adding entity instances to bundles” on page 336
- “Committing a bundle explicitly in very rare cases” on page 339
- “Running code in an entirely new bundle” on page 344

Overview of bundles

To manage database transactions, Guidewire applications group entity instances in groups called *bundles*. A bundle is a collection of in-memory entity instances that represent rows in the database. The application transmits and saves all entity instances in the bundle to the database in one transaction. A bundle includes changed entities, new entities, and entities to delete from the database. Gosu represents a bundle with the class `gw.transaction.Bundle`.

Guidewire refers to the process of sending the entities to the database as *committing the bundle* to the database. If a bundle commit attempt completely succeeds, all database changes happen in a single database transaction. If the commit attempt fails in any way, the entire update fails and Gosu throws an exception.

A bundle is not thread-safe. Be aware of any concurrency issues when accessing entity instances in a bundle or committing those instances to the database. If multiple users can access the entity instances in a bundle simultaneously, you must use external synchronization to ensure reading and writing correct data, such as for property values.

WARNING Use external synchronization to ensure data integrity for concurrent access to entity instances.

The two basic types of bundles are read-only bundles and writable bundles. A database query places the results of the query in a temporary read-only bundle. To change any data, you must copy the contents of a read-only bundle to a writable bundle.

Not all writable bundles eventually commit to the database. For example:

- A user might start to make data changes in the user interface but abandon the task.
- A user might start to make data changes in the user interface, try to save them, but errors prevent completion. Eventually, the user might cancel the action before fixing and completing the action.
- A batch process might attempt a database change, but errors prevent completing the action.
- A web service call might attempt a database change, but errors prevent completing the action.

If any code destroys a bundle that has uncommitted changes, no entity data in the database changes.

WARNING The base configuration contains visible usages of the entity instance method `setFieldValue`. The `setFieldValue` method is reserved for Guidewire internal use. Calling this method can cause serious data corruption or errors in application logic that may not be immediately apparent. You must obtain explicit prior approval from Guidewire Support to call `setFieldValue` as part of a specific approved use.

See also

- “Making an entity instance writable by adding to a bundle” on page 337
- “Adding entity instances to bundles” on page 336
- “Getting the bundle of an existing entity instance” on page 338
- “Getting an entity instance from a public ID or a key” on page 338
- “Creating new entity instances in specific bundles” on page 339
- “Committing a bundle explicitly in very rare cases” on page 339
- “Determining what data changed in a bundle” on page 341
- “Concurrency and thread safety” on page 365

Accessing the current bundle

In most programming contexts, the application has already prepared a current bundle, for example, in all rule execution contexts, and typical PCF user interface code. When there is a current bundle, get the current bundle using the code:

```
gw.transaction.Transaction.getCurrent()
```

In batch processes and WS-I web service implementations there is no current bundle. You need to create a new bundle for your data changes.

Warning about transaction class confusion

PolicyCenter provides more than one `Transaction` type in Gosu. This topic is about the class in the `gw.transaction` package, `gw.transaction.Transaction`. Do not confuse the `gw.transaction.Transaction` class with the `Transaction` entity type or type list.

If you use the `gw.transaction.Transaction` API, you can use a Gosu `uses` statement such as the following:

```
uses gw.transaction.Transaction
```

Adding entity instances to bundles

A set of entity instances that a query retrieve from the database is not writable. You must add an entity instance to a writable bundle to make the instance writable.

Making an entity instance writable by adding to a bundle

It is common to query the database for specific entity instances and then make changes to the data. The direct results of a database query are in a temporary read-only bundle. You must copy objects to a writable bundle to change objects or delete objects.

To add an entity instance to a bundle, pass the object reference to the `bundle.add(obj)` method and save the return result. It is extremely important to save the return value of this method. The return result of the `add` method is a copy of the object that exists inside the new bundle. Only use that return result, not what you passed to the `add` method. Never modify the original entity reference. Avoid keeping any references to the original entity instance.

The recommended pattern is to set a variable with the original entity reference, and then set its value to the result of the `add` method. For example:

```
obj = bundle.add(obj)
```

IMPORTANT You must save the return result of the `add` method and use the result for any changes. Carefully avoid keeping any references to the original entity instance.

The following example gets the current bundle and moves an entity instance to it

```
uses gw.transaction.Transaction

var bundle = Transaction.getCurrent() // Get the current bundle

obj = bundle.add(obj)                // Move the object to the new bundle and save the result
```

You can choose to modify the object:

```
obj.Prop1 = "NewValue"
```

You can choose to delete the object from the database. Take care to avoid dangling references to the deleted object if you perform this action:

```
bundle.delete(obj)
```

In most programming contexts, it is critical to not explicitly commit the bundle after your changes. For example, in typical Rules and PCF contexts, it is dangerous and unsupported to explicitly commit the bundle. Let PolicyCenter perform its default bundle commit behavior.

See also

- “When to use database transaction APIs” on page 333
- “Committing a bundle explicitly in very rare cases” on page 339
- “Removing entity instances from the database” on page 340
- “Running code in an entirely new bundle” on page 344
- *Integration Guide*

Moving a writable entity instance to a new writable bundle

You can add a read-only entity instance to a writable bundle. You can also add an entity instance to a writable bundle even if the original entity instance is in a writable bundle already. For this action to succeed, the original entity instance must be unmodified. The entity instance must not be newly added, changed, or deleted in its existing bundle.

If you try to add a modified entity instance to a new bundle, Gosu throws the exception `IllegalBundleTransferException`. To avoid this exception, be careful not to modify the entity in more than one bundle.

Be aware that even if the entity instance is unmodified at the time you copy it to the new bundle, problems can occur later. Only one bundle can try to commit a change to the same entity instance based on the same revision of the database row. If an entity instance was modified in more than one bundle and both bundles commit, the second commit fails with a concurrent data change exception.

The failed commit attempt might be in code other than your code, such as PCF user interface code that is editing the same object.

Gosu attempts to avoid this problem by preventing adding an already-modified entity to a new bundle. In some cases, user interface code internally refreshes an entity instance, such as when switching from view mode to edit mode. Refreshing an entity is an internal process that discards a cached version and gets the latest version from the database. This process reduces the likelihood of concurrent data change exceptions. Gosu does not provide public API to refresh entity instances.

See also

- “Making an entity instance writable by adding to a bundle” on page 337

Getting the bundle of an existing entity instance

If you have a reference to an entity instance in a bundle, you can use the `object.Bundle` property to get a reference to its bundle. Use this property if you want entity changes to happen with existing changes that you know are in a writable bundle.

For example, you can use the bundle reference and call its `add` method to include additional entity instances. When PolicyCenter commits that bundle, all changes to the database happen in one database transaction.

See also

- “Making an entity instance writable by adding to a bundle” on page 337

Getting an entity instance from a public ID or a key

In some cases, your code might not have a reference to an entity instance, but has a reference to one of the following:

The object public ID

A public ID is the `PublicID` property of an entity instance. Integration code must frequently access objects by using the public ID.

The object key

An object key represents the internal `Id` property entity instance, which is unique for that entity type. In most cases your code does not access objects by this key. The `Id` property is the underlying primary key for the object and the contents of a typical foreign key reference. In the underlying database row, this column is just a number.

In Gosu, the key object contains both the entity type and the ID value.

To load an entity by a key, use the `Bundle` method `loadBean`, which takes a `gw.pl.persistence.core.Key` object. Create a `Key` object by using a constructor that takes the entity type and the numeric ID. For example:

```
var myAddress = gw.transaction.Transaction.getCurrent().loadBean(new Key(Address, 123))
```

To load an entity by a public ID, use the query builder API. For example:

```
uses gw.api.database.Query
uses gw.api.database.Relop

var myPublicId = "abc"
var query = Query.make(Policy).compare(Policy#PublicID, Relop.Equals, myPublicId)
    .select().AtMostOneRow

print(query)
```

If you use the query builder API, the entity reference that is returned is in a read-only bundle. If you need to modify the data and save changes to the database, you must first add the entity instance to a writable bundle.

[See also](#)

- “Adding entity instances to bundles” on page 336
- *Integration Guide*

Creating new entity instances in specific bundles

If you pass arguments to the new operator, Gosu calls a specific constructor defined for that Java type. If there are multiple constructors, Gosu uses the number and types of the arguments that you pass to choose the correct constructor. For Guidewire business entities such as `Policy` and `User`, typical code passes no arguments to create the entity. If you pass no arguments, Gosu creates the entity in the current bundle. Using the current bundle is usually the best approach.

In special cases, pass a single argument with the new operator to override the bundle in which to create the new entity instance. Pass an existing entity instance to create the new entity instance in the same bundle. Pass a bundle to explicitly use for the new entity instance. The following table compares different arguments to the new operator.

Arguments to new operator	Example	Result
No arguments	<code>new Note()</code>	Constructs a new <code>Note</code> entity in the current bundle. Changes related to the current code submit to the database at the same time as any other changes in the current bundle. This approach is the typical approach for most programming contexts, such as rule set code or plugin implementation code. This approach requires that there be a current bundle. In some programming contexts, such as batch processes and WS-I web service implementations, there is no automatic current bundle.
An entity instance	<code>new Note(myPolicy)</code>	Constructs a new <code>Note</code> entity in the same bundle as a given <code>Policy</code> entity instance. Changes to the <code>Policy</code> are committed to the database at the same time as the new note, and all other changes in the bundle.
A bundle	<code>new Note(myPolicy.bundle)</code>	Same as previous table row

[See also](#)

- “Using the operator new in object expressions” on page 83
- “Running code in an entirely new bundle” on page 344

Committing a bundle explicitly in very rare cases

Committing a bundle sends all changes for entity instances in this group of entities to the database. A successful commit can perform any combination of adding, changing, or removing entity instances from the database. In typical code, do not explicitly commit a bundle.

For data changes, typically one of the following is true:

- The application has a default bundle management life cycle and commits the bundle at the appropriate time, for example, in rule set execution or typical PCF edit screens. There is no reason to explicitly commit this bundle.
- In some cases, you must create an entirely new bundle. Do not explicitly commit the bundle. The `runWithNewBundle` API commits the bundle automatically when your code completes.

WARNING Only commit a bundle if you are sure it is appropriate for that programming context. Otherwise, you could cause data integrity problems. For example, in Rule sets or PCF code, it is typically dangerous to commit a bundle explicitly. Contact Customer Support if you have questions.

For web service implementations that need to change data, use the `runWithNewBundle` API method. The `runWithNewBundle` API method commits the bundle for you when your code completes.

If you are sure that committing the bundle is appropriate, use the method `bundle.commit()`. If the attempt fails, Gosu throws an exception. The entire commit process fails. The database remains unchanged.

Get the current bundle by calling the `getCurrent` static method on the `Transaction` class:

```
uses gw.transaction.Transaction

var bundle = Transaction.getCurrent()
bundle.commit()
```

If you have an entity instance reference, use the `entity.bundle` property to get its bundle. Committing a bundle commits everything in the bundle, not just the instance from which you got the bundle reference. For example:

```
var bundle = myPolicy.bundle
bundle.commit() // Commit all entities in the bundle
```

See also

- “When to use database transaction APIs” on page 333
- “Running code in an entirely new bundle” on page 344

Removing entity instances from the database

WARNING Guidewire strongly recommends never to delete entities from the database. An entity instance can include foreign key links to other entity instances and other entity instances can link to the deleted entity instance. Removal of an entity instance can compromise data integrity. If you must delete an entity instance, using the `remove` method on the entity instance is strongly preferable to using the `Bundle` interface’s `delete` method.

Entity instance remove method

Mark an entity instance for removal from the database by using the `remove` method on the entity.

When the bundle is committed, the action that PolicyCenter takes depends on whether the entity is retireable. If the entity’s data model configuration does not specify that the entity is `Retireable`, the entity instance is deleted from the database. If the data model specifies that the entity is `Retireable`, PolicyCenter retires the entity instance rather than deleting it. Additional actions of the `remove` method depend on the entity type. Typically, `remove` deletes owned child entity instances to which arrays in that instance refer even if those entity types are `Retireable`. Some entity types perform additional clean-up actions as part of their `remove` implementation.

For example:

```
uses gw.transaction.Transaction

// Make a query of Address instances
var query = Query.make(Address)

// Query for addresses created today
query.compare(
    DBFunction.DateFromTimestamp(query.getColumnRef("CreateTime")), Equals, DateUtil.currentDate())

for (address in query.select()) {
    address = bundle.add(address)
    address.remove()
}
```

Bundle interface delete method

The `Bundle` interface's `delete` method also provides the functionality to delete an entity instance from the database. This method does not perform any additional actions to ensure database integrity. The `remove` method on some entity types does perform additional clean-up. For this reason, if you must delete an entity instance, using the `remove` method on the entity instance is strongly preferable to using the `Bundle` interface's `delete` method.

When the bundle is committed, the entity instance is deleted from the database. If the entity's data model configuration declares the entity as `Retireable`, the entity is not deleted, but is retired instead. The `delete` method removes only the specified entity instance from the database, not linked entity instances.

See also

- *Configuration Guide*

Determining what data changed in a bundle

In rule set code, you sometimes need to identify data that recently changed. You might need to compare the most recent entity data from the database with the latest version of an entity in a locally modified bundle. In the most common case, rule sets contain conditions and rule actions that must detect which if any properties just changed. For example, if a certain object property changed, you might want to validate properties, recalculate related properties, log changes, or send messages to external systems.

To detect data changes, call the object's `isFieldChanged` method. If it changed, call the entity's `getOriginalValue` method to get the original value loaded from the database. You must cast the original value to the expected type with the syntax `"as TYPENAME"`.

Depending on the type of value stored in that property, the `isFieldChanged` method behaves differently. The following table describes the behavior based on the property type.

Type of property	Behavior of <code>getOriginalValue</code>	Behavior of <code>isFieldChanged</code>
Scalar value	Returns the original simple value, such as a String value like "John Smith", a number like 234, or any other non-entity and non-array type.	Returns true if and only if the actual property value changed to a different actual value. If the value changed to a different value and then back to the original value, <code>isFieldChanged</code> would return false.
Entity	Guidewire applications represent links to entity subobjects as a foreign key called the <i>internal ID</i>. This foreign key is the <code>entity.Id</code> property on the destination entity. Note that the <code>Id</code> property is different than the <code>PublicID</code> property. If you call <code>getOriginalValue</code> with a foreign key field, you get an internal ID as a Key object. A Key object encapsulates the entity type and the unique ID value for that object. To get a reference to the entity instance with the original ID use the bundle method <code>loadBean</code>. For example: <pre>var k = e.getOriginalValue("Address1") var bundle = Transaction.getCurrent() var addy = bundle.loadBean(k) as Address</pre> Remember to call the <code>getOriginalValue</code> method on the correct entity instance.	Returns true if and only if the foreign key <code>Id</code> value for the subobject is the same in the original entity. Remember to call the <code>isFieldChanged</code> on the correct entity instance. Keep in mind that checks for a changed entity could return false even if there are changes on properties of subobjects. Be aware that the foreign key is a link to the <code>Id</code> property of the target object. The <code>PublicID</code> property of the target object is not examined.
Array	Returns an array containing the set of array elements that were originally persisted.	Returns true if any elements were added or removed to the array.

Type of property	Behavior of <code>getOriginalValue</code>	Behavior of <code>isFieldChanged</code>
	If any properties changed on those elements, the bundle contains the new values, not the original values. If you need the original properties on those array elements, you must examine each combination of element and properties by calling <code>getOriginalValue</code> on each array element for each property.	Additionally, if the array is defined in the data model configuration as an owned array, if any properties on an array element changed, <code>isFieldChanged</code> returns true. This does not apply to non-owned arrays. Note: Because entity array order is not meaningful, the order is not checked by <code>isFieldChanged</code>. In this case, <code>isFieldChanged</code> returns the same value as the array method <code>isArrayElementAddedOrRemoved</code>.

Scalar value property example

For a scalar value example, the `Address` entity contains a `String` value property called `City`. If the `address.City` value is different from the original entity, `address.isFieldChanged("City")` returns true and `address.getOriginalValue("City")` returns the original value for this property of type `String`.

Entity property example 1

For an entity property example, the `ClaimCenter` application includes a `Claim` entity with a `LossLocation` property containing an `Address` entity.

If that property points to an entirely different entity instance than the original entity loaded from the database, it is considered changed:

- The expression `claim.isFieldChanged("LossLocation")` returns true
- The expression `claim.getOriginalValue("LossLocation")` returns the ID of the original entity instance as a key. Do not assume that the original entity instance is unchanged. It may be loaded and already changed.

For example:

```
if (claim.isFieldChanged("LossLocation")) {
    // Get original entity
    var id = claim.getOriginalValue("LossLocation") as Key
    var originalAddress = claim.Bundle.loadBean(id)

    // Get original property on original entity
    var origAddLine1 = originalAddress.getOriginalValue("AddressLine1")
}
```

In contrast, if the `claim.isFieldChanged("LossLocation")` returns false, then the entity foreign key is unchanged but that does not mean necessarily that data in a subobject is unchanged. To check properties on subobjects, you must test specific properties on the subobject using the `isFieldChanged` method. For example:

```
if (not claim.isFieldChanged("LossLocation")) {
    if (claim.LossLocation.isFieldChanged("City")) {
        var origAddLine1 = claim.LossLocation.getOriginalValue("AddressLine1")
    }
}
```

Entity property example 2

The following example gets the original value for an entity property and gets a property on that original object

```
var originalPolicyID = account.getOriginalValue("Policy") as Key
var originalPolicy = account.Bundle.loadBean(originalPolicyID)
var originalPolicyNumber = originalPolicy.getOriginalValue("PolicyNumber") as String
```

See also

- “Getting an entity instance from a public ID or a key” on page 338

Detecting property changes on an entity instance

From your rule set code, you can use the entity instance read-only property `Changed` to check if the entity changed at all. This property returns a Boolean value of `true` if the entity instance has one or more properties that changed.

```
if (myAddress.Changed) {
    // Your code here...
}
```

From your rule set code, you can get the set of changed properties on an entity instance using the `ChangedFields` property. That property value has the type `java.util.Set`.

The following example uses a Gosu block that iterates across the set of changed properties:

```
if (myAddress.Changed) {
    myAddress.ChangedFields.each( \ e -> print("Address has changed property: " + e))
}
```

See also

- “What are blocks?” on page 181
- “Collections” on page 189

Getting changes to entity arrays in the current bundle

In addition to entity array properties supporting the `isFieldChanged` and `getOriginalValue` methods, you can test for changes to entity array contents in the current bundle. You use methods to compare the array on the persisted entity instance to the array on the cached version in the bundle.

The following methods are available only for properties directly defined on an entity in the data model configuration files as entity arrays. These methods do not work on other types of properties or methods. For example:

- These methods do not work with Gosu enhancement properties that return entity arrays.
- These methods do not work with other virtual properties such as those implemented in internal Java code.

Use the *Data Dictionary* to check whether a property is a database-backed property or a virtual property.

Entity methods for getting changes to array properties

The following table lists the methods on an entity instance that you can use to get entity array changes.

Entity method	Argument	Return type	Returns
<code>getAddedArrayElements</code>	Array property name	Entity array	A list of array elements that were added. If there are none, returns an empty array.
<code>getChangedArrayElements</code>	Array property name	Entity array	A list of array elements that were changed. If there are none, returns an empty array.
<code>getRemovedArrayElements</code>	Array property name	Entity array	A list of array elements that are marked for deletion if the bundle commits. If there are none, returns an empty array.
<code>isArrayElementChanged</code>	Array property name	boolean	true if and only if an element in an array changes, in other words if one or more properties change on the entity. If no existing element changes but elements add or remove, this returns false.
<code>isArrayElementAddedOrRemoved</code>	Array property name	boolean	true if and only if any array elements were added or removed.

Getting added, changed, or deleted entities in a bundle

A bundle's set of entity instances might include multiple types of entities. You can get the set of changes by using properties on the bundle object.

Bundle methods for getting added, changed, or deleted entities

The following table lists the methods that you can use to find bundle changes.

Type of entities to re- turn	Bundle API method	Description
New entity instances	<code>bundle.InsertedBeans</code>	Entity instances in this bundle that are entirely new objects, which have never been committed to the database, appear in this set. This bundle property returns an iterator that iterates across the set of entity instances.
Changed entity instances	<code>bundle.ChangedBeans</code>	Entity instances in this bundle that had one or more properties change appear in this set. This bundle property returns an iterator that iterates across the set of entity instances.
Removed entities	<code>bundle.RemovedBeans</code>	Some entities may be marked for deletion if this bundle commits. If the entity is retireable, the row remains in the database rather than deleted but the entity is marked as retired. This bundle property returns an iterator that iterates across the set of entity instances.
Unmodified entity instances in the current bundle	Not available	An entity might be added to a writable bundle but not yet modified. There is no API method to get the list of unmodified entities in the bundle.

Running code in an entirely new bundle

Typical Gosu code interacts with database transactions by using the current bundle set up for rules execution, plugin code, or PCF user interface code. In such cases, PolicyCenter automatically manage the low level parts of database transactions. Only in rare cases your code must create or commit a bundle explicitly, such as batch processes, WS-I web service implementation classes, and actions triggered from workflows.

In rare cases, you need to run Gosu code in an entirely new transaction, so you create a new bundle. You might need a new transaction because there is no current bundle or because you need changes in a different transaction from the current transaction.

Create your own bundle only in the following cases:

- If your code modifies entities to commit independent of the success of the surrounding code. For example, some workflow asynchronous actions or unusual PCF user interface code might need to create a new bundle.
- If your code represents a separate asynchronous actions from surrounding code.
- If your Gosu calls Java code that spawns a new thread within the server, such as a timer or scheduled event. It is important that no two threads share a bundle.

To run code within a separate transaction, you must create a Gosu block, which is an in-line function that you define in-line within another function. You can pass a block to other functions to invoke as appropriate.

To create a new bundle, call the static method `runWithNewBundle` on the `gw.transaction.Transaction` class. As a method argument, pass a Gosu block that takes one argument, which is a bundle (`Bundle`). Gosu creates an entirely new bundle just for your code within the block.

The syntax is:

```
uses gw.transaction.Transaction
...
Transaction.runWithNewBundle( \ bundle -> YOUR_BLOCK_CODE_HERE )
```


Your Gosu code within the block can add entity instances to the bundle by using the `bundle.add(entity)` method. Remember to save the return result of the add method.

Gosu immediately runs this code synchronously. If the block succeeds with no exceptions, `runWithNewBundle` commits the new bundle after your code completes. For the most concise and readable code, Guidewire encourages you to use this automatic commit behavior rather than committing the bundle explicitly.

If your code detects error conditions, throw an exception from the Gosu block. Gosu discards the bundle and does not commit the changes.

The following example demonstrates how to create a new bundle to run code that creates a new entity instance and modifies some fields. The current transaction is an argument to the block.

If you are adding entity instances, in typical code you do not need to use the bundle explicitly. Gosu sets the current bundle inside the block to this new block automatically. You can use the no-argument version of the new operator for entity types, which creates the object in the current bundle:

```
gw.transaction.Transaction.runWithNewBundle( \b -> {
    var me = new MyEntity()

    me.FirstName = firstName
    me.LastName = lastName
    me.PrimaryAddress = address
    me.EmailAddress1 = email
    me.WorkPhone = workPhone
    me.PrimaryPhone = primaryPhoneType
    me.TaxID = taxId
    me.FaxPhone = faxPhone
} )
```

If you perform database queries, you can use the bundle reference to add entity instances to the new writable bundle.

WARNING Only commit a bundle if you are sure that it is appropriate for that programming context. Otherwise, you could cause data integrity problems. For example, in Rule sets or PCF code, it is typically dangerous to commit a bundle explicitly, including with this API. Contact Customer Support if you have questions.

Using variable capturing in your block

The following example method from a web service implementation demonstrates how to create a new bundle to call a utility function that does the core part of the work.

```
...

public function myAction(customerID : String) : void {
    Transaction.runWithNewBundle( \ bundle -> MyClass.doSomething(customerID, bundle))
}

...
```

This apparently simple block is not self-contained because the code uses `customerID`, which is the argument to the `myAction` method and is declared outside the scope of the block. The ability of a block to use variables from its creation context is called *variable capturing*.

In some cases, you need to return information from your block. You can return information by more advanced use of variable capturing. The following example demonstrates variable capturing by creating a local variable in the outer scope (`myAction`) and then setting its value from within the block. The outer scope can use that value as a return value from the method.

```
public function myAction(customerID : String) : boolean {
    var res : boolean

    // The local variable "res" and the parameter "customerID" are captured by the block.
    // Both variables are shared between the calling function and the block
    Transaction.runWithNewBundle( \ bundle -> { res = MyClass.doSomething(customerID, bundle) })

    // Return the value that was set in the block using the shared (captured) variable
    return res
}
```

See also

- “Blocks” on page 181
- “Variable scope and capturing variables in blocks” on page 184
- “Adding entity instances to bundles” on page 336
- “Getting changes to entity arrays in the current bundle” on page 343

Creating a bundle for a specific PolicyCenter user

The `gw.transaction.Transaction` class provides a signature of the `runWithNewBundle` method to associate a specific user with the newly created bundle. You use this method signature in contexts in which there is no built-in user context, such in the startable plugins feature. The method signature is:

```
gw.transaction.Transaction.runWithNewBundle(\ bundle -> YOUR_BLOCK_BODY, user)
```

For the second argument to the method, you can pass either a `User` entity instance or the user name as a `String`. Guidewire strongly recommends that you do not use the Guidewire sys user (System User), or any other default user, to create a new bundle.

Effects of exception handling on database commits

Typically, if your code throws an exception, Gosu does not commit the current database transaction. This behavior applies to rule sets, PCF code, and other contexts that have a current bundle.

Throw an exception from your code to get this behavior. If you catch exceptions in your code, to ensure that the current bundle is not committed in the current action, rethrow the exception.

Bundles and published web services

Web service implementation classes must explicitly manage all bundles, for both reading and writing entity data. Because web services do not support entity instances as arguments or return types, the interaction between web service implementation classes and bundles is straightforward.

Entity cache versioning, locking, and entity refreshing

The application caches entity data for faster access. The `gw.transaction.Transaction` API methods that refer to original entity instances check against the database version as of the time the bundle loaded that entity. The time of loading might not have been immediately before your code checks the original entity data. The application may have loaded the database row long before the recent application change. Although the database data may have changed since the time of loading, some safeguards prevent concurrent data access in most cases. For typical entity access, the server prevents committing an entity instance if the instance changed in the database after the entity loaded in the bundle and before the bundle commits.

Entity instance versioning and the entity touch API

PolicyCenter protects entity instances from concurrent access through a version property that exists on all versionable entities. Almost all entity types in the system are versionable. If you load a versionable entity instance into a bundle, the application loads the version number and stores it with the data.

If a bundle commits, PolicyCenter checks this version number property in each entity instance with the latest version in the database. PolicyCenter confirms that the cached entity instance is up to date. If the version numbers match, the commit can proceed.

If the version numbers do not match, the entire commit attempt fails for the entire bundle. In other words, if the recent change for an object relies on an out-of-date database row, it is unsafe to commit this recent change. Gosu throws an exception during the database commit.

During the final commit, the version number is increased by one from its previous value. In typical code, you need to add entity instances to a bundle and then modify them. PolicyCenter automatically increments the version number.

In rare cases, it may be desirable to force the version number of an entity to increment even if there is no known change to the entity yet.

For example, suppose your code reads values from four entity instances (A, B, C, and D) that have a tight conceptual relationship. Your code reads the values and changes most or all of the objects in one database transaction. In one case, the code makes changes to only B, C, and D. There remains one object (A) for which you did not change properties. Therefore, A is not in the same bundle as B, C, and D. In this case, neither does the database row for A change nor does its version number increment, which will not be a problem in some cases.

However, suppose the three objects you change are related to the current properties on A. The default behavior may be undesirable compared to updating A, B, C, and D together:

- You might want to protect against other threads on the current server, including the user interface, from making concurrent changes on A that make the other changes make no sense.
- You might want to protect against other servers in the PolicyCenter cluster from making concurrent changes on A that make the other changes make no sense.
- You might want the last modified time of A to match the last modified time of B, C, and D.
- You might want to force preupdate rules to run for object A.

To force PolicyCenter to increment the entity version number (`obj.BeanVersion`), update the modified time, and force preupdate rules to run, call the `touch` method on the entity instance. The method takes no arguments:

```
obj.touch()
```

Record locking for concurrent data access

In addition to version protections, the system locks the database briefly during a commit. The server throws concurrent data change exceptions if two different threads or two different servers in a cluster simultaneously modify the same entity at the same time.

Concurrent data change exceptions always occur if two application users attempt to change the same record from the application user interface.

If all of the following conditions are true, a concurrent data change exception does not occur:

- A batch process or messaging plugin made the update that conflicts with a change from the user interface.
- The user that made the update is a system user. In the base configuration, in the User record of a system user, the `SystemUserType` property is `sysadmin` or `syservices`.
- The `BeanVersion` field in the database record differs only by 1. In this case, only the batch process or messaging plugin has updated the record.
- The changes that the concurrent change makes are not in fields that the batch process or messaging plugin updated.

User interface bundle refreshes

In some cases, the PolicyCenter user interface internally refreshes bundle entity data with the latest version from the database. For example, PolicyCenter refreshes the data when changing from view-only to edit mode.

Gosu does not provide a public API for you to programmatically refresh a bundle's entity data.

Details of bundle commit steps

PolicyCenter commits a bundle to save data to the database by performing the following steps.

Note: Bundle commit does not perform the following preupdate rule sets and validation rule sets: archiving, geocoding changes, messaging reply, messaging preprocessor, upgrade actions, and XML import, including administrative data import. For this reason, there are no messaging events nor concurrent data change exceptions for these actions. These actions always overwrite an entity instance in the database with the instance in the bundle.

1. PolicyCenter reserves a connection from the connection pool.
2. Internally, PolicyCenter saves the state of the bundle. PolicyCenter uses this snapshot to revert to the current state of data if writing to the database fails.
3. PolicyCenter runs preupdate rule sets.
4. PolicyCenter refreshes all entity instances that already existed but are unchanged in the bundle. This action ensures that validation rules, which run next, get the newest versions of those entity instances.
5. PolicyCenter triggers validation rule sets.
6. PolicyCenter sets properties that exist on editable and versionable entities: `updateTime`, `createTime`, and `user`.
7. PolicyCenter increments the version number on each modified entity instance.
8. Any entity instances that are new (not yet in the database) have a temporary internal ID. An internal ID is the `entity.Id` property. PolicyCenter creates the permanent internal ID for each new entity instance and assigns the value to the `entity.Id` property. PolicyCenter sets any foreign key references to refer to the new internal ID value rather than the temporary ID.
9. PolicyCenter computes the set of new, changed, and removed entity instances in the bundle.
10. PolicyCenter determines the set of messaging events that are raised by changes in the bundle.
11. For each messaging destination that is registered to listen for messaging events, PolicyCenter triggers Event Fired rule sets once for each messaging destination. For example, `ENTITYNAMEAdded`, `ENTITYNAMEChanged`, or `ENTITYNAMERemoved` events. If more than one destination listens for the same event, the Event Fired rule set executes once for each messaging destination.
12. PolicyCenter writes all changed entities to the database connection. During this step, PolicyCenter checks for concurrent data change exceptions.
13. PolicyCenter attempts to commit the database connection. This action either completely succeeds or fails.
14. On success, PolicyCenter updates the global entity cache. The global entity cache speeds up entity data access by caching recently used data in memory on each server in a PolicyCenter cluster.

After updating the cache, the current server signals other servers in the cluster about the change. This signal tells the other servers to remove data from the cache for entity instances that just updated or deleted. After a server receives this signal, entity instances in a remote entity cache immediately show the data as requiring an update, but do not immediately reload data from the database. Any local existing references to the old data on the other servers are out of date and will never commit to the database. The entity data is safe from accidental changes of this type.

See also

- “Entity instance versioning and the entity touch API” on page 346
- *Integration Guide*
- *System Administration Guide*

Type system

Gosu provides several ways to gather information about an object or other type. Use this information to debug or to determine program behavior at run time.

You can find the runtime type of an object. You can also test whether the type of an object matches a particular type or one of that type's subtypes.

The `Type` property of a type provides access to the metadata of that type.

You can also use reflection to gather information about an object and run methods on that object.

Basic type checking

Every Gosu object and value has a type. For example, the type of an instance of a class is the class itself.

Gosu provides the following operators to retrieve or test an item's type.

- The `typeof` operator retrieves the run-time type of an item.
- The `typeis` operator tests the type of an item.

Retrieving the type by using `typeof`

Use the `typeof` operator to retrieve the run-time type of a data item.

The syntax of the `typeof` expression is shown below.

```
typeof expression
```

The example below demonstrates the syntax. The sample `typeof` expression evaluates to `true`.

```
var myVar : Integer = 10
if (typeof myVar == Integer) print("true") // typeof myVar == Integer
```

Comparing the retrieved type to a subtype of the retrieved type evaluates to `false`. For example, the `Integer` type is a subclass of the `Number` class, which itself is a subclass of the `Object` class. All of the `typeof` expressions shown below evaluate to `false`.

```
if (typeof myVar == Number) print("false") // typeof myVar != Number
if (typeof myVar == Object) print("false") // typeof myVar != Object
```

The type of an entity or typekey is a parameterized type of `Type<t>`.

The following example shows some `Type<t>` types of a `Producer` entity and a `NoteSecurityType` typekey. The `typeof` expressions shown below evaluate to `true`.

```
if (typeof Producer == Type<Producer>) print("true")
if (typeof Producer == Type<entity.Producer>) print("true")
```

```
if (typeof NoteSecurityType == Type<NoteSecurityType>) print("true")
if (typeof NoteSecurityType == Type<typekey.NoteSecurityType>) print("true")
```

The `Type<t>` syntax must be used in comparison expressions of entities or typekeys that use the `typeof` operator. The expressions shown below evaluate to `false` because they do not use the required `Type<t>` syntax.

```
if (typeof Producer == Producer) print("false")
if (typeof Producer == entity.Producer) print("false")
if (typeof NoteSecurityType == NoteSecurityType) print("false")
if (typeof NoteSecurityType == typekey.NoteSecurityType) print("false")
```

The example below demonstrates how the run-time type retrieved by the `typeof` operator can differ from the type specified when the variable was defined.

```
var myVar : Object = 10
print(typeof myVar)           // Run-time type is java.lang.Integer
```

The run-time type of a null value is the void type as demonstrated below.

```
var myVar : Boolean = null
print(typeof myVar)           // Run-time type of the null value is void
```

Testing the type by using typeis

A `typeis` expression tests the type of an object. If the object is of either the tested type or a subtype of the tested type, the expression evaluates to `true`.

The syntax of a `typeis` expression is shown below.

```
OBJECT typeis TYPE
```

The definition of a `myVar` variable of type `Integer` is shown below. The `Integer` type is a subclass of the `Number` class, which itself is a subclass of the `Object` class. Thus, all the `typeis` expressions shown below evaluate to `true`.

```
var myVar : Integer = 10

if (myVar typeis Integer) { print("true") } // true because myVar is of type Integer
if (myVar typeis Number) { print("true") }  // true because Integer is a subtype of Number
if (myVar typeis Object) { print("true") }  // true because Integer is a subtype of Object
```

The type of an entity or typekey is a parameterized type of `Type<T>`.

The following example shows some `Type<T>` types of a `Producer` entity. The `Producer` class is a subclass of the `BeanBase` class. All of the `typeis` expressions shown below evaluate to `true`.

```
if (Producer typeis Type<Producer>) { print("true") }
if (Producer typeis Type<entity.Producer>) { print("true") }
if (Producer typeis Type<BeanBase>) { print("true") } // true because Producer is a subtype of BeanBase
```

The following example shows some `Type<T>` types of a `NoteSecurityType` typekey. The `NoteSecurityType` typekey is a subclass of the `Object` class. All of the `typeis` expressions shown below evaluate to `true`.

```
if (NoteSecurityType typeis Type<NoteSecurityType>) { print("true") }
if (NoteSecurityType typeis Type<typekey.NoteSecurityType>) { print("true") }
if (NoteSecurityType typeis Object) { print("true") }
```

Some methods, parameters, and return values use the interface type `IType`, which is analogous to the parameterized `Type<T>`.

If the compiler determines that a particular `typeis` expression can never evaluate to `true` at run time, a compile error is generated. This behavior is demonstrated in the following example that generates a compile error because the `Boolean` variable can never hold an `Integer` value.

```
var myVar : Boolean

if (myVar typeis Integer) { print("This message never prints") } // Compile error because Boolean can never be Integer
```

Basic type coercion

Gosu provides coercion or casting syntax to cast an expression to a specific type. The syntax is shown below.

```
Coercions.makeTYPEFrom(expression)
```

The expression must be compatible with the type. The following table shows the results of casting a simple numeric expression into one of the Gosu-supported data types.

Expression	Data type	Result or error
Coercions.makeBooleanFrom(5 * 4)	Boolean	true
Coercions.makeStringFrom(5 * 4)	String	"20"
Coercions.makeArrayFrom(5 * 4)	Not applicable	Does not compile. Method does not exist.

The `as` keyword can be used to cast an array type to another compatible array type. For example, `String` is a subtype of `Object`, so an array of type `String` can be cast to an array of type `Object`.

```
var objArray : Object[]
objArray = new String[] { "a", "b" } as Object[] // Can cast String[] to Object[]
```

The reverse operation of casting an `Object` to a `String` is invalid, as demonstrated below.

```
var objArray2 = new Object[1]
objArray2[0] = "aString"
var strArray = objArray2 as String[] // Run-time error attempting to cast Object[] to String[]
```

The compiler can detect and produce a compilation error when two referenced types are incompatible. For example, an `Integer[]` can never be converted to a `String[]`, so the following code statements produce a compilation error.

```
var strArray2 : String[]
strArray2 = new Integer[] { 2, 3 } as String[] // Compilation error attempting to cast Integer[] to String[]
```

In some cases, you know that all the objects in an array are of a particular subtype of the defined type of the array. In other cases, you filter an array to retrieve only objects of a particular subtype. As shown in a preceding code example, you cannot use the `as` keyword to convert an array to an array of a subtype. Instead, you can use the Gosu enhancement method `cast` to make an array of the subtype. You can then perform operations on the new array that are specific to the subtype and not available to the parent type. If you filter an array by element type and do not use the `cast` method, the result retains the type of the original array.

The following code demonstrates how to use the `cast` method.

```
var objArray3 = new Object[1]
objArray3[0] = "aString"
var strArray3 = objArray3.cast(String) // String array

var objArray4 = new Object[2]
objArray4[0] = "anotherString"
objArray4[1] = 42
var strArray4 = objArray4.where{\aid -> aid typeis String}.cast(String) // String array
var objArray5 = objArray4.where{\aid -> aid typeis String} // Object array
```

You cannot use the `cast` method to convert incompatible types. The compiler does not detect this type of error. The following code produces a run-time error.

```
var strArray5 : String[]
strArray5 = new Integer[] { 2, 3 }.cast(String) // Run-time error attempting to cast Integer[] to String[]
```

Automatic type downcasting

To improve the readability of Gosu code, Gosu automatically downcasts a type after a `typeis` or `typeof` expression if the type is a subtype of the specified type. Automatic type downcasting is particularly valuable for `if` and `switch` code blocks. Gosu confirms that the typed object has the more specific subtype within the code block. Accordingly,

Gosu implicitly downcasts the variable's type to the relevant subtype for the block. There is no need to explicitly cast the type with an *as TYPE* statement.

A common pattern for this feature is shown below.

```
var VARIABLE_NAME : TYPE_NAME

// This code requires SUBTYPE_NAME to be a subtype of TYPE_NAME
if (VARIABLE_NAME typeis SUBTYPE_NAME) {
    // Use the VARIABLE_NAME variable as a SUBTYPE_NAME without explicit casting
    ...
}
```

For example, the following code shows a variable declared as an *Object*, but automatically downcast to *String* when referenced within the *if* code block.

```
var x : Object = "nice"
var strlen = 0

if ( x typeis String ) {
    strlen = x.length      // x automatically downcast to a String which has a length property
}
```

Note that *length* is a property on *String*, not *Object*. The automatic downcasting of the variable eliminates the need to explicitly cast it. The following code is equivalent to the previous code, but has an unnecessary explicit cast.

```
var x : Object = "nice"
var strlen = 0

if ( x typeis String ) {
    strlen = (x as String).length    // Do not need to explicitly cast x as a String
}
```

Best practice recommends utilizing automatic downcasting when it occurs and eliminating unnecessary explicit casting.

When automatic downcasting occurs

Automatic downcasting occurs when executing the following types of statements.

- *if* statements
- *switch* statements

```
uses java.util.Date

var x : Object = "neat"
switch ( typeof x ){
    case String :
        print( x.charAt( 0 ) )    // x is automatically downcast to type String
        break
    case Date :
        print( x.Time )           // x is automatically downcast to type Date
        break
}
```

- The true portion of ternary conditional expressions

```
x typeis String ? x.length : 0    // x is automatically downcast to type String
```

The automatic downcast occurs only for the true portion of the ternary expression. In the false portion, the variable reverts to its original type.

The automatic downcast is terminated and the object is interpreted as being of its original type when any of the following conditions occurs.

- Reaching the end of the code block's scope
 - The end of an `if` code block
 - The end of a case code block within a `switch` statement
 - The end of the `true` portion of a ternary conditional expression
- Assigning any value to the variable that you checked with `typeis` or `typeof`. This behavior applies only to `if` and `switch` statements.
- Assigning any value to any part of an entity path that you checked with `typeis` or `typeof`. This behavior applies only to `if` and `switch` statements.
- An `or` keyword in a logical expression
- The end of an expression negated with the `not` keyword
- In a `switch` statement, a case section does not use automatic downcasting if the previous case section is not terminated by a `break` statement. The following `switch` statement demonstrates this rare behavior.

```
uses java.util.Date

var x : Object = "neat"
switch( typeof x ){
  case String :
    print( x.charAt( 0 ) ) // x is automatically downcast to type String
  case Date :             // Code execution falls through to next case
    print( x.Time )       // x reverts to original type of Object which does not have Time property
    break
}
```

When the `x` object of type `Object` attempts to access a non-existent `Time` property, a compile error occurs. To work around this behavior, explicitly cast the `x` object to a type `Date` as shown below.

```
print( (x as Date).Time ) // Explicitly cast x to type Date to access the Time property
```

Type metadata properties and methods

The type `Type` is the root type of all types. Every `Type` includes metadata properties and methods, such as the `DisplayName` property and the `isAssignableFrom` method. You use the `Type` property of a type to access the metadata. Although the metadata properties are available directly on the type, best practice is to use the `Type` property because static properties on a type override `Type` properties of the same name.

General type metadata

General type metadata is available for all types of data. Some of the properties and methods available in general type metadata are described in the following list. For a complete list of properties and methods, refer to the PolicyCenter Javadoc.

Abstract

If the type is abstract, this property is `true`.

Array

If the type is an array, this property is `true`.

DisplayName

The human-readable name of the type.

Final

If the type is final, this property is `true`.

GenericType

If the type is a parameterized type, this property is the raw generic type of the type.

If the type is a raw generic type, this property is this type.

If the type is neither a parameterized type nor a raw generic type, this property is `null`.

For example, for a type of `ArrayList<String>`, the `GenericType` property is `java.util.ArrayList`.

Interface

If the type is an interface, this property is true.

isAssignableFrom

If the type is assignable from a variable of a specified type, this method returns true.

Boxed types, such as `Boolean` and `Integer`, can be assigned to and from other boxed types only. If either variable type referenced in the `isAssignableFrom` expression is a primitive, such as `boolean` or `int`, the method returns false.

isGenericType

If the type is a raw generic type, this method returns true.

For example, for the type `ArrayList<String>`, this method returns false. For the type `java.util.ArrayList`, this method returns true.

ParameterizedType

If the type is parameterized, this property is true.

For example, for the type `ArrayList<String>`, this property is true. For the raw generic type of `java.util.ArrayList`, this property is false.

Primitive

If the type is primitive, this property is true.

Supertype

The supertype of the type. If no supertype exists, this property is null.

TypeInfo

Contains metadata about properties and methods of the type.

Type metadata for entities and typekeys

Gosu typekeys and some Gosu entities have additional type metadata that is not included with the general metadata. For example, Gosu typekeys have a metadata method called `getTypeKeysByCategories` that is available only on typekey data types. To access the entity and typekey metadata, reference the data's `TYPE` property and get method. The code sample below demonstrates the appropriate syntax.

```
var myTypekeys = new TypeKey[10]
NoteSecurityType.TYPE.get().getTypeKeysByCategories(myTypekeys)
```

The `TYPE` property is available only on Gosu entity and typekey data types.

Accessing general type metadata

To access general type metadata, retrieve the `Type` property of the data by using the `typeof` operator. The following example accesses various type metadata values on the `java.util.ArrayList` type.

```
var stringList = {"cat", "dog"}
var t = typeof stringList

print("Name = " + t.DisplayName)
print("Supertype = " + t.Supertype.DisplayName)
print("isAssignableFrom(Object) = " + t.isAssignableFrom(Object))
print("Example method name = " + t.TypeInfo.Methods[0].DisplayName)
```

Alternatively, the `Type` property of a data type can be referenced directly to access the type's metadata. The following example demonstrates the syntax to access the metadata of the `ArrayList` class.

```
var alt = ArrayList.Type
print("Name = " + alt.DisplayName)
print("Supertype = " + alt.Supertype.DisplayName)
print("isAssignableFrom(Object) = " + alt.isAssignableFrom(Object))
print("Example method name = " + alt.TypeInfo.Methods[0].DisplayName)
```

The output from the example code is identical for both of these techniques.

```
Name = java.util.ArrayList<java.lang.Object>
Supertype = java.util.AbstractList<java.lang.Object>
isAssignableFrom(Object) = false
Example method name = clone
```

Using feature literals

Gosu feature literals provide a way to statically refer to a type's features such as methods and properties. You can use feature literals to pass class property and method members to API methods. Gosu validates feature literals at compile time, preventing common errors that might occur if you pass members by using their names as `String` values. To refer to the feature of a type or object, you use the `#` operator after the type or object, followed by the feature name. The syntax is similar to the feature syntax of the Javadoc `@link` syntax.

You can use feature literals in API methods, such as in the following types of layers.

- Mapping layers, where you are mapping between properties of two types
- Data-binding layers
- Type-safe object paths for a query layer

Consider the following Gosu class.

```
class Employee {
    var _boss : Employee as Boss
    var _name : String as Name
    var _age : int as Age

    function update(name : String, age : int) {
        _name = name
        _age = age
    }

    function greeting() {
        print("Hello world")
    }
}
```

Given this class, you can refer to its features by using the `#` operator.

```
// Get property feature literal
var nameProp = Employee#Name

// Get method feature literal
var updateFunc = Employee#update(String, int)
```

These variables contain feature literal references. By using these references, you can access the underlying property or method information.

You can define or use Gosu methods that use feature literals as arguments or return values.

Chaining feature literals by using feature literal syntax

Feature literals support type-safe chaining, so this code is valid.

```
var bossesNameRef = anEmp#Boss#Name
```

The chained feature literal refers to the name of the boss of the object in the `anEmp` variable.

Binding a property to an instance by using feature literal syntax

You can bind a feature literal to a specific object instance. This technique is the equivalent of using the compile-time type to request the property by name for a particular instance of the object.

Using the feature literal code from the previous example, this code creates a reference to a property on an employee instance.

```
var anEmp = new Employee() { :Name = "Joe", :Age = 32 }
```

```
// Get the property feature literal for a specific instance
var namePropForAnEmp = anEmp#Name
```

Binding argument values by using feature literal syntax

You can bind argument values in method references that use feature literal syntax.

Using this technique, you can refer to a method invocation with a particular set of arguments. Note that the second line in the following example does not invoke the `update` method. Instead, it gets a reference that you can use later to refer to the method on that instance.

```
var anEmp = new Employee() { :Name = "Joe", :Age = 32 }
var updateFuncForAnEmp = anEmp#update("Ed", 34)

print(anEmp.Name) // Prints "Joe". Code has not yet invoked the method reference.
```

Using feature literal syntax to convert method references to blocks

You can convert a method reference to a block by using the `toBlock` method.

The following example creates a block object for a call to the `update` method on the `anEmp` object.

```
var aBlock = anEmp#update("Ed", 34).toBlock()
```

You can use this technique to pass a method reference as an argument to a method that takes a block parameter.

Primitive and boxed types

In Gosu, primitive types such as `int` and `boolean` exist primarily for compatibility with the Java language. Gosu uses these Java primitive types to support extending Java classes and implementing Java interfaces.

From a Gosu language perspective, primitives are different only in subtle ways from object-based types such as `Integer` and `Boolean`. Primitive types can be automatically coerced to non-primitive versions or back again by the Gosu language in almost all cases. For example, `int` can be cast to `Integer` and `Boolean` cast to `boolean`. In typical code, you do not need to know the differences.

The `boolean` type is a Java primitive which is also called an “unboxed” type. In contrast, `Boolean` is a class that implements a “boxed” type of the `boolean` primitive. A boxed type is a primitive type wrapped in the shell of a class. Boxed types are useful for code that requires all values to have the common ancestor type `Object`. For example, Gosu collection data types typically require members to have types that descend from `Object`. Thus, collections can contain `Integer` and `Boolean`, but not the primitives `int` or `boolean`.

The Gosu boxed types use the Java primitive types. The boxed types are defined in the `java.lang` package, such as `java.lang.Integer`. Code execution performance is slightly improved when using a primitive rather than its boxed version.

There is an important difference between primitive and boxed types when handling uninitialized values. Variables declared of a primitive type cannot hold the `null` value. However, `null` can be assigned to variables of type `Object` and any subtype of `Object`.

Compound types

To implement other features, Gosu supports a special type called a compound type. A compound type combines an optional base class and additional interfaces. Typical usage of compound types occurs only when Gosu automatically creates a variable with a compound type because of type inference. In extremely rare cases, Gosu allows a variable to be declared explicitly with a compound type.

Suppose you use the following code to initialize list values with different types of objects. When used with initializers, the run-time type is always `ArrayList<Object, Object>`. The compile-time type depends on what you pass to the initializer. Gosu infers the type of the result list to be the least upper bound of the components of the list. If you pass different types of objects, Gosu finds the most specific type that includes all of the items in the list.

If the types implement interfaces, Gosu attempts to preserve commonality of interface support in the list type. This behavior ensures that your list acts as expected with APIs that rely on support for the interface. In some cases, the resulting type is a compound type, which combines:

- Zero classes or one class, which might be `java.lang.Object`, if no other common class is found
- One or more interfaces

At compile time, the list and its elements use the compound type. You can use the list and its elements with APIs that expect those interfaces or the ancestor element.

For example, the following code creates classes that extend a parent class and implement an interface.

```
interface HasHello {
    function hello()
}

class TestParent {
    function unusedMethod() {}
}

class TestA extends TestParent implements HasHello{
    function hello() {print("Hi A!")}
}

class TestB extends TestParent implements HasHello{
    function hello() {print("Hi B!")}
}

var newList = { new TestA(), new TestB()}
print("Run-time type = " + typeof newList)
```

This code prints the following line.

```
Run-time type = java.util.ArrayList<java.lang.Object>
```

The compile-time type is a combination of:

- The class `TestParent`, which is the most specific type in common for `TestA` and `TestB`
- The interface `HasHello`, which both `TestA` and `TestB` implement

Gosu also creates compound types in the special case of using the `delegate` keyword with multiple interfaces.

Using reflection

If you know what type an object is, you can use reflection to get information about the type or to perform actions on the object. *Reflection* means using type introspection to query or modify objects at run time. You can use reflection to retrieve metadata about classes, interfaces, fields, and methods at run time without knowing their names at compile time. You can also instantiate new objects and call methods with this information. For example, instead of calling an object method, you can get a list of methods from its type, or use a `String` runtime value to call a method by name.

Only use reflection if there is no other way to do what you need. Avoid using reflection to get properties or call methods. In almost all cases, you can write Gosu code to avoid reflection. Using reflection dramatically limits how Gosu and Guidewire Studio can alert you to problems at compile time. Detecting errors at compile time is better than encountering unexpected behavior at run time.

The following list summarizes the packages that support reflection in Gosu:

- `gw.lang.reflect`
Base reflection utilities for inspecting types, methods, annotations, and builders. The primary class is `ReflectUtil`, which provides object creation and method calls.
- `gw.lang.reflect.features.*`
Reference packages for inspecting properties, methods, constructors, and blocks.
- `gw.lang.reflect.gs`
Inspecting Gosu classes, methods, and properties.
- `gw.lang.reflect.interval`
Reflection iterators, intervals, and sequences.

```
gw.lang.reflect.java
    Inspecting Java classes, methods, and properties.
gw.lang.reflect.json
    Inspecting JSON objects.
gw.lang.reflect.module
    Inspecting modules and files.
```

To use reflection from Java or Gosu, use the utility class `gw.lang.reflect.ReflectUtil`, which contains methods to retrieve information about a class, including its properties and methods, and other methods to invoke object methods.

Getting properties by using reflection

The `type.TypeInfo` object includes a property called `Properties`, which contains a list of type properties. Each item in that list includes metadata properties such as the name (`Name`) and a short description (`ShortDescription`). Because of performance implications and the inability to find errors at compile time, only use reflection if there is no other way to do what you need.

To examine how reflection provides information about class properties, paste the following code into the Gosu Scratchpad.

```
var object = "This is a string"
var propNames = ""
var props = (typeof object).TypeInfo.Properties

for (prop in props) {
    propNames = propNames + prop.Name + " "
}

print(propNames)
```

This code prints lines similar to the following output.

```
Class Bytes Empty CASE_INSENSITIVE_ORDER Digits AsBigDecimal length size HasContent
NotBlank Alpha AlphaSpace Alphanumeric AlphanumericSpace Numeric NumericSpace Whitespace
```

Array syntax for getting a property by using reflection

Gosu also provides a concise *associative array* syntax to access properties by reflection. This syntax uses square brackets to access properties, similar to using arrays. The property name is case sensitive.

Note: The `[]` operator applies to all objects and has the typical meaning for `List`, `Array`, and `Map` objects. For other objects, the `[]` syntax performs a reflective look-up.

For example, paste the following code into the Gosu Scratchpad.

```
var dateObject = new Date()
var propertyName = "Hour"

// Get a property name using reflection
print(dateObject[propertyName])
```

If the time is currently 5:00 a.m., this code prints the following output.

```
5
```

Example

Consider the following Gosu class:

```
public class ObjClass {
    property get MyProperty() : String {
        return "Default value for MyProperty"
    }
}
```

The following code declares an object of class `ObjClass` and uses array syntax to access the `MyProperty` property by reflection:

```
var obj = new ObjClass()
var val = obj["MyProperty"]
```

This code is equivalent to the following:

```
var obj = new ObjClass()
var val = (typeof obj).TypeInfo.getProperty("MyProperty").Accessor.getValue(obj)
```

Retrieving methods by using reflection

You can use reflection to find all the methods that a class provides and to get information about individual methods. You can even call methods by name (given a `String` for the method name) and pass a list of parameters as object values. You can call a method using the method's `CallHandler` property, which contains a `handleCall` method.

Because of performance implications and the inability to find errors at compile time, only use reflection if there is no other way to do what you need.

To examine how reflection provides information about class methods, paste the following code into the Gosu Scratchpad.

```
var object = "This is a string"
var methodNames = ""
var methods = (typeof object).TypeInfo.Methods

for (method in methods) {
    methodNames = methodNames + method.Name + " "
}

print(methodNames)
```

This code prints lines similar to the following output, which is truncated for brevity.

```
wait() wait( long, int ) wait( long ) hashCode() getClass() equals( java.lang.Object )
toString() notify() notifyAll() @itype() compareTo( java.lang.String ) charAt( int )
length() subSequence( int, int ) indexOf( java.lang.String, int ) indexOf( java.lang.String )
indexOf( int ) indexOf( int, int ) codePointAt( int ) codePointBefore( int )
```

The following example gets a method by name and then calls that method. This example uses the `String` class and its `compareTo` method, which returns 0, 1, or -1. Paste the following code into the Gosu Scratchpad.

```
// Initialize test data
var objectText = "a"
var argumentText = "b"

// Get the list of methods
var methodInfos = String.TypeInfo.Methods

// Find a specific method by name.
// More than one method signature may exist for the same name, so the return type is an array
// of results with each method having different argument numbers or types
var compareToMethodName = methodInfos.firstWhere( \ elt -> elt.DisplayName == "compareTo" )
if (compareToMethodName == null) { throw "Could not find method by name" }

print("Full method name is: " + compareToMethodName.Name)
print("Number of parameters is: " + compareToMethodName.Parameters.length)
print("Name of first parameter is: " + compareToMethodName.Parameters[0].DisplayName)
print("Type of first parameter is " + compareToMethodName.Parameters[0].OwnersType.toString())

var res = compareToMethodName.CallHandler.handleCall( objectText, { argumentText } )
print("Call compareTo() using reflection: " + res)

var res2 = objectText.compareTo(argumentText)
print("Call compareTo() the normal way (with static typing): " + res2)
```

The code prints the following output.

```
Full method name is: compareTo( java.lang.String )
Number of parameters is: 1
Name of first parameter is: String
Call compareTo() using reflection: -1
Call compareTo() the normal way (with static typing): -1
```

Comparing types by using reflection

You can compare the types of two objects in several ways.

You can use the equality (==) operator to test types. However, the equality operator is almost always inappropriate because it returns true only for exact type matches. This operator returns false if one type is a subtype of the other or if the types are in different packages.

Use the `sourceType.Type.isAssignableFrom(destinationType)` method to determine whether types are compatible for assignment. This method considers the possibility of subtypes, such as subclasses, which the equality operator does not. The method determines if the destination type argument is either the same as, or a superclass or superinterface of the source type. The `sourceType.Type.isAssignableFrom(destinationType)` method examines only the supertypes of the source type. Although Gosu statements can assign a value of one unrelated type to another by using coercion, the `isAssignableFrom` method always returns false if coercion of the data is necessary. For example, although Gosu can use coercion to convert boolean to String or String to boolean, using those types with the `isAssignableFrom` method returns false.

Gosu provides a variant of the functionality that tests assignability with the Gosu `typeis` operator. The operator `typeis` operates between an object and a type. The method `type.Type.isAssignableFrom(...)` operates between a type and another type.

For example, paste the following code into the Gosu Scratchpad.

```
var str: String = "hello" // Explicit declaration of "String" is optional, but is shown for clarity
var objString: Object = "hello"
var objDouble: Object = 2.345
var bool: Boolean = true

print("Typeof str: " + (typeof str).Name)
print("Typeof objString: " + (typeof objString).Name)
print("Typeof objDouble: " + (typeof objDouble).Name)

// The following line would be a compile error! String is not a subtype or supertype of Boolean.
// tempTypeIs = (bool typeis String)

print("String typeis Object: " + (str typeis Object) )

// Notice the different run time behavior of these two lines,
// Note that both objString and obj3 are Object at compile time,
// but have different run-time types
print("Object objString typeis String: " + (objString typeis String) )
print("Object objDouble typeis String: " + (objDouble typeis String) )
print("Boolean assignable from String: " + (typeof str).isAssignableFrom((typeof bool)))
print("String assignable from Boolean: " + (typeof bool).isAssignableFrom((typeof str)))
print("Object assignable from String: " + ((Object).Type.isAssignableFrom( String )) )
print("String assignable from Object: " + ((String).Type.isAssignableFrom( Object )) )

// For typical use cases, DO NOT compare types with the == operator
// Using == to compare types returns false if one is a subtype.
// Instead, typically it is best to use the 'typeis' operator
print("Compare a string to object using ==: " + ( (typeof str) == Object ) )
```

This code prints the following output.

```
Typeof str: java.lang.String
Typeof objString: java.lang.String
Typeof objDouble: java.lang.Double
String typeis Object: true
Object objString typeis String: true
Object objDouble typeis String: false
Boolean assignable from String: false
String assignable from Boolean: false
Object assignable from String: true
String assignable from Object: false
Compare a string to object using ==: false
```

Java type reflection

Gosu implements a dynamic type system that is designed to be extended beyond its native objects. A dynamic type system enables Gosu to work with a variety of types. This capability is not the same as items being dynamically typed. Gosu is a statically typed language.

These types include Gosu classes, Java classes, business entity objects, typelists, XML types, SOAP types, and other types. These different types plug into Gosu's type system similarly to how Gosu business entities connect to the type system.

In almost all ways, Gosu does not differentiate between a Java class instance and a native Gosu object. Both sets of objects are exposed to the language through the same abstract type system. You can use Java types directly in your Gosu code. You can even extend Java classes by writing Gosu types that are subtypes of Java types. Similarly, you can implement or extend Java interfaces from Gosu.

The Gosu language transparently exposes and uses Java classes as Gosu objects through the use of Java `BeanInfo` objects. Java `BeanInfo` objects provide analogous information to what a Gosu `TypeInfo` object provides. Both `BeanInfo` and `TypeInfo` objects encapsulate type metadata, including properties and methods. All Java classes have `BeanInfo` information either explicitly provided with the Java class or dynamically constructed at run time. Gosu examines a Java class's `BeanInfo` and determines how to expose this type to Gosu. Because of this behavior, your Gosu code can use the Gosu reflection API with Java types.

In rare cases, you might need to do reflection on the underlying implementation Java class of a Gosu type rather than the Gosu type itself. The Java implementation of the Gosu class is also known as the backing class. For example, suppose you need to inspect each field in the implementing Java class, rather than the Gosu type. You can get the backing class by casting the type to `IHasJavaClass` as shown in the following example.

```
uses gw.lang.reflect.IHasJavaClass

// Get the Java backing class

var theType = Policy
var javaClass = (theType.Type as IHasJavaClass).BackingClass

print(javaClass.toString())
print(javaClass.Fields.length) // always zero

// With that class, you can iterate across the native Java fields at run time
for (field in javaClass.Fields index i) {
    print("Field ${i}:\t" + field.Name)
}
```

This code prints the output similar to the following lines.

```
class entity.Policy
110
Field 0:    TYPE
Field 1:    ACCOUNT_PROP
Field 2:    ARCHIVEDATE_PROP
Field 3:    ARCHIVEFAILURE_PROP
Field 4:    ARCHIVEFAILUREDETAILS_PROP
Field 5:    ARCHIVEPARTITION_PROP
Field 6:    ARCHIVESCHEMAINFO_PROP
...
```

The `IHasJavaClass` interface is implemented only by types that have a Java backing class.

Type system class

You can use the class `gw.lang.reflect.TypeSystem` for advanced type system introspection. For example, its `getByFullName` method returns a `Type` object from a `String` containing the fully qualified name of the type. You can perform further actions on the `Type` object, to get further information about the type or to instantiate an object of the type. Although each `Type` object itself has properties and methods, you typically use the properties and methods on the `type.TypeInfo` object. For example, you can get a type's complete set of properties and methods at run time by getting the `TypeInfo` object.

For example, the following code gets a type by a `String` version of its fully qualified name and instantiates an object of that type by using the type information.

```
var myFullClassName = "com.mycompany.MyType"
var type = TypeSystem.getByFullName( myFullClassName )
var instance = type.TypeInfo.getConstructor( null ).Constructor.newInstance( null )
```

Calling Gosu from Java by using reflection

From Java, you use reflection to access Gosu types. *Reflection* means asking the type system about types at run time to get data, set data, or invoke methods. Reflection is a powerful way to access type information at run time, but is not type safe. For example, language elements such as class names and method names are manipulated as `String` values. Be careful to correctly type the names of classes and methods because the Java compiler cannot validate the names at compile time.

To use reflection from Java, you use the same utility class `gw.lang.reflect.ReflectUtil` that provides reflection to Gosu classes. The `ReflectUtil` class contains methods to get information about a class, get information about members (properties and methods), and invoke static class methods or instance methods.

To access Gosu API classes from Java, including the classes that you use to implement reflection, you must add `gosu-core-api-release.jar` to your Java project's class path.

To run the Java code that accesses Gosu classes, you must be in the PolicyCenter application context. You cannot run that code from your Java IDE because the IDE project does not initialize Gosu. When PolicyCenter starts, it performs that initialization. You can test the Java class and its methods by running them in Guidewire Studio with a running server.

Improving type safety in using reflection

If you need to call methods on a Gosu class from Java, the following general steps increase the degree of type safety.

1. Create a Java interface containing only those methods that you need to call from Java. For getting and setting properties, define the interface using getter and setter methods, such as `getMyField` and `setMyField`.

```
package doc.example.reflection.java;

public interface MyJavaInterface {
    public String getMyField();
    public void setMyField(String value);
}
```

2. Compile the interface and place the class file in its complete hierarchy in the `PolicyCenter/modules/configuration/plugins/Gosu/classes` folder. If the interface extends a plugin interface, use the `PolicyCenter/modules/configuration/plugins/shared/classes` folder instead.
3. Create a Gosu class that implements that interface. The Gosu class can contain additional methods as well as the interface methods.

```
package doc.example.reflection.gosu

class MyGosuClass implements doc.example.reflection.java.MyJavaInterface {
    public static function myMethod(input : String) : String {
        return "MyGosuClass returns the value ${input} as the result!"
    }

    override property get MyField():String{
        print("Get MyField called")
        return "test"
    }
    override property set MyField(value:String){
        print("hello")
    }
}
```

4. In code that requires a reference to the object, use `ReflectUtil` to create an instance of the Gosu class. The following approaches are available.
 - Create an instance by using the `constructGosuClassInstance` method in `ReflectUtil`.

```
package doc.example.reflection.java;

import gw.lang.reflect.ReflectUtil;

public class MyJavaClass {
    ...

    public void accessGosuClass(String gosuClassName) {
```

```

        Object myGosuObject = ReflectUtil.constructGosuClassInstance(gosuClassName);
    }
}

```

- Define a method on an object that creates an instance, and call that method with the `invokeMethod` or `invokeStaticMethod` method in `ReflectUtil`, as appropriate.

To see a list of available methods on the created instance, use `ReflectUtil.invokeMethod(myGosuObject, "@IntrinsicType").getClass().getMethods()`. Getting the value of a property requires the `@` symbol before the property name. Alternatively, use the `getProperty` method.

Note: In both cases, any new object instances at compile time that are returned by `ReflectUtil` have the type `Object`.

5. Downcast the new instance to your new Java interface and assign a new variable of that interface type.

```
MyJavaInterface myJavaInterface = (MyJavaInterface) myGosuObject;
```

You now have an instance of an object that conforms to your interface.

6. Call methods on the Gosu instance as you normally would from Gosu. The getters, setters, and other method names are defined in your interface, so the method calls are type safe. For example, the Java compiler can protect against misspelled method names.

```
myJavaInterface.setMyField("New value");
```

Calling a Gosu method from Java by reflection

If you need to call a method by reflection without using the previous technique, you can pass parameter values as additional arguments to the `invokeMethod` or `invokeStaticMethod` method in `ReflectUtil`.

The following example Gosu class defines a static method:

```

package doc.example.reflection.gosu

class MySimpleGosuClass {
    public static function myMethod(input : String) : String {
        return "MyGosuClass method myMethod returns the value ${input} as the result!"
    }
}

```

The following Java code calls the static method defined in `MyGosuClass` by calling the `ReflectUtil.invokeStaticMethod` method:

```

package doc.example.reflection.java;

import gw.lang.reflect.ReflectUtil;

public class MyNonTypeSafeJavaClass {

    public void callStaticGosuMethod() {
        // Call a static method on a Gosu class
        Object o = ReflectUtil.invokeStaticMethod("doc.example.reflection.gosu.MySimpleGosuClass", "myMethod", "hello") ;

        // Print the return result
        System.out.println((String) o);
    }
}

```


Concurrency and thread safety

Gosu provides APIs that protect shared data from access from multiple threads.

If more than one Gosu thread interacts with data structures that another thread needs, you must ensure that you protect data access to avoid data corruption. Because this topic involves concurrent access from multiple threads, this issue is called *concurrency*. Code that is designed to safely get or set concurrently accessed data is called *thread safe*.

The most common situation that requires concurrency handling is the use of data in class static variables. Static variables are variables that are stored once for a class rather than once for each instance of the class. If multiple threads on the same Java virtual machine access this class, you must ensure that simultaneous accesses to this data safely get or set the data.

If you are certain that static variables or other shared data are necessary, you must ensure that you synchronize access to static variables. *Synchronization* refers to locking access between threads to shared resources such as static variables. For example, if you need to manage a single local memory cache that multiple threads use, you must carefully synchronize all reads and writes to shared data.

WARNING Static variables can be extremely dangerous in a multi-threaded environment. Using static variables in a multi-threaded environment can cause problems in a production deployment if you do not properly synchronize access. If such problems occur, they are extremely difficult to diagnose and debug. Timing in an multi-user multi-threaded environment is difficult, if not impossible, to control in a testing environment.

In a Guidewire application, some contexts always require proper synchronization. For example, in a plugin implementation exactly one instance of that plugin exists in the Java virtual machine on each server. Your plugin code must have the following attributes:

- Your plugin must support multiple simultaneous calls to the same plugin method from different threads. You must ensure multiple calls to the plugin never access the same shared data. Alternatively, protect access to shared resources so that two threads never access it simultaneously.
- Your code must support multiple simultaneous calls to a plugin instance. For example, PolicyCenter might call two different plugin methods at the same time. You must ensure multiple method calls to the plugin never access the same shared data. Alternatively, protect access to shared resources so that two threads never actually access it simultaneously.
- Your plugin implementation must support multiple user sessions. Do not assume shared data or temporary storage is unique to one user request, which is one HTTP request from a single user.

Gosu provides the following types of concurrency APIs to support writing thread-safe code:

Request and session variables

The classes `RequestVar` and `SessionVar` in the package `gw.api.web` synchronize and protect access to shared data. These APIs return a variable that is stored in either the request or the session.

Lazy concurrent variables

The `LockingLazyVar` class (in `gw.util.concurrent`) implements what is known as a lazy variable. Gosu does not construct a *lazy variable* until the first time any code uses the variable. Because the `LockingLazyVar` class uses the Java concurrency libraries, access to the lazy variable is thread-safe. The `LockingLazyVar` class wraps the double-checked locking pattern in a type-safe holder.

Concurrent cache

The `Cache` class in `gw.util.concurrent` declares a cache of values that you can look up quickly and in a thread-safe way. It declares a concurrent cache similar to a Least Recently Used (LRU) cache. Because the `Cache` class uses the Java concurrency libraries, access to the concurrent cache is thread-safe.

WARNING Caches can cause subtle problems. Use caches only as a last resort for performance.

Support for Java monitor locks, reentrant locks, and custom reentrant objects

Gosu provides access to Java-based classes for monitor locks and reentrant locks in the Java package `java.util.concurrent`. Gosu using clauses provide access to these classes and properly handle cleanup if exceptions occur. Gosu also provides a readable syntax for creating custom objects for reentrant object handling.

Concurrency API does not synchronize across clusters

None of these concurrency APIs affect shared data across clusters. In practice, the only data shared across clusters is entity data. PolicyCenter includes built-in systems to synchronize and intelligently cache entity data across a cluster.

Do not attempt to synchronize Guidewire entities

Never try to synchronize access to Guidewire entities defined in the data model configuration files. PolicyCenter automatically manages synchronized access and entity loading and caching, both locally and across a cluster if you use clusters.

A transaction bundle based on the class `gw.transaction.Bundle` is not thread-safe. Be aware of any concurrency issues when accessing entity instances in a bundle or committing those instances to the database. If multiple users can access the entity instances in a bundle simultaneously, you must use external synchronization to ensure reading and writing correct data, such as for property values.

WARNING Use external synchronization to ensure data integrity for concurrent access to entity instances in a transaction bundle.

See also

- “Request and session scoped variables” on page 367
- “Concurrent lazy variables” on page 368
- “Concurrent cache” on page 369
- “Concurrency with monitor locks and reentrant objects” on page 370
- “Bundles and database transactions” on page 333
- *Integration Guide*

Request and session scoped variables

To create a variable that can safely be accessed in the request or session scope, use the following two classes in the `gw.api.web` package:

- `RequestVar` – Instantiate this class to create a request-scoped variable stored in the web request
- `SessionVar` – Instantiate this class to create a session-scoped variable stored in the web session

These classes create a variable with a well-defined life cycle and attachment point of either the request or session. Use the `set` method to set the variable value. Use the `get` method to get the variable value.

In the base configuration, these classes do not work in the context of web service implementation or Gosu servlet classes. To enable web services and Gosu servlets as valid sources for variables in a `RequestVar` object, enable the `ScopedVariableSupportedForAllCapturedRequests` configuration parameter.

For both objects, Gosu provides a `RequestAvailable` property. Check the value of this boolean property to see if the `RequestVar` object has a request or the `SessionVar` object has a session available in this programming context. For example, if the code was called from a non-web unit test or a batch process, these properties return `false`. If `RequestAvailable` returns `false`, it is unsafe to call either `get` or `set` methods on the property. From Java, these properties appear as the `isRequestAvailable` method on each object.

WARNING Rule sets, plugin code, and other Gosu code can be invoked through various code contexts. You may not encounter contexts other than the user interface until late in system testing. When getting values `RequestVar` and `SessionVar`, always use `RequestAvailable` before trying to access the variable. Always handle the variable not being available or having a value of `null`.

The recommended pattern for concurrent use is to store the instance of the variable as a class variable declared with the `static` keyword. By using the `static` keyword on the class, many programming contexts can access the same session variable or request variable. Use the generics syntax `SessionVar<TYPE>` when you define the variable. For example, to store a `String` object, define the type as `SessionVar<String>`.

For example:

```
class MyClass {
    static var _varStoredInSession = new SessionVar<String>()

    static property get MySessionVar() : String {
        if (_varStoredInSession.RequestAvailable) {
            return _varStoredInSession.get()
        } else {
            return null // ENSURE THE CALLER CHECKS FOR NULL
        }
    }

    static property set MySessionVar(value: String) {
        if (_varStoredInSession.RequestAvailable) {
            _varStoredInSession.set(value)
        } else {
            // decide what to do if the request is not available
        }
    }
}
```

In any other part of the application, you can write code that sets this property:

```
MyClass.MySessionVar = "hello"
```

In another part of the application, you can write code that gets this property:

```
print( MyClass.MySessionVar )
```

This example explicitly checks the `RequestAvailable` property. In business code, for example to support your batch processes, you must decide what to do if `RequestAvailable` returns `false`.

Using `RequestVar` and `SessionVar` is strongly recommended, rather than the Java thread local API `java.lang.ThreadLocal<TYPE>`. Because application servers pool their threads, using `ThreadLocal` increases the risk of data remaining forever as a memory leak, because the thread to which you attached it never terminates. Also, pooled threads can preserve stale data from a previous request, because the server thread pool reuses the thread for

future requests. If you ever use a `ThreadLocal`, it is critical to be very careful to clean up all code with a `try/finally` block. Better practise is to convert that code to use `RequestVar` and `SessionVar`.

See also

- “Generics” on page 241
- *Integration Guide*

Concurrent lazy variables

In addition to using the Java native concurrency classes, Gosu includes utility classes that provide additional concurrency functionality. The `LockingLazyVar` class implements what is known as a lazy variable. Gosu constructs a *lazy variable* the first time some code uses the variable and not at the variable declaration. Because the `LockingLazyVar` class uses the Java concurrency libraries, access to the lazy variable is thread-safe. The `LockingLazyVar` class wraps the double-checked locking pattern in a type-safe holder.

In Gosu, the `LockingLazyVar.make(gw.util.concurrent.LockingLazyVar.LazyVarInit)` method signature returns the lazy variable object. This method requires a Gosu block that creates an object. Gosu runs this block on the first access of the `LockingLazyVar` value. The following example demonstrates the use of this method and clarifies the method signature:

```
var _lazy = LockingLazyVar.make( \-> new ArrayList<String>() )
```

The example passes a block as an argument to `LockingLazyVar.make(...)`. That block creates a new `ArrayList` that is parameterized to the `String` class. The parameter is a block that creates a new object. In this case, the block returns a new `ArrayList` object. You can create any object. In your business-use code, this block might be very resource-intensive in creating or loading this object.

It is best to let Gosu infer the correct type of the block and the result of the `make` method, as shown in this example. Using Gosu type inference simplifies your code because you do not need to use explicit Gosu generics syntax to define the block type, such as the following version:

```
var _lazy : LockingLazyVar<List<String>> = LockingLazyVar.make( \-> new ArrayList<String>() )
```

To use the lazy variable, call its `get` method:

```
var i = _lazy.get()
```

If the block has not yet run, Gosu runs the block when you access the lazy variable. If the block has already run, Gosu returns the cached value of the lazy variable and does not rerun the block.

A good approach to using a lazy variable is to define the variable as static and then define a property accessor function to abstract the implementation of the variable. The following code shows an example inside a Gosu class definition:

```
class MyClass {
    // Lazy variable using a block that calls a resource-intensive operation that returns a String
    var _lazy = LockingLazyVar.make( \-> veryExpensiveMethodThatReturnsAString() )

    // Define a property get function that gets this value
    property get MyLazyString() : String {
        return _lazy.get()
    }
}
```

If any code accesses the property `MyLazyString`, Gosu calls its property accessor function. The property accessor always calls the `get` method on the object. Gosu runs the very expensive method only once, the first time that code accesses the lazy variable value. If any code accesses this property again, Gosu uses the cached value and does not execute the block again. This behavior is useful in cases where you want a system to start quickly and pay only incremental costs for resource-intensive value calculations.

Optional non-locking lazy variables

Gosu also provides a non-locking variant of `LockingLazyVar` called `LocklessLazyVar`. Use `LocklessLazyVar` if you do not need the code to be thread-safe. Because this class does not lock, it performs faster and has less chance of

server deadlock. However, this class is unsafe in any context that has potential concurrent access and thus requires thread safety.

Concurrent cache

A similar class to the `LockingLazyVar` class is the `Cache` class. An instance of this class declares a concurrent cache of values that you can look up quickly and in a thread-safe way. The cache is similar to a Least Recently Used (LRU) cache. Because the `Cache` class uses the Java concurrency libraries, access to the cache is thread-safe. Each entry in the cache is defined by a key, which is also called an input value. A block uses the value of the key to calculate the value of the cache entry.

The constructor for a cache requires:

- A name as a `String`. The implementation uses this name to generate logging for cache misses.
- The size, as a number of slots.
- A block that defines a function to calculate a value from a key. Typically, this calculation is resource-intensive.

Use the key and value types to parameterize the `Cache` type using Gosu generics syntax. For example, if you need to pass a `String` to the cache and get an `Integer` back, create a new `Cache<String, Integer>`.

To use the cache, call the `get` method and pass an input value, which is the key. If the value for the key is in the cache, `get` returns that value. If the value is not cached, Gosu calls the block to calculate the value from the key and then caches the result. On any subsequent call of the `get` method, the value is the same but Gosu uses the cached value, if it is still in the cache. If too many items were added to the cache and the required item is unavailable, Gosu reruns the block to regenerate the value. Gosu then caches the result again.

To use a cache within another class, you can define a static instance of the cache. The static variable definition defines your block. Because the `Cache` class uses the Java concurrency libraries, this static implementation is thread-safe. For example, in your class definition, define a static variable like this:

```
static var _myCache = new Cache<String, Integer>( "StrToInt", 1000, \ str -> getMyInt( str ) )
```

To use your cache, get a value from the cache using code like the following. In this example, `inputString` is a `String` variable that may or may not contain a `String` that you used before with this cache:

```
var fastValue = _myCache.get( inputString )
```

The first time you call the `get` method, Gosu calls the block to generate the `Integer` value.

On any subsequent call of the `get` method, the value is the same but Gosu uses the cached value, if it is still in the cache. If too many items were added to the cache and the required item is unavailable, Gosu reruns the block to regenerate the value. Gosu then caches the result again in the concurrent cache object.

An even better way to use the cache is to abstract the cache implementation into a property accessor function. A private static object `Cache` object, as shown in the previous example, can handle the actual cache. For example, define a property accessor function such as:

```
static property get function MyInt( str : String ) {  
    return _myCache.get( str )  
}
```

These code examples are demonstrations that have a simple operation in the block. Typically, the overhead of maintaining the cache is appropriate if your calculation is resource-intensive and you expect repeated access with the same input values.

WARNING Incorrect implementation of caching can lead to run-time errors and data corruption. Only use caches as a last resort for performance issues.

Create a thread-safe cache

Procedure

1. Decide the key and value types for your cache based on input data.

For example, you need to pass a `String` and get an `Integer` back from the cache.

2. Construct a new cache.

For example:

```
// A cache of string values to their upper case values
var myCache = new Cache<String, String>( "My Uppercase Cache", 100, \ s -> s.toUpperCase() )
```

3. To use the cache, call the `get` method and pass the input value, which is the key. If the value is in the cache, `get` returns the value from the cache. If the value is not cached, Gosu calls the block and calculates the value from the key and then caches the result.

For example:

```
print(myCache.get("Hello world"))
print(myCache.get("Hello world"))
```

This code prints:

```
"HELLO WORLD"
"HELLO WORLD"
```

Result

In this example, the first call of the `get` method calls the block to generate the uppercase value. On the second call of the `get` method, the value is the same but Gosu uses the cached value.

Concurrency with monitor locks and reentrant objects

From Gosu, you can use the Java 1.5 concurrency classes in the package `java.util.concurrent` to synchronize a variable's data to prevent simultaneous access to the data.

The most straightforward implementation of a lock is to define a static variable for the lock in your class definition. Next, define a property get accessor function that uses the lock and calls another method to perform the task that you must synchronize. This approach uses a Gosu `using` clause with reentrant objects to simplify concurrent access to shared data. The `using` statement syntax for lock objects causes Gosu to consider the object reentrant.

A *reentrant* object supports multiple concurrent uses of a single lock in a single thread. Reentrancy maintains a count of the number of times the thread sets and releases the lock. In reentrant lock use, the thread does not deadlock on setting an already locked lock, nor release the lock when other code in the thread still retains that lock. Reentrant objects help manage safe access to data that is shared by reentrant or concurrent code execution. For example, if you must store data that is shared by multiple threads, ensure that you protect against concurrent access from multiple threads to prevent data corruption. The most prominent type of shared data is class static variables, which are variables that are stored on the Gosu class itself.

For Gosu to recognize a valid reentrant object, the object must have at least one of the following attributes:

- The object implements the `java.util.concurrent.locks.Lock` interface. For example, the following Java classes in that package implement this interface: `ReentrantLock`, `ReadWriteLock`, `Condition`.
- You cast the object to the Gosu interface `IMonitorLock`. You can cast any arbitrary object to `IMonitorLock`. It is useful to cast Java monitor locks to this Gosu interface.
- The object implements the Gosu class `gw.lang.IReentrant`. This interface contains two methods with no arguments: `enter` and `exit`. Your implementation code must properly lock or synchronize data access as appropriate during the `enter` method and release any locks in the `exit` method.

The profiler tag class (`gw.api.profiler.ProfilerTag`) implements the `IReentrant` interface. This class adds hints to the Gosu profiler about what actions happen in a block of code.

For blocks of code that use locks by implementing `java.util.concurrent.locks.Lock`, a `using` clause simplifies the code. The `using` statement always cleans up the lock, even if the code throws an exception.

```
uses java.util.concurrent

// In your class variable definitions...
var _lock : ReentrantLock = new ReentrantLock()

...

function useReentrantLockNew() {
    using ( _lock ) {
        // Do your main work here
    }
}
```

Similarly, you can cast any object to a monitor lock by adding `as IMonitorLock` after the object. For example, the following method code uses the object itself, by using the keyword `this`, as the monitor lock:

```
function monitorLock() {
    using ( this as IMonitorLock ) {
        // Do stuff
    }
}
```

This approach is equivalent to a `synchronized` block in the Java language.

The following code uses the `java.util.concurrent.locks.ReentrantLock` class using more verbose `try` and `finally` statements. This approach is not recommended:

```
// In your class variable definitions...
var _lock : ReentrantLock = new ReentrantLock()

function useReentrantLockOld() {
    _lock.lock()
    try {
        // Do your main work here
    }
    finally {
        lock.unlock()
    }
}
```

Alternatively, you can do your change in a Gosu block. Guidewire does not recommend this approach. Returning the value from a block imposes more restrictions on how you implement return statements. It is typically better to use the `using` statement structure shown earlier in this topic.

See also

- “Object life-cycle management with using clauses” on page 120
- [http://en.wikipedia.org/wiki/Monitor_\(synchronization\)](http://en.wikipedia.org/wiki/Monitor_(synchronization))
- <http://docs.oracle.com/javase/tutorial/essential/concurrency/index.html>

Checksums

Gosu provides an API for generating checksums. A *checksum* is a calculated number to detect modification of data in transit. Such modification can be either accidental or malicious. For example, you can use a checksum to detect corrupted stored data or errors in a communication channel. Longer checksums such as 64-bit checksums are also known as *fingerprints*.

To improve detection of accidental modification of data in transit, you can use checksums. A checksum is a computed value generated from an arbitrary block of digital source data. To check the integrity of the data at a later time, recompute the checksum and compare it with the stored checksum. If the checksums do not match, the data was altered either intentionally or unintentionally. For example, this technique can help detection of physical data corruption or errors in a communication channel.

Be aware that checksums cannot perfectly protect against intentional corruption by a malicious agent. A malicious attacker could modify the data in a way that preserves its checksum value or, depending on transport integrity, substitute a new checksum. To guard against malicious changes, use encryption at the data level with a cryptographic hash or the transport level, such as SSL/HTTPS.

WARNING Checksums improve detection from accidental modification of data but cannot detect all intentional corruption by a malicious agent. If you need that level of protection, use encryption instead of checksums, or in addition to checksums.

You can also use fingerprints to design caching and synchronizing algorithms that check whether data changed since the last cached copy. You can save the fingerprint of the cached copy and an external system can generate a fingerprint of its most current data. If you have both fingerprints, compare them to determine if you must resynchronize the data. To work effectively, the fingerprint algorithm must provide near certainty that a real-world change would change the fingerprint. Having two different values generate a matching fingerprint is called a *collision*. A fingerprint uniquely identifies the data for most practical purposes, although changed data causing a collision is theoretically possible.

Gosu provides support for 64-bit checksums in the class `FP64` in the package `gw.util.fingerprint`.

The `FP64` class provides methods for computing 64-bit fingerprints of the following kinds of data:

- String objects
- Character arrays
- Byte arrays
- Input streams

Fingerprints provide a probabilistic guarantee that defines a mathematical upper bound on the probability of a collision. A *collision* occurs if two different strings have the same fingerprint. Using 64-bit fingerprints, the odds of a collision are extremely small. The odds of a collision between two randomly chosen texts a million characters long are less than 1 in a trillion.

Suppose you have a set S of n distinct strings each of which is at most m characters long. The odds of any two different strings in S having the same fingerprint is described by the following equation where k is the number of bits in the fingerprint:

$$(nm^2) / 2^k$$

For practical purposes, you can treat fingerprints as uniquely identifying the bytes that produced them. The following mathematical notation demonstrates this assumption, given two `String` variables `s1` and `s2`, using the $\rightarrow$ symbol to mean “implies”:

```
new FP64(s1).equals(new FP64(s2))  $\rightarrow$  s1.equals(s2)
```

Do not fingerprint the value of a fingerprint by fingerprinting the output of the `FP64` methods `toBytes` and `toHexString`. Using this type of fingerprint might lead to unexpected collisions. The shorter length of the fingerprint itself invalidates the probabilistic guarantee that a collision is unlikely to occur.

Creating a fingerprint

To create a fingerprint object, instantiate the `gw.util.fingerprint.FP64` object and pass one of the supported object types to the constructor:

- A `String` object:

```
var s = "hello"
var f = new FP64(s)
```

- A character array:

```
var s = "hello"
var ca : char[] = {s[0], s[1], s[2], s[3], s[4]}
var f = new FP64(ca)
```

An alternative method signature takes extra parameters for start position and length of the series of characters in the array.

- A byte array:

```
var ba = "hello".Bytes // Or use "hello".getBytes()
var f = new FP64(ba)
```

An alternative method signature takes extra parameters for start position and length of the series of bytes in the array.

- A stream object:

```
var s = "testInputStreamConstructor"
new FP64(new ByteArrayInputStream(gw.util.StreamUtil.toBytes(s)));
```

- An input stream:

```
var s = "testInputStreamConstructor"
new FP64(new StringBuffer(g));
```

- Another `FP64` fingerprint object to duplicate the fingerprint:

```
var s = "hello"
var f = new FP64(s)
var f2 = new FP64(f)
```

Generating the fingerprint binary data

To generate output data from a fingerprint, use the `FP64` method `toBytes()`, which returns the value of this fingerprint as a newly allocated array of 8 bytes.

Instead of the no-argument method, you can also use the alternative method signature that takes a byte array buffer into which the method writes the bytes. The buffer must have a length of at least 8 bytes.

Alternatively, you can use a method `toHexString()`. This method returns the fingerprint as an unsigned integer encoded in base 16 (hexadecimal) and padded with leading zeros to a total length of 16 characters.

Fingerprints as keys in hash maps

Because the `FP64` class overrides the Java methods `equals` and `hashCode`, you can use `FP64` objects as keys in hash tables. For example, use `FP64` values as keys in a `java.util.HashMap` instance.

Extending a fingerprint

The `FP64` class provides methods for extending an existing fingerprint by more bytes or characters. This functionality is useful if the only change to the source data was appending a known series of bytes to the end of the original `String` data.

To produce a fingerprint equivalent to the fingerprint of the concatenation of two `String` objects, you can extend the fingerprint created from one `String` using another `String`. Given the two `String` variables `s1` and `s2`, the following is true:

```
new FP64(s1 + s2).equals( new FP64(s1).extend(s2) )
```

The same logic is true for character arrays, not just `String` objects.

All operations for extending a fingerprint are destructive. The operations modify the fingerprint object directly, in place. All operations return the resulting `FP64` object, so you can chain method calls together, such as in the following code:

```
new FP64("x").extend(foo).extend(92))
```

If you need to make a copy of a fingerprint, instantiate the `FP64` object and use the `FP64` object to copy as the constructor argument:

```
var original = new FP64("Hello world")
var copy = new FP64(original) // A duplicate of the original fingerprint
```


Gosu programs and the command-prompt tool

A *Gosu program* is a file with a `.gsp` file extension that you can run directly from the Gosu command-prompt tool. This tool encapsulates the Gosu language engine.

You can use the Gosu language command-prompt tool outside the PolicyCenter server to perform the following tasks:

- Invoke Gosu programs (`.gsp` files), which can use Gosu classes, Gosu extensions, and Java classes
- Evaluate Gosu expressions passed to the command prompt.

Accessing entities and other types from the Gosu command-prompt tool

Gosu programs that you run with the Gosu command-prompt tool cannot access some types available from PolicyCenter Studio. For example, you cannot access entity types, PCF types, and plugin types. Some types have different sets of methods because some Gosu enhancements are unavailable in the command-prompt tool.

IMPORTANT Guidewire does not support the import of built-in PolicyCenter application JAR files into Gosu command-prompt tool programs.

To add, edit, delete, or query PolicyCenter entity instances from a Gosu program, implement the majority of your code as a PolicyCenter WS-I web service. From your Gosu program, read the command-prompt arguments, and then call the web service.

See also

- “Command-prompt arguments” on page 380
- *Integration Guide*

Gosu programs

The structure of a Gosu program

A Gosu program includes one or more lines that contain Gosu statements. A Gosu program can also include the following elements:

- A metaline
- One or more arguments from the command prompt
- One or more functions

See also

- “Command-prompt arguments” on page 380
- “Create and run a Gosu program that processes arguments” on page 380

Metaline as first line

Gosu programs support a single line at the beginning of the program for specifying the executable with which to run a file. This line is for compliance with the UNIX standard for shell script files. The metaline is optional. If present, this line must be the first line of the program. The metaline looks like the following:

```
#!/usr/bin/env gosu
```

The # character that begins the metaline does not start a comment line in Gosu programs. The # character is not a valid line-comment start symbol.

Functions in a Gosu program

Your Gosu program can define functions in the same file and call those functions.

For example, the following program creates a function and calls the function twice:

```
print(sum(10, 4, 7));
print(sum(222, 4, 3));

function sum(a: int, b: int, c: int) : int {
    return a + b + c;
}
```

Running this program prints:

```
21
229
```

Write and run a Gosu program

You can run a Gosu program from a command prompt. The Gosu program uses the same keywords and syntax as a Gosu class.

Before you begin

The following instructions describe running a basic Gosu program after you set up the gosu command-prompt tool.

See also

- “Setting up the Gosu command-prompt tool” on page 379

Procedure

1. Create a file called `myprogram.gsp` containing only the following line:

```
print("Hello World")
```

2. Open a command prompt.
3. Change your working directory to the directory that contains your program.
4. Type the following command:

```
gosu myprogram.gsp
```

If you have not yet added the gosu executable folder to your system path, instead type the full path to the gosu executable.

The tool runs the program.

The program prints the following output:

```
Hello World
```

Gosu command-prompt-tool

Setting up the Gosu command-prompt tool

The PolicyCenter installation directory includes the Gosu tool as a subdirectory named `admin`. The `admin` directory includes the following files and directories:

Path	Purpose
<code>/bin/gosu.cmd</code>	The Windows tool that invokes Gosu
<code>/src/gw/cmdline/util/*.gs</code>	Gosu classes for the command-prompt tool
<code>/lib/*.jar</code>	Java archive (JAR) files that contain core Gosu libraries

To use the Gosu tool on another computer, copy the entire `admin` directory to that computer. Ensure that the computer has a supported version of the Java run time.

IMPORTANT You are licensed to use the Gosu tool only to work with Guidewire applications.

You can change your system's path to add the Gosu tool `bin` directory so that the command prompt can find the gosu command. In the base configuration, this directory is `PolicyCenter/admin/bin`.

On Windows, modify the system Path variable by going to the **Start** menu and choosing **Control Panel**→**System**→**Advanced System Settings**→**Environment Variables**. In **System variables**, choose the Path variable. At the end of the variable value, type a semicolon and the full path to the `bin` directory in the Gosu tool directory. For example, suppose you copied the contents of the `admin` directory to the path:

```
C:\gosu\
```

Add the following to the system path:

```
;C:\gosu\bin
```

To test this path, close any existing command-prompt windows, and then open a new command-prompt window. Type the following command:

```
gosu -help
```

If the Gosu help information appears, the Gosu tool is installed correctly.

Command-prompt tool and options

To run the Gosu command-prompt tool, you use the `gosu` command.

The following table lists the command-prompt tool parameters and options.

Task	Options	Example
Default behavior of the command prompt tool with no options is to run Gosu Lab.		<code>gosu</code>
Run a Gosu program. Include the <code>.gsp</code> file extension when specifying the file name.	<code>filename.gsp</code>	<code>gosu myprogram.gsp</code>
Enable checking for numeric overflow in arithmetic operations. This option requires a Gosu program to run.	<code>-checkedArithmetic filename.gsp</code>	<code>gosu -checkedArithmetic myprogram.gsp</code>
Add more paths to the search path for Java classes or Gosu classes. Separate paths with semicolons.	<code>-classpath path</code>	<code>gosu -classpath C:\gosu\projects\libs</code>
Evaluate a Gosu expression at the command prompt. Surround the entire expression with quotation marks. For any quotation mark in the expression, replace it with three quotation marks. For other special DOS characters such as <code>></code> and <code><</code> , precede them with a caret (^) symbol.	<code>-e expression</code> <code>-eval expression</code>	<code>gosu -e "new java.util.Date()"</code> <code>gosu -e """"a"""+""""b""""</code>
Print help information for this tool.	<code>-h</code> <code>-help</code>	<code>gosu -h</code>
Show Gosu version	<code>-v</code> <code>-version</code>	<code>gosu -version</code>

See also

- “Checked arithmetic for add, subtract, and multiply” on page 76

Command-prompt arguments

You can access command-prompt arguments to Gosu programs by manipulating raw arguments. You can parse the full list of arguments from the command line as positional parameters. Alternatively, you can write a custom parser to identify named options and their values. For example, you can write a parser for an option that has multiple parts that are separated by space characters, such as `-username jsmith`. In this example, each of the `-username` and the `jsmith` components is a separate raw argument.

To get the full list of command-prompt arguments as a list of `String` values, use the `RawArgs` property of the `gw.lang.Gosu` class. This property provides an array of `String` values. You can access the values in this array in the same way as any other array. For example, you can use Gosu code like the following lines.

```
// TestArgs.gsp
var myArgs = Gosu.RawArgs
var numArgs = myArgs.Count
print("${myArgs} ${numArgs}")
for (str in myArgs) {
    print("${str}")
}
```

You can run the program with the following command.

```
TestArgs one two three
```

This code prints the following lines.

```
[one, two, three] 3
one
two
three
```

Create and run a Gosu program that processes arguments

A Gosu program that you run from the command prompt can read arguments from the command line and report errors in the values of those arguments.

Procedure

1. Choose a directory in which to save your command-prompt tool. In this directory, create a subdirectory called `test`, which is the package name.
2. Create a Gosu class that defines your properties. The following code defines two properties, one `String` property named `Name` and a `boolean` property named `Hidden`:

```
package test

class Args {
    // String argument
    static var _name : String as Name = "No name"

    // boolean argument -- No value to set on the command line
    static var _hidden : boolean as Hidden = false
}
```

3. Save this Gosu class file as the file `Args.gs` in the `test` directory.
4. Create a Gosu program, using the following code:

```
uses test.*
var args = Gosu.RawArgs
var numArgs = args.Count

try {
    if (arg.startsWith( "-" )) {
        if ((arg == "-name") and (i< numArgs-1)) {Args.Name = args[i+1]}
        if (arg == "-hidden") {Args.Hidden = (i< numArgs-1) and (args[i+1] == "false")?false:true}
    }
} catch (Exception) {
    print("Error parsing args: " + args)
}

print("hello " + Args.Name)
print("you are " + (Args.Hidden ? "hidden" : "visible") + "!!!!")
```

5. Click **Save As** and save this new command-prompt tool as `myaction.gsp` in the directory that contains the `test` subdirectory.
6. Open a command-prompt window and navigate to the directory that contains `myaction.gsp`.
7. In the command-prompt window, enter the following command

```
gosu -classpath "." myaction.gsp -name John -hidden
```

This command prints:

```
hello John
you are hidden!!!!
```

